

Mathematical inference

science and logic

Jelmer de Boer

Master Thesis Artificial Intelligence

2026



Open Universiteit

*“I got bored one day and I put everything on a bagel...
everything: All my hopes and dreams, my old report
cards, every breed of dog, every last personal ad on
Craigslist, sesame, poppy seed, salt... and it collapsed
in on itself. ’Cause you see, when you really put
everything on a bagel, it becomes this: The truth.”*

*Jobu Tupaki in *Everything Everywhere All At Once**

Contents

1	Prelude: Andante sostenuto	1
1.1	Inspiration	2
1.2	Introduction	2
1.3	Outline	6
1.4	Idiosyncrasies	6
2	First movement: Allegro con brio	9
2.1	Solid foundations for mathematics	10
2.2	Solid foundations for statistics	11
2.3	A good and honest attitude for doing these sciences	12
2.4	Mathematical manifesto	13
3	Second movement: Andante con moto	15
3.1	The Jaynes–Cox programme	16
3.2	Objections	17
4	Third movement: Adagio	25
4.1	Different types of logic	26
4.2	Type theory	27
4.3	Martin-Löf-style type theory	29
5	Fourth movement: Allegro maestoso	31
5.1	Context manipulation in Martin-Löf-style type theory	32
5.2	Modalities inside Martin-Löf-style type theory	32
5.3	Probability types	33
5.4	Structural causal models in Martin-Löf-style type theory	35
5.5	Lean 4 implementation	40
5.6	Intermediate results	44
6	Fifth movement: Presto con fuoco	45
6.1	Questionable research practices on a large scale	47
6.2	Start of a research career	49
6.3	Reporting in journals	50
6.4	Peer review and editorial process	51
6.5	Career advancement	52
6.6	Reform proposals	53

Contents

Acknowledgements	59
Appendices	61
A Definitions	63
A.1 Type-theoretic definitions	65
A.2 Modal definitions	83
A.3 Probabilistic definitions	87
A.4 Finite causal models	105
A.5 Do-calculus	128
A.6 Counterfactual semantics	135
A.7 Modal causal representation	138
A.8 Examples	146
B Proofs and arguments	149
B.1 Causality argument	150
B.2 Physics argument	151
B.3 Excluded middle is not derivable	152
B.4 Lean verification and foundational boundary	154
B.5 Clayton–Waddington probability proof	156
B.6 Finite causal-model correspondence	166
B.7 Modal and structural causal execution	171
B.8 Counterfactual and multiworld execution	176
B.9 Finite-source certificate transport	179
B.10 Do-calculus identification and completeness transport	180
B.11 Counterfactual completeness transport	186
B.12 Finite SCM construction and latent projection	187
B.13 Completeness consequences and boundary	188
Bibliography/sources	191

CHAPTER 1

Prelude: Andante sostenuto

“Science, including philosophy, is intrinsically polemical, and the polemical cannot be separated from the agonistic. In epochs in which great new things arise, the agonistic factor usually comes to the fore”

Johan Huizinga in *Homo Ludens*

1.1 Inspiration

When I was an undergraduate studying physics, probability theory often felt oddly under-defined. In statistical physics, and even more sharply in quantum physics, probability seemed to appear sometimes as ignorance, sometimes as frequency, sometimes as a physical tendency, and sometimes merely as a useful rule for calculation. The mathematical formalism worked well enough; that was not the issue. The problem was that there didn't seem to be a coherent interpretation, so the underlying logical story remained inconsistent.

After my studies I discovered E. T. Jaynes' *Probability Theory: The Logic of Science* [Jay03]. That book was, for me, the book I would have liked to have read much earlier. Jaynes, also trained as a physicist, refused to treat probability as a bag of unrelated conventions. He presented it, instead, as a disciplined extension of logic: a way of saying what one should believe, given what one already knows. The attraction of this view was not only technical: it also restored a kind of intellectual honesty to the subject, since hidden assumptions and convenient interpretations could no longer be smuggled in without being noticed.

Yet, Jaynes' book stops before the story is finished. Part of this is literal: the book was posthumously released on the basis of an unfinished manuscript. Part of it is conceptual: Jaynes gave a powerful solution for probability, following and extending Cox's programme [Cox46; Cox61], but several of his broader complaints about mathematics, subjectivity, causality and scientific practice remain only partially resolved. This thesis is therefore both a tribute and an attempt at continuation. It asks what Jaynes' programme might look like if it were done in an epistemological, constructive, type-theoretic and causal setting.

1.2 Introduction

The central concern of this work is the process by which new knowledge is added to existing information. I will call this process "mathematical inference". It is not a marginal technicality; it is, or at the very least should be, the core of scientific reasoning. Jaynes was therefore not simply concerned with the correctness of a few specific statistical formulae: he thought that the core of science itself was at stake, because large parts of mathematical and statistical practice were built on foundations that were too permissive, too tolerant. His polemical tone makes sense in this context: if the problem is pervasive and foundational, then a mere technical correction would miss part of the point.

The urgency that is felt in Jaynes' text also perfectly fits the opening quotation from Huizinga on the polemical and agonistic nature of defending new scientific paradigms [Hui49]. This urgency becomes even more justified if one looks at two interrelated phenomena: independent analysts reaching divergent conclusions from the same data [Bot+20; Bre+22; Sch+21; Sil+18], and large replication projects that reveal failures to replicate important and accepted scientific findings [Cam+16; Kle+14; Ope15]. Based on these phenomena, it becomes abundantly clear that a well-defined theory and formalism for mathematical inference is not a mere theoretical exercise, but an urgent matter.

Three concerns run through this thesis:

1. Mathematics should have foundations that make assumptions explicit and prevent uncontrolled use of ideal objects like infinity;
2. Mathematical inference should be treated as a principled extension of logic, while also being honest about the limits of that derivation;
3. Scientific practice should be organised in such a way that good inference is encouraged rather than punished.

Jaynes addresses all three, but solves only part of the second. The present work takes these concerns together, since separating them would miss part of the problem.

The formal answer developed here starts from Martin-Löf-style type theory [DP24; Mar84; NPS90]. This is not chosen for foundational purity: it is chosen because constructive type theory forces one to be explicit about what is being constructed, in which context, and by which rules. It also connects mathematics to computation, so that the reconstruction is not just a philosophical exercise, but computationally checkable. For this reason, it is also a natural environment for rebuilding finite probability theory, especially in light of finitary refinements of Cox-style arguments [CW17].

Moreover, if probability is called subjective, then subjectivity should not merely mean that one may freely choose a prior – as is the case in Jaynes’ story. A subjective theory should also be able to represent which questions are in scope, which facts are remembered, which possibilities are being imagined, and how relations between concepts change. Also for this reason, Martin-Löf-style type theory proves to be an excellent basis, as its intuitionistic and epistemological foundations naturally support these operations on contexts [DP24; SN25]. However, to explicate that specific operations on the context change someone’s knowledge, someone’s *epistemic state*, modes and modalities are introduced. This introduction is made possible by modal dependent type theory, which extends Martin-Löf-style type theory so that these modes can be described natively [Gra+21; Gra23; GSB19].

The probabilistic part is not the whole story: Jaynes presents probability theory as a universal logic of science, but the mature formal theory of causal models was developed largely outside his work. To extend Jaynes’ probabilistic framework beyond prediction, Pearl’s structural causal models and do-calculus therefore have to be incorporated into the framework [Pea09; PM18].

Finally, inference is not performed in a vacuum. McElreath’s presentation of Bayesian modelling is valuable not only because it teaches calculations, but also because it keeps statistical inference close to modelling choices, scientific questions and workflow [McE20]. The reproducibility crisis and the diverging conclusions from competent researchers mentioned earlier illustrate that even technically competent inference can become unreliable inside a bad scientific pipeline [GL13; Ioa05]. Good science needs good inference, but also a good environment in which inference takes place.

Scope and assumptions

This is a thesis in artificial intelligence, where artificial intelligence is understood broadly as the formalisation and implementation of inference. In this sense probability theory, logic, causality and proof checking all fall naturally within its

1. Prelude: Andante sostenuto

scope. The aim is not to build a large applied AI system, but to investigate what kind of mathematical inference such systems should ideally perform when they reason under uncertainty, act under causal assumptions, or revise an epistemic state.

The thesis looks at inference on both the mathematical and sociological level. Mathematically, it asks which formal rules should ideally govern uncertain, subjective and causal inference. Sociologically, it asks in what scientific environment mathematical and statistical inferences are generally made. This distinction matters, because the proposed formal machinery may not fit comfortably inside every existing scientific habit; and if that turns out to be the case, the problem may lie – at least partly – with those habits.

The mathematical scope is deliberately restricted: finite probability spaces; finite causal models; explicitly representable contexts and epistemic states; and central objects that are constructible and implementable. Thus the technical development is discrete throughout, but more narrowly so: the results below are finite results, not a general theory of arbitrary discrete spaces. This restriction is not merely a convenience: it follows Jaynes’ suspicion of uncontrolled infinities; it follows the constructive spirit of Martin-Löf-style type theory; and it fits the computational aspect of artificial intelligence.

Selected formal parts are implemented in Lean 4 [MU21] where this is useful and feasible. Lean is used here as a proof checker for representative definitions, proofs and examples, not as a promise that every philosophical claim in the thesis will be mechanised: the point is more modest and more important. The central mathematical constructions should be expressible in a form precise enough for independent mechanical checking. Recent discussions of digitised proofs and AI-assisted mathematics make clear that this is becoming an increasingly relevant part of mathematical practice [Cas26; Kak26; Orn26; Slo26].

Several limitations follow directly from this scope: this work does not reconstruct all of classical probability theory; it does not provide a general theory of continuous probability spaces; it does not attempt to solve measure theory; it does not claim that Martin-Löf-style type theory is the only acceptable foundation for mathematics; it does not claim that Pearl’s framework captures everything that can meaningfully be called causality; and it does not pretend to give a complete empirical sociology of science. The aim is narrower: to construct a coherent finite framework in which probability, subjectivity and causality can be represented with as little hand-waving as possible, and to fit this framework within its practical context.

Research questions

The thesis is guided by the following research questions:

1. Can Jaynes’ probabilistic framework be cast in the language of Martin-Löf-style type theory?
2. Can Jaynes’ probabilistic framework be extended so as to natively include subjectivity?

3. Can Jaynes' probabilistic framework be extended so as to include Pearl-style causal reasoning?
4. Can Jaynes' commentary on bad scientific practice be placed in a wider institutional context?

Expected contributions

My first hope is to clarify Jaynes' programme. Jaynes' probability theory will be presented as an attempt to treat probability as extended logic, while the thesis carefully separates what he actually derives from what he claims. The distinction matters, because some of Jaynes' strongest claims – around universality, subjectivity and completeness – are exactly where later objections become sharpest [Col04; CW17; Hal99a; Hal99b; Sha04; Van03].

Second, the thesis offers a reconstruction of the relevant slice of Jaynes' programme inside a Martin-Löf-style type-theoretic setting. In this setting, probability assignments should be representable as explicit objects, and the familiar rules of probability should either be derived or obeyed by construction. Probability types are the natural vehicle for this, and they connect naturally to finitary versions of Cox-style reasoning [CW17].

A constructive account of causality follows. Pearl-style structural causal models are not replaced; they are embedded in a broader type-theoretic and epistemic setting. Interventions, a representative model-level counterfactual calculation and changes of causal structure are then treated as changes between explicitly represented epistemic states.

Alongside causality comes a more explicit account of subjectivity. Subjectivity is not reduced to the choice of a prior – instead, epistemic position is encoded through modes and transitions between them. This makes it possible to speak formally about learning, forgetting, interest and hypothesising, which matters especially for artificial intelligence, where agents may have bounded resources, limited memory and different contexts [HP07; NR19; Sim55].

A limited Lean 4 formalisation accompanies the formal chapters. This is not solely decorative; it is a guard against hidden assumptions. When a definition or proof cannot be made precise, even in the finite setting, this indicates not an implementation nuisance, but mathematical information.

The thesis closes with a diagnosis of scientific practice that connects back to the formal work. Hidden assumptions in mathematics and hidden incentives in science are different phenomena, but they are intertwined: both make it possible for conclusions to look cleaner than the process that produced them. The thesis therefore ends with practical suggestions concerning policies, incentives and research culture.

Compactly put: the formal part succeeds if finite Jaynes-style probabilistic updating and finite Pearl-style causal reasoning can be represented in the chosen Martin-Löf-style framework, with representative constructions checked in Lean 4. The wider thesis succeeds if this formal work can be convincingly connected to the claim that good science needs both good inference and a good environment in which this inference takes place.

Relevance

The relevance for artificial intelligence is direct. If artificial intelligence is, at least in part, the formalisation and implementation of inference, then its reliability depends on the quality of the reasoning methods on which it is built. A finite, constructive and computational account of probability and causality would provide a clearer route from abstract reasoning to executable systems.

The causal extension is especially important. Modern intelligent systems are increasingly expected not only to predict, but to answer questions about intervention, responsibility and alternative outcomes. Prediction alone is too weak for that task. Pearl’s causal framework shows why prediction has to be supplemented by intervention and counterfactual reasoning [Pea09; PM18]. This thesis asks how such reasoning can be embedded in a more explicitly constructive and subjective framework.

The broader scientific-practice component is also relevant to artificial intelligence. AI systems trained on, used within, or used to accelerate science inherit the weaknesses of the scientific pipeline. If the surrounding environment rewards fragile results, opaque reasoning or publication gamesmanship, then faster inference may simply produce bad science more efficiently. The formal and institutional questions therefore have to be considered together.

1.3 Outline

The thesis is structured as follows:

The first movement expounds Jaynes’ central claims: it discusses his demand for solid foundations in mathematics, solid foundations in statistics, and a good and honest attitude for doing these sciences. It also introduces the mathematical manifesto that will guide the rest of the work.

The second movement discusses Jaynes’ solution for statistics and the problems surrounding it: it reconstructs the Jaynes–Cox view, separates derivation from interpretation, and discusses objections concerning consistency, reasonableness, universality, subjectivity and causality.

The third movement proposes a more suitable mathematical setting: it discusses alternative logics and type theory, then presents the Martin-Löf-style type theory used in the constructive parts of the thesis.

The fourth movement uses this mathematical setting to address the problems raised earlier: it develops context manipulation, modalities, probability types and structural causal models inside the chosen type-theoretic framework. It also collects the formal results.

The fifth movement returns to scientific practice: it expands Jaynes’ complaints about bad attitudes into a wider analysis of perverse incentives in modern science. It also proposes practical changes.

The appendices contain definitions, proofs, arguments and code-oriented material that would interrupt the main line of the text. They are not less important, but they are placed where they can be checked without interrupting the narrative.

1.4 Idiosyncrasies

Before continuing, a few habits of this text deserve some attention:

The structure is intentionally unconventional. The thesis is arranged as a symphony, with movements rather than ordinary parts. This is inspired by Huizinga’s insistence on the playful character of culture and science, and by the suspicion that some of this playfulness has been replaced by a more anxious and institutionalised competition. The musical structure is therefore not only decorative: it is a reminder that scientific thought is not just a sequence of deliverables, but also a playful activity. The tempo markings are mood cues rather than a strict musical programme: the Prelude is deliberate, the first movement polemical, the second reflective, the third slow and foundational, the fourth broad and constructive, and the fifth quick and heated.

Acronyms will be avoided where they do not genuinely help. Science already contains enough abbreviations that sound useful mainly because they sound fundable. Likewise, ordinary terms will not be visually inflated with unnecessary capital letters. This means there will be no grand acronym for “Mathematical Inference as a Logical Extension” (MILE); there will simply be “mathematical inference”.

The thesis is written with active AI collaboration: ChatGPT is being used for source discovery and general framing; Lumo is being used for stylistic assistance; Claude and ChatGPT are used for coding and proof-checking work; Claude is also co-responsible for the figures and diagrams; ChatGPT made the front-cover. Having agents assisting mathematical projects belongs to the wider movement towards AI-assisted formal mathematics [Cas26; Kak26; Orn26; Sku26; Slo26]. The whole storyline, critical analysis of the literature and verification and improvement of prose and proofs, is entirely done by me. This thesis is in AI is therefore, rather appropriately, 100% AI-made, 100% human-reviewed. Every paragraph was (re-)written, weighed and revised by me; the intellectual responsibility, including responsibility for errors, remains completely mine. The work is therefore made completely in line with the Leiden declaration on artificial intelligence and mathematics [Alp+26].

The intended audience is broad, and this is risky. The thesis lies at the crossing of logic, mathematics, artificial intelligence, philosophy of probability, causality and the sociology of science. Readers from any one of these domains may sometimes want more detail exactly where another reader needs less. The compromise adopted here is narrative-first: the main text preserves the flow of ideas, while technical definitions, proofs and code are moved to the appendices whenever they would otherwise break the story.

These idiosyncrasies are not meant as a decorative ornament: they are part of the method. A thesis about explicit reasoning, constructive mathematics and scientific honesty should itself try to be explicit about the way it reasons and the choices it makes. The reader is invited to follow this movement.

CHAPTER 2

First movement: Allegro con brio

“Indeed, an insistence that all of mathematics be stated in that language all of the time can place unnecessary burdens on a theory, particularly one intended for application in the real world. It can also degenerate into an affectation, used only linguistically rather than functionally. To give every old, familiar notion a new, impressive name and symbol unknown to Gauss and Cauchy has nothing to do with rigor. It is, more often than not, a form of gamesmanship whose real purpose is to conceal the Mickey Mouse triviality of what is being done. One would blush to state it in plain English”

E. T. Jaynes in *Probability Theory: The Logic of Science*

2.1 Solid foundations for mathematics

Jaynes' first complaint is directed at the foundations of mathematics. His target is not mathematics as such, and certainly not calculation, experiment, modelling or honest abstraction. His target is a particular kind of mathematical culture: a culture in which formal derivation is too easily confused with scientific usefulness, and in which the ability to write down a consistent-looking construction is mistaken for the ability to explain anything worth explaining, see the opening quote from Jaynes.

The most visible symptom is what is called “infinite paradoxing” by Jaynes. Modern mathematics is riddled with results that actively discover the limits of infinity (pun intended): Banach–Tarski decompositions, non-measurable Vitali sets, the Borel–Kolmogorov paradox, supertasks such as Thomson’s lamp, and Ross–Littlewood-style urn puzzles [AK91; AK95; BT24; GHR17; Tho54; Vit05]. These examples do not all have the same status: some are genuine theorems; some are philosophical stress tests; some are warnings about limiting procedures.

The deeper objection is therefore methodological. Axiomatic set theory, especially in the Zermelo–Fraenkel with choice tradition, easily allows for deeply abstract constructions. Its ease of use is the main reason for its success. By the same token, it is also deceptively easy to create nonsensical or non-applicable objects. The trained mathematician may admire the sheer possibility; the physicist may ask how it can be measured. Jaynes was a classically-trained physicist: start with what can be stated, counted, constructed or approximated, and move to limits only with a clear account of what the limiting process achieves. If the point of mathematics in science is mathematical inference, then a method that converts finite observations into uncontrolled ideal objects should inherently be treated with suspicion. It may still be useful, but usefulness is not conferred by grandeur.

It is worth noting that, although Jaynes actively fulminates against using Zermelo–Fraenkel with choice theory as a basis for mathematics, he uses it himself, albeit in a rather conservative way. This is therefore an objection he extensively discusses, but is unable to ‘solve’. He proposes a more conservative approach to using the theory as it stands, but that approach asks users deliberately not to explore the full capability of the theory – an argument that is rather unconvincing to the mathematical community.

In this thesis I side with the physicists: for the formal reconstruction, the default mathematical environment will be constructive and finitary. This is not primarily motivated by some form of mathematical purity, but much more by efficiency. A framework that forces objects to be constructed, contexts to be explicitly declared and dependencies to be visible is better suited to the kind of scientific reasoning Jaynes wanted. Rather than running the risk of spending valuable resources and research facilities on obtaining results that are questionable to applied research, it can be more efficient to construct everything in a more limiting environment, where results are more likely to be directly applicable and computationally checkable.

This is why the third movement turns to alternative logics and Martin–Löf-style type theory. The point is not to escape into yet another formalism, but to find a setting in which constructions, contexts and dependencies have to be stated openly. The fourth movement then uses this setting for the constructive

reconstruction itself: probability types, epistemic modalities and structural causal models are all built there within this setting.

2.2 Solid foundations for statistics

Jaynes’ second complaint is more direct and more devastating. In his view, twentieth-century statistics had become a patchwork of procedures, tests, conventions and reputations. Some procedures worked well in practice; some were defended by tradition; some were inherited because nobody wanted to reopen old discussions. What was clearly missing was a single account of why statistical inference should have the form it had.

The force of Jaynes’ criticism becomes clearer when placed next to its possible consequences. Independent teams can receive the same data and reach sharply different conclusions, as shown in Figure 2.1 [Bot+20; Bre+22; Sch+21; Sil+18]. This poses a serious problem, because it threatens the foundations of statistical claims in economics, psychology, sociology and other sciences, as large replication studies show; see Figure 2.2 [Cam+16; Kle+14; Ope15]. This objection therefore bears even more serious consequences than the previous objection: not only are time and energy potentially wasted, but there is also an acute risk of consciously propagating incorrect conclusions, which is directly contrary to doing science. This clearly illustrates that inference is fragile when assumptions are hidden, workflows are underdescribed and technical choices are treated as harmless details [GL13; Ioa05].

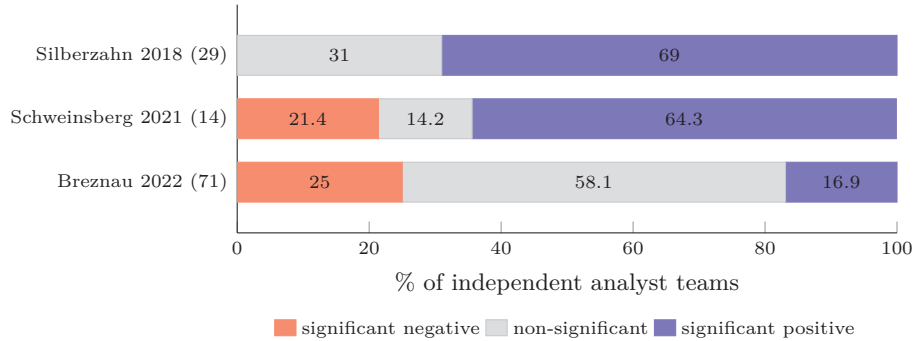


Figure 2.1: Three many-analyst projects in which independent teams analysed one dataset, classified by the *direction* of the reported result. Silberzahn et al. (29 teams): no significant-negative result, 31% non-significant, 69% significant-positive [Sil+18]; Schweinsberg et al. (Hypothesis 1, 14 analysts with known direction): 21.4% / 14.2% / 64.3% [Sch+21]; Breznau et al. (71 teams, estimate-level): about a quarter (25%) significant-negative, a little over half (58.1%) non-significant, 16.9% significant-positive [Bre+22]. Botvinik-Nezer et al.’s neuroimaging study is discussed in the text but omitted here, since its 70 teams answered nine yes/no hypotheses with no directional split [Bot+20].

Cox’s programme offered Jaynes exactly the kind of answer he wanted: if plausibility is represented numerically, if logically equivalent states of information are assigned equivalent plausibilities, and if reasoning is required

2. First movement: Allegro con brio

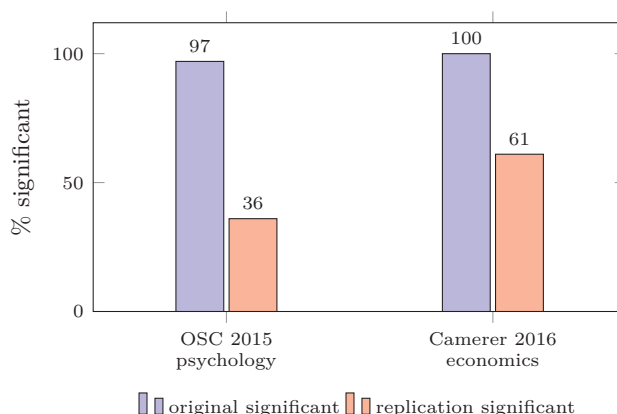


Figure 2.2: Two large replication projects. Of 100 psychology studies, 97% of originals reported a statistically significant result versus 36% of replications [Ope15]; of 18 economics experiments, all reported a positive original finding and 61% (11 of 18) replicated as a significant effect in the same direction [Cam+16].

to be consistent across different routes, then the familiar rules of probability can be recovered after a suitable rescaling [Cox46; Cox61; Jay03]. Probability is then not a bag of recipes, it is extended logic: the calculus of consistent reasoning under uncertainty.

This is the great attraction of the Jaynes–Cox view: it offers a way to say that statistical inference is neither a social convention, nor a computational convenience. It is what one must do if one wants uncertainty to obey the same coherence that ordinary deductive logic obeys in the limiting case of absolute certainty. Keynes, Ramsey and Jeffreys had already made probability philosophically serious as a calculus of partial belief; Cox and Jaynes gave the programme its sharpest logical form [Jef61; Key21; Ram31].

Of the three complaints filed in this movement, this one can be considered the most ‘solved’ and the most complete by Jaynes. I therefore see Jaynes’ solution as both hugely important, but still partly incomplete: important because it gives a principled answer to a real problem (probability as a form of coherent inference); and incomplete because his derivations depend themselves on assumptions that are not completely innocent, and because his derivations lack causality and a native representation of subjectivity.

The second movement will therefore critically assess the Jaynesian programme: what it derives, what it assumes and where its claims become too large. The fourth movement then rederives the finite Bayesian core and expands it such that subjectivity and causality are made explicit within the formal language.

2.3 A good and honest attitude for doing these sciences

The third complaint is less formal, but no less central. Jaynes does not only object to bad foundations: he objects to bad habits. His prose is rather polemical

and, at times, even develops into full-blown personal attacks¹. Since, for Jaynes, the very heart of scientific praxis is at stake – with an impractical basis for mathematics and a non-rigorous basis for statistics – this objection is absolutely the most serious. It directly takes aim at people who, in his view, can be considered personally responsible for the bad state of science as it stands.

The habits Jaynes attacks can be arranged under five headings:

1. **Gamesmanship**: the desire to display technical virtuosity, rather than to clarify a problem;
2. **Cutting corners**: the focus on polished results, without explaining the painstaking work that led to these outcomes;
3. **Hidden assumptions**: the silent transfer of what should have been stated into what is merely presumed;
4. **Hyperfocus**: the habit to only investigate an incredibly specialised subfield, whilst forgetting the wider context in which this research sits;
5. **Theoretical navel-gazing**: the production of mathematics that neither explains, predicts, nor helps anyone build or test anything practical.

For this complaint, too, Jaynes addresses the problem abundantly, but is unable to ‘solve’ it. He treats this complaint rather personally, while I believe it deserves a more institutional view. The vices mentioned by Jaynes are not limited to ‘pure’ mathematics and statistics, and they are, unfortunately, not limited to Jaynes’ opponents². They are temptations built into scholarly life.

The fifth movement returns to these vices in a more institutional setting. I will argue there that they appear whenever professional reward becomes too detached from intellectual responsibility. The formal reconstruction of the intervening movements is therefore not separate from the sociological diagnosis: both are attempts to make hidden assumptions harder to hide.

However, that is not all. In software development, there has been some attempt to establish workflow standards as part of an effort to optimise operations, regardless of the particulars of each separate project. In mathematics, and science in general, there seems to be a general lack of such efforts. In the following, I will put forward a standard that I will adhere to, that is aimed at preventing most of these critiques. This thesis is therefore to be read as an attempt to obey that standard: a mathematical manifesto.

2.4 Mathematical manifesto

From these complaints a small mathematical manifesto follows. It is not a new foundation, nor a grand acronym, nor a shiny institutional programme. It borrows its form from the Agile Manifesto and from later workflow frameworks such as Scrum: a short set of comparative preferences, meant to guide practice [Bec+01; SS20]. For the rest of the thesis, the discipline is this manifesto:

¹Especially Ronald Fisher, but also William Feller, Nicolas Bourbaki/the Bourbaki group and others take personal blows.

²Cutting corners also appears inside Jaynes’ own work: some of his strongest claims about universality, subjectivity and completeness move faster than the derivations can justify. This is not a reason to discard Jaynes: it is a reason to take him seriously enough to apply his own standard to him.

2. First movement: Allegro con brio

I am uncovering better ways of developing mathematical theories by doing it and helping others do it. Through this work I have come to value:

Algorithms over abstractions

Narrative over formulae

Explicit over implicit

Simple over complex

That is, while there is value in the items on the right, I value the items on the left more.

Algorithms over abstractions does not mean that abstraction is bad, but it means that abstraction should remain connected to construction, computation or procedure. If a concept cannot in principle be implemented, checked, traced or approximated, then its scientific role should be handled with care.

Narrative over formulae does not mean that formulae are ornamental, but it means that formulae should serve the movement of thought. A derivation is not a heap of symbols; it is a story about why one statement follows from another. When the story is hidden, the formulae may still be true, but they become harder to inspect.

Explicit over implicit is the most Jaynesian rule: assumptions should be visible, contexts should be named and dependencies should be tracked. If a proof requires a specific domain, a decidable event, a specific logic or a specific causal graph, that requirement should not be secretly smuggled in. The constructive framework used later is chosen in this spirit: not because it is the easiest language at first glance, but because it makes hidden commitments harder to hide.

Simple over complex is not a cult of simplicity: it is a preference for the smallest mechanism that does the job. Complexity is sometimes necessary, but it should be paid for consciously. If a simpler derivation, formula, or even plain English sentence does the job equally well, then the more complex solution requires a justification.

CHAPTER 3

Second movement: Andante con moto

“A story is needed before one can set sail. Look at us. For the sake of the story, we are sitting here aboard a water taxi on our way to the Holy Grail, the unknown, and adventure. People think life becomes more enjoyable through stories. That is true enough, but it is not the point. Without stories, it becomes meaningless. Sense consists of sentences. The words that tell the story lay a reassuring relation of cause and effect over random facts and events. People crave plot because a plot reduces the unbearable and unmanageable chaos of the sublunary world to a human scale, and to a chain of initiatives and consequences that a person is able to comprehend. A plot gives a sense of control, provenance and destination, origin and direction”

Ilja Leonard Pfeijffer in *Grand Hotel Europa*

3.1 The Jaynes–Cox programme

Desiderata

The Jaynes–Cox programme begins with a modest question: what rules should govern reasonable degrees of belief [Cox46; Cox61; Jay03]? The exposition starts with three desiderata that, according to Jaynes, are the minimal, reasonable requirements that a calculus of plausible reasoning ought to satisfy. In Jaynes' own words, in the compact formulation [Jay03]:

- (I) Representation of degrees of plausibility by real numbers;
- (II) Qualitative correspondence with common sense;
- (III) Consistency.

In the ensuing discussion, Jaynes unpacks consistency into three demands: a conclusion should not depend on the route by which it is reached; relevant evidence should not be ignored; and equivalent states of knowledge should receive equivalent plausibility assignments. The first desideratum translates to divisibility and comparability: plausibility should be represented on a single ordered numerical scale, so that one state of information can be compared with another. The second is qualitative agreement with common sense: ordinary Aristotelian logic should appear as the limiting case in which propositions are known with certainty, and small changes in the premises should not produce arbitrary jumps in the conclusions. The third is consistency: if a plausibility can be obtained by two legitimate routes, the two answers should agree. These desiderata are then via functional equations translated in the familiar axioms of probability theory.

Derived calculus

Cox's formal move is to apply these desiderata to the logical operations of negation and conjunction. If A and B are propositions and I is the background information, then the plausibility of $\neg A$ should be determined by the plausibility of A , and the plausibility of $A \wedge B$ should be determined by the plausibility of A and the plausibility of B once A is assumed. Under the usual regularity assumptions, one can rescale the plausibility function to a probability function P satisfying

$$P(A \wedge B | I) = P(A | I)P(B | A, I)$$

and

$$P(A | I) + P(\neg A | I) = 1.$$

The first equation is the product rule; the second is the sum rule for complements.

Bayes' theorem then follows by applying the product rule in both orders:

$$P(A | B, I) = \frac{P(B | A, I)P(A | I)}{P(B | I)}$$

whenever $P(B | I)$ is non-zero. Here $P(A | I)$ is the prior probability of A relative to background information I , $P(B | A, I)$ is the likelihood of observing B under A , $P(B | I)$ is the evidence or normalising term, and $P(A | B, I)$ is the posterior probability after B has been taken into account. In general, many probabilities, such as the likelihood, are often clear from the context of the

problem and therefore treated as relatively objective. The prior distribution is usually harder to fix, and is therefore often treated as more subjective. This degree of freedom makes this theory notoriously controversial, but allows the theory to introduce ‘subjectivity’ as the ability to freely choose one’s prior. The finite constructive version of conditional probability and Bayes’ theorem used later is stated in Definition A.3.12 and proved in Theorem B.5.16.

With these ingredients, one recovers the ordinary finite probability calculus: non-negativity, normalisation and finite additivity. In the classical measure-theoretic presentation, this calculus is extended to Kolmogorov’s axioms. In this classical reading, a probability function P satisfies Kolmogorov’s axioms if:

1. *Non-negativity*: $P(A) \geq 0$ for every $A \in \mathcal{F}$;
2. *Normalisation*: $P(\Omega) = 1$;
3. *Countable additivity*: for every countable pairwise disjoint family $(A_i)_{i=1}^\infty$ in $\mathcal{F}$,

$$P\left(\bigcup_{i=1}^\infty A_i\right) = \sum_{i=1}^\infty P(A_i).$$

Here $\mathcal{F}$ is a σ -algebra of events. The finite analogue used later is stated in Remark A.3.11 and proved in Theorem B.5.12.

This derivation does not mean that Jaynes invented Bayesian reasoning, as Keynes, Jeffreys, Ramsey, Cox and others are key ancestors of the project, and Jaynes explicitly inherits the Coxian line [Cox46; Cox61; Jef61; Key21; Ram31]. Jaynes’ distinct contribution is the clarity of the synthesis: probability as extended logic, statistical inference as disciplined common sense, and scientific disagreement as often traceable to hidden assumptions or silently disregarded desiderata. The philosophical gain is large: what appears to be a specialised derivation sets the norm for coherent reasoning under uncertainty. Probability is then more than a matter of repeated experiments: it is about what follows from information.

Jaynes really maintained that he uncovered a universal and logical theory of doing science and that everything deviating from either Bayes’ rule or his logical desiderata should be considered fundamentally illogical and – even more strongly so – fundamentally unscientific. The attraction of this view is obvious. If correct, the programme gives mathematical inference a foundation. The remaining sections critically assess the programme and ask whether all its assumptions and conclusions can, in fact, be justified.

3.2 Objections

The objections to Jaynes do not all have the same focus: some are technical; some are philosophical; some concern scope. I will treat them in an order that moves from the derivation itself to the larger claims built on top of it: consistency, reasonableness, universality, subjectivity and causality.

The guiding distinction is the one already announced in the Prelude: what is derived, and what is claimed. Jaynes derives a powerful representation result under assumptions. He sometimes writes as if he has also derived a universal account of rationality, subjectivity and scientific method. The route through these objections is summarised in Table 3.1.

3. Second movement: Andante con moto

Objection	Target	Problem for Jaynes	Thesis response
Consistency	Cox-style derivation	Naive formulations of Cox's assumptions can fail on finite or insufficiently rich domains.	Treat the counterexamples as design constraints and rebuild the finite assumptions explicitly.
Reasonableness	Jaynes' desiderata	A consistent assumption can still be too technical, too strong or too poorly motivated.	Use the finite urn principles of Clayton and Waddington as the constructive route.
Universality	Probability as universal logic	A theorem inside one background logic is not automatically a logic of all uncertainty or all science.	Put probability inside a constructive and epistemological setting with explicit scope.
Subjectivity	Priors and background information	The subject is represented mainly by I and the prior, while the agent's position remains implicit.	Represent epistemic position through contexts, modes and modalities.
Causality	Prediction by probability	Statistical dependence does not by itself distinguish seeing, doing and imagining.	Add finite structural causal models and transport the relevant causal results.

Table 3.1: Route through the objections to the Jaynes–Cox programme.

Consistency

The first worry is whether Cox-style assumptions are even consistent in the form needed for Jaynes' conclusion. Halpern showed that naive formulations of Cox's theorem can fail – finite domains and insufficiently rich plausibility ranges can produce counterexamples to the intended representation [Hal99a; Hal99b].

Later work repairs the situation. Arnborg and Sjödin treat finite models and argue for additional principles under which Bayesian rules can be recovered or justified [AS00; AS01; AS99]. Van Horn reconstructs the assumptions needed for a continuous Cox-style theorem more carefully, and Snow defends finite-domain reasonableness from a related angle [Sno98; Van03].

These results illustrate that Cox-style derivations require additional assumptions: domain richness, regularity assumptions and a precise account of what counts as a permissible plausibility assignment. For this thesis, these critiques are taken as instructive sources of information: if probability is to be rebuilt constructively, the needed assumptions must be made precise and checkable.

This is the role of the finite reconstruction in Section 5.3: the assumptions are not treated as background atmosphere, but are gathered into explicit finite rules whose proof route is summarised in Table 5.1. Representative finite checks are later collected in Table 5.3.

Reasonableness

A subtler worry is whether the assumptions are reasonable to begin with. An assumption can be consistent and still be philosophically unattractive. Shafer objects, among other things, to the idea that reasonable expectation must always be represented by a single real number [Sha04], and Van Horn’s reconstruction – although technically precise – does not by itself make every assumption compelling¹ [Van03].

Clayton and Waddington are especially important here because they aim to design explicit logical and intuitive rules that are more in line with Jaynes’ intentions. Instead of simply postulating technical conditions such as universality and strict monotonicity on large domains, they derive the needed structure from urn-based rules [CW17]. In the reconstruction below, finite urn existence is not an additional assumption: the type theory supplies $\text{Fin}(N)$, and a proof that $N > 0$ makes this cell type nonempty. The remaining assumptions needed here are:

- *qualitative coherence*: qualitative equivalence is an equivalence relation and strict comparison is a compatible strict order;
- *count invariance*: events with the same number of selected cells are qualitatively equivalent;
- *strict monotonicity*: in a symmetric urn, more favourable cells means greater plausibility;
- *refinement invariance*: splitting every cell uniformly preserves plausibility.

Logical equivalence and relabelling invariance follow from count invariance, while product urns are constructed from Cartesian products of finite cell types. Some additional rules and derivations are needed in the original paper, as it operates in the classical axiomatic set-theoretic domain. The thesis does not reproduce Clayton and Waddington’s full route. Instead, the finite assumptions needed later are listed in Definition A.3.22.

As Clayton and Waddington’s account of probability is perfectly in harmony with the mathematical manifesto stated earlier, this thesis bases its derivation on their work. The reconstruction in this thesis only keeps the finite urn principles and changes the order of their derivations somewhat, but the idea remains the same. It is important not to take Jaynes’ desiderata as sacred, immutable tablets, but as a solid basis on which one can sensibly vary. Clayton and Waddington’s account might make the resulting probability theory less grand, but more explicit and, therefore, directly checkable and less evasive.

The constructive proof in Section 5.3 follows this lesson; its route is summarised in Table 5.1 and carried out in Section B.5. It does not try to preserve every grand Coxian claim; it preserves the finite ingredients that can be made explicit, checked and used.

Universality

Jaynes often writes that his derived probability theory is the universal logic of science. Shafer and Colyvan give useful warnings against this overreach: Cox-style arguments presuppose a background logic, and the choice of that logic matters [Col04; Sha04]. A theorem about plausible inference inside classical

¹An objection explicitly mentioned by Van Horn himself.

3. Second movement: Andante con moto

propositional logic is not automatically a theorem about all forms of uncertainty, all domains of discourse or all scientific reasoning.

The point becomes clearer once one distinguishes ontological and epistemological questions. An ontological logic asks what is the case; an epistemological logic asks what is known, knowable or justified from a given position [Hof23; SN25]. The question “Does God exist?” illustrates the contrast nicely: an ontological view forces one to give an immediate answer; an epistemological stance may answer that the question is simply not answerable.

Jaynes’ framework is epistemological in temperament – as he considers inference as the process of adding knowledge to a given, personal state of information – but formally ontological. He wants probabilities to describe states of knowledge, yet his derivation usually assumes a classical propositional background in which every proposition is immediate (true or false). However, vagueness, fictional discourse, undecided mathematics, limited interest and forgotten information are not well represented by simply assigning a number to such a classical proposition, as also maintained by Van Horn [Van03].

The response is not to abandon probability, but to abandon the claim that probability, derived once in one background logic, is the final logic of everything. The present thesis uses a constructive and epistemological setting, but remains modest about its applications to other domains. Probability itself remains central in this work, but it is placed inside a richer account of contexts, modes and epistemic states. This richer account is, then, better suited for natively representing someone’s knowledge, rather than being an afterthought. The thesis is not about making probability a ‘universal logic’, but about making its domain more explicit and its applications more checkable.

Subjectivity

In the terminology of Bayes’ theorem above, Jaynes’ subjectivity enters mainly through the background information I and the prior $P(A \mid I)$: different agents may start from different information and therefore assign different probabilities. But the subject is still strangely absent from the formalism. The prior may vary from one agent to another, while the structure of the agent’s position is not itself represented.

For a truly subjective representation, it is essential to know what information, per agent, is available. This means knowing what problems are in scope; what information has been forgotten; what information is of no interest; what connections are believed to hold true; and what can be imagined, based on the available information. These are not merely psychological decorations, but they affect what can be asked, what can be inferred and what counts as an identical problem. These questions should be part of the mathematical formalism itself, rather than being added as an afterthought.

Jaynes rightly objects to treating probability as a property of objects in isolation, a stance often defended by frequentists. In this view, repeating a measurement until one ‘uncovers the True Probability’ simply means, as Jaynes correctly points out, an ill-defined procedure, as one can make a repeated coin flip have any experimental outcome one likes². A coin does not carry

²Jaynes often refers to this phenomenon as the ‘projection fallacy’, where personal missing information is projected on a situation as a whole – like with the coin flip – or even is seen as being *fundamentally* missing – as with quantum mechanics.

a metaphysical label “one half”. A physical procedure, a personal state of information and a model of the experiment are often covertly doing the heavy work. However, stating this in prose does not yet build subjectivity into the mathematics.

Interest is a clear example, somewhat alluded to by Colyvan [Col04]. Classical logic lets every proposition sit in the same universe of discourse. Real agents do not work like that. To a real agent, a proposition may be true, false, unknown, irrelevant, forgotten or outside the present domain of concern. One may not care whether Achilles was considered the fastest among mortals by the ancients, nor what ending Gogol’s *Dead Souls* would have had if he had lived a bit longer, while caring deeply about one’s own medical condition. A logic of subjectivity should be able to represent this asymmetry without pretending that every statement is waiting for some kind of ‘objective’ probability to happen.

The constructive proposal in later sections is to make this epistemic position explicit. Subjectivity is then no longer a label placed on Bayesian calculus after the fact: it becomes part of the machinery that is developed in Sections 5.1 and 5.2.

Causality

Jaynes’ probability theory can express statistical dependence, but dependence is not causation, as the famous adage goes. That is, a barometer and a storm can be tightly associated without the barometer causing the storm to happen. I give an argument in Section B.1 for why a working theory of scientific reasoning – under which mathematical inference falls – should include causality. The main components of this argument are already present in the opening quote by Pfeijffer, albeit there much more eloquently than I am capable of. This is a philosophical argument from desirability, but it does not yield usable clues on how causality should be mathematically integrated.

Pearl’s framework supplies the missing formal technology. This framework distinguishes three levels of causal inquiry. At the bottom sits association: questions such as $P(Y \mid X)$, where one asks how observations influence each other. Above that comes intervention, with questions such as $P(Y \mid \text{do}(X = x))$, asking what would happen if variables are forced to a specific value. At the top lives counterfactual reasoning — questions such as $Y_x(u)$, where one asks what the outcome for a particular unit hypothetically would have been under a different intervention, see Figure 3.1 [Pea09; PM18]. These three levels are not stylistic variations. They require different assumptions, and they answer different questions.

One needs more mathematical machinery for these three levels. At the first level, I can still make do with the Bayesian calculus that Jaynes derived: separate variables that are functionally related. At the second level, I would need connections between variables, as I need to explicitly know what variable is influencing which variable if I were to intervene in a situation. This is neatly represented by causal Bayesian networks and causal graphs: a causal graph is a directed acyclic graph that may also carry bidirected edges for unobserved common causes. These graphs obey the causal Markov condition: each variable is independent of its non-descendants once its parents are known. Pearl introduces the do-operator to mark the difference between observing a variable and setting one. The formal definitions are collected in Definitions A.4.7, A.4.17, A.4.19,

3. Second movement: Andante con moto

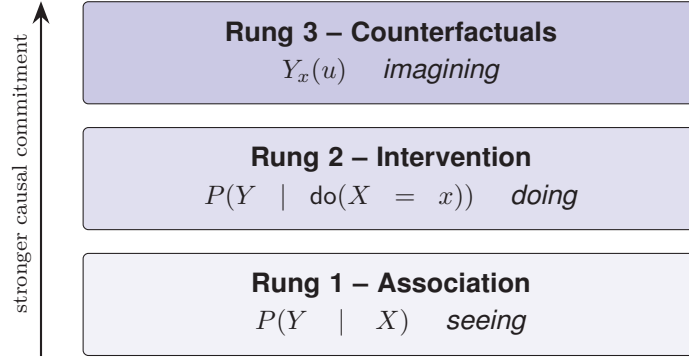


Figure 3.1: Pearl’s ladder of causation: each rung answers a strictly stronger class of question [PM18].

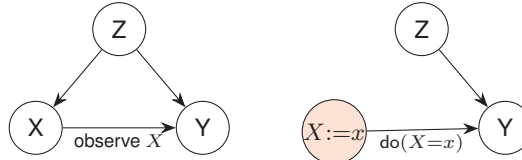


Figure 3.2: Observing X leaves the confounding path through Z intact; intervening on X severs its incoming edges (Definition A.4.21).

A.4.21, A.4.22, A.4.24 and A.5.10, where Pearl’s three do-calculus rules are the transformation rules that connect observations and interventions under specific conditions [Pea09; Pea12]. Observing leaves the ordinary causal mechanism in place; intervening replaces the mechanism for a variable by an imposed value, see Figure 3.2. This is exactly the distinction Jaynes’ probabilistic calculus cannot express by itself.

At the counterfactual level, a Bayesian network is no longer sufficient. Here, a structural causal model is necessary: endogenous and exogenous variables in the sense of Definition A.4.11, structural equations and a solution map as in Definitions A.4.14 and A.4.15, and an intervention as in Definition A.4.22. Given evidence, the first step is to update the distribution over the exogenous variables; for a hypothetical, imagined action, a twin network is connected to these exogenous variables and the intervened, endogenous variables are set to a constant. Within this twin network, it is then possible to evaluate a hypothetical outcome in the intervened model, see Figure 3.3. The relevant finitary definitions are collected in Definitions A.6.1 and A.6.3 and Proposition B.8.1, where the described procedure follows Pearl’s abduction–action–prediction procedure.

For the ordinary interventional fragment, Pearl’s framework is not merely convenient notation: do-calculus, together with ordinary probability manipulations, is complete for identifying the relevant class of causal effects in the standard settings [HV06; Pea12; SP06; TS09].

In my research I consider Pearl’s account of causality a paramount extension of Jaynes’ work. Not only do the structural causal models allow for hypothesising *per se*, but they also provide an ideal representation of a personal state of

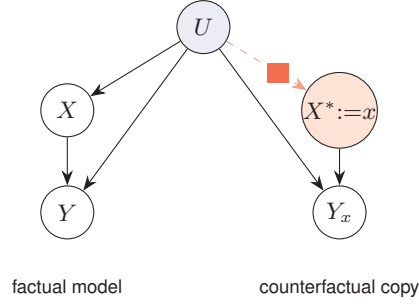


Figure 3.3: A counterfactual copy of a structural causal model. The factual and counterfactual sectors share the same exogenous background U ; the counterfactual intervention replaces the mechanism for X^* by the imposed value x , and Y_x is then evaluated in the copied model.

information. These models naturally capture what propositions are in scope, and what relations exist between them. Furthermore, by dynamically transitioning between different structural causal models, it is easy to capture epistemological events like forgetting, learning, connecting, hypothesising, and others. Pearl-style causal models will therefore be embedded in the constructive, epistemic setting developed later, which buys one extra freedom: causal structures need not be treated as eternally fixed.

This thesis does not reprove these completeness results or the global Markov theorem underlying do-calculus soundness. It proves a conditional finite transport theorem: once the applicable classical finite-rational soundness and completeness specializations are supplied explicitly, the restricted finite structural-causal-model correspondence carries inspectable derivation certificates and hedge counterexamples into the type-theoretic presentation and reflects certificate soundness back to identifiability, in the sense of Definitions A.5.6, A.5.9 and A.5.12. This transfer is developed in Section 5.4 and summarised in Table 5.2. A separate extension checks both an explicit shared-latent twin reference model and a general occurrence-indexed multiworld SCM in the sense of Definition A.6.4, executed by proof-carrying atomic modal edits. It transports supported finite multiworld counterfactual identification when an exact source-native specialization of the published counterfactual result is supplied. It does not formalise the published **ID***/**IDC*** algorithms or construct that external specialization, and it does not imply completeness for every operation made possible by the epistemic construction.

CHAPTER 4

Third movement: Adagio

“A common mistake that people make when trying to design something completely foolproof is to underestimate the ingenuity of complete fools”

Douglas Adams in *Mostly Harmless*

4. Third movement: Adagio

This movement prepares the formal language used in the fourth movement: first by sorting the relevant logics, then by moving through type theory, and finally by fixing the Martin-Löf-style fragment on which contexts, modalities, probability types and causal models will rest.

4.1 Different types of logic

In light of the overly broad universality claims posited by Jaynes, the next step is to enquire which logic is appropriate for which task. Classical logic is still magnificent when its assumptions fit: propositions are true or false; excluded middle is accepted (as propositions are always either true or false); contradiction is fatal; and the inferential world is crisp as all propositions are always available to everyone. However, reasoning is not always that crisp, and artificial intelligence does not have the luxury of pretending so.

Fuzzy and many-valued logics deal with predicates that do not behave as if they have sharp, binary borders. “Young”, “tall”, “similar” and “healthy” are not always best captured as if they only had two options, but rather a whole range of values or a degree of ‘truthness’ [CFN22]. Probability is sometimes misused here: a degree of truth is not the same thing as a degree of belief. Van Horn makes this distinction sharper, emphasising that plausible or probabilistic inference concerns uncertainty about propositions, but does not concern assigning gradients to propositions [Van03].

Modal logic adds structure by allowing truth to vary across worlds, states or situations. In Kripke-style semantics, one does not merely ask whether a proposition is true; one asks whether it is true at a specific world, and how different worlds are related [Gar24]. This is already close to the language needed later. An epistemic state can be treated like a world; moving between states can represent learning, forgetting or intervention; modal structure can record which movements are legitimate.

The contrast between **ontological** and **epistemological** logic is more important for this thesis. The above logics are often framed in an ontological setting – although this is not strictly necessary, as will be seen later with modal logic. Ontological logic speaks as if it were describing what *is* or what *exists in reality*; Epistemological logic speaks about *knowledge*: what is known and what can be induced from that [Hof23; SN25]. Jaynes’ language is deeply epistemological, as it frequently refers to the personal knowledge of an agent, but his formal derivation remains close to a classical background.

Within epistemology there are a few candidates to consider. **Temporal** and **dynamic** logics are useful for reasoning about change and actions [BR16; GR25]. Both of these contain valuable components, and the later implementation can be considered a special flavour of a dynamic logic, where modalities govern transitions between epistemic states. **Intuitionism** is an epistemological logic in the constructive tradition: everything needs to be explicitly derived. It therefore does not accept a proposition because its negation cannot be proven, but it needs a positive proof of its truth. It immediately follows that the law of excluded middle is not generally accepted in this logic¹. Furthermore, real

¹Which is based on the premise that all propositions are either true or false. In the classical case, it then follows that if a proposition is explicitly not one, it must be automatically the other, e.g. if something is explicitly *not* false, then it *must* be true. This does not automatically

Logic	Type theory
proposition	type
proof	program / term
$A \wedge B$	$A \times B$ (pair)
$A \vee B$	$A + B$ (sum)
$A \Rightarrow B$	$A \rightarrow B$ (function)
$\top, \perp$	1 , 0 (unit, empty)
$\forall x. P(x)$	Π -type
$\exists x. P(x)$	Σ -type

Figure 4.1: The Curry–Howard correspondence: proofs are programs, propositions are types.

numbers, in the classical sense, are also not accepted, as these are, in axiomatic set theory, completed infinite objects, and not explicitly constructed; only numbers that are completely constructible are accepted [Iem24]. Especially this last point is attractive, as it aligns perfectly with Jaynes’ philosophy of explicit limits and constructions.

The philosophical stance I take here is agnostic about ontology and constructive about epistemology, therefore largely aligning with intuitionism. The explicit argument for this is laid down in Section B.2. In the following, I do not decide what reality is made of; I ask how the knowledge of finite agents is influenced by finite measurements and finite updates.

However, to be completely aligned with the mathematical manifesto, more is needed: an explicit and computational part. This is provided by **type theory**, which deserves its own section, following next.

4.2 Type theory

Type theory is the natural technical home for this stance. It connects mathematics to computation through the Curry–Howard correspondence: propositions can be read as types, and proofs as terms inhabiting those types, known as propositions-as-types, see Figure 4.1. To prove a proposition is, in this sense, to construct a proof, or, even more explicitly: compute a proof [Coq25].

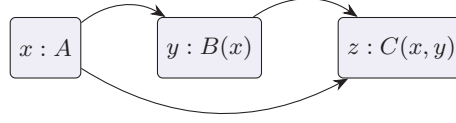
A **type** may be understood, broadly, as a hypothesis or, specifically, as a domain of construction. A term $a : A$ says that a is an **inhabitant** of the type A , which means that a is a “witness” within the domain A . Within type theory, a **context**, Γ , records the assumptions and variables currently available. Importantly, these contexts are ordered: later entries in the context may depend on previous entries, but not vice versa², see Figure 4.2. What can be derived, given a specific context, is called a **judgement**. For example, a judgement of the form

$$\Gamma \vdash a : A$$

follow in the intuitionistic case, where either value needs an explicit confirmation.

²This ordering makes these contexts natural vehicles for encoding structural causal models, since these are directed acyclic structures that are only ‘forward-looking’ in terms of dependencies, as will be seen later.

4. Third movement: Adagio



order matters: later entries may depend on earlier ones

Figure 4.2: A context $\Gamma = (x : A, y : B(x), z : C(x, y))$ as an ordered telescope of dependencies — the ordering that later mirrors a causal directed acyclic graph.

says that, in context Γ , it can be derived that the term a has type A . Furthermore, types themselves can be considered a term of even more general types, $A : B : C \dots$, creating a hierarchy, or **universe**, of types. The more formal vocabulary for types, contexts, judgements and universes is given in Definitions A.1.1, A.1.2 and A.1.6. The above already brings subjectivity nearer: what can be derived depends heavily on the specific context considered.

Substitution explains how a term depending on a variable changes when that variable is replaced by a particular term. **Function types** transform inhabitants of one type into inhabitants of another. **Lambda abstractions** construct such functions, and **applications** use them. **Pair types** and **sum types** function like containers for two types at once³. In the ensuing substitution and application steps, **equality** is not a purely metaphysical relation; it has computational content, for instance through **beta-reduction**. The more formal definitions and machinery are laid down in Definitions A.1.4, A.1.5, A.1.9, A.1.10, A.1.12 and A.1.28.

A small example can be illuminating, which follows below:

Example 4.2.1 (A tenure-track offer as a type). Suppose a hiring committee is considering one candidate, Mira, for a tenure-track position. Classically, the proposition

$$\text{Offer}(\text{Mira})$$

is immediately subject to excluded middle:

$$\text{Offer}(\text{Mira}) \vee \neg \text{Offer}(\text{Mira}).$$

Moreover, classical reasoning validates double-negation elimination:

$$\neg \neg \text{Offer}(\text{Mira}) \rightarrow \text{Offer}(\text{Mira}).$$

In the propositions-as-types reading, by contrast, $\text{Offer}(\text{Mira})$ is a type. A proof that Mira receives the offer is a term

$$\ell : \text{Offer}(\text{Mira}),$$

which may be imagined as the constructed offer letter or the verified decision record. In this constructive reading, negation is interpreted as:

$$\neg A := A \rightarrow \perp$$

³Function types roughly correspond to implication, $\implies$, pair types roughly correspond to conjunction, $\wedge$, and sum types roughly to disjunction, $\vee$.

where $\perp$ is the empty type; see Definitions A.1.25 and A.1.28 for a formal definition. Thus a proof of $\neg A$ transforms a witness of A into contradiction. From a positive witness one can construct double negation:

$$\lambda \ell. \lambda n. n(\ell) : \text{Offer}(\text{Mira}) \rightarrow \neg\neg\text{Offer}(\text{Mira}).$$

The converse direction, double-negation elimination, is valid classically but is not derivable in the constructive base theory used here. Thus, from double negation it is not generally possible to prove that Mira received an offer, unless this is explicitly added to the theory.

Having the mathematical machinery of type theory available is powerful, but does not automatically yield an epistemological theory *per se*. However, an epistemological type theory is available and known as **Martin-Löf-style type theory** [DP24]. This type theory is a combination of intuitionism – explained in the previous section – within the framework of type theory. The following section explains more details about this specific type theory.

4.3 Martin-Löf-style type theory

Martin-Löf-style type theory, or intuitionistic type theory, develops type theory as a constructive foundation for mathematics [DP24; Mar84; NPS90]. As with regular type theory, one begins with judgements about contexts, types, terms and equality.

Dependent types are a powerful extension: within this theory, a type may depend on a term. If A is a type and $B(x)$ is a type depending on $x : A$, then one can form dependent function and pair types. Dependent function types, Π , and dependent pair types, Σ , generalise the definitions of their ‘simpler’ versions above⁴. Furthermore, these functions and pairs can then themselves also be considered terms of yet more general types or functions and pairs. It is worth being explicit that this extension makes these functions and pairs extremely flexible⁵. The more formal definitions are presented in Definitions A.1.9 and A.1.10.

The law of excluded middle is not derivable, which is desirable within the intuitionistic discipline. If excluded middle is needed for a particular argument, it must be introduced as an assumption and paid for openly. A small demonstration of non-derivability is made explicit in Proposition B.3.1.

The above are generally accepted for Martin-Löf-style type theory, but there are some specific choices that can be made in the application of this theory, especially with respect to **identity types** and **inductive types**. Identity types internalise propositional equality, whereas judgemental equality records equality by computation or definition. The intensional/extensional distinction is not a distinction between “small” and “large” amounts of computation: for example, $3 + 1 \equiv 4$ is judgemental by numeral computation in the ordinary intensional theory as well. The relevant difference appears when one has variables $a, b : A$ and a term $p : \text{Id}_A(a, b)$. Intensional type theory permits transport along p

⁴Dependent function types roughly correspond to universal quantification, $\forall$, dependent pair types roughly correspond to existential quantification, $\exists$.

⁵Indeed, this way of considering functions – as terms themselves – forms the basis of functional programming languages, in which functions can be passed around as terms.

4. Third movement: Adagio

through the identity eliminator J , but does not thereby turn a and b into judgementally equal expressions. An extensional theory with equality reflection may infer $a \equiv b$ from p . Inductive types allow familiar mathematical objects to be generated by constructors: natural numbers from zero and successor, finite types from explicit labels, lists from empty lists and cons, and so on. The more formal definitions can be found in Definitions A.1.13 and A.1.15.

In order to develop Martin-Löf-style type theory into a modal logic in the next movement, the extension by Gratzer and collaborators is employed [Gra+21; Gra23; GSB19]. For this reason, my design for Martin-Löf-style type theory is partly governed by the design of their theory. These choices are made explicit in Definitions A.1.6, A.1.7, A.1.13, A.1.15 and A.1.19.

This concludes the philosophical and mathematical setting for the constructive part of the thesis. The Martin-Löf-style type theory used here is intuitionistic in its logic, epistemological in its interpretation, finite enough to avoid infinite paradoxing, and computational enough to connect with proof checking. It therefore answers Jaynes' demand for explicit constructions without inheriting his overly broad universality claim: the framework tracks what can be formed, proved and computed from a given context, while leaving ontology deliberately open.

CHAPTER 5

Fourth movement: Allegro maestoso

*“Scientists have calculated that the chance of anything so
patently absurd actually existing are millions to one.
But magicians have calculated that million-to-one chances crop
up nine times out of ten”*

Terry Pratchett in *Mort*

5. Fourth movement: Allegro maestoso

The previous movement left three issues to address:

1. **Subjectivity**: has to become part of the formal language rather than a slogan about priors.
2. **Probability**: has to be reconstructed over finite, decidable objects rather than arbitrary classical propositions.
3. **Causality**: has to enter as intervention and model change, not merely as statistical dependence.

The fourth movement takes these in order: *contexts and modalities* first, *probability types* next, and *finite structural causal models* last.

5.1 Context manipulation in Martin-Löf-style type theory

Contexts are the quiet machinery of subjectivity. A context records what is currently available: variables, assumptions, dependencies and previously constructed terms. In ordinary prose one says “given what we know”, in type theory one writes a context. The formal definitions can be found in Definition A.1.2 and Remark A.1.3.

Several operations on contexts have immediate epistemic readings. *Extending a context by a new variable* corresponds to learning or introducing a new assumption. *Allowing a later variable to depend on an earlier one* records directed dependence. *Deleting an unused variable* corresponds to forgetting or ignoring something that nothing else depends on. Furthermore, *if a question depends on a variable absent from the current context*, the appropriate answer is not false but unavailable – not in this context, not from here, not with this information. What is in scope can be used. What is out of scope cannot even be formed as a judgement. These operations give a first, austere account of subjectivity. A later causal notion of forgetting is stricter: it inverts a recorded learning step, rather than deleting an arbitrary unused variable.

This also gives a first account of intervention. *To intervene* is not merely to observe a value; it is to set a variable and sever or alter the dependencies that normally determine it. In a context, that means changing the dependency structure. *To hypothesise* is similar but more theatrical: one extends the current context with alternative variables, changes something there, and asks what follows under that altered arrangement.

Context manipulation alone is not yet the full, subjective theory. It gives the raw operations, but not the classification of epistemic states as such. For that, the thesis treats certain context changes as transitions between modes. Strictly speaking, ordinary type theory could already perform the operations. However, subjectivity becomes clearer when specific operations are marked as movements between states of knowing. A **mode theory** supplies that marking, which will be discussed in the following section.

5.2 Modalities inside Martin-Löf-style type theory

A mode theory is the smallest extra structure that records an epistemic location: it declares a small collection of epistemic states (*modes*), together with the admissible transitions between them (*modalities*), and indexes every judgement

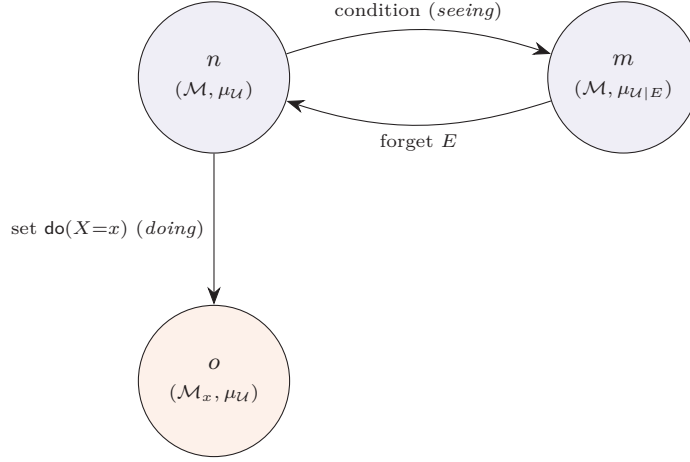


Figure 5.1: Modes as epistemic states; modalities as admissible transitions. Conditioning updates the exogenous distribution (abduction); setting replaces a mechanism (intervention). The fragment is introduction-only: modalities *mark* admissible moves rather than inspect the context (Definition A.2.4 and Remark A.2.10).

by the mode at which it is made. A mode can be read, for present purposes, as an epistemic state. A modality can be read as a structured way of moving from one epistemic state to another, see Figure 5.1. The price is one extra index on every judgement; the benefit is that learning, forgetting, hypothesising and intervening become (a combination of) formal moves rather than informal annotations.

Gratzer’s theory provides an extension of Martin-Löf-style type theory, where one works not only with contexts and types, but with modes and modalities between modes [Gra+21; Gra23; GSB19]. Gratzer’s theory is rather extensive and provides many additional proofs and formal definitions. For the purposes of this thesis, the technical ambition is rather limited, as I only need *specific* modes and modalities to function properly, rather than a generalised case. The formal fragment used here is therefore rather modest in scope and made explicit in Definitions A.2.1 to A.2.5 and A.2.7. Its finite causal records, fixed-signature transitions and model-changing edits are stated in Definitions A.7.1, A.7.3 and A.7.4; their preservation, conditioning and cancellation facts are in Propositions B.7.2, B.7.4 and B.7.5.

With subjectivity explicitly formalised, the next step is to allow uncertainty within the epistemic states themselves; that is, uncertainty over the available propositions within a given context. For this, a new type needs to be introduced: **probability types**. These are discussed next.

5.3 Probability types

The next task is to allow for uncertainty within each context. The guiding idea is simple: a probability assignment should be an explicit object attached to a finite and, ideally, decidable domain. It should therefore be represented

5. Fourth movement: Allegro maestoso

by a type whose inhabitants carry the required structure, as laid down in Definitions A.3.1 and A.3.7 to A.3.9.

In Martin-Löf-style type theory, however, an arbitrary proposition is *not* decidable, because excluded middle is not available. Probability in this thesis is therefore assigned only to explicitly decidable events on finite types. When the subject matter is the proof status of a proposition, a chosen finite inspection procedure may classify each available outcome as proved, refuted or unresolved. The three tags, and hence the resulting status events, are decidable even when the proposition is not. A constant procedure returning *unresolved* is always available, so this construction does not smuggle in a decision of the proposition; informative proved or refuted outputs require the corresponding soundness witnesses. This distinction is made precise in Definitions A.3.2 and A.3.4.

Clayton and Waddington provide the bridge from Cox-style reasoning to a finite, constructive reconstruction [CW17]. The rules named in Section 3.2 are made formal in Definition A.3.22. The order of use is slightly different from the original Clayton–Waddington route. Furthermore, as already mentioned, universality, continuum regularity and arbitrary-proposition complementation are not used in the constructive proof, but are part of the original paper, as it operates on a different domain.

Step	Ingredient or result	Role in the argument	Where
1	Finite resolution-status events	Replaces probability on arbitrary unresolved propositions by a decidable three-way event partition.	Proposition B.5.5 and Corollary B.5.15
2	Finite urn principles	Give the order-preserving rescaling $U_N(k) \mapsto k/N$; decidable urn complements then satisfy the finite complement rule.	Proposition B.5.7
3	Constructed product urns	Cartesian products of finite cell types give multiplication for rectangular events after the ratio rescaling.	Proposition B.5.8
4	Record–urn equivalence	Shows that urn presentations and common-denominator finite rational records agree on every decidable event.	Proposition B.5.10
5	Finite event calculus	Proves non-negativity, normalisation and finite additivity, with monotonicity and inclusion–exclusion as corollaries.	Remark A.3.11, Theorem B.5.12, and Corollaries B.5.13 and B.5.14
6	Conditional probability	Proves Bayes’ theorem and Bayesian update for decidable events with positive conditioning probability.	Definition A.3.12, Theorem B.5.16, and Propositions B.5.18 and B.5.19

Table 5.1: Route through the finite probability reconstruction.

The full proof route is summarised in Table 5.1 and carried out in Section B.5. Starting from the rules of Clayton and Waddington, the construction recovers both the finite Kolmogorov laws and Bayes’ theorem. Since these rules are appealing and intuitive, and since the final result appears in an epistemological and constructive setting, the construction captures many features desired by both Jaynes and his critics. The main victory is conceptual: probability is made into something fundamentally constructible and checkable.

For a complete formalism, causality still needs to be explicitly introduced. In the next section, **structural causal models** will serve both as an implementation of causality and as a vehicle for epistemic states.

5.4 Structural causal models in Martin-Löf-style type theory

Pearl-style structural causal models can now be rebuilt from the same basis. A finite structural causal model consists of variables, exogenous noise, structural functions and a finite rational probability distribution over the exogenous variables [Pea09; PM18]. The formal definition can be found in Definitions A.3.14, A.3.15 and A.4.14. In the constructive setting, each component can be represented explicitly: variables as finite types or indexed families, structural functions as terms, exogenous uncertainty as a probability type – defined in the previous section – and interventions as transformations of the structural functions.

The ordered shape of a type-theoretic context is doing real work here: later declarations may depend on earlier ones; earlier declarations cannot depend on later ones. Thus, acyclicity is not merely an external graph condition in the restricted fragment: it is reflected in what the structural term is allowed to mention. This is the ordered-telescope reading of finite recursive structural causal models.

The modal reading gives structural causal models their epistemic interpretation. A causal model is more than an abstract graph: it represents a genuine state of knowledge, namely what variables are distinguished, what mechanisms are assumed, what interventions are meaningful, and which counterfactual questions can be asked. Moving from one structural causal model to another is therefore a change of epistemic state; see Definition A.7.2 for a formal definition. Modes containing these structural causal models can represent these states; modalities can represent admissible transitions between them, see Figure 5.2.

Causal reasoning fits this modal reading compositionally. This means that an intervention or twin-network construction is not treated as an unexplained modal primitive. A structural intervention replaces selected mechanisms by constants and then removes the selected variables’ incoming endogenous and exogenous links one at a time. A twin construction creates the two observed sectors and the shared exogenous roots, relates the roots and copied observed variables according to the two graphs, and sets the intervention in the counterfactual sector. This is why I do not introduce a separate ‘do modality’ or ‘twin-network modality’. The point of the modal machinery introduced in Section 5.2 is precisely that smaller, causally meaningful edits can be organised into larger constructions. In Gratzner’s theory, composites of modalities are again legitimate

5. Fourth movement: Allegro maestoso

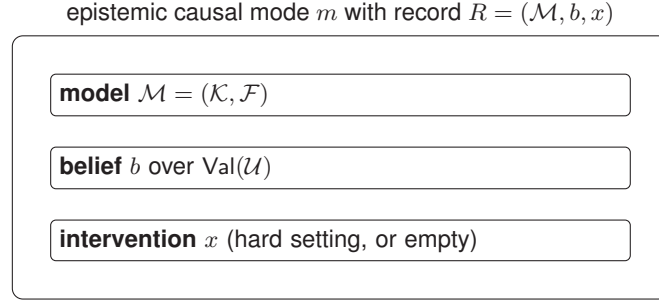


Figure 5.2: An epistemic causal mode is a name together with a record $(\mathcal{M}, b, x)$: the current structural causal model, the current latent belief, and the current hard intervention (Definition A.7.2). Moving between modes may change any of these three fields.

modalities [Gra+21; Gra23]. Lean checks an ordered atomic intervention path and proves it semantically equivalent to compact hard-intervention surgery. For finite counterfactual events it also executes two occurrence-indexed paths: one appends the counterfactual occurrences to a fresh factual mode, while the other creates every exogenous root, factual node and counterfactual node from a genuinely empty mode. Both paths install causal links and equations, reindex the source belief explicitly and compile the occurrence actions from the same primitive edits. Their dependent endpoints need not be definitionally equal; Lean instead proves pointwise agreement after coordinate transport and equivalence of their event probabilities. For a one-action query, the executed endpoint is additionally proved equivalent to the independently defined **TwinNetwork**. It does not claim that arbitrary dependent-signature duplication is definitionally equal to repeated applications of the restricted terminal-learning constructor.

Seen in this way, the current version of Pearl’s framework suggests transitions beyond ordinary causal queries. In an ordinary query, a structural causal model is usually held fixed; in scientific reasoning, however, the model itself may change: a mechanism may be discovered; a variable may be split in two; an intervention may be reclassified; a hidden confounder may be introduced; or a previous assumption may be withdrawn. Treating causal models as epistemic modes makes such changes visible as transitions between states of knowledge. The present thesis checks a limited calculus of such changes — typed directed and latent-input linking, terminal endogenous and exogenous creation, provenance-guided deletion and structural intervention — but does not claim a complete general theory of arbitrary model revision.

It is worth running a concrete example before presenting more general results. The natural choice is to revisit Mira’s situation from Example 4.2.1, but now from the hiring committee’s side. Officially, the committee claims to select on research dossier strength alone; in practice, candidates from prestigious institutions tend to do better, fashionable topics tend to attract committee members with matching expertise, and funding lines have their own seasonality. The interesting question is not whether such a committee is biased — almost every committee is — but whether the bias visible at the output reflects a stable causal mechanism or merely a hidden common cause. The model below is small

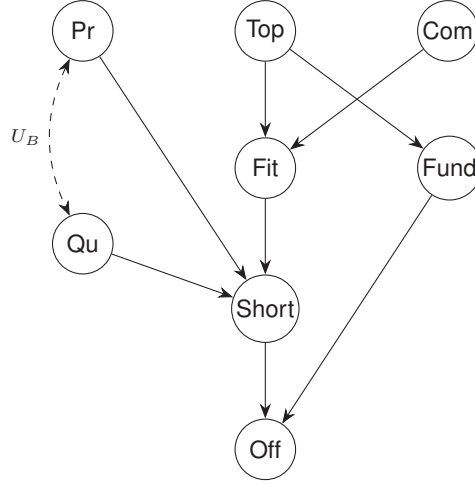


Figure 5.3: Mira's tenure-track structural causal model. The triples $\text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$, $\text{Pr} \rightarrow \text{Short} \leftarrow \text{Fit}$ and $\text{Short} \rightarrow \text{Off} \leftarrow \text{Fund}$ are colliders; the dashed edge is the latent common cause U_B inducing $\text{Pr} \leftrightarrow \text{Qu}$ (Example 5.4.1).

enough to inspect line by line and rich enough to exhibit the standard causal phenomena, see Figure 5.3. The type-level proposition $\text{Offer}(\text{Mira})$ now becomes the Boolean endogenous variable Off , and the three resolution statuses *proved*, *refuted* and *unresolved* from Definition A.3.4 become decidable events on the joint exogenous assignment.

Example 5.4.1 (Mira's tenure-track search, as a structural causal model). Returning to Mira from Example 4.2.1, the running example is now a stylised tenure-track hiring decision viewed from the search committee's side. The finite probabilities below should be read as a toy model for a candidate already under serious committee consideration, not as empirical base rates for all tenure-track applications. All endogenous variables are Boolean. Let Pr denote institutional prestige, Qu research dossier strength, Top topic fashionability, Com committee expertise, Fit committee fit, Short being shortlisted, Fund funding-line availability and Off receiving the tenure-track offer; the previous proposition $\text{Offer}(\text{Mira})$ is the event $\text{Off} = 1$ on Mira's joint exogenous assignment. One finite structural causal model is given by independent Boolean exogenous variables $U_B, U_{\text{Pr}}, U_{\text{Qu}}, U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}$ and structural equations

$$\begin{aligned}
 \text{Pr} &:= U_B \vee U_{\text{Pr}}, & \text{Qu} &:= U_B \vee U_{\text{Qu}}, \\
 \text{Top} &:= U_{\text{Top}}, & \text{Com} &:= U_{\text{Com}}, \\
 \text{Fit} &:= \text{Top} \wedge \text{Com}, & \text{Short} &:= \text{Qu} \wedge (\text{Pr} \vee \text{Fit}), \\
 \text{Fund} &:= \text{Top} \vee U_{\text{Fund}}, & \text{Off} &:= \text{Short} \wedge \text{Fund}.
 \end{aligned}$$

The directed structure contains

$$\begin{aligned}
 U_B &\rightarrow \text{Pr}, \quad U_B \rightarrow \text{Qu}, \quad \text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}, \\
 \text{Pr} &\rightarrow \text{Short} \leftarrow \text{Fit}, \quad \text{Qu} \rightarrow \text{Short}, \quad \text{Top} \rightarrow \text{Fund}, \\
 \text{Short} &\rightarrow \text{Off} \leftarrow \text{Fund}.
 \end{aligned}$$

5. Fourth movement: Allegro maestoso

If U_B is not observed, the observed semi-Markovian projection (in the sense of Definition A.4.12) has the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$. Thus the example contains an unobserved common cause, colliders, ordinary directed chains and the graph predicates used later for d-separation and c-components (Definitions A.4.18 and A.5.7). With the finite rational probabilities in Example A.8.1, one obtains, for instance,

$$P(\text{Qu} = 1 \mid \text{Pr} = 1) = \frac{3}{4}, \quad P(\text{Qu} = 1 \mid \text{do}(\text{Pr} = 1)) = \frac{1}{2}.$$

Seeing prestige gives evidence about the hidden background variable that also affects dossier strength. Setting prestige by intervention does not change that background variable and therefore does not by itself improve the dossier.

The same finite model also supports a representative counterfactual calculation. Suppose the actual evidence is that the candidate had no prestige, had a strong dossier, and received no offer:

$$E := \{\text{Pr} = 0, \text{Qu} = 1, \text{Off} = 0\}.$$

After conditioning the exogenous distribution on E , replacing the equation for Pr by the constant value 1, and evaluating the offer variable in the intervened model, one obtains

$$P(\text{Off}_{\text{Pr}:=1} = 1 \mid E) = \frac{1}{2}.$$

This is a genuine abduction–action–prediction calculation for the concrete finite model, checked again in Example A.8.1; it is not a general counterfactual-identification theorem from a graph alone.

In the tenure-track example the epistemic reading means that the model is not treated as background prose, but takes a more active role. Schematically, at the tenure-track mode there is a judgement

$$\Gamma_{\text{ten}} \vdash \mathcal{M}_{\text{ten}} : \text{FinSCM}@m_{\text{ten}},$$

where FinSCM is the type of finite recursive structural causal models in the restricted fragment, as introduced in Definition A.5.6. An intervention such as $\text{do}(\text{Pr} = 1)$ has two checked presentations. The compact evaluator retains $\mathcal{M}_{\text{ten}}$ together with the hard-intervention assignment $\text{Pr} := 1$ and overrides that equation during evaluation. The structural presentation constructs a target mode whose graph has every incoming directed and latent link into Pr removed and whose mechanism for Pr is the constant 1. Lean proves these presentations pointwise equivalent, so both denote $\mathcal{M}_{\text{ten}, \text{Pr}:=1}$. A calculation such as

$$\Gamma_{\text{ten}} \vdash P_{\mathcal{M}_{\text{ten}, \text{Pr}:=1}}(\text{Qu} = 1) = \frac{1}{2}@m_{\text{ten}}$$

is therefore a judgement about a finite probability expression inside the current epistemic causal mode.

In this movement, I am not claiming to rebuild all of Pearl’s theory from scratch, but I am making a finite correspondence claim: in the finite, discrete and recursive case, the ordinary ingredients of a structural causal model can be represented inside the type-theoretic setting used here. Variables become finite types; structural equations become terms; exogenous uncertainty becomes

5.4. Structural causal models in Martin-Löf-style type theory

a finite rational probability type; interventions become transformations of the relevant structural equations; and the graph tests used by do-calculus remain finite combinatorial checks.

Once this bridge is in place, the classical finite results do not need to be reinvented. The appendix proves the model and identifiability correspondence, defines an executable finite d-separation test and an inspectable derivation language, and then takes the applicable classical soundness and completeness specializations as explicit theorem parameters; see Table 5.2. The point is not to reprove Pearl–Tian–Shpitser completeness or the global Markov theorem from scratch, but to state exactly what must be supplied from those results and to check the resulting transport [HV06; Pea12; SP06; SP08; TS09].

Step	Ingredient or result	Role in the argument	Where
1	Finite source and target records	Fixes an independently executable finite-table source and its intrinsic dependent target fragment.	Definition A.5.5
2	Model correspondence	Proves pointwise agreement of source-table and target evaluation, probabilities, interventions and graph data, and equivalence of the identification notions.	Theorem B.6.4
3	Probability, graph and derivation data	Preserves event probabilities and graph predicates; computes finite d-separation and represents probability-algebra and do-calculus steps as inductive certificates.	Propositions B.6.7 and B.6.8 and Definition A.5.12
4	Query, identifiability and modal transport	Transfers local interventional queries and identifiability claims across the source interpretation, annotates every derivation by action/observation locks and Pearl-rule cells, and indexes the resulting certificate by a compatible proof-carrying mode.	Corollary B.8.4 and Propositions B.7.3, B.9.1 and B.10.6
5	Soundness and completeness transport	Takes the finite classical soundness/completeness specializations as explicit data and transports inspectable certificates and hedge counterexamples.	Proposition B.9.3 and Section B.10
6	Counterfactual extension	Checks the shared-latent twin reference, occurrence-indexed multiworld endpoint records executed either from a factual mode or from an empty mode, structural abduction–action–prediction and finite multiworld source–target semantics; conditionally transports a supplied source-native ID^*/IDC^* reduction and its ordinary observationalization through the executed modal endpoint semantics.	Propositions B.8.1 and B.8.3 and Theorem B.11.1

Table 5.2: Route through the finite causal correspondence and completeness transfer.

The transport argument is mostly bookkeeping, but this is precisely why it matters, as the architecture is now explicit: probability types supply uncertainty;

5. Fourth movement: Allegro maestoso

modal dependent type theory supplies epistemic movement; structural causal models supply intervention and counterfactual vocabulary; and Lean checks representative finite pieces of the construction. The Lean results are discussed in the following section.

5.5 Lean 4 implementation

The mathematical manifesto asks for explicitness, and in this work Lean 4 is used as a proof checker [MU21]. The default library consists of the stable facades `lean/Thesis/Probability.lean`, `lean/Thesis/Causality.lean` and `lean/Thesis/CausalTransport.lean`, their implementation modules, and checked examples under `lean/Thesis/Examples/`. The full eight-variable tenure-track calculation now uses this same default library; there is no separate legacy implementation. The aim is not to mechanise every philosophical claim of the text, but to make the exact finite transport boundary independently checkable.

The boundary of the formalisation is just as important. Strictly speaking, a Lean proof is not a proof in raw intensional Martin-Löf type theory. Lean 4 is a proof assistant based on dependent type theory [MU21], but it has its own kernel, inductive types and conventions. The project source uses no explicit `Classical.*`, no global excluded-middle principle, no explicit `Classical.choice` and no `native_decide`. The focused `#print axioms` audit in `lean/Thesis/AxiomAudit.lean` reports only Lean’s constructive extensional infrastructure, `propext` and `Quot.sound`, for those audited declarations that are not axiom-free. The development must therefore be read as constructive-extensional Lean, not strict raw intensional Martin-Löf-style type theory. This is the deliberate fallback boundary: none of the audited probability, source–target, modal, soundness, transport or tenure-track declarations depends on excluded middle or classical choice. Obtaining a literally axiom-free intensional development would require rebuilding ordinary library arithmetic and function/setoid infrastructure and is not claimed here. The detailed proof-status and foundational conventions are given in Section B.4.

The modal files formalise proof-carrying finite record transitions, typed causal link edits, endogenous and exogenous creation, provenance-guided forgetting, structural intervention, action and observation locks in the sense of Definition A.5.13, operation-sensitive Pearl-rule cells, modal derivation traces, modal counterfactual worlds and a generic abduction–action–prediction bridge compatible with the Gratzner-style discipline described in Definitions A.2.1, A.2.4 and A.2.5; they do not formalise Gratzner’s full modal dependent type theory [Gra+21; Gra23; GSB19]. Likewise, the causal files do not reprove the published `ID`, `IDC`, `ID*` or `IDC*` algorithms, the global Markov theorem or the equivalence between path blocking and ancestral moralisation. They define the finite computations, partial probability-expression and multiworld semantics and certificate syntax; isolate the path criterion in `DSeparationCorrectness` and use it to compile path-based certificates; take the applicable classical completeness, primitive semantic soundness, counterfactual-reduction and latent-projection results as explicit arguments; and check the model correspondence and the resulting transports. This distinction is made precise in Section B.10.

The flavour of the Lean code is best seen in an example, here the record–urn equivalence theorem:

Listing 5.1: Record–urn extensionality (`lean/Thesis/Probability.lean`).

```
theorem urn_to_record_extensional (m : UrnProb 0) :
  forall E : Event 0, QProb.Equiv ((ofUrn m).probVal E) (m.probVal E) :=
  ofUrn_agrees m

theorem record_to_urn_extensional (R : FiniteProbRecord 0) :
  forall E : Event 0, QProb.Equiv (R.probVal E) (R.toUrn.probVal E) :=
  R.toUrn_agrees
```

Read informally, the two statements say that converting an urn to a finite record, or a finite record to an urn, does not change the probability assigned to any decidable event. The proof is not deep, and that is exactly the sort of bookkeeping that prose often hides: counts are preserved when atoms are expanded, unit-weight atoms count as one cell, and the resulting rational probabilities agree by cross multiplication. Lean is used here not for theorem-proving acrobatics, but to keep the finite construction honest.

The Lean implementation is best read as a test of explicitness. The theorem-facing modules check node-dependent finite value types, explicit latent roots, independently executable finite source tables, structurally generated hidden-DAG models, direct-root and hidden-path projections, generic rational conditional-probability-table functionalisation, recursive evaluation, hard and structural interventions, an explicit shared-latent twin network, finite multiworld query semantics, executable finite d-separation, separate source and target probability-expression semantics, finite c-forests and hedges, concrete derivation certificates, source–target identifiability transport, operation-realized modal traces, typed causal structure edits, terminal learning and forgetting, and modal abduction–action–prediction. These correspondences are collected in Table 5.3.

5. Fourth movement: Allegro maestoso

Module below Thesis/	What it checks	Where it connects
Probability.lean; Probability/Core.lean; Reindexing.lean; FiniteRecord.lean; Urn.lean; Construction.lean	The stable facade and its finite rational core, records, urns, common-denominator construction, dependent products, finite-sum reindexing, Kolmogorov laws, rescaling and constructive conditioning.	Section 5.3; Table 5.1; Section B.5
Causality.lean; Causality/ Graph.lean; Model.lean; Core.lean; Identification.lean	The stable intrinsic-causality facade; node-dependent finite values, directed–bidirected graphs, explicit latent roots, product priors, typed mechanisms, well-founded evaluation, exact models, hard interventions and graph-indexed query semantics.	Definitions A.4.12 and A.5.5
Causality/ Counterfactual.lean; Multiworld.lean; ExecutedMultiworld/*.lean; ModalCounterfactual.lean	A finite two-world reference, occurrence-indexed multiworld query semantics, ordinary finite-SCM encoding, literal from-factual and from-empty atomic execution, endpoint-record probability preservation and generic abduction–action–prediction.	Definitions A.6.1 to A.6.4 and A.7.8 and Propositions B.8.1 and B.8.2
Causality/ Derivation.lean; Semantics.lean; ModalDerivation.lean	Executable d-separation, Pearl’s rules and derivation trees, partial finite-rational denotation, recursive support witnesses, and operation-sensitive query locks, rule cells and modal traces.	Definitions A.5.12 and A.5.13

5.5. Lean 4 implementation

Module below Thesis/	What it checks	Where it connects
Causality/ Modalities.lean; Structural/*.lean; ModalRealization.lean; EndpointSemantics.lean; EndpointAgreement.lean	Core proof-carrying transitions, one indexed causal-edit family, executable lock and atomic-kernel realizations, typed linking, variable creation, structural intervention and current-state endpoint semantics.	Propositions B.7.5 and B.7.8
Causality/ Learning.lean; Reductions.lean; PairRoot.lean	Conservative terminal extension, hidden-path projection, rational CPT functionalisation and the graph-only pair-root expansion.	Propositions B.6.3 and B.7.6 and Lemma B.12.1
CausalTransport.lean; CausalTransport/ FiniteSource.lean; DSeparation.lean; Correspondence.lean; Modal.lean; ModalRealization.lean; Learning.lean	The stable transport facade; independent finite-table source evaluation, source-native expression/support semantics, explicit source–target preservation, active-path compilation, distributional queries, hedges, modal certificates and conservative-learning certificate transport.	Theorems B.6.4 and B.13.3
CausalTransport/ Counterfactual.lean; ModalCounterfactual.lean	Independent source semantics for arbitrary finite multiworld events, source–target support and identifiability correspondence, and mode-indexed conditional transport from a source-native counterfactual reduction plus ordinary do-calculus observationalization.	Proposition B.8.3 and Theorem B.11.1
CausalTransport/ HiddenDAGModel.lean; HiddenDAG.lean; Construction.lean	Structurally generated full hidden-DAG SCMs, the external latent-projection interface, and the CPT-construction-to-transport composition theorem.	Proposition B.12.2 and Theorem B.13.3
Examples/ TenureTrack.lean	The full eight-variable SCM, shared latent confounder, observational and conditional probabilities, interventions, twin-network and occurrence-indexed counterfactuals and the structural modal abduction–action–prediction calculation, all using the default finite machinery.	Example A.8.1 and Corollary B.8.5

5. Fourth movement: Allegro maestoso

Table 5.3: Lean modules and their checked finite constructions.

5.6 Intermediate results

The research questions postulated in the Prelude were:

1. Can Jaynes' probabilistic framework be cast in the language of Martin-Löf-style type theory?
2. Can Jaynes' probabilistic framework be extended so as to natively include subjectivity?
3. Can Jaynes' probabilistic framework be extended so as to include Pearl-style causal reasoning?
4. Can Jaynes' commentary on bad scientific practice be placed in a wider institutional context?

This movement answers the first three questions in the restricted finite setting of this thesis. First, Jaynes' probabilistic framework can be cast in Martin-Löf-style type-theoretic language, provided probability is assigned to decidable events on finite sample types rather than to arbitrary propositions. This reconstruction is developed in Section 5.3, while the proof route is summarised in Table 5.1.

Second, the framework can be extended with a native account of subjectivity, where subjectivity is no longer reduced to the choice of a prior. Contexts, modes and modalities make the epistemic position itself part of the formal language. This account is developed in Sections 5.1 and 5.2, while the corresponding finite transition layer is summarised in Table 5.3.

Third, Pearl-style causal reasoning can be embedded in the same finite constructive setting. Structural causal models, interventions, explicit twin semantics and representative counterfactual calculations are developed in Section 5.4, while the correspondence and conditional completeness-transfer routes, including their external theorem parameters, are summarised in Table 5.2.

The fourth question is deliberately left open here. It belongs to Chapter 6, where the formal concern with hidden assumptions is connected to the institutional conditions under which scientific inference is produced.

CHAPTER 6

Fifth movement: Presto con fuoco

“For far too long economists have sought to define themselves in terms of their supposedly scientific methods. In fact, those methods rely on an immoderate use of mathematical models, which are frequently no more than an excuse for occupying the terrain and masking the vacuity of the content. Too much energy has been and still is being wasted on pure theoretical speculation without a clear specification of the economic facts one is trying to explain or the social and political problems one is trying to resolve”

Thomas Piketty in *Capital in the Twenty-First Century*

6.1 Questionable research practices on a large scale

Jaynes' criticism of bad scientific attitude can be read as a criticism of individual character, as was seen in Section 2.3:

1. **Gamesmanship:** the desire to display technical virtuosity, rather than to clarify a problem;
2. **Cutting corners:** the focus on polished results, without explaining the painstaking work that led to these outcomes;
3. **Hidden assumptions:** the silent transfer of what should have been stated into what is merely presumed;
4. **Hyperfocus:** the habit to only investigate an incredibly specialised subfield, whilst forgetting the wider context in which this research sits;
5. **Theoretical navel-gazing:** the production of mathematics that neither explains, predicts, nor helps anyone build or test anything practical.

In his text he then continues to blame Fisher for having all these vices, while Jeffreys is presented as a model scientist. Besides the fact that this is rather simplistic view of both quite complicated characters, the following observation by Jaynes is at least as interesting: Jaynes laments that although Jeffreys was correct in his view on probability theory¹, he only trained one Ph.D. student during his whole career². Meanwhile, Fisher, who according to Jaynes was utterly wrong on his views on probability theory, trained several tens of influential Ph.D. students [Jay03]. Although in itself completely anecdotal, this observation suggests that perhaps the mechanisms by which scientific ideas and scientific careers spread do not select primarily for truthfulness.

The business language that has become central to science matters here. Discovery is naturally highly nonlinear, but is nowadays often translated into a linear framework of targets, deliverables and measurable output. This also makes the risk more precise: when funding, promotion, job security, visas and prestige all depend directly or indirectly on publication rate, publication can become the object that is optimised, rather than any 'truthfulness' as such. The phrase 'publish or perish' is therefore not merely a complaint about stress, but it highlights a selection environment. As such, it is advisable to consider scientific inheritance, promotion and prestige as part of the machinery of inquiry as well. McElreath's reminder from the Prelude is therefore central here: inference is only one component of science [McE20]. A formally correct inference engine cannot rescue a pipeline in which claims are not selected for their truthfulness, but rather for fashionability, publishability and speed.

Surveys and meta-analyses of questionable research practices and misconduct already suggest that scientifically questionable behaviour is widespread, and even more so since the official record understates the problem, as (self-)reporting is naturally limited by stigma and personal risk [Fan09; JLP12; MAV05]. Retraction and paper-mill studies emphasise the same problem from another

¹Jeffreys is widely known for having resisted the theory of plate tectonics throughout his life.

²Maybe not completely coincidentally, also Jaynes only had one Ph.D. student during his career, giving this observation a rather personal dimension.

6. Fifth movement: Presto con fuoco

angle [Agr+25a; FSC12; Ric+25]. More recent reports on the sheer scale of industrialised fraud, fake authorship markets, pseudo-scientific journals and batch retractions make the same point even more urgently [Bri25; Nad26; NDR18; OGr25].

Study	Field	What was checked	Reported signal
Open Science Collaboration [Ope15]	Psychology	100 studies from three psychology journals	97% of original studies reported significant results; 36% of replications did so, 39% were subjectively judged to replicate, and replication effects were about half the original size.
Klein et al. [Kle+14]	Social psychology	13 effects tested across 36 samples in the Many Labs project	10 effects replicated consistently, one received weak support, and two did not replicate.
Camerer et al. [Cam+16]	Economics	18 laboratory experiments from top economics journals	11 of 18 replications, or 61.1%, found a significant effect in the same direction; replicated effect sizes averaged 66% of the originals.
Begley and Ellis [BE12]	Preclinical oncology	53 landmark cancer findings checked in an industrial drug-development setting	Scientific findings were confirmed in only 6 cases, roughly 11%, well enough to continue the development programme.
Button et al. [But+13]	Neuroscience	Statistical power and its consequences for neuroscience research	The central result is not a direct replication rate: low power lowers the chance that a significant result reflects a true effect and inflates effect-size estimates.

Table 6.1: Selected reproducibility signals across fields. The table is intentionally heterogeneous: significance rates, subjective replication judgements, confirmed findings and statistical power all expose different parts of the same institutional problem.

The scale of the consequences is difficult to explain by individual vice alone – contrary to what Jaynes posited. The studies in Table 6.1 all point to the same direction: when published claims are put under more explicit pressure, much of their apparent certainty disappears. Smaldino and McElreath’s point is precisely that bad methods can spread by selection even when most individuals are not trying to corrupt the record [SM16]. Bad science is therefore not only a matter of bad people; it is also a matter of bad selection pressures. The better conclusion to draw based on this data is not that science is full of vicious personalities, but that the current system actively rewards behaviour that is adjacent to fraud.

Explicit fraud cases are useful here, because they make the institutional pressures visible in extreme form. Stapel, Schön, Hwang, Obokata, Wakefield, the LaCour case involving Green’s coauthorship, Pandey, Raoult and Fujii are not typical in every detail, but they reveal familiar failure modes: fabricated or copied data, ethical shortcuts, failure to declare conflict of interest, salami-sliced output, hypothesising after the results are known (HARKing), reviewer manipulation, author manipulation, prestige dependence, citation games and highly inadequate correction mechanisms [BDK02; Cyr06; GSM11; Kon15;

Los11; MO15; OGr24; The16; The24]. Where the incentive of such behaviour stems from is the subject of the next sections.

6.2 Start of a research career

The pressure begins before a researcher has the power to resist it. A young researcher needs publications to obtain funding, funding to obtain positions, positions to obtain independence, and independence to obtain the time required for careful work. This is the catch-22 of grants and careers: one needs resources to do the work that justifies receiving resources [Ibá99; NI12]. The result favours rich institutions, already visible researchers and projects close enough to fashion that committees can recognise them. The inequality is not only personal: if ambitious research increasingly requires expensive equipment, time, grant-writing capacity and institutional prestige, then rich entities can attempt speculative projects while poorer entities either support the existing agenda or operate with much less room for risk. As a result, a conservative funding environment has an opportunity cost: low-quality but recognisable work is generally supported while high-quality but risky work is suppressed or delayed [Agr+25b; Lan+22; PLF23].

This structure makes positive findings unusually valuable. The incentive is not simply to do good work, but to produce something that can be written up as a finding. Nosek, Spies and Motyl describe the conflict directly: incentives for publishable results can pull against incentives for accurate results [NSM12]. Replication studies, negative findings and careful methodological work are all part of science, but they have often carried less immediate career value than mere novelty of results. Furthermore, the penalty for an irreproducible study is also often diffuse and delayed, whereas the reward for a publishable claim can be immediate.

The tragedy is that the system can then select against precisely the kind of slow work it later celebrates. Peter Higgs remarked that, under present productivity expectations, he would probably not have been considered productive enough to keep his post [Ait13; Al-14]. That is not an argument against funding or accountability, but it *is* an argument against confusing accountability with throughput.

As mentioned, the pressure to publish does not need bad intent in order to create bad science. Small samples, underpowered designs, flexible stopping rules and flexible analyses allow researchers to obtain publishable results without consciously committing fraud [But+13; SNS11]. Gelman and Loken's garden of forking paths emphasises that a study can be kneaded by many unnoticed analytical choices to obtain the desired results, even when nobody is deliberately 'fishing' [GL13; GL14]. A study can be made more publishable by post-hoc hypothesising, selective reporting, choosing the positive analysis among many alternatives, cutting power because more data takes more time, or replacing careful modelling by the quickest defensible model. These are not rare choices, but ordinary research decisions that can subconsciously support a publication economy that rewards significance, clarity and speed [Ioa05; NSM12; SM16; SNS11]. Gamesmanship, corner-cutting, not specifying hidden assumptions, hyperfocus and theoretical navel-gazing do not have to be taught, they can emerge as adaptive responses to the environment.

6. Fifth movement: Presto con fuoco

Additionally, it should be noted that aforementioned pressure can be especially sharp for researchers whose employment or residence status depends on visible output. This should be stated carefully: visa pressure does not make a researcher fraudulent, but when the right to stay in a country or the ability to support a family is linked to publication metrics, fraudulent intermediaries obtain a ready market [OGr25; Ric+25]. It can only be concluded that with the advent of more protectionist research environments the coming years, together with intensifying competition, fraud cases and retracted studies related to visas, permits and direct livelihoods will most likely increase.

6.3 Reporting in journals

Firstly, the current ‘winner-takes-all’ economy within science actively damages collaboration, as everyone competes for the same (limited) publishing space. When the first team to publish the positive result wins the reward, independent verification becomes a less attractive use of time. This is especially dangerous in fields where the underlying process of discovery is slow, uncertain and nonlinear. Artificial intelligence can intensify the problem by increasing the speed at which manuscripts, variations and near-duplicates can be produced [Zha26]. The journal article then turns an uncertain process into a clean product, and the market rewards the product more directly than the process that produced it, or any corrections that come after.

Journals are not merely neutral containers for such products, as they are also competing for attention, subscriptions, impact and prestige. Randy Schekman, a Nobel laureate in biology, has openly criticised prestige journals for distorting incentives [Sch13]. The justified worry in a system where journals have a conflict of interest with the truth is that surprise, novelty and controversy can be rewarded before reliability is known. As such, correction can remain slow or absent even after reliability is doubted or proofs are provided of clear misconduct [Bik26a; FSC12]. In practice, investigations by journals can be pushed back onto the same institutions, laboratories and even the very same people who are under investigation [OGr24; OM23; The24]. As such, even when some of the behaviours leading to publication are formally prohibited, the consequences are often lighter, slower or less predictable than the original reward [Cam98; MAV05; Smi06].

Once publication itself becomes a currency, a market grows around that currency: salami slicing, extreme self-citation, exploit editorial and review roles for citation, citation brokers, ghostwritten papers, fake authorship advertisements, paper mills, predatory or pseudo-scientific journals, unusual review processes, identity manipulation and fake reviewer accounts all exploit the fact that careers, visas, rankings and institutional prestige depend on visible output [Agr+25a; Bri25; Cam98; MAV05; Nad26; NDR18; OGr25; Smi06]. Also here remains true that gamesmanship, corner-cutting, not specifying hidden assumptions, hyperfocus and theoretical navel-gazing do not have to be taught as principles, but they appear as responses to the environment. At this point the problem is no longer only a set of individual lapses, as it can be readily concluded there has been built an entire industry around the weaknesses of current publication system.

6.4 Peer review and editorial process

Peer review is supposed to prevent many of the described behaviours, but the task assigned to it is almost impossible. Reviewers are asked to judge novelty, correctness, significance, ethics, method, relevance and sometimes fraud, usually without enough time, sufficient training, necessary data or any reward to do so [Smi06]. Furthermore, without proper training and a good definition of the process, there are many problems directly identifiable: inconsistency between reviewers; rivalry between reviewers and authors; bias, conservatism and self-interest of reviewers; and lack of clear standards for what defines a good review [Cam98].

As a result, review actively stimulates conservatism and review systems can punish risk while actually claiming to value innovation [AS14; GB24; Lan+22]. Reviewers and researchers are trained to be critical, and any paper will reveal weaknesses if pressed hard enough. Since the process is also structurally conflicted – as reviewers and authors often compete for similar publication space, funding and prestige – referees can ask for extra citations, editors can choose reviewers with predictable objections, and anonymous review can leave authors with little recourse against poor or self-interested judgement [Cam98; SR14]. In a sense, the garden of forking paths that is present for authors, exists for referees and editors as well. The broader evidence that papers and patents have become less disruptive over time gives this observation a historical perspective [PLF23].

Conservatism also gives disproportionate power to senior researchers and prestigious institutes [AS14; Cam98; Lan+22]. Their names can help a paper survive review; their views can shape what is judged plausible; and their time, networks and authority make them more likely to be involved in evaluation. In extreme cases this allows for coercive citation, inappropriate authorship demands and reviewer manipulation, or even citation cartels on larger scales [Agr+25a; Ric+25]. This can slow the ordinary replacement of old ideas by new ones considerably; a process which is natural in society and one of the cornerstones of the scientific method. A radical paper may be rejected because the shared assumptions of the review community make it hard to read fairly. Gamesmanship, corner-cutting, not specifying hidden assumptions, hyperfocus and theoretical navel-gazing can then be reproduced by peer review rather than exposed by it.

The standards for reviewing are often also surprisingly thin. Authors are usually given detailed formal obligations on how and what to publish, but reviewers and editors often receive much less training and fewer explicit standards. Proper reviewing is therefore partly learned at the expense of authors: an anonymous referee can be careless, arbitrary or self-interested while the author carries the cost [Cam98; Smi06]. As reviewers are often anonymous, often competing researchers and often exceptionally time-pressured, the mentioned consequences for not having a clear review process are not a fringe phenomenon. Additionally, the remarkable fact that reviewers are not obliged to disclose their conflicts of interest is a source of concern, especially in fields where industry and academics heavily overlap. With such a system, it should not come as a surprise if both the false positive rate, as well as the false negative rate, would be considerable, even though both are extremely difficult to assess in practice [Ioa05; SNS11].

6. Fifth movement: Presto con fuoco

Finally, the hopes are that peer review could function as a first defence against serious offences like fraud, however: fabricated data, plagiarism, image manipulation, conflicts of interest, salami slicing, p-hacking, hypothesising after the results are known (HARKing), paper-mill products, reviewer fraud, unusual editorial processes and lost original data are often invisible to (other) reviewers working with a submitted manuscript and a deadline. Therefore, this turns out to be an insufficient guardrail in practice. The very fact that platforms for post-publication scrutiny, such as: Retraction Watch, PubPeer, Science Integrity Digest, Clear Skies, Elisabeth Bik's blog and the Black Spatula Project exist, is precisely because pre-publication review can demonstrably not carry the whole burden [Bik26a; Bik26b; Cle26; Pub26; Ret26; The26]. As such, it can be concluded that the general expectations for the review system are unrealistically high, while the current state of affairs indicate that the result are in fact not as intended.

6.5 Career advancement

As was alluded to above, researchers are rewarded for publication counts, citation counts, grants, prestige and managerial expansion more directly than for careful reasoning. Salami slicing, excessive self-citation, strategic authorship, rushed review, exaggerated claims and selective reporting can then quietly turn into professional habits. In order to be successful in every successive step, mostly 'important' publications literally count, and thus quantity can quietly devour quality. In fact, when read carefully, the analysis of Martinson, Anderson and de Vries seems to imply that the occurrence of scientifically questionable behaviour increases between successive stages of a scientific career [MAV05]. If this extrapolation turns out to be correct, the system as a whole can be considered to actively support gamesmanship, corner-cutting, not specifying hidden assumptions, hyperfocus and theoretical navel-gazing.

For researchers the sheer growth of the scientific literature makes careful participation even harder. Keeping up with one's own field can in itself already become a full-time job, and reviewing every relevant paper with equal care is downright impossible [BHM21; Lan16]. Reports of sharply rising reviewing loads and batch retractions are therefore not side details, but a clear indication of an overstretched system [Bri25]. Artificial intelligence may help with search, drafting and checking, but it can also increase output, intensify competition and create a loop in which machine-generated manuscripts meet machine-assisted review [Zha26]. The expected result for the coming years is not more serious science, but simply more similar throughput.

Contrarily, researchers at poorer institutions, or outside wealthy national systems, can be blocked from reading the literature to which they could meaningfully contribute. This directly adds a severe form of inequality to career advancement, and actively pushes such researchers towards lower-quality publication venues and fraudulent intermediaries [Ric+25; Spa18]. This means that, fundamentally, the researchers situated at better-off institutions have a significant advantage over their peers situated at poorer institutions in terms of: publication venues, citation counts, prestige, grants and, ultimately, career advancement. Furthermore, a public research system that hides publicly funded knowledge behind tolls weakens its own corrective capacity, and actively hampers

potentially fruitful collaborations.

When focusing on the scientific career path itself, one sees that, rather remarkably, the role of the researcher fundamentally changes whilst advancing in successive stages. Researchers are – as the word gives away – trained to do research, but as their career progresses, they are silently turning into managers of human resources, finance, public relations, grant strategy and institutional survival. Some researchers genuinely flourish in their new role, but it is worth to emphasise two things:

1. This career path is not efficient, something that might matter as often substantial public funding is involved;
2. There is often no good alternative for researchers that genuinely wish to specialise – and remain specialised – in doing research.

As there is a clear mismatch between training and occupation, and there is often no link between funding and managerial behaviour, unintended or unprofessional behaviour could result from this mismatch. The mismatch matters, because badly supported operations can directly result into gamesmanship, shortcutting, delegation without oversight and prestige management. As such, it should be concluded that if academia would like to improve the quality of its output, it should also reflect on the career path of its researchers.

6.6 Reform proposals

Science can be imagined in at least two noble ways: as the sincere search for truth, and as the organised attempt to solve important societal problems. As can be concluded from previous analysis, a system increasingly governed by key performance indicators should not automatically inherit the name ‘science’, as it does not align with these noble goals [Gel14; NSM12]. That is why current set-up fully deserves the name *funded research* instead. As it happens, some funded research is excellent science, however, most of it is merely funded research.

For funded research to fundamentally become *science* once more, several structural changes are highly recommended. It is not claimed that following recommendations are exhaustive, but they can be considered a fruitful starting point. The focus in the following paragraphs will be about funding, peer reviewing, publication, correction and task division inside research groups.

Funding should become more pluralistic. An argument in favour of current construction can be that it maintains a relative autonomy for researchers. Research autonomy greatly matters, but it should be emphasised that this autonomy does not move in neutral space. It is pulled by whoever has the clearest agenda and the strongest purse, which can, directly and indirectly, simply be the industry within a country [Agr+25b; SM16; Spa18]. It is therefore worthwhile, if science is to be directed towards solving important societal problems, to consider other sources of funding too:

- **Citizen-directed funding** can support local and neglected problems;
- **Democratic funding priorities**, that is: parties or democratically elected bodies that decide on funding priorities;

6. Fifth movement: Presto con fuoco

- **Specialist peer funding** can protect difficult work whose value is not obvious at first sight.

Such a system where funding can be obtained in more than one significant way rewards more kinds of contribution than only publishable results. Education, outreach, public problem-selection, good reviewing, replication, negative findings, careful methods, useful tools and well-curated data all create scientific and societal value. Funding and assessment should therefore be allowed to count these objects directly rather than pretending that their value only exists once it has been converted into a publication [BE12; DOR12; Mun+17; Nos+15].

Publication should be rebuilt around public access and public scrutiny. As an argument in favour of open access, it can be noted that now often considerable amounts of public spending are involved in creating publications that are publicly closed. With this argument in mind, the Council of the European Union has called for transparent, equitable and open access to scholarly publications [Cou23]. Indeed, it is justified that if public money pays for research, the public should not pay again to read it. Diamond open-access infrastructure, such as the programme described by DIAMAS, points in the right direction [DIA26]. A digital, public publication infrastructure should also make the older ‘limited space’ argument obsolete. Such a system can host well-defined, well-executed and correct work even when it is not necessarily novel.

Even more importantly, as space limitations are less stringent in the digital age, a finished paper should not be the only recognised scientific object shared. Indeed, such a platform can also host the parts of research that are usually hidden: protocols, raw data, analysis plans, code, materials, failed attempts, reviewer reports and later corrections. Public platforms such as the Open Science Framework, Wellcome Open Research and Open Research Europe show how protocols, data, methods, tools and results can all be made part of the public record [Cen26a; Eur26; Wel26]. If any specific aspect of a study, like data or a model, cannot be made fully public for ethical or proprietary reasons, the reason should itself be explicit. The repeated excuse that original data were lost should not be treated as a harmless administrative accident; in several fraud cases, missing raw data were central to the difficulty of correction [BDK02; BE12; NSM12; The16]. Registered reports, open materials, open data, better research assessment and explicit rewards for replication would make correction an integral part of the ordinary scientific cycle, rather than an embarrassing afterthought [Cen26b; DOR12; Mun+17; Nat26; Nos+15; NSM12].

Naturally, the same infrastructure would be most beneficial if linking were easy. A replication should be (visibly) connected to the original study; extensions should be connected to methods and datasets they reuse; comments and post-publication discussions should remain attached to the relevant research object; and reliability signals should be able to change as evidence accumulates. Standard researcher identifiers, such as ORCID-style identifiers, would strengthen accountability by making authors, reviewers and conflicts of interest of either harder to obscure. Templates, documentation and research-design tools could then help researchers find relevant methods, datasets and recent methodological improvements before embarking on a new study. Such an open system also changes the meaning of failure: a study that looked promising but is weakened by a later analysis, a failed replication or a better design need

not disappear, but can remain in the record as a corrected step in a growing process. That is closer to how knowledge actually develops, and it lowers the burden on pre-publication review to get everything right at once. In this way, a false start does not remain there without correction, linkage or context [Nat26; NSM12].

Review should become more open, more cumulative and more clearly credited. Anonymous review may still be useful in some settings, for example in swift double-blind review that the original author can request, to quickly assess the quality of the to-be published object. However, after its initial release, the system should also support signed reviews, public post-publication discussion and links between replications and originals, so that usability or credibility of scientific work can change over time. Researchers should, as such, be able to build reputations as solid reviewers and contributors. This would make review and replication itself a recognised scholarly contribution rather than invisible service work. In such a system a direct impetus would appear for institutions to offer serious training to reviewers, as good reviewers would become reputable members of the scientific community. For this to work properly, it is essential that, next to authors, also reviewers would be subjected to declare their conflict-of-interest with regards to the objects they are reviewing. Such a review system would considerably lower the burden on pre-publication review, while increasing the appreciation of review as a serious scholarly contribution.

Research groups should be organised around a research-focused and an operational part. The former concerns questions, methods and intellectual direction; the latter concerns people, budgets, compliance, project support, data stewardship and institutional logistics. It would therefore be commendable if different people within a research group would be considered for these different tasks. The head of research would be responsible for the research-focused part, while the head of operations would be responsible for the operational part. With a clear separation in place, two interrelated consequences would be:

1. Researchers would be able to focus on their own research, something which they *are* properly trained for and, as such, this set-up would arguably be more efficient than current situation;
2. As considerable public spending is involved in general research projects, it is important to ensure that the selected solution is efficient and transparent.

As a result, researchers would not have to become operations managers, something that often takes away a lot of time and attention and they did *not* receive proper training for. As budgets within research institutions are often tight, one could even imagine a set-up where the head of operations is shared between different groups or across an institute. Naturally, as a new role is defined, this could be a role that one specialises in during their tenure, but it should really be considered as such: a different specialisation, that requires a different set of skills and knowledge.

It should be enforced by law that public research institutes make their income and expenses publicly inspectable. As public research is publicly funded, serious financial interests, contribution claims and institutional

6. Fifth movement: Presto con fuoco

dependencies should be visible enough that failure to disclose them becomes practically and legally meaningful rather than a polite administrative courtesy. There are several fraud cases that could have been revealed much earlier if the conflict of interest were immediately disclosed, and even in these cases the conflict of interest itself was often a mere afterthought rather than the main reason for the fraud [GSM11; OGr24; OM23; The24]. It should always be clearly remembered that these institutes are *public* and, as such, hold a serious obligation towards society to generate objects that serve the general interest. Sponsorship need not automatically imply a direct mingling in the end-results, however they increase the odds for serious interference. Cases in the past where researchers accepted funding to steer the scientific debate around, e.g.: the active support around harmful treatments or products, the suppression of helpful treatments, climate change causes or effects, the effects of social media on psychological well-being, or the active support for increasing financial and socioeconomic inequality, are pointers to the fact that researchers are mere human beings that can be influenced by power or money [Bol25; Gol12; OC10; PM18; Zub19]. Rather than blindly trust institutions to self-correct when such processes take place, it is wiser and more efficient to be ahead of it.

Synthesis. Above proposals create an environment where more sources for funding will be made available to a researcher. Furthermore, the online environment where the research would be developed in, would actively stimulate collaboration and sharing of a much broader set of scientific objects. In this system, rather than all developing in prolific producers of publishable papers, researchers are allowed to specialise in different roles: Enthusiastic teachers, thorough reviewers, precise replicators, exceptional data-collectors, deep methodologists, gifted public speakers, creative out-of-the-box thinkers, and others. As research groups would have at least one dedicated employee focusing on operations and funding, the researcher can indeed focus on those aspects of science that the community and society needs most. By ensuring that any form of interference can be readily detected, society also ensures that the interests of the researchers stays aligned with them. As a result, it can be expected that the achieved results can be faithfully judged on their current reliability or truthfulness.

Circling back on the habits that Jaynes mentioned as bad scientific behaviours, the new system is expected to reduce these greatly:

1. **Gamesmanship:** would be counteracted by an incentive to increase linkage to the rest of the scientific web;
2. **Cutting corners:** would be placed under much heavier scrutiny by reviewers and replication studies;
3. **Hidden assumptions:** would be placed under much heavier scrutiny by reviewers and replication studies;
4. **Hyperfocus:** would be counteracted by an incentive to increase linkage to the rest of the scientific web;
5. **Theoretical navel-gazing:** would be discouraged in citizen- or democratically-directed projects.

By unlocking a wider view and a more society-driven approach among researchers, more society-focused results can be expected. It is likely that as a direct result the mutual trust between researchers and society will increase, to the benefit of all. Whatever one's definition of science, an activity that serves both the pursuit of truthfulness and society at large, is entitled to receive this qualification. As such, above system would rightfully deserve the name *science* once again.

Of course it cannot be expected that above system eradicates fraud, vanity or ignorance. However, by changing what the system rewards, it makes replication, correction, negative results, careful review, public access, transparent data practices, operational competence and public problem-selection part of the normal scientific economy. It also reduces the value of a single spectacular publication by increasing the value of the surrounding process that checks, links and corrects it. That is the institutional version of the formal demand made throughout this thesis: if scientific bad habits are the enemy of good inference, then scientific institutions should not be stimulated to develop them.

Acknowledgements

“Teleutias truly sailed home under the happiest of circumstances. When he went down to the sea to begin his journey home, there was not a soldier who did not clasp his hand. One placed a wreath on his head, another put a fillet around him, and those who came too late, when he was already putting out to sea, still threw wreaths into the water and wished him every good thing.

I realize that I am not describing any great expenditure, any danger, or any strategy worth mentioning; but by Zeus, I do think it is important for a man to consider what kind of conduct enabled Teleutias to evoke such a response from his subordinates, for that is, beyond any amount of money or danger, a man’s most noteworthy achievement”

Xenophon in *Hellenica*

Appendices

APPENDIX A

Definitions

“So if you are an expert and, as I said just now, have deceived me with your promise to give a demonstration about Homer, you are playing a trick on me. But if you are not an expert, but are possessed by Homer through a divine gift and, without knowledge, say many beautiful things about the poet, as I claimed is the case with you, then you are not playing a trick on me. So you need only choose how you wish us to regard you: as a man who plays tricks, or as a man governed by the gods”

Socrates in *Ion*

A.1 Type-theoretic definitions

This section presents the judgemental syntax and rules of the fragment of Martin-Löf-style type theory used in the thesis; it does not give a model semantics for Martin-Löf-style type theory. The definitions fix the fragment used in the main text. They are intentionally finite and judgemental. The type-theoretic notation follows the Martin-Löf tradition and the presentations used by Nordström–Petersson–Smith and Gratzer [Gra23; Mar84; NPS90].

Judgements, contexts and equality

Definition A.1.1 (Type-theoretic judgements). The basic judgemental forms used here are

$$\Gamma \text{ ctx}, \quad \Gamma \vdash A \text{ type}, \quad \Gamma \vdash a : A, \quad \Gamma \vdash \delta : \Delta,$$

together with definitional equality judgements

$$\Gamma \vdash A \equiv B \text{ type}, \quad \Gamma \vdash a \equiv b : A, \quad \Gamma \vdash \delta \equiv \epsilon : \Delta.$$

The judgements in the first line are read as follows: Γ is a well-formed context; A is a type in context Γ , that is, a domain of possible constructions available in that context; a is a term of type A in context Γ ; and δ is a substitution from Γ to Δ . Under the later reading of propositions as types, such a term a is also evidence for the proposition represented by A .

The judgements in the second line state definitional equality: A and B are definitionally equal types in context Γ ; a and b are definitionally equal terms of type A in context Γ ; and δ and ϵ are definitionally equal substitutions from Γ to Δ . Here $\equiv$ denotes judgemental equality by computation or unfolding, not a term of an identity type. The symbol Γ is always a context; δ and ϵ are substitutions.

Definition A.1.2 (Context). A context is a finite list of typed variable declarations

$$\Gamma := x_1 : A_1, x_2 : A_2(x_1), \dots, x_n : A_n(x_1, \dots, x_{n-1}),$$

in which each type A_{i+1} may depend on the earlier variables $x_1, \dots, x_i$. Such a dependently-typed sequence is called a *telescope* in the de Bruijn–Jacobs tradition. Contexts are generated from the empty context by context extension:

$$\frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type}}{\Gamma, x : A \text{ ctx}}.$$

Later declarations may depend on earlier declarations; earlier declarations may not depend on later ones.

Remark A.1.3 (Structural use of contexts). In the fragment used here, weakening, exchange and forgetting are admissible only when the dependency order permits them. Thus a fresh declaration $x : A$ may be added to a context when $\Gamma \vdash A \text{ type}$; two declarations may be exchanged only if neither dependency is broken; and a declaration $x : A$ may be forgotten from $\Gamma, x : A, \Delta$ only when neither Δ nor the target judgement contains a free occurrence of x .

A. Definitions

Definition A.1.4 (Substitution). A context written $\Gamma, x : A, \Delta$ is one concatenated context with three consecutive parts:

$$\underbrace{\Gamma}_{\text{initial context}}, \quad \underbrace{x : A}_{\text{declaration to be replaced}}, \quad \underbrace{\Delta}_{\text{later context suffix}}$$

The prefix Γ contains the declarations available before x is introduced. The suffix Δ is a possibly dependent sequence of later declarations: its types may refer to variables from Γ , to x , and to preceding declarations within Δ .

Suppose that, using only the declarations in the initial context Γ , one can construct a term

$$\Gamma \vdash a : A.$$

Suppose also that an arbitrary judgement $\mathcal{J}$ is derivable in the complete concatenated context:

$$\Gamma, x : A, \Delta \vdash \mathcal{J}.$$

Substitution replaces x by the already available term a :

$$\frac{\Gamma \vdash a : A \quad \Gamma, x : A, \Delta \vdash \mathcal{J}}{\Gamma, \Delta[a/x] \vdash \mathcal{J}[a/x]}.$$

The term a is therefore derived from the part of the context preceding x ; it cannot depend on x or on any declaration in Δ . Every free occurrence of x is replaced by a , both in the later declarations Δ and in the judgement $\mathcal{J}$. The declaration $x : A$ disappears because its role has been taken over by the already constructed term a . No declaration $a : A$ is added: a is a term available under Γ , not a new variable.

For example, let $\Gamma = (z : A)$, let $B(x)$ be a type in context $z : A, x : A$, and consider

$$z : A, x : A, y : B(x) \vdash y : B(x).$$

Since

$$z : A \vdash z : A,$$

substituting z for x transforms this judgement into

$$z : A, y : B(z) \vdash y : B(z).$$

The notation above presents substitution metatheoretically as the operation $[a/x]$ on expressions.

In an explicit-substitution presentation, as used by Gratzer, substitutions are instead included as a separate class of syntactic objects, with judgements

$$\Gamma \vdash \delta : \Delta.$$

Here Γ and Δ are complete contexts. A substitution δ supplies, using terms available in Γ , a term for every variable declared by Δ , respecting the dependencies among those declarations. It consequently sends a judgement over Δ ,

$$\Delta \vdash b : B,$$

to a judgement over Γ :

$$\Gamma \vdash b[\delta] : B[\delta].$$

For example, if

$$\Delta = (x : A, y : B(x)),$$

then a substitution $\Gamma \vdash \delta : \Delta$ may be written $\delta = (a, b)$, where

$$\Gamma \vdash a : A, \quad \Gamma \vdash b : B(a).$$

Applying δ replaces x by a and y by b simultaneously.

Explicit substitutions also have identity and composition operations, together with rules describing their effect on contexts, types and terms. This thesis uses only their ordinary action on types and terms and does not present the full explicit-substitution calculus. In the modal fragment introduced later, substitutions are restricted to one mode at a time; see Definition A.2.3.

Definition A.1.5 (Definitional equality). Definitional equality, also called *judgemental equality*, is written $\equiv$. Its principal forms are

$$\Gamma \vdash A \equiv B \text{ type}, \quad \Gamma \vdash a \equiv b : A, \quad \Gamma \vdash \delta \equiv \epsilon : \Delta.$$

These judgements say that the type theory itself treats the displayed types, terms or substitutions as interchangeable. They do not require an additional term witnessing the equality.

Definitional equality is generated by the computation and unfolding rules of the theory, together with equations declared as judgemental computation rules. It is closed under reflexivity, symmetry and transitivity, and is preserved by substitution and by the type and term constructors. A judgemental equation is fixed in advance as part of the theory's computation rules; it cannot be introduced as an arbitrary assumption.

Examples of such computation rules include beta-reduction

$$(\lambda x. b)(a) \equiv b[a/x]$$

from Definition A.1.9, and the projection equation

$$\text{pr}_1(a, b) \equiv a$$

from Definition A.1.10. If c is introduced as an abbreviation for t , unfolding the definition of c yields t , and hence the theory judges $c \equiv t$.

Definitional equality supports conversion. If

$$\Gamma \vdash a : A \quad \text{and} \quad \Gamma \vdash A \equiv B \text{ type},$$

then the same term may be used at the definitionally equal type:

$$\Gamma \vdash a : B.$$

Definitional equality therefore does not require expressions to be written identically: expressions are also definitionally equal when the theory's computation or unfolding rules identify them.

Definitional equality is part of the judgemental structure of the type theory, not a proposition represented by a type. Equality represented within the theory is instead expressed by the identity type $\text{Id}_A(a, b)$ introduced in Definition A.1.13. The relationship between these two notions of equality is explained in Remark A.1.14.

Universes and rule schemas

Definition A.1.6 (Universe). A universe $\mathcal{U}_i$ is a type whose terms are codes for the *small types* at level i . Here, *small* is a classification of type-theoretic size; it does not mean that the type has finitely many inhabitants.

The correspondence between codes and small types is given by two operations. First, a term of the universe can be decoded into the type that it represents:

$$\Gamma \vdash M : \mathcal{U}_i \Rightarrow \Gamma \vdash \text{El}(M) \text{ type}_i.$$

Thus M is a term serving as a code, whereas $\text{El}(M)$ is the corresponding type.

Conversely, a small type can be encoded as a term of the universe:

$$\Gamma \vdash A \text{ type}_i \Rightarrow \Gamma \vdash \text{Code}(A) : \mathcal{U}_i.$$

The two operations are inverse by definitional equality:

$$\text{El}(\text{Code}(A)) \equiv A, \quad \text{Code}(\text{El}(M)) \equiv M.$$

Consequently, encoding a small type and then decoding its code computes back to the original type, while decoding and then re-encoding a universe term computes back to the original code. These are judgemental computation rules and therefore require no inhabitant of an identity type.

The restriction to small types is essential: allowing the universe to contain codes for absolutely every type, including itself, would admit Girard's paradox and thereby undermine the consistency of the theory [Gra+21]. The universe itself is therefore kept outside the class of small types that it codes, or assigned a higher size in a stratified presentation. The index i records the chosen universe level. No principle of cumulativity between different universe levels is required in this thesis.

This is the Coquand-style universe presentation used by Gratzer: types and terms of the universe are connected by the explicit operations `Code` and `El` rather than being identified silently.

Definition A.1.7 (Formation, introduction, elimination and computation). The rules for a type former are organised around four questions:

A.1.7.1. *Formation*: under which assumptions is the type well formed?

A.1.7.2. *Introduction*: how can an inhabitant of the type be constructed?

A.1.7.3. *Elimination*: how can an inhabitant of the type be used?

A.1.7.4. *Computation*: how does an elimination applied to an introduced inhabitant simplify by definitional equality?

For example, the dependent function type introduced in Definition A.1.9 answers these questions as follows:

A.1.7.1. *Formation*: if A is a type in context Γ and $B(x)$ is a type in the extended context $\Gamma, x : A$, then $\Pi_{x:A} B(x)$ is a type in context Γ .

A.1.7.2. *Introduction*: from a term $b(x) : B(x)$ in context $\Gamma, x : A$, one constructs the dependent function $\lambda x. b(x) : \Pi_{x:A} B(x)$.

A.1.7.3. *Elimination*: given a dependent function $f : \Pi_{x:A} B(x)$ and an argument $a : A$, one obtains $f(a) : B(a)$.

A.1.7.4. *Computation*: applying an introduced dependent function to an argument reduces by definitional equality:

$$(\lambda x. b(x))(a) \equiv b(a).$$

Remark A.1.8. The four-part organisation in Definition A.1.7 belongs to the metalanguage used to present the type theory. It is a recurring template for describing the concrete rules of the individual type formers, not an additional type or collection of rule objects internal to Martin-Löf-style type theory. Proofs in this thesis may therefore apply the particular formation, introduction, elimination and computation rules stated for a type former, but they do not quantify over an internal type of all such rules.

Fundamental type formers

Definition A.1.9 (Dependent function type). A dependent type over A in context Γ is a family specified by a judgement $\Gamma, x : A \vdash B(x)$ type; it assigns a type $B(a)$ to each term $\Gamma \vdash a : A$ by substitution.

Formation. Given $\Gamma \vdash A$ type and $\Gamma, x : A \vdash B(x)$ type, the dependent function type is formed by

$$\Gamma \vdash \Pi_{x:A} B(x) \text{ type.}$$

Introduction. A term $b(x) : B(x)$ depending on $x : A$ determines a dependent function:

$$\frac{\Gamma, x : A \vdash b(x) : B(x)}{\Gamma \vdash \lambda x. b(x) : \Pi_{x:A} B(x)}.$$

Elimination. A dependent function is used by applying it to an argument:

$$\frac{\Gamma \vdash f : \Pi_{x:A} B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}.$$

Computation. Applying an introduced function reduces by beta-reduction:

$$(\lambda x. b(x))(a) \equiv b(a).$$

Here, beta-reduction is the conventional name for the lambda-calculus computation that replaces the occurrences of the parameter x in the body b by the argument a ; in Martin-Löf-style type theory, it is stipulated as the judgemental computation rule connecting lambda abstraction and application.

When B does not depend on x , this is the ordinary non-dependent function type $\Gamma \vdash A \rightarrow B$ type, with the same introduction, elimination and computation rules and the usual beta-reduction $(\lambda x. b)(a) \equiv b[a/x]$.

Definition A.1.10 (Dependent pair type). A dependent pair packages a term $a : A$ together with a term $b : B(a)$, whose type may depend on the first component. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma, x : A \vdash B(x)$ type, then

$$\Gamma \vdash \Sigma_{x:A} B(x) \text{ type.}$$

A. Definitions

Introduction. Its canonical inhabitants are pairs:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B(a)}{\Gamma \vdash (a, b) : \Sigma_{x:A} B(x)}.$$

Elimination. Suppose

$$\Gamma, z : \Sigma_{x:A} B(x) \vdash C(z) \text{ type}$$

and

$$\Gamma, x : A, y : B(x) \vdash d(x, y) : C((x, y)).$$

For $\Gamma \vdash p : \Sigma_{x:A} B(x)$, dependent pair elimination gives

$$\Gamma \vdash \text{split}_C(d, p) : C(p).$$

In particular, the projections produce $\text{pr}_1(p) : A$ and $\text{pr}_2(p) : B(\text{pr}_1(p))$.

Computation. Elimination of an introduced pair reduces to its defining case:

$$\text{split}_C(d, (a, b)) \equiv d(a, b).$$

The projections consequently satisfy

$$\text{pr}_1(a, b) \equiv a, \quad \text{pr}_2(a, b) \equiv b.$$

When B does not depend on x , this is the ordinary non-dependent product type $\Gamma \vdash A \times B$ type, with pair inhabitants (a, b) and the same projections.

Remark A.1.11 (Finite records and named projections). A finite record type is an iterated dependent-pair type with named fields. For example, suppose a record type **Record** has fields

$$a : A, \quad b : B(a), \quad c : C(a, b).$$

If $r : \text{Record}$, its named field projections are

$$a(r) : A, \quad b(r) : B(a(r)), \quad c(r) : C(a(r), b(r)).$$

These are the corresponding dependent-pair projections with readable names; they add no new type former.

Definition A.1.12 (Sum type). A sum type records a choice between a term of A and a term of B , together with which of the two alternatives was chosen. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma \vdash B$ type, then

$$\Gamma \vdash A + B \text{ type.}$$

Introduction. The two injections construct inhabitants:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B}, \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}.$$

Elimination. Suppose

$$\Gamma, z : A + B \vdash C(z) \text{ type,}$$

together with a left case $\Gamma, x : A \vdash \ell(x) : C(\text{inl}(x))$ and a right case $\Gamma, y : B \vdash r(y) : C(\text{inr}(y))$. For $\Gamma \vdash s : A + B$, case analysis gives

$$\Gamma \vdash \text{case}_C(\ell, r, s) : C(s).$$

Computation. Case analysis reduces according to the injection used:

$$\text{case}_C(\ell, r, \text{inl}(a)) \equiv \ell(a), \quad \text{case}_C(\ell, r, \text{inr}(b)) \equiv r(b).$$

Under the propositions-as-types interpretation of Definition A.1.26, $A + B$ represents logical disjunction. Both A and B may be inhabited, although each canonical inhabitant of $A + B$ is introduced through either inl or inr .

Definition A.1.13 (Identity type). The identity type $\text{Id}_A(a, b)$ represents equality between two terms $a, b : A$; its inhabitants are proofs of that equality. Its rules are as follows:

Formation. If $\Gamma \vdash A$ type and $\Gamma \vdash a, b : A$, then

$$\Gamma \vdash \text{Id}_A(a, b) \text{ type}.$$

Introduction. Its canonical introduction is reflexivity:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a)}.$$

Elimination. Its eliminator is identity induction, or J . If

$$\Gamma, x : A, y : A, p : \text{Id}_A(x, y) \vdash C(x, y, p) \text{ type}$$

and

$$\Gamma, x : A \vdash d(x) : C(x, x, \text{refl}_x),$$

then

$$\Gamma, a, b : A, p : \text{Id}_A(a, b) \vdash J_C(d, a, b, p) : C(a, b, p),$$

Computation. Identity elimination reduces on reflexivity:

$$J_C(d, a, a, \text{refl}_a) \equiv d(a).$$

Thus it suffices to prove a property of equality proofs in the reflexivity case.

Remark A.1.14 (Definitional equality and identity). Definitional equality and identity are related, but they are not the same judgement. Suppose

$$\Gamma \vdash a \equiv b : A.$$

Reflexivity gives

$$\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a).$$

Because $a \equiv b$, the types $\text{Id}_A(a, a)$ and $\text{Id}_A(a, b)$ are definitionally equal. Conversion therefore allows the same term to be used as

$$\Gamma \vdash \text{refl}_a : \text{Id}_A(a, b).$$

Thus definitional equality always gives an inhabitant of the corresponding identity type.

A. Definitions

The converse does not hold in general. In the intensional Martin-Löf-style type theory used in this thesis, a term

$$\Gamma \vdash p : \text{Id}_A(a, b)$$

does not necessarily imply

$$\Gamma \vdash a \equiv b : A.$$

Adding such an implication would amount to the equality-reflection rule

$$\frac{\Gamma \vdash p : \text{Id}_A(a, b)}{\Gamma \vdash a \equiv b : A},$$

which belongs to an extensional presentation and is not assumed in this thesis.

Definitional equality is established by the computation and unfolding rules of the theory. An identity term is instead an object within the theory and may be used through the identity eliminator J . Consequently, two terms may be propositionally identical even when computation does not make them definitionally equal.

Inductive and finite types

Definition A.1.15 (Inductive type and indexed family). The following definitions require types whose terms are generated by explicit constructors and whose inhabitants can be processed by structural recursion or induction. Rather than repeat this common pattern separately for each example, the present definition introduces inductive types and indexed inductive families in general. Concrete instances are worked out in Definitions A.1.16, A.1.17 and A.1.23: natural numbers represent counts and sizes; finite index types provide the canonical positions used for the urn cells of Definition A.3.7 and for indexing the finite families of variables in Definition A.3.15; and lists represent the finite sequences used in the finite-support probability records of Definition A.3.9.

An inductive definition specifies a type by giving canonical ways to construct its terms and a rule for defining or proving something about every term so constructed. Constructors may use previously constructed terms recursively. The eliminator expresses that it is sufficient to handle the constructor cases, and the computation rules state how the resulting definition evaluates on each constructor.

An *ordinary inductive type* I is a single type. Given an already available indexing type

$$\Gamma \vdash K \text{ type},$$

an *indexed inductive family*

$$\Gamma, i : K \vdash I(i) \text{ type}$$

assigns a type to each index $i : K$. A constructor of an indexed family may determine the particular index at which its result lies. An index is therefore not necessarily a natural number: it is a term of the indexing type K specified by the definition.

Formation. The formation rule declares when the type or family is well formed:

$$\Gamma \vdash I \text{ type} \quad \text{or} \quad \Gamma, i : K \vdash I(i) \text{ type}.$$

Parameters shared by the entire definition are declarations already present in the surrounding context Γ ; they remain fixed while the indices and inhabitants vary.

Introduction. The constructors give the canonical ways to build terms. A constructor of an ordinary inductive type has result type I ; a constructor of an indexed family has result type $I(i)$, where the result index $i : K$ is determined by that constructor and its arguments. Constructor arguments may themselves contain previously constructed terms of the type or family being defined.

Recursive occurrences are permitted only in *strictly positive* positions. A direct recursive constructor argument $r : I$ is permitted. A constructor argument that is itself a function consuming I , as in a hypothetical constructor $\text{bad} : (I \rightarrow B) \rightarrow I$, is not. This restriction ensures that the constructors support well-founded recursion and induction rather than unrestricted self-reference.

Elimination. For an ordinary inductive type, first choose a family

$$\Gamma, x : I \vdash P(x) \text{ type.}$$

To define a dependent function

$$\Gamma \vdash f : \Pi_{x:I} P(x),$$

it is sufficient to give one term of P for each constructor case. Whenever a constructor has a recursive argument, its case also receives the corresponding induction hypothesis.

For an indexed family, the property may depend on both the index and the inhabitant:

$$\Gamma, i : K, x : I(i) \vdash P(i, x) \text{ type.}$$

Each constructor case must establish P at the index selected by that constructor, again receiving induction hypotheses for its recursive arguments.

Computation. Applying the eliminator to a constructor reduces definitionally to the case supplied for that constructor. In a recursive case, the induction hypotheses compute by applying the same eliminator to the recursive arguments. Thus the eliminator computes constructor by constructor.

Definition A.1.16 (Natural-number type). The natural-number type is the ordinary inductive type generated by zero and successor. It supplies the counts and sizes used by the finite types and probability constructions below. Its rules are as follows:

Formation. In every context Γ ,

$$\Gamma \vdash \mathbb{N} \text{ type.}$$

Introduction. Zero is a natural number, and the successor of a natural number is again a natural number:

$$\Gamma \vdash 0 : \mathbb{N}, \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}.$$

Elimination. Given a family

$$\Gamma, n : \mathbb{N} \vdash P(n) \text{ type,}$$

A. Definitions

a zero case

$$\Gamma \vdash p_0 : P(0),$$

and a successor case

$$\Gamma, n : \mathbb{N}, q : P(n) \vdash p_s(n, q) : P(\text{succ}(n)),$$

natural-number induction gives

$$\Gamma, n : \mathbb{N} \vdash \text{natElim}_P(p_0, p_s, n) : P(n).$$

Computation. Natural-number induction reduces on zero and successor:

$$\text{natElim}_P(p_0, p_s, 0) \equiv p_0,$$

and

$$\text{natElim}_P(p_0, p_s, \text{succ}(n)) \equiv p_s(n, \text{natElim}_P(p_0, p_s, n)).$$

The numerals $1, 2, \dots$ abbreviate iterated successors of 0. Addition, multiplication and the other primitive-recursive operations on natural numbers are defined using this eliminator.

Definition A.1.17 (Finite index type). The family Fin is indexed by the natural-number type of Definition A.1.16. It assigns to each natural number n a finite type whose canonical inhabitants are the n positions $0, \dots, n - 1$. Its rules are as follows:

Formation. A natural number indexes a finite type:

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Fin}(n) \text{ type}}.$$

Introduction. The constructors are

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{fzero}_n : \text{Fin}(\text{succ}(n))}, \quad \frac{\Gamma \vdash i : \text{Fin}(n)}{\Gamma \vdash \text{fsucc}(i) : \text{Fin}(\text{succ}(n))}.$$

For a numeral n , their iterated canonical forms are conventionally written $0, \dots, n - 1$; in particular, $\text{Fin}(0)$ has no canonical inhabitants.

Elimination. Given a family

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n) \vdash P(n, i) \text{ type},$$

a zero case

$$\Gamma, n : \mathbb{N} \vdash p_0(n) : P(\text{succ}(n), \text{fzero}_n),$$

and a successor case

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n), q : P(n, i) \vdash p_s(n, i, q) : P(\text{succ}(n), \text{fsucc}(i)),$$

finite induction gives

$$\Gamma, n : \mathbb{N}, i : \text{Fin}(n) \vdash \text{finElim}_P(p_0, p_s, n, i) : P(n, i).$$

Computation. Finite induction reduces according to the constructor:

$$\text{finElim}_P(p_0, p_s, \text{succ}(n), \text{fzero}_n) \equiv p_0(n),$$

and

$$\text{finElim}_P(p_0, p_s, \text{succ}(n), \text{fsucc}(i)) \equiv p_s(n, i, \text{finElim}_P(p_0, p_s, n, i)).$$

Remark A.1.18 (Canonical rank and order on finite indices). The natural numbers carry the strict-order family $m <_{\mathbb{N}} n$, generated by

$$\text{zeroLess}_n : 0 <_{\mathbb{N}} \text{succ}(n)$$

and

$$\text{succLess}_{m,n} : m <_{\mathbb{N}} n \rightarrow \text{succ}(m) <_{\mathbb{N}} \text{succ}(n),$$

with no other constructors. Finite elimination defines the canonical rank map

$$\text{rank}_n : \text{Fin}(n) \rightarrow \mathbb{N}$$

by the computation rules

$$\begin{aligned} \text{rank}_{\text{succ}(n)}(\text{fzero}_n) &\equiv 0, \\ \text{rank}_{\text{succ}(n)}(\text{fsucc}(i)) &\equiv \text{succ}(\text{rank}_n(i)). \end{aligned}$$

Consequently, the canonical strict order on a finite index type is

$$i <_{\text{Fin}(n)} j \quad := \quad \text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j).$$

In Lean’s structure-based presentation of **Fin**, the projection **i.val** plays the role of this rank map.

Definition A.1.19 (Finite type). Let A be a small type. A *finiteness witness* is explicit data demonstrating that A is finite. It consists of a natural number

$$n : \mathbb{N},$$

two maps

$$\text{encode} : A \rightarrow \text{Fin}(n), \quad \text{decode} : \text{Fin}(n) \rightarrow A,$$

and the round-trip identity proofs

$$\eta : \prod_{a:A} \text{Id}_A(\text{decode}(\text{encode}(a)), a)$$

and

$$\varepsilon : \prod_{i:\text{Fin}(n)} \text{Id}_{\text{Fin}(n)}(\text{encode}(\text{decode}(i)), i).$$

The first identity says that encoding and then decoding a term of A returns the original term. The second says that decoding and then encoding a finite index returns the original index. A type A is called finite when one has specified this natural number, these two maps and these two round-trip proofs. Together, they match the inhabitants of A exactly with the n canonical positions of $\text{Fin}(n)$.

The type of these finiteness witnesses is written $\text{Finite}(A)$. Thus a term $h : \text{Finite}(A)$ carries the natural number, the two maps and both round-trip proofs displayed above.

Definition A.1.20 (Boolean type). The Boolean type is the two-valued inductive type whose canonical inhabitants are **true** and **false**. Its rules are as follows:

Formation. In every context Γ ,

$$\Gamma \vdash \text{Bool type}.$$

Introduction. It has two constructors:

$$\Gamma \vdash \text{true} : \text{Bool}, \quad \Gamma \vdash \text{false} : \text{Bool}.$$

A. Definitions

Elimination. Given a family $\Gamma, b : \text{Bool} \vdash P(b)$ type and terms $\Gamma \vdash p_{\text{true}} : P(\text{true})$ and $\Gamma \vdash p_{\text{false}} : P(\text{false})$, Boolean case analysis defines

$$\Gamma, b : \text{Bool} \vdash \text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, b) : P(b).$$

Computation. Boolean elimination reduces according to the constructor:

$$\text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, \text{true}) \equiv p_{\text{true}},$$

and

$$\text{boolElim}_P(p_{\text{true}}, p_{\text{false}}, \text{false}) \equiv p_{\text{false}}.$$

Boolean conjunction is the derived operation

$$\text{and}_{\text{Bool}} : \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool},$$

defined by case analysis on its first input:

$$\text{and}_{\text{Bool}}(b, c) := \text{boolElim}_{\lambda_.\text{Bool}}(c, \text{false}, b).$$

Consequently,

$$\text{and}_{\text{Bool}}(\text{true}, c) \equiv c, \quad \text{and}_{\text{Bool}}(\text{false}, c) \equiv \text{false};$$

it returns **true** exactly when both inputs are **true**.

Boolean negation and disjunction are derived in the same way:

$$\text{not}_{\text{Bool}}(b) := \text{boolElim}_{\lambda_.\text{Bool}}(\text{false}, \text{true}, b),$$

$$\text{or}_{\text{Bool}}(b, c) := \text{boolElim}_{\lambda_.\text{Bool}}(\text{true}, c, b).$$

Thus not_{Bool} exchanges **true** and **false**, while $\text{or}_{\text{Bool}}(b, c)$ returns **true** exactly when at least one of b and c is **true**.

Remark A.1.21 (Examples of finite types). The finite index types and the Boolean type give two basic examples of finite types in the sense of Definition A.1.19:

- Every finite index type $\text{Fin}(n)$ from Definition A.1.17 is trivially finite: take both maps in Definition A.1.19 to be the identity and both round-trip proofs to be reflexivity.
- The Boolean type of Definition A.1.20 is finite with $n = 2$: encode **false** as 0 and **true** as 1, and decode those two indices back to the corresponding Boolean terms. The two round-trip identities follow by case analysis.

Remark A.1.22 (Closure of finite types). Let A and B be finite types, and let $C : A \rightarrow \mathcal{U}$ be a type family for which every $C(a)$ is finite. Then:

- the sum type $A + B$ of Definition A.1.12 is finite;
- the dependent pair type

$$\Sigma_{a:A} C(a)$$

of Definition A.1.10 is finite. In particular, the ordinary product $A \times B$, obtained by taking the constant family $C(a) := B$, is finite;

- if the inhabitants of A are enumerated as $a_0, \dots, a_{n-1}$, then the finite telescope

$$C(a_0) \times \dots \times C(a_{n-1})$$

is finite. With pointwise or function-extensional equality, the same coordinate data may be represented by the dependent function type

$$\prod_{a:A} C(a)$$

of Definition A.1.9.

The required encodings, decodings and round-trip proofs are constructed in Proposition B.5.1.

Definition A.1.23 (List type). The list type is an ordinary inductive type whose terms are finite sequences. It is the list construction used in the weighted finite-support representation of Definition A.3.9. Its rules are as follows:

Formation. If A is a type, then finite lists over A form a type:

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash \text{List}(A) \text{ type}}.$$

Introduction. The empty list and the operation that places an element in front of an existing list are the constructors:

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash \text{nil} : \text{List}(A)}, \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash xs : \text{List}(A)}{\Gamma \vdash \text{cons}(a, xs) : \text{List}(A)}.$$

Elimination. Given a family

$$\Gamma, xs : \text{List}(A) \vdash P(xs) \text{ type},$$

an empty-list case

$$\Gamma \vdash p_{\text{nil}} : P(\text{nil}),$$

and a cons case

$$\Gamma, a : A, xs : \text{List}(A), q : P(xs) \vdash p_{\text{cons}}(a, xs, q) : P(\text{cons}(a, xs)),$$

list induction gives

$$\Gamma, xs : \text{List}(A) \vdash \text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, xs) : P(xs).$$

Computation. List induction reduces according to the constructor:

$$\text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, \text{nil}) \equiv p_{\text{nil}},$$

and

$$\text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, \text{cons}(a, xs)) \equiv p_{\text{cons}}(a, xs, \text{listElim}_P(p_{\text{nil}}, p_{\text{cons}}, xs)).$$

Remark A.1.24 (Optional list endpoints). The operations below must distinguish an empty list, which has no endpoint, from a nonempty list, which supplies one. For any type A , define the optional-value type

$$\text{Option}(A) := \text{Fin}(1) + A.$$

A. Definitions

An inhabitant of $\text{Option}(A)$ therefore represents either the absence of a value of A or one supplied value $a : A$. The corresponding terms are

$$\text{none} := \text{inl}(0), \quad \text{some}(a) := \text{inr}(a).$$

List elimination defines operations returning the first and last entries when they exist:

$$\text{head?}, \text{last?} : \text{List}(A) \rightarrow \text{Option}(A),$$

with computation rules

$$\begin{aligned} \text{head?}(\text{nil}) &\equiv \text{none}, \\ \text{head?}(\text{cons}(a, p)) &\equiv \text{some}(a), \\ \text{last?}(\text{nil}) &\equiv \text{none}, \\ \text{last?}(\text{cons}(a, \text{nil})) &\equiv \text{some}(a), \\ \text{last?}(\text{cons}(a, \text{cons}(b, p))) &\equiv \text{last?}(\text{cons}(b, p)). \end{aligned}$$

Thus an identity $\text{head?}(p) = \text{some}(X)$ or $\text{last?}(p) = \text{some}(Y)$ also supplies evidence that p is nonempty.

Definition A.1.25 (Empty type). The empty type is a type with no constructors and hence no canonical inhabitants. In this thesis, it is obtained as the zero-index case of the finite family. Its rules are as follows:

Formation. Define

$$\perp := \text{Fin}(0).$$

Consequently,

$$\Gamma \vdash \perp \text{ type}.$$

Unfolding the definition gives $\perp \equiv \text{Fin}(0)$ judgementally. By the rules of Definition A.1.17, this is the inductive type with no constructors.

Introduction. There are no introduction rules and hence no canonical inhabitants of $\perp$.

Elimination. Given a family

$$\Gamma, x : \perp \vdash P(x) \text{ type}$$

and $\Gamma \vdash e : \perp$, elimination requires no constructor cases and yields

$$\Gamma \vdash \text{absurd}_P(e) : P(e).$$

For a family that does not depend on x , this gives the non-dependent function $\text{absurd}_P : \perp \rightarrow P$.

Computation. There are no constructor computation equations to state, because there is no introduced inhabitant on which absurd_P could compute.

Constructive logic

Definition A.1.26 (Propositions as types). The propositions-as-types interpretation is the type-theoretic instance of the Curry–Howard correspondence: propositions are represented by types, and proofs are represented by terms of those types [Mar84; NPS90]. A context Γ may consequently be read as a

sequence of assumptions: a declaration $x : A$ records that x is available as evidence for A . The judgement

$$\Gamma \vdash a : A$$

then says that, under the assumptions in Γ , the term a supplies constructive evidence for A . Equivalently,

$$A \text{ is proved in } \Gamma \iff \Gamma \vdash a : A \text{ for some term } a.$$

Because proofs are represented by terms, they may carry computational content. For example, a term $f : A \rightarrow B$ transforms any proof $a : A$ into a proof $f(a) : B$. The same reading extends to the remaining logical connectives through their corresponding type formers.

Remark A.1.27. The propositions-as-types reading is used informally throughout the thesis but is not invoked as a rule inside the formal transfer of Theorem B.13.3; it serves as a background interpretation of the constructive judgemental discipline.

Definition A.1.28 (Logical connectives under propositions-as-types). Under Definition A.1.26, the standard logical connectives correspond to type formers:

$$\begin{aligned} A \wedge B &:= A \times B, & A \vee B &:= A + B, \\ A \Rightarrow B &:= A \rightarrow B, & \neg A &:= A \rightarrow \perp, \\ \top &:= \text{Fin}(1). \end{aligned}$$

The constructive content of a logical statement is then a term of the corresponding type. In particular, $A \wedge B$ requires both a witness of A and a witness of B ; $A \vee B$ requires a witness of one side together with evidence of which side; and $\neg A$ requires a procedure converting any witness of A into an inhabitant of the empty type.

Remark A.1.29 (Constructive negation and double negation). Since $\neg A := A \rightarrow \perp$, a term $n : \neg A$ may be used as a function $n : A \rightarrow \perp$. Given also a term $a : A$, applying n to a yields $n(a) : \perp$. Consequently, A and $\neg A$ cannot both be inhabited in a consistent theory: a pair $(a, n) : A \times \neg A$ would produce an inhabitant of the empty type. Equivalently, Martin-Löf-style type theory constructively proves $\neg(A \wedge \neg A)$.

The same application also gives a constructive proof of $A \rightarrow \neg\neg A$. Recall the function-introduction rule from Definition A.1.9:

$$\frac{\Gamma, p : P \vdash q : Q}{\Gamma \vdash \lambda p. q : P \rightarrow Q}.$$

The assumption $p : P$ is thereby removed from the surrounding context and retained as a bound function input; this does not assert that P is inhabited.

For double negation, work temporarily under the assumptions

$$a : A, \quad n : A \rightarrow \perp.$$

Under these assumptions, function application gives the body

$$n(a) : \perp.$$

A. Definitions

First bind n as the input of a function. The type of n becomes the input type, while the type of the body $n(a)$ remains the output type:

$$a : A \vdash \lambda n. n(a) : \underbrace{(A \rightarrow \perp)}_{\text{type of the input } n} \rightarrow \underbrace{\perp}_{\text{type of the body } n(a)}.$$

By the definition of negation,

$$(A \rightarrow \perp) \rightarrow \perp \equiv \neg\neg A.$$

Hence, under the remaining assumption $a : A$,

$$\lambda n. n(a) : \neg\neg A.$$

Finally, bind a as the input of an outer function. Its type A becomes the new input type, while the previously constructed function type remains the output type:

$$\lambda a. \lambda n. n(a) : \underbrace{A}_{\text{type of the input } a} \rightarrow \underbrace{((A \rightarrow \perp) \rightarrow \perp)}_{\text{type of the inner function}}.$$

Since the inner function type is $\neg\neg A$, this gives

$$\lambda a. \lambda n. n(a) : A \rightarrow \neg\neg A.$$

Thus both temporary assumptions become inputs of the constructed function. Constructing the function does not claim that either input has actually been supplied.

Decidability and classification

Definition A.1.30 (Decidable type). A type A is *decidable* when the sum type $A + \neg A$ is inhabited, using Definitions A.1.12 and A.1.28. A term

$$d : A + \neg A$$

is a *decision procedure* for A . Its left branch contains a witness $a : A$; its right branch contains a refutation $n : \neg A$. A negative result must therefore explain why any hypothetical witness of A would yield an inhabitant of the empty type; merely failing to find a witness does not decide A .

Remark A.1.31 (Decidability and double-negation elimination). The construction in Remark A.1.29 showed that $A \rightarrow \neg\neg A$ is constructively valid for every type A . The converse is not available for an arbitrary A , but it can be constructed when A is decidable in the sense of Definition A.1.30.

Fix a decision procedure

$$d : A + \neg A.$$

To construct a term of $\neg\neg A \rightarrow A$, temporarily assume an input

$$q : \neg\neg A \equiv (A \rightarrow \perp) \rightarrow \perp.$$

Under this assumption, construct a term of A by case analysis on d :

A.1.31.1. If d contains a witness $a : A$, return a .

A.1.31.2. If d contains a refutation $n : \neg A$, then $q(n) : \perp$. Empty-type elimination consequently produces a term of A .

Let $r_d(q) : A$ denote the term produced by this case analysis. Binding q as the input of the resulting function then gives

$$\lambda q. r_d(q) : \neg\neg A \rightarrow A.$$

The construction does not assert that $\neg\neg A$ is already inhabited; it describes how any supplied term $q : \neg\neg A$ is converted into a term of A .

This construction depends on the supplied decision procedure $d : A \rightarrow \mathcal{U}$. It does not provide double-negation elimination for arbitrary types. The law of excluded middle is the stronger uniform principle that supplies such a decision for every proposition A . Uniform double-negation elimination and the law of excluded middle are equivalent, and neither is assumed in this thesis; see Proposition B.3.1.

Definition A.1.32 (Decidable predicate). A predicate $P : X \rightarrow \mathcal{U}$ is *decidable* when it has a uniform decision procedure:

$$d_P : \prod_{x:X} (P(x) + \neg P(x)).$$

Given any $x : X$, the term $d_P(x)$ therefore returns either a witness of $P(x)$ or a refutation of $P(x)$.

Definition A.1.33 (Boolean classifier). A Boolean classifier is an equivalent Boolean-valued presentation of the decidability defined in Definition A.1.32. For a predicate $P : X \rightarrow \mathcal{U}$, it consists of a function $b : X \rightarrow \text{Bool}$ together with correctness evidence for both branches:

$$\begin{aligned} \text{trueCorrect}_b &: \prod_{x:X} ((b(x) = \text{true}) \rightarrow P(x)), \\ \text{falseCorrect}_b &: \prod_{x:X} ((b(x) = \text{false}) \rightarrow \neg P(x)). \end{aligned}$$

A Boolean classifier and a decision procedure express the same information in different forms. Given a classifier b and an input $x : X$, inspect $b(x)$. If it returns **true**, the first correctness map supplies a witness of $P(x)$; if it returns **false**, the second supplies a refutation of $P(x)$. This gives a term of $P(x) + \neg P(x)$. Conversely, given a decision procedure for $P(x)$, return **true** when its left branch contains a witness of $P(x)$, and return **false** when its right branch contains a refutation. The contents of these branches provide the required correctness evidence. Hence a predicate is decidable in the sense of Definition A.1.32 if and only if it has a Boolean classifier.

For example, take $X := \text{Fin}(3)$ and define the predicate $P(i) := \text{Id}(i, 0)$. It is decided by the classifier $b_P : \text{Fin}(3) \rightarrow \text{Bool}$ that returns **true** exactly at 0. When $i = 0$, the true branch supplies the witness $\text{refl}_0 : \text{Id}_{\text{Fin}(3)}(0, 0)$ using the reflexivity constructor of Definition A.1.13; when $i = 1$ or $i = 2$, the false branch supplies a proof that i is not equal to 0.

Definition A.1.34 (Indicator). The indicator numerically recodes the Boolean component of a classifier from Definition A.1.33: it sends **true** to 1 and **false** to 0 so that event membership can contribute to a finite sum. The construction itself applies to any Boolean-valued function.

A. Definitions

Data. Let X be a type and let $E : X \rightarrow \mathbf{Bool}$ be a Boolean-valued function.

Definition. Boolean elimination first defines the Boolean-to-number function

$$\mathbf{boolToNat} : \mathbf{Bool} \rightarrow \mathbb{N}$$

by

$$\mathbf{boolToNat}(b) := \mathbf{boolElim}_{\lambda b. \mathbb{N}}(1, 0, b),$$

which sends **true** to 1 and **false** to 0. The indicator of E is the composite

$$\mathbb{1}_E := \mathbf{boolToNat} \circ E : X \rightarrow \mathbb{N}.$$

Equivalently, for each $x : X$,

$$\mathbb{1}_E(x) := \mathbf{boolElim}_{\lambda b. \mathbb{N}}(1, 0, E(x)).$$

Computation. The computation rules for the Boolean eliminator give

$$\mathbb{1}_E(x) \equiv 1 \quad \text{when} \quad E(x) \equiv \mathbf{true}, \quad \mathbb{1}_E(x) \equiv 0 \quad \text{when} \quad E(x) \equiv \mathbf{false}.$$

Example. For the classifier b_P from the example in Definition A.1.33, the indicator has the values

$$\mathbb{1}_{b_P}(0) \equiv 1, \quad \mathbb{1}_{b_P}(1) \equiv 0, \quad \mathbb{1}_{b_P}(2) \equiv 0.$$

Purpose. In the probability constructions of Section A.3, summing $\mathbb{1}_E$ counts the cells at which E holds, while multiplying by $\mathbb{1}_E$ restricts a finite sum to those cells.

Definition A.1.35 (Decidable equality). A type A has *decidable equality* when its identity predicate from Definition A.1.13 is decidable in the sense of Definition A.1.32, that is, when there is a term:

$$\mathbf{decEq}_A : \prod_{a:A} \prod_{b:A} (\mathbf{Id}_A(a, b) + \neg \mathbf{Id}_A(a, b)).$$

Thus, for every $a, b : A$, the procedure determines whether they are identical: it returns either an identity $p : \mathbf{Id}_A(a, b)$ or a refutation $n : \neg \mathbf{Id}_A(a, b)$.

Remark A.1.36 (Finite types have decidable equality). For every finite type A in the sense of Definition A.1.19, its finiteness witness determines an explicit procedure for deciding equality between terms of A . To be precise, let $a, b : A$. Equality between them is decided as follows:

A.1.36.1. Encode both terms:

$$i := \mathbf{encode}(a), \quad j := \mathbf{encode}(b).$$

A.1.36.2. Use decidable equality on $\mathbf{Fin}(n)$ to compare i and j . This comparison is always available because the constructors of $\mathbf{Fin}(n)$ can be compared recursively.

A.1.36.3. If i and j are identical, decode their common index. The first round-trip proof says that decoding i returns a , while decoding j returns b . Because i and j are identical, these decoded terms are identical, and hence a and b are identical.

A.1.36.4. If i and j differ, then a and b must differ as well. Otherwise, because `encode` is a function, identical inputs a and b would have identical outputs i and j , contradicting their comparison.

The procedure therefore returns either an identity between a and b or a refutation of such an identity. Hence every finite type has decidable equality.

The equality procedure uses only the first round-trip proof η , whereas the second round-trip proof ε remains part of the finiteness witness because it ensures that every index represents an inhabitant of A .

A.2 Modal definitions

This section introduces the mode-theoretic indexing structure and modal syntax used in the thesis. The modal notation follows Gratzer’s multimodal dependent type theory: a mode theory is a small strict 2-category; objects are modes; morphisms are modalities; and 2-cells record transformations between modalities [Gra+21; Gra23; GSB19]. Its epistemic reading is thesis-specific; it is neither a complete semantics for Gratzer’s multimodal dependent type theory nor a reproduction of every rule of Gratzer’s full systems.

Modes, modalities and judgements

Definition A.2.1 (Mode theory). A mode theory organizes the modes at which judgements may be formed, the modalities relating those modes, and transformations between modalities. Formally, it is a small strict 2-category $\mathcal{M}$. Its objects $m, n, o \in \mathcal{M}$ are modes. A modality $\mu : n \rightarrow m$ is a morphism from mode n to mode m . The identity morphism $1_m : m \rightarrow m$ is the identity modality at m , while categorical composition combines successive modalities. A 2-cell $\alpha : \mu \Rightarrow \nu$ records a transformation between parallel modalities $\mu, \nu : n \rightarrow m$. The thesis uses modes, identity modalities and composition directly. No argument depends on a non-identity 2-cell, but the 2-cell structure is retained as part of the standard mode-theoretic framework of Gratzer and collaborators [Gra+21; Gra23].

Definition A.2.2 (Mode). A mode is an object m of a mode theory $\mathcal{M}$. In this thesis a mode is interpreted as an epistemic location: the index at which a judgement is formed. What variables, assumptions, questions and causal models are available is recorded by the context Γ at that mode, not by the mode object itself.

Definition A.2.3 (Modal judgement). For each mode m , judgements are indexed by that mode:

$$\Gamma \text{ ctx } @m, \quad \Gamma \vdash A \text{ type } @m, \quad \Gamma \vdash a : A @m, \quad \Gamma \vdash \delta : \Delta @m.$$

The annotation $@m$ belongs to the whole judgement, not to the type expression A alone. It records the mode at which the context, type, term or substitution is being considered. In the epistemic reading of this thesis, it therefore identifies the epistemic state from which the judgement is made.

Gratzer’s full multimodal type theory provides a richer modal-substitution discipline [Gra+21; Gra23]. This thesis uses substitutions within one mode at

A. Definitions

a time. Movement between modes is expressed by locked contexts and a modal type former, defined next.

Definition A.2.4 (Modality). Let n and m be modes of a mode theory $\mathcal{M}$. A modality $\mu : n \rightarrow m$ is a morphism of $\mathcal{M}$ with source n and target m . It will index locked contexts and the modal type former below. How the direction of μ appears in judgements is stated there.

Locks and modal types

Definition A.2.5 (Locked context). Let $\mu : n \rightarrow m$ be a modality in a mode theory $\mathcal{M}$. Appending the lock $\{\mu\}$ to a context at mode m changes the mode at which subsequent context entries are read:

$$\frac{\Gamma \text{ ctx @}m \quad \mu : n \rightarrow m}{\Gamma.\{\mu\} \text{ ctx @}n}.$$

The context is read from left to right. If a declaration $x : A$ is subsequently appended, its position relative to the lock determines the mode at which it is checked:

$$\underbrace{\Gamma}_{\text{original context at } m} \quad , \quad \underbrace{\{\mu\}}_{\text{changes the current mode from } m \text{ to } n} \quad , \quad \underbrace{x : A}_{\text{checked at } n}.$$

The lock therefore acts as a delimiter. The declarations already contained in Γ remain on its m -side, while declarations appended to its right are checked at n . In particular, placing $x : A$ to the left of the lock would make it a declaration at m ; placing it to the right makes it a declaration at n . The lock records this change of checking mode without altering the declarations already present in Γ .

Accordingly, once the lock has been appended, the context may be extended at its new current mode n using the ordinary variable rule:

$$\frac{\Gamma.\{\mu\} \text{ ctx @}n \quad \Gamma.\{\mu\} \vdash A \text{ type @}n}{\Gamma.\{\mu\}, x : A \text{ ctx @}n}.$$

Thus $x : A$ is an ordinary declaration at mode n . Its modal location is already recorded by its position after $\{\mu\}$, so no additional modal annotation on the declaration is required.

Locks compose in the same order as their modalities. Given $\mu : n \rightarrow m$ and $\nu : o \rightarrow n$, appending both locks changes the current mode first to n and then to o . Strict composition gives

$$\Gamma.\{\mu\}.\{\nu\} \equiv \Gamma.\{\mu \circ \nu\}, \quad \Gamma.\{1_m\} \equiv \Gamma,$$

where $\mu \circ \nu : o \rightarrow m$ is the composite modality and $1_m : m \rightarrow m$ is the identity modality at m [Gra+21; Gra23].

Remark A.2.6 (Direction of a modality and its lock). At first sight, the modality and its lock may appear to run in opposite directions. Given a modality $\mu : n \rightarrow m$, they have two different roles:

A.2.6.1. The modality records the source-to-target direction from n to m in which modal information may be presented.

A.2.6.2. The lock is appended to a context already at the target mode m and opens a region in which subsequent assumptions are checked at the source mode n :

$$\Gamma \text{ ctx } @m \quad \longmapsto \quad \Gamma.\{\mu\} \text{ ctx } @n.$$

For example, let n describe Alice before she opens a sealed message, and let m describe her after she has read the secret inside. A modality $\mu : n \rightarrow m$ records this transition from not knowing the secret to knowing it. Suppose A is a claim considered before the message is opened, such as “the message is still sealed”. While working at m , placing the lock $\{\mu\}$ in the context opens a region in which subsequent assumptions are checked at n . The lock does not reveal the message, erase the secret or reverse the transition; it only changes the mode at which that part of the context is checked. Thus the modality points from n to m , whereas applying its lock takes a context at m to a context at n .

Definition A.2.7 (Modal type former). The modal type former packages a type or term checked in a locked context as a modal type or term at the original mode, while retaining the modality in the resulting type. Its rules are as follows:

Data. Let $\mu : n \rightarrow m$ be a modality, and let

$$\Gamma \text{ ctx } @m.$$

Appending the lock from Definition A.2.5 gives

$$\Gamma.\{\mu\} \text{ ctx } @n.$$

The lock makes n the mode at which subsequent types and terms are checked.

Formation. If A is a type in this locked context,

$$\Gamma.\{\mu\} \vdash A \text{ type } @n,$$

then its μ -modal type is formed at the original mode m :

$$\Gamma \vdash \langle \mu \mid A \rangle \text{ type } @m.$$

The expression $\langle \mu \mid A \rangle$ is a new type at mode m ; it is not an unmarked judgement that A itself is a type at m . The modality remains visible and records that A is available through the locked mode n .

Introduction. If the locked context contains a term

$$\Gamma.\{\mu\} \vdash a : A @n,$$

then this term can be introduced at mode m under the same modal marking:

$$\Gamma \vdash \text{mod}_\mu(a) : \langle \mu \mid A \rangle @m.$$

Thus $\text{mod}_\mu(a)$ is the canonical way to construct an inhabitant of the modal type from an inhabitant available under the lock.

Reading. In the epistemic interpretation, mode n is an endpoint at which A or $a : A$ is available, while $\langle \mu \mid A \rangle$ records that availability from mode m through μ . Neither the lock nor modal introduction changes an endpoint record; the corresponding record transformations are described in Remark A.2.10.

A. Definitions

Remark A.2.8 (Modal fragment is introduction-only). Definition A.2.7 gives only formation and introduction for $\langle \mu \mid A \rangle$. The thesis does not reproduce the corresponding modal elimination rule, usually written as **letmod** or modal pattern matching, the action of 2-cells $\alpha : \mu \Rightarrow \nu$ on modal terms, or the full inter-mode substitution discipline, because no definition or proof eliminates a term $\text{mod}_\mu(a)$, applies a 2-cell to a term, or transports a substitution across modes [Gra+21; Gra23].

This thesis therefore uses only a small, introduction-only fragment of Gratzner’s full multimodal type theory. The rules stated here operate in one direction:

$$\Gamma.\{\mu\} \vdash a : A @n \implies \Gamma \vdash \text{mod}_\mu(a) : \langle \mu \mid A \rangle @m.$$

A term checked under the lock at n can thus be packaged at m , but it is not moved across unchanged: the result has the modal type $\langle \mu \mid A \rangle$, not A . This fragment provides neither an elimination rule for unpacking an arbitrary modal term nor a variable-access rule for carrying declarations from the m -side of the lock into the n -side. Gratzner’s full theory supplies controlled modal elimination, cross-lock variable access and inter-mode substitution, allowing modal information to be used in ways unavailable in the present fragment. The cited works develop the additional machinery required for more general multimodal reasoning.

Epistemic interpretation

Remark A.2.9 (Interpretive terminology). The terms *subjectivity*, *epistemic position* and *interest* are interpretive shorthand, not additional objects of the calculus. In this thesis, an epistemic position is represented by the current mode together with its context; interest by the variables and questions in scope within that context; and subjectivity by the fact that every judgement is formed relative to a mode and context. No later definition or proof treats these terms as separate formal entities.

Remark A.2.10 (Epistemic modalities mark record transitions). Just as the terms in Remark A.2.9 do not add new objects to the calculus, the names learning, forgetting, conditioning, relating, unrelating, setting and unsetting do not introduce additional modal operators; rather, they describe transitions between already specified modes.

More specifically, a modality $\mu : n \rightarrow m$ witnesses that the transition from n to m is admitted by the mode theory; it does not inspect the context at n , perform the corresponding operation or construct the target at m . Instead, a separately defined context or record transformation constructs the target, and a proof certifies that this target is the result of the stated operation. The modality therefore marks the transition, while the accompanying transformation determines how its endpoint records differ.

The causal records and proof-carrying operations that those names describe are introduced after structural causal models.

A.3 Probabilistic definitions

This section defines constructive finite probability structures and operations within the type-theoretic setting. These are object-level mathematical constructions, not a model semantics for the judgemental syntax of Section A.1.

Events and resolution status

Definition A.3.1 (Finite sample type and decidable event). A finite sample type is a finite type Ω , in the sense of Definition A.1.19, whose inhabitants are read as possible outcomes. Its finiteness witness enumerates those outcomes and permits finite sums over them. By Remark A.1.36, every finite sample type has decidable equality, so outcomes can be compared and singleton events can be decided.

A decidable event on Ω is represented by a Boolean classifier

$$E : \Omega \rightarrow \text{Bool}$$

in the sense of Definitions A.1.20, A.1.32 and A.1.33. For each outcome $\omega : \Omega$, the value $E(\omega) = \text{true}$ means that ω belongs to the event, while $E(\omega) = \text{false}$ means that it does not. A decidable event is therefore a decidable finite subset of Ω . The notation $E \subseteq \Omega$ is used interchangeably with the Boolean-classifier notation.

Intersection, union, difference and complement are defined pointwise on Boolean values.

Definition A.3.2 (Resolution status). A resolution-status package records the result of a finite inspection procedure together with evidence that every reported proof or refutation is sound. Its components are as follows:

Data. Let Ω be a finite sample type. Let

$$A : \Omega \rightarrow \mathcal{U}$$

be an arbitrary proposition-valued family on Ω . Equivalently, A is a predicate assigning to every outcome $\omega : \Omega$ a proposition $A(\omega)$. No decision procedure for $A(\omega)$ is assumed.

Status type. The resolution-status type for A is the three-constructor finite type

Status_A generated by `proved`, `refuted`, `unresolved`.

Inspection procedure. A finite inspection procedure for A is a total function

$$\iota_A : \Omega \rightarrow \text{Status}_A.$$

For every input $\omega : \Omega$, the procedure therefore returns exactly one of the three status values.

Soundness. To interpret the returned values as genuine proof status, the procedure must be equipped with

$$\text{sound}_{\text{proved}} : \prod_{\omega : \Omega} (\iota_A(\omega) = \text{proved}) \rightarrow A(\omega),$$

and

$$\text{sound}_{\text{refuted}} : \prod_{\omega : \Omega} (\iota_A(\omega) = \text{refuted}) \rightarrow \neg A(\omega),$$

A. Definitions

where $\neg A(\omega) := A(\omega) \rightarrow \perp$ as in Definition A.1.28. Thus a **proved** result supplies a witness of $A(\omega)$, while a **refuted** result supplies a refutation of $A(\omega)$.

Interpretation. The value **unresolved** means only that the chosen inspection procedure supplies neither kind of evidence. It does not claim that no proof or refutation exists.

The family A is not assumed to be decidable. The finite output of ι_A is decidable instead: whether the returned status is **proved**, **refuted** or **unresolved** can always be determined. These three decidable predicates induce the resolution-status events of Definition A.3.4.

Remark A.3.3 (Why the unresolved case is necessary). A sound resolution-status package is available for every family $A : \Omega \rightarrow \mathcal{U}$: define $\iota_A(\omega) := \text{unresolved}$ for every $\omega : \Omega$. The premises of the proved and refuted soundness maps are then impossible, so those maps follow by empty-type elimination.

By contrast, a sound procedure guaranteed never to return **unresolved** would make A a decidable predicate in the sense of Definition A.1.32: for every $\omega : \Omega$, its result would supply either a witness of $A(\omega)$ or a refutation of $A(\omega)$. The unresolved case is therefore what permits a total inspection procedure without assuming that the underlying family of propositions is decidable.

Definition A.3.4 (Resolution-status events). Let Ω , $A : \Omega \rightarrow \mathcal{U}$ and $\iota_A : \Omega \rightarrow \text{Status}_A$ be the data of Definition A.3.2. The three resolution-status events associated with A are

$$\text{Prov}_A(\omega) := \begin{cases} \text{true}, & \iota_A(\omega) = \text{proved}, \\ \text{false}, & \text{otherwise}, \end{cases} \quad \text{Ref}_A(\omega) := \begin{cases} \text{true}, & \iota_A(\omega) = \text{refuted}, \\ \text{false}, & \text{otherwise}, \end{cases}$$

and

$$\text{Unr}_A(\omega) := \begin{cases} \text{true}, & \iota_A(\omega) = \text{unresolved}, \\ \text{false}, & \text{otherwise}. \end{cases}$$

Formally, these are Boolean classifiers in the sense of Definition A.1.33 for the predicates that $\iota_A(\omega)$ equals **proved**, **refuted** or **unresolved**, respectively. They are obtained by case analysis on the finite resolution-status type, with the equality tests justified by decidable equality in the sense of Definition A.1.35. Their correctness evidence follows from the corresponding cases. The three classifiers are shown in Proposition B.5.5 to form a partition of Ω : every $\omega : \Omega$ lies in exactly one of them.

Finite probability presentations

Definition A.3.5 (Finite summation). Let A be a finite type in the sense of Definition A.1.19, with $n : \mathbb{N}$ and

$$\text{decode} : \text{Fin}(n) \rightarrow A$$

supplied by its finiteness witness. Let M be a type with an equivalence relation $\approx$, an addition operation and an additive identity 0 . Assume that addition respects $\approx$, and that associativity, commutativity and the identity laws hold up to $\approx$.

For a function $g : A \rightarrow M$, define

$$\sum_{a:A} g(a) := \sum_{i:\text{Fin}(n)} g(\text{decode}(i)),$$

where the sum on the right is computed recursively over the n indices, beginning with zero.

The sum is unchanged up to $\approx$ when the indices are permuted, as proved in Proposition B.5.3. It therefore gives the same value, up to $\approx$, regardless of the order in which the terms are enumerated. Here $\approx$ expresses the appropriate notion of equality for M : for natural-number sums it is ordinary identity, whereas for the rational presentations of Definition A.3.6 it is the cross-multiplication relation $\sim$.

Definition A.3.6 (Nonnegative rational values). The presentation type of nonnegative rational values is

$$\mathbb{Q}_{\geq 0} := \Sigma_{a:\mathbb{N}} \Sigma_{b:\mathbb{N}} (b > 0).$$

An inhabitant $q = (a, b, h) : \mathbb{Q}_{\geq 0}$ consists of a numerator $a : \mathbb{N}$, a denominator $b : \mathbb{N}$, and a proof $h : b > 0$ that the denominator is positive. It represents the fraction a/b .

Two presentations

$$q = (a, b, h) \quad \text{and} \quad r = (c, d, k)$$

represent the same rational value when

$$q \sim r \iff ad = cb.$$

The same presentations are ordered by cross multiplication:

$$\frac{a}{b} \leq \frac{c}{d} \iff ad \leq cb.$$

This is the relation **QProb.LE**. Equality between values of $\mathbb{Q}_{\geq 0}$ is read extensionally through $\sim$, not as identity of their presentations. For example, with proofs $h : 5 > 0$ and $k : 10 > 0$, the presentations $(3, 5, h)$ and $(6, 10, k)$ represent the same rational value because $3 \cdot 10 = 6 \cdot 5$, although the two triples are not definitionally equal. The thesis therefore keeps the presentations together with the explicit relation $\sim$, rather than forming a quotient type.

Natural-number counts are regarded as rational values with denominator 1: the natural number m is represented by

$$(m, 1, \star) : \mathbb{Q}_{\geq 0},$$

where $\star$ is the immediate proof that $1 > 0$. This allows natural-valued indicators and cell counts to be used in rational probability expressions.

Addition, multiplication and division by a positive value use the usual fraction operations:

$$\frac{a}{b} + \frac{c}{d} := \frac{ad + cb}{bd}, \quad \frac{a}{b} \frac{c}{d} := \frac{ac}{bd}, \quad \frac{a/b}{c/d} := \frac{ad}{bc} \quad \text{when } c > 0.$$

The required proofs that the resulting denominators are positive follow from the supplied positive denominators. These operations respect $\sim$, as recorded by the Lean correspondence in Proposition B.5.4.

A. Definitions

Definition A.3.7 (Urn-generated probability type). An urn-generated probability assignment is built from a nonempty finite urn whose cells are treated as equiprobable and labelled by outcomes in a chosen type. Its data and event evaluation are as follows:

Data. Let X be any type. Define

$$\text{Prob}_{\mathbb{Q}}^{\text{urn}}(X) := \Sigma_{N:\mathbb{N}} \Sigma_{h:N>0} (\text{Fin}(N) \rightarrow X).$$

An inhabitant

$$\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$$

consists of a number N of cells, a proof h that the urn is nonempty, and a function

$$f : \text{Fin}(N) \rightarrow X$$

assigning an outcome in X to every cell.

Event probability. Let $E : X \rightarrow \text{Bool}$ be any Boolean-valued function. First count the cells whose labels satisfy E :

$$m_E := \sum_{i:\text{Fin}(N)} \mathbb{1}_E(f(i)),$$

using the finite summation of Definition A.3.5 and the indicator of Definition A.1.34. Its probability is the nonnegative rational presentation

$$P_{\mu}(E) := (m_E, N, h) : \mathbb{Q}_{\geq 0},$$

which represents the proportion m_E/N according to Definition A.3.6. When X is instantiated by a finite sample type Ω in the sense of Definition A.3.1, the same function $E : \Omega \rightarrow \text{Bool}$ is a decidable event.

Interpretation. Every cell has probability represented by $(1, N, h)$, conventionally written $1/N$. The map f need not be injective: when several cells have the same label, their contributions add together. Nor need f be surjective: labels outside its image do not contribute to any probability sum. Consequently, X itself need not be finite. Only the source urn is finite, and every event probability is determined by the finitely many labels occurring in that urn.

This is the urn presentation of a probability assignment: outcome weights are encoded indirectly by repeated labels among equiprobable cells. Definition A.3.8 introduces the direct normalized-weight presentation used for finite sample types.

Example. Take $X := \text{Colour}$, where Colour is the finite type generated by red, blue and pink. Let $N := 5$, let $h : 5 > 0$ be the canonical proof, and define $f : \text{Fin}(5) \rightarrow \text{Colour}$ by

$$(f(0), f(1), f(2), f(3), f(4)) := (\text{red}, \text{blue}, \text{red}, \text{red}, \text{blue}).$$

These data determine

$$\mu := (5, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Colour}).$$

Since several cells are labelled red or blue, while no cell is labelled pink, the map f is neither injective nor surjective. Let $E_{\text{red}} : \text{Colour} \rightarrow \text{Bool}$ return true on red

and false on blue and pink. Three of the five indicator values are 1, so $P_\mu(E_{\text{red}})$ computes to

$$(3, 5, h),$$

which represents $3/5$. The repeated labels produce a non-uniform probability assignment on the outcomes even though the five urn cells themselves are equiprobable.

Definition A.3.8 (Finite probability type). The urn presentation of Definition A.3.7 encodes probabilities indirectly through the multiplicities of labels among equiprobable cells. On a finite sample type, probabilities can instead be recorded directly by assigning a nonnegative rational weight to every outcome and proving that the weights sum to 1. This direct record presentation is defined below.

Data. Let Ω be a finite sample type. Probability values lie in the presentation type $\mathbb{Q}_{\geq 0}$ of Definition A.3.6. All sums below use the finite summation of Definition A.3.5.

The record form of a probability type over Ω is

$$\text{Prob}_{\mathbb{Q}}(\Omega) := \Sigma_{p:\Omega \rightarrow \mathbb{Q}_{\geq 0}} \left(\sum_{\omega:\Omega} p(\omega) \sim 1 \right).$$

An inhabitant

$$\mu = (p, h) : \text{Prob}_{\mathbb{Q}}(\Omega)$$

consists of a function

$$p : \Omega \rightarrow \mathbb{Q}_{\geq 0}$$

assigning a nonnegative rational weight to every outcome, together with a proof

$$h : \sum_{\omega:\Omega} p(\omega) \sim 1$$

that these weights are normalized.

Event probability. If $\mu = (p, h) : \text{Prob}_{\mathbb{Q}}(\Omega)$ and $E \subseteq \Omega$, then

$$P_\mu(E) := \sum_{\omega:\Omega} \mathbb{1}_E(\omega) p(\omega),$$

using the indicator of Definition A.1.34 and the embedding of natural numbers into $\mathbb{Q}_{\geq 0}$ from Definition A.3.6. If $E \cap F = \emptyset$, then

$$P_\mu(E \cup F) \sim P_\mu(E) + P_\mu(F)$$

because disjointness gives the pointwise indicator identity

$$\mathbb{1}_{E \cup F}(\omega) = \mathbb{1}_E(\omega) + \mathbb{1}_F(\omega),$$

after which distributivity and finite summation separate the two contributions. This is the direct-record form of the finite additivity proved in Theorem B.5.12; the passage from a direct record through a common-denominator record to an urn is justified by Propositions B.5.9 and B.5.10. When only one probability assignment is in scope, the subscript is sometimes dropped: $P(E) := P_\mu(E)$.

A. Definitions

Interpretation. The function p records outcome weights directly rather than encoding them through repeated urn labels. Finiteness of Ω makes every event sum finite, while the proof h ensures that the total weight is extensionally equal to 1. Unlike the urn presentation of Definition A.3.7, this direct presentation requires the outcome type itself to be finite.

Example. Use the finite type `Colour` from the example following Definition A.3.7, and let $d : 5 > 0$ be the canonical proof. Define $p : \text{Colour} \rightarrow \mathbb{Q}_{\geq 0}$ by

$$p(\text{red}) := (3, 5, d), \quad p(\text{blue}) := (2, 5, d), \quad p(\text{pink}) := (0, 5, d).$$

These three weights sum extensionally to 1. Together with the corresponding normalization proof h , they determine

$$\mu_{\text{Colour}} := (p, h) : \text{Prob}_{\mathbb{Q}}(\text{Colour}).$$

For the event E_{red} of the preceding example,

$$P_{\mu_{\text{Colour}}}(E_{\text{red}}) \sim (3, 5, d).$$

Thus the direct mass function represents the same assignment as the five-cell urn.

Definition A.3.9 (Common-denominator probability record). In the direct presentation of Definition A.3.8, different outcomes may initially have different denominators. Because its sample type is finite, these fractions can be rewritten over one positive common denominator, as proved in Proposition B.5.9. Once finite support is recorded explicitly as a list, however, the codomain itself need not be finite. The common-denominator representation is structured as follows:

Data. Let X be any type. A common-denominator probability record over X has the form

$$\begin{aligned} R &= (a, D, d, h), \\ a &: \text{List}(X \times \mathbb{N}), \\ D &: \mathbb{N}, \\ d &: D > 0, \\ h &: \sum_{(\omega, m) \in a} m = D. \end{aligned}$$

The list a contains finitely many explicitly supplied labels $\omega : X$, each paired with a natural weight m . No equality test on X is part of the record; if the same label occurs in several entries, each corresponding weight contributes separately. The natural number D is the common denominator, d proves that it is positive, and h proves normalization by showing that the total weight of the list is exactly D .

Event probability. Let $E : X \rightarrow \text{Bool}$ be a Boolean-valued function. Evaluate E by traversing a and summing the weights of precisely those entries for which $E(\omega) = \text{true}$; this requires no decidable equality on X . Denote the selected weight and the resulting probability by

$$\begin{aligned} m_E &:= \sum_{\substack{(\omega, m) \in a \\ E(\omega) = \text{true}}} m, \\ P_R(E) &:= (m_E, D, d) : \mathbb{Q}_{\geq 0}. \end{aligned}$$

Here m_E is the numerator and D is the common denominator, so $P_R(E)$ represents the fraction m_E/D .

Interpretation. The list a is a compressed finite urn: the weight m records how many unit cells carry the label ω . The total event selects every entry and therefore has weight D by h , so its probability is represented by D/D and satisfies $D/D \sim 1$.

Lean implementation. This is the structure `FiniteProbRecord` in `lean/Thesis/Probability/FiniteRecord.lean`: its fields `atoms`, `den`, `den_pos` and `total_mass` correspond respectively to a , D , d and h . The functions `FiniteProbRecord.eventMass` and `FiniteProbRecord.probVal` implement m_E and $P_R(E)$.

Example. For the same assignment on `Colour`, take

$$a := [(\text{red}, 3), (\text{blue}, 2), (\text{pink}, 0)], \quad D := 5.$$

Let $d : 5 > 0$ be the canonical proof and let $h : 3 + 2 + 0 = 5$ be the normalization proof. These data determine

$$R_{\text{Colour}} := (a, 5, d, h).$$

For E_{red} , the selected weight is $m_{E_{\text{red}}} = 3$, so $P_{R_{\text{Colour}}}(E_{\text{red}})$ computes to

$$(3, 5, d).$$

Remark A.3.10 (Relationship between finite probability presentations). The urn, direct-weight and common-denominator presentations describe the same finite rational probability assignments, but the conversions between them use slightly different assumptions.

From an urn to direct weights. Let

$$\nu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega),$$

where Ω is a finite sample type. Here N is the number of urn cells, $h : N > 0$ proves that the urn is nonempty, and

$$f : \text{Fin}(N) \rightarrow \Omega$$

gives the outcome labelling each cell. Using decidable equality on Ω , let

$$E_{\omega} : \Omega \rightarrow \text{Bool}$$

be the singleton classifier that returns `true` exactly at ω . The number of cells labelled ω is the event count from Definition A.3.7:

$$m_{E_{\omega}} := \sum_{i : \text{Fin}(N)} \mathbb{1}_{E_{\omega}}(f(i)).$$

Define

$$p_{\nu}(\omega) := (m_{E_{\omega}}, N, h) : \mathbb{Q}_{\geq 0}.$$

Every cell has exactly one label, so summing these counts over all outcomes gives

$$\sum_{\omega : \Omega} m_{E_{\omega}} = N.$$

A. Definitions

Consequently, the weights $p_\nu(\omega)$ sum extensionally to 1 and determine an inhabitant of $\mathbf{Prob}_\mathbb{Q}(\Omega)$ in the sense of Definition A.3.8.

From direct weights to a common denominator. Suppose

$$\mu = (p, h) : \mathbf{Prob}_\mathbb{Q}(\Omega).$$

Because Ω is finite and every $p(\omega)$ is rational, one can choose a positive denominator D , a proof $d : D > 0$, and natural weights m_ω such that

$$p(\omega) \sim (m_\omega, D, d) \quad \text{for every } \omega : \Omega$$

and

$$\sum_{\omega : \Omega} m_\omega = D.$$

These data produce a common-denominator record of Definition A.3.9. Proposition B.5.9 proves that this conversion preserves the probability of every Boolean-valued event.

Between common-denominator records and urns. An urn $\nu = (N, h, f)$ determines a common-denominator record by taking denominator N and listing its cells with unit weight:

$$[(f(i), 1)]_{i : \mathbf{Fin}(N)}.$$

Conversely, a weighted entry (ω, m) in a common-denominator record can be expanded into m separate urn cells labelled ω . Proposition B.5.10 proves both directions and shows that the record and resulting urn assign extensionally equal probabilities to every Boolean-valued event.

Therefore, when Ω is a finite sample type, all three presentations can be converted into one another without changing the probabilities they assign to events. For an arbitrary type X , only the urn–common-denominator correspondence is available, because those presentations carry their finite support explicitly; the direct presentation requires the whole outcome type to be a finite sample type.

Remark A.3.11 (Finite probability laws). Every urn-generated probability assignment

$$\mu : \mathbf{Prob}_\mathbb{Q}^{\text{urn}}(X)$$

satisfies the finite counterparts of the Kolmogorov laws. For every decidable event $E : X \rightarrow \mathbf{Bool}$:

$$P_\mu(E) \geq 0.$$

If $\top_X : X \rightarrow \mathbf{Bool}$ is the total event defined by $\top_X(x) := \mathbf{true}$, then:

$$P_\mu(\top_X) \sim 1.$$

Finally, if $E, F : X \rightarrow \mathbf{Bool}$ are disjoint decidable events, then:

$$P_\mu(E \cup F) \sim P_\mu(E) + P_\mu(F).$$

The same additivity law extends to every finite pairwise disjoint family of decidable events.

These laws are consequences of finite cell counting and the normalization carried by the probability assignment; they are not additional assumptions or record fields. They are proved in Theorem B.5.12. Through the presentation equivalences of Remark A.3.10, the same laws hold for the direct-weight and common-denominator forms. No countable-additivity principle is asserted.

Conditioning and independence

Definition A.3.12 (Conditional probability and Bayesian update). Conditioning is defined directly on the urn-generated assignments of Definition A.3.7. Its components are as follows:

Data. Let X be any type, let

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X),$$

and let $A, B : X \rightarrow \text{Bool}$ be decidable events. Write m_A and m_B for their respective selected-cell counts in μ , as in Definition A.3.7. Their intersection is the event whose classifier returns **true** exactly when both A and B return **true**:

$$(A \cap B)(x) := \text{and}_{\text{Bool}}(A(x), B(x)).$$

Here and_{Bool} is the Boolean-conjunction operation defined in Definition A.1.20. Write $m_{A \cap B}$ for the selected-cell count of this intersection in μ .

Conditional probability. Suppose $m_B : m_B > 0$; equivalently, B has positive probability under μ . Then:

$$P_{\mu}(A \mid B) := (m_{A \cap B}, m_B, h_B) : \mathbb{Q}_{\geq 0}.$$

Indeed,

$$P_{\mu}(A \cap B) \sim \frac{m_{A \cap B}}{N}, \quad P_{\mu}(B) \sim \frac{m_B}{N},$$

and therefore, because $m_B > 0$,

$$\frac{P_{\mu}(A \cap B)}{P_{\mu}(B)} \sim \frac{m_{A \cap B}/N}{m_B/N} = \frac{m_{A \cap B}}{m_B}.$$

Thus $(m_{A \cap B}, m_B, h_B)$ presents the conditional probability obtained after the common urn denominator N cancels. Equivalently,

$$\boxed{P_{\mu}(A \mid B) \sim \frac{P_{\mu}(A \cap B)}{P_{\mu}(B)}}.$$

Bayes' theorem. If both A and B have positive probability, then the preceding definition satisfies the derived identity

$$\boxed{P_{\mu}(A \mid B) \sim \frac{P_{\mu}(B \mid A)P_{\mu}(A)}{P_{\mu}(B)}}.$$

This is Bayes' theorem for decidable finite events. It is not an additional defining rule: it follows by expanding both conditional probabilities into finite event counts and using $A \cap B = B \cap A$. The calculation is proved in Theorem B.5.16.

Bayesian update. Regard μ as the probability assignment before receiving new evidence, and let the event $E : X \rightarrow \text{Bool}$ represent that evidence. Write m_E for its selected-cell count in μ , as in Definition A.3.7, and suppose $m_E : m_E > 0$, so at least one cell of μ satisfies E . Filter the ordered list of urn cells, retaining exactly those cells $i : \text{Fin}(N)$ for which $E(f(i)) = \text{true}$. Renumbering the resulting m_E retained cells in their inherited order gives an enumeration

$$f_E : \text{Fin}(m_E) \rightarrow X$$

A. Definitions

and hence the filtered, Bayesian-updated assignment

$$\mu_E := (m_E, h_E, f_E) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X).$$

With E now taken as given evidence, let $F : X \rightarrow \text{Bool}$ be any decidable query event. The updated assignment then satisfies

$$P_{\mu_E}(F) \sim P_{\mu}(F \mid E),$$

as proved in Proposition B.5.18.

When X is a finite sample type and μ corresponds to a direct assignment $(p, h) : \text{Prob}_{\mathbb{Q}}(X)$ through Remark A.3.10, converting the filtered urn gives the familiar posterior mass function

$$p_E(x) := \frac{\mathbb{1}_E(x)p(x)}{P_{\mu}(E)}.$$

Both conditional probability and Bayesian update are partial: they are defined only when the conditioning event has positive probability under μ .

Definition A.3.13 (Independence and conditional independence of decidable events). Let X be any type, let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $A, B, C : X \rightarrow \text{Bool}$ be decidable events.

Unconditional independence. The events A and B are *independent under μ* , written $A \perp_{\mu} B$, when

$$P_{\mu}(A \cap B) \sim P_{\mu}(A) \cdot P_{\mu}(B).$$

Equivalently, when $P_{\mu}(B) > 0$, $P_{\mu}(A \mid B) \sim P_{\mu}(A)$; and when $P_{\mu}(A) > 0$, $P_{\mu}(B \mid A) \sim P_{\mu}(B)$.

Conditional independence. The events A and B are *conditionally independent given C under μ* , written $A \perp_{\mu} B \mid C$, when

$$P_{\mu}(A \cap B \cap C) \cdot P_{\mu}(C) \sim P_{\mu}(A \cap C) \cdot P_{\mu}(B \cap C).$$

Equivalently, when $P_{\mu}(C) > 0$, $P_{\mu}(A \cap B \mid C) \sim P_{\mu}(A \mid C) \cdot P_{\mu}(B \mid C)$; and when additionally $P_{\mu}(B \cap C) > 0$, $P_{\mu}(A \mid B \cap C) \sim P_{\mu}(A \mid C)$. These unconditional and conditional characterisations are proved in Proposition B.5.17.

The product forms are primary because they remain defined when conditioning events have zero probability, whereas the corresponding conditional probabilities do not. This is why they are used in the structural causal model proofs of Sections B.6 and B.10, including Lemma B.10.1.

Variables and event projections

Definition A.3.14 (Value type). Let I be a type of variable names. A finite value-type assignment is a family

$$\text{Val} : I \rightarrow \mathcal{U}$$

for which every $\text{Val}(i)$ is finite in the sense of Definition A.1.19. For a name $X : I$, the type $\text{Val}(X)$ is the value type of X , and its inhabitants are the admissible values that X may take. By Remark A.1.36, every value type has decidable equality in the sense of Definition A.1.35.

Example. If distinct names $C, D : I$ denote the outcomes of a coin toss and a six-sided die, one may take

$$\text{Val}(C) := \text{CoinFace}, \quad \text{Val}(D) := \text{DieFace}.$$

The two inhabitants of CoinFace are `heads` and `tails`, while the six inhabitants of DieFace are

$$\text{faceUp}(1), \dots, \text{faceUp}(6).$$

Their finiteness witnesses identify these terms with the indices of $\text{Fin}(2)$ and $\text{Fin}(6)$, respectively. The value types specify which outcomes are admissible; they do not select the outcome of a particular toss or roll.

Definition A.3.15 (Finite variable and joint assignment). A finite family of variables $\mathcal{V}$ consists of a finite type I of variable names together with a finite value-type assignment $\text{Val} : I \rightarrow \mathcal{U}$ from Definition A.3.14. Foundationally, its joint assignments form the finite telescope of Remark A.1.22. For readability, write the same coordinate data pointwise as

$$\text{Val}(\mathcal{V}) := \Pi_{i:I} \text{Val}(i)$$

for its joint-assignment type. An inhabitant $v : \text{Val}(\mathcal{V})$ chooses one admissible value $v(i) : \text{Val}(i)$ for every $i : I$. By Proposition B.5.1, this type is finite and hence is a finite sample type. A joint event is therefore a decidable event

$$E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$$

in the sense of Definition A.3.1.

Example. Let I be the two-element finite type whose variable names are C and D , and use the coin and die value types from Definition A.3.14. A joint assignment chooses one coin face and one die face. For example, $(\text{heads}, \text{faceUp}(2))$ is the assignment with $C = \text{heads}$ and $D = \text{faceUp}(2)$. Every coin-and-die pair is a possible joint assignment. A joint event may return `true` for any chosen collection of these pairs; for example, one event may return `true` exactly on

$$(\text{heads}, \text{faceUp}(2)) \quad \text{and} \quad (\text{tails}, \text{faceUp}(3)).$$

Definition A.3.16 (Cylinder event). Let $\mathcal{V}$, I and

$$\Omega := \text{Val}(\mathcal{V})$$

be as in Definition A.3.15. Thus Ω is the full joint-assignment type and, by that definition, a finite sample type. A selected subfamily is represented by a natural number $m : \mathbb{N}$ and an embedding

$$j : \text{Fin}(m) \rightarrow I,$$

meaning a map equipped with a proof that $j(k) = j(\ell)$ implies $k = \ell$. The inhabitants of $\text{Fin}(m)$ enumerate the selected positions, while j specifies which variable name occupies each position. The selected joint-assignment type is

$$\text{Val}(j) := \Pi_{k:\text{Fin}(m)} \text{Val}(j(k)).$$

Projection. The selected coordinates of a full assignment are obtained by

$$\pi_j : \Omega \rightarrow \text{Val}(j), \quad \pi_j(v)(k) := v(j(k)).$$

A. Definitions

Event lifting. A decidable event $E : \text{Val}(j) \rightarrow \text{Bool}$ determines the cylinder event

$$E^\uparrow : \Omega \rightarrow \text{Bool}, \quad E^\uparrow(v) := E(\pi_j(v)).$$

Interpretation. The projection forgets all coordinates outside the selected subfamily, while the cylinder event lifts an event in the opposite direction by testing only the retained coordinates. Equivalently, $E^\uparrow$ is the preimage $\pi_j^{-1}(E)$.

Example. Continue with the coin variable C and die variable D from Definition A.3.15. Take $m := 1$ and select only the die coordinate by setting $j(0) := D$. Under the resulting identification of $\text{Val}(j)$ with DieFace , the projection discards the coin outcome:

$$\pi_j(\text{heads}, \text{faceUp}(2)) = \text{faceUp}(2), \quad \pi_j(\text{tails}, \text{faceUp}(2)) = \text{faceUp}(2).$$

Let $E_D : \text{DieFace} \rightarrow \text{Bool}$ return `true` exactly on `faceUp(2)`. Its cylinder lift is

$$E_D^\uparrow(c, d) := E_D(d).$$

It therefore contains both $(\text{heads}, \text{faceUp}(2))$ and $(\text{tails}, \text{faceUp}(2))$: the lifted event tests the die coordinate and ignores the coin coordinate.

Notation. In later formulas, $\mathcal{X} \subseteq \mathcal{V}$ abbreviates a natural number $m : \mathbb{N}$ together with an embedding $j_{\mathcal{X}} : \text{Fin}(m) \rightarrow I$. It specifies a selection of coordinates, not a set-theoretic subset of terms. Write $\text{Val}(\mathcal{X}) := \text{Val}(j_{\mathcal{X}})$. The selection data $j_{\mathcal{X}}$, together with their injectivity proof, determine the projection

$$\pi_{\mathcal{X}} : \text{Val}(\mathcal{V}) \rightarrow \text{Val}(\mathcal{X}), \quad \pi_{\mathcal{X}}(v)(k) := v(j_{\mathcal{X}}(k)).$$

Thus $\mathcal{X} \subseteq \mathcal{V}$ specifies which projection is used. Once an event $E : \text{Val}(\mathcal{X}) \rightarrow \text{Bool}$ has been chosen, this projection determines its cylinder lift $E^\uparrow : \Omega \rightarrow \text{Bool}$.

Remark A.3.17 (Fibres and cylinder events). Let $\mathcal{Y} \subseteq \mathcal{V}$ specify the projection

$$\pi_{\mathcal{Y}} : \Omega \rightarrow \text{Val}(\mathcal{Y})$$

of Definition A.3.16. Since $\text{Val}(\mathcal{Y})$ is finite, its equality is decidable by Remark A.1.36. Every retained assignment $y : \text{Val}(\mathcal{Y})$ therefore determines the decidable *fibre event*:

$$F_y : \Omega \rightarrow \text{Bool},$$

which returns `true` exactly when $\pi_{\mathcal{Y}}(v) = y$. The fibre over y consists of the full assignments selected by F_y .

The events $\{F_y\}_{y : \text{Val}(\mathcal{Y})}$ are pairwise disjoint and jointly exhaustive: every full assignment $v : \Omega$ belongs to exactly the fibre indexed by $\pi_{\mathcal{Y}}(v)$. Moreover, a cylinder event $E^\uparrow$ is the union of precisely those fibres whose retained assignment satisfies E , which gives the pointwise identity:

$$E^\uparrow(v) = \text{true} \iff E(\pi_{\mathcal{Y}}(v)) = \text{true}.$$

This finite fibre partition is used to calculate marginal probabilities in Definition A.3.21.

Remark A.3.18 (Rectangular events). Let $\mathcal{V}$ be a finite family of variables with index type I and joint-assignment type $\Omega = \text{Val}(\mathcal{V})$, as in Definitions A.3.15

and A.3.16. A rectangular event is the special joint event obtained from a dependent product of coordinate events

$$A : \Pi_{i:I}(\text{Val}(i) \rightarrow \text{Bool}).$$

Thus A_i is the event selected at coordinate i . Define the joint classifier

$$\text{Rect}_A : \Omega \rightarrow \text{Bool}$$

by finite elimination over the canonical enumeration of I . The empty family returns **true**; adjoining one final coordinate uses and_{Bool} from Definition A.1.20:

$$\begin{aligned} \text{Rect}_\emptyset(\star) &:= \text{true}, \\ \text{Rect}_{(A',A)}(\mathbf{v}', x) &:= \text{and}_{\text{Bool}}(\text{Rect}_{A'}(\mathbf{v}'), A(x)). \end{aligned}$$

Here $\star$ is the unique assignment to the empty telescope.

Characterization. For a complete assignment $\mathbf{v} : \Omega$, the recursive definition gives

$$\text{Id}_{\text{Bool}}(\text{Rect}_A(\mathbf{v}), \text{true}) \iff \Pi_{i:I} \text{Id}_{\text{Bool}}(A_i(\mathbf{v}_i), \text{true}).$$

This finite characterization is proved in Proposition B.5.2. Equivalently, lift each A_i to a cylinder event on Ω using Definition A.3.16; then Rect_A is their finite intersection. The dependent product on the right contains coordinatewise evidence; it is not the rectangular event itself, which remains a Boolean classifier.

Example. Let $I = \text{Fin}(2)$ and let both coordinates have Boolean value type. Suppose A_0 accepts only **true**, while A_1 accepts both Boolean values. The resulting rectangular event selects exactly **(true, false)** and **(true, true)**. It is therefore not a singleton. The event selecting exactly **(false, false)** and **(true, true)** is not rectangular.

Definition A.3.19 (Independence of variable families). The definition is structured as follows:

Data. Let $\mathcal{V}$ be a finite family of variables and let

$$\Omega := \text{Val}(\mathcal{V})$$

be its finite joint-assignment type from Definition A.3.16. Choose an urn-generated probability assignment

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

in the sense of Definition A.3.7. The finiteness of Ω supplies the finite domain of assignments and decidable events; the assignment μ is additional probabilistic data, whose labelling map $f : \text{Fin}(N) \rightarrow \Omega$ labels every urn cell with a full joint assignment.

Independence. Two selected variable families $\mathcal{X}$ and $\mathcal{Y}$ are independent under μ when every pair of joint events

$$E : \text{Val}(\mathcal{X}) \rightarrow \text{Bool}, \quad F : \text{Val}(\mathcal{Y}) \rightarrow \text{Bool}$$

satisfies the event-independence condition of Definition A.3.13 after cylinder lifting:

$$E^\uparrow \perp_\mu F^\uparrow.$$

A. Definitions

Here the arrows denote the cylinder lifts of Definition A.3.16. Unfolding the event-independence notation of Definition A.3.13, the displayed condition is

$$P_\mu(E^\uparrow \cap F^\uparrow) \sim P_\mu(E^\uparrow)P_\mu(F^\uparrow).$$

Conditional independence. The families $\mathcal{X}$ and $\mathcal{Y}$ are conditionally independent given a selected family $\mathcal{Z}$, written

$$\mathcal{X} \perp\!\!\!\perp_\mu \mathcal{Y} \mid \mathcal{Z},$$

when, for every $z : \text{Val}(\mathcal{Z})$ and every pair E, F as above,

$$E^\uparrow \perp\!\!\!\perp_\mu F^\uparrow \mid H_z^\uparrow.$$

The event $H_z : \text{Val}(\mathcal{Z}) \rightarrow \text{Bool}$ is the singleton event that returns `true` exactly on z . It is decidable because $\text{Val}(\mathcal{Z})$ is finite, and $H_z^\uparrow$ is its cylinder lift to Ω . Unfolding the conditional-independence notation of Definition A.3.13, the displayed condition is

$$P_\mu(E^\uparrow \cap F^\uparrow \cap H_z^\uparrow)P_\mu(H_z^\uparrow) \sim P_\mu(E^\uparrow \cap H_z^\uparrow)P_\mu(F^\uparrow \cap H_z^\uparrow).$$

Example. Continue with the coin variable C and die variable D . Let μ_C be the two-cell urn containing `heads` and `tails` once each, and let μ_D be the six-cell urn containing each die face once. Under the coin-and-die identification of the full assignment type from Definition A.3.15, their product urn

$$\mu := \mu_C \otimes \mu_D : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

contains each of the twelve coin-and-die assignments once. Let H_C be the event that the coin is `heads` and let S_D be the event that the die is `faceUp(6)`. Their cylinder lifts satisfy

$$P_\mu(H_C^\uparrow) \sim \frac{1}{2}, \quad P_\mu(S_D^\uparrow) \sim \frac{1}{6},$$

while their intersection selects exactly one of the twelve assignments:

$$\begin{aligned} P_\mu(H_C^\uparrow \cap S_D^\uparrow) &\sim \frac{1}{12} \\ &\sim \frac{1}{2} \cdot \frac{1}{6} \\ &\sim P_\mu(H_C^\uparrow)P_\mu(S_D^\uparrow). \end{aligned}$$

By Definition A.3.13, this identity means that $H_C^\uparrow$ and $S_D^\uparrow$ are independent under μ . This verifies one pair of events. More generally, Proposition B.5.8 proves the same product identity for every pair of coin and die events; hence the coin and die variable families are independent under μ in the sense of this definition.

Remark A.3.20 (Independence notation). The symbol $\perp\!\!\!\perp$ has two readings in this thesis. A probability subscript denotes probabilistic independence: Definition A.3.13 defines it for events, and Definition A.3.19 lifts it to variable families. A graph subscript instead denotes graphical d-separation, for example $(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}}$, as in Definitions A.4.18 and A.5.10. The subscript therefore determines which meaning is intended. When the probability assignment is induced by a structural causal model $\mathcal{M}$, it is written $\mu_{\mathcal{M}}$ as in Definitions A.4.16 and A.4.19.

Definition A.3.21 (Marginalisation). Marginalisation forgets selected coordinates of a finite joint assignment while retaining their contribution to the probability of the remaining coordinates. Its components are as follows:

Data. Let $\mathcal{V}$ and

$$\Omega := \text{Val}(\mathcal{V})$$

be as in Definition A.3.16, and choose an urn-generated probability assignment

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$$

as in Definition A.3.19. For a selected family $\mathcal{Y} \subseteq \mathcal{V}$, Definition A.3.16 supplies the projection $\pi_{\mathcal{Y}} : \Omega \rightarrow \text{Val}(\mathcal{Y})$.

Marginal assignment. Retain the same urn cells and relabel each one through the projection:

$$\mu_{\mathcal{Y}} := (N, h_N, \pi_{\mathcal{Y}} \circ f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{Y})).$$

Event probability. This construction is the urn pushforward of Proposition B.5.11. For every decidable event $E : \text{Val}(\mathcal{Y}) \rightarrow \text{Bool}$, both E under $\mu_{\mathcal{Y}}$ and its cylinder lift $E^{\uparrow} = E \circ \pi_{\mathcal{Y}}$ under μ select cell $i : \text{Fin}(N)$ exactly when

$$E(\pi_{\mathcal{Y}}(f(i))) = \text{true}.$$

That proposition therefore gives

$$P_{\mu_{\mathcal{Y}}}(E) \sim P_{\mu}(E^{\uparrow}).$$

Finite-sum interpretation. For $y : \text{Val}(\mathcal{Y})$, let H_y be the singleton event that returns **true** exactly on y , and let F_y be the fibre event of Remark A.3.17. Finite additivity gives

$$P_{\mu_{\mathcal{Y}}}(H_y) \sim P_{\mu}(F_y) \sim \sum_{\substack{v : \Omega \\ F_y(v) = \text{true}}} P_{\mu}(H_v),$$

where H_v is the singleton event on Ω selecting v . The final sum ranges over the finite fibre of $\pi_{\mathcal{Y}}$ above y and is the finite-additivity calculation of Theorem B.5.12. This variable-projection construction is the form used inside the do-calculus derivation steps of Definition A.5.12 and in the structural causal model proofs of Sections B.6 and B.10.

Constructive urn assumptions

Definition A.3.22 (Constructive Clayton–Waddington urn assumptions). Definition A.3.7 assigns the ratio k/N to an event selecting k cells from an N -cell urn. The present definition records the qualitative assumptions under which the rescaling from represented urn plausibilities to these rational ratios is well-defined and reflects strict plausibility comparisons.

In this thesis, these assumptions form a finite constructive adaptation of the urn-based argument of Clayton and Waddington [CW17]. Current assumptions are not a verbatim restatement of the original argument; their main differences are summarised in Table B.3, and the corresponding finite proof is developed in Section B.5.

A. Definitions

Data. Let Plaus be a type of qualitative plausibility values. For every $N : \mathbb{N}$, let

$$\text{plaus}_N : (\text{Fin}(N) \rightarrow \text{Bool}) \rightarrow \text{Plaus}$$

assign a qualitative value to every decidable event on the N -cell type. Equip Plaus with two type-valued binary relations

$$\approx_U, \prec_U : \text{Plaus} \rightarrow \text{Plaus} \rightarrow \mathcal{U},$$

read respectively as ‘qualitative equivalence’ and ‘strict qualitative comparison’.

Canonical events. Let

$$N, k : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N.$$

and define the canonical decidable k -cell event

$$C_{N,k} : \text{Fin}(N) \rightarrow \text{Bool}$$

to return `true` on the first k indices and `false` on the remaining indices. Define

$$U_N(k) := \text{plaus}_N(C_{N,k}) : \text{Plaus},$$

where $U_N(k)$ denotes the qualitative plausibility of the canonical event; the link with probability is established after the assumptions.

Assumptions. The constructive Clayton–Waddington assumptions used here are:

A.3.22.1. Qualitative coherence: The relation $\approx_U$ is an equivalence relation, and $\prec_U$ is irreflexive and transitive. Moreover, strict comparison respects qualitative equivalence in both arguments. More explicitly, for all $a, a', b, b' : \text{Plaus}$,

$$\begin{aligned} a \approx_U a' \wedge a \prec_U b &\implies a' \prec_U b, \\ b \approx_U b' \wedge a \prec_U b &\implies a \prec_U b'. \end{aligned}$$

Since $\approx_U$ is symmetric, the same substitutions can be made in the reverse direction. Consequently, if $a \approx_U a'$ and $b \approx_U b'$, then:

$$a \prec_U b \iff a' \prec_U b'.$$

It follows that strict comparison depends only on the qualitative plausibilities represented, not on the particular descriptions chosen for them. As such, strict comparison depends only on the $\approx_U$ -equivalence classes of its arguments: replacing either argument by an equivalent representative does not change the comparison. Equivalently, $\prec_U$ induces a well-defined strict order on the qualitative-equivalence classes.

A.3.22.2. Count invariance: let $N : \mathbb{N}$ and let $E, F : \text{Fin}(N) \rightarrow \text{Bool}$ be arbitrary Boolean-valued functions. Similar to Definition A.3.7, let:

$$m_E := \sum_{i:\text{Fin}(N)} \mathbb{1}_E(i), \quad m_F := \sum_{i:\text{Fin}(N)} \mathbb{1}_F(i),$$

denote the numbers of cells selected by E and F , respectively. Events with the same selected-cell count receive qualitatively equivalent plausibilities:

$$m_E = m_F \implies \text{plaus}_N(E) \approx_U \text{plaus}_N(F).$$

Hence, for a fixed urn size N , only the number of selected cells matters, not which particular cells are selected. Consequently, replacing an event by a pointwise equal event or permuting the labels of its cells does not change its qualitative plausibility, so every event is qualitatively equivalent to its canonical same-count event; see Proposition B.5.6.

A.3.22.3. Strict monotonicity: let

$$N, k, \ell : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N, \quad h_\ell : \ell \leq N.$$

If $k < \ell$, then

$$U_N(k) \prec_U U_N(\ell);$$

selecting more cells in the same urn is therefore strictly more plausible.

A.3.22.4. Refinement invariance: let

$$N, k, c : \mathbb{N}, \quad h_N : N > 0, \quad h_k : k \leq N, \quad h_c : c > 0.$$

Replacing every cell by c indistinguishable subcells preserves plausibility:

$$U_N(k) \approx_U U_{cN}(ck).$$

This permits two urn ratios to be compared after passing to a common refined urn without changing either represented plausibility.

Lean implementation. The finite cell-product definitions are in `lean/Thesis/Probability/FiniteCellProduct.lean`. The four assumptions are implemented in the structure `QualitativeUrnScale`: qualitative coherence, same-count invariance, strict monotonicity and refinement invariance. The product urn defined in Remark A.3.23 is constructed by `UrnRatio.product`. Within the `FiniteCellProduct` namespace, `equivalence` gives the row-major equivalence between product cells and MN cells, while `encodedRectangularEvent` transports a rectangular Boolean event to the latter cell type. The agreement between the encoded and canonical cell enumerations, together with the resulting rectangular count, is proved in Proposition B.5.8. The urn-ratio definitions are in `lean/Thesis/Probability/Urn.lean`.

Remark A.3.23 (Consequences of the urn assumptions). The four clauses of Definition A.3.22 are the primitive constructive Clayton–Waddington assumptions. The following establishes the link between the qualitative and rational values of represented urn events.

Rational rescaling. Define the urn sizes and cell counts:

$$N, M, k, \ell : \mathbb{N}, \quad h_N : N > 0, h_M : M > 0, \quad h_k : k \leq N, h_\ell : \ell \leq M,$$

then the qualitative assumptions yield:

$$\begin{aligned} U_N(k) \approx_U U_M(\ell) &\iff kM = \ell N, \\ U_N(k) \prec_U U_M(\ell) &\iff kM < \ell N. \end{aligned}$$

Consequently, the rational rescaling:

$$\rho(U_N(k)) := (k, N, h_N) : \mathbb{Q}_{\geq 0}$$

A. Definitions

is well-defined up to $\sim$ on the $\approx_U$ -equivalence classes and reflects the strict cross-multiplication order of represented ratios. These order characterisations and the well-definedness of ρ are proved in Proposition B.5.7.

Connection to labelled urns. Following Definition A.3.7, let X be any type and let μ be an urn-generated probability assignment:

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X).$$

For any Boolean-valued function $E : X \rightarrow \text{Bool}$, define the induced cell event:

$$E_f : \text{Fin}(N) \rightarrow \text{Bool}, \quad E_f(i) := E(f(i));$$

its selected-cell count is:

$$m_E := \sum_{i:\text{Fin}(N)} \mathbb{1}_{E_f}(i) = \sum_{i:\text{Fin}(N)} \mathbb{1}_E(f(i)).$$

Count invariance gives $\text{plaus}_N(E_f) \approx_U U_N(m_E)$, and the exact bridge to the event probability $P_\mu(E)$ from Definition A.3.7 is:

$$\boxed{P_\mu(E) \sim \rho(U_N(m_E))}.$$

It follows that for a fixed urn size N , the qualitative and rational values depend on E and f only through the selected-cell count m_E . The reduction to this count is proved in Proposition B.5.6; combining it with the rescaling of Proposition B.5.7 gives the displayed bridge.

Note that no finiteness or decidable equality on X is required; it is only required that X is inhabited. Once X contains a term x , an urn-generated probability assignment for every positive N can be made using the trivial labelling $f(i) := x$. Naturally, trivial labelling leads to a trivial distribution; nontrivial distributions require additional labels.

Finite complements. The rescaling ρ is defined only for represented finite urn events; it does not assign a probability to an arbitrary, possibly undecidable proposition A . For a decidable urn event E , however, the complement $E^{\complement}$ is again decidable, and:

$$P_\mu(E) + P_\mu(E^{\complement}) \sim 1.$$

The finite-complement result is proved in Proposition B.5.7.

Product urns. Multiplication requires no additional qualitative assumption. Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ be an M -cell urn assignment and let $\nu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y)$ be an N -cell urn assignment. If $E : X \rightarrow \text{Bool}$ selects k cells of μ and $F : Y \rightarrow \text{Bool}$ selects ℓ cells of ν , then the product type:

$$\text{Fin}(M) \times \text{Fin}(N),$$

serves as the cell type of the constructed product assignment $\mu \otimes \nu$. The rectangular event is defined by

$$(E \times F)(x, y) := \text{and}_{\text{Bool}}(E(x), F(y)).$$

It selects $k\ell$ of the MN product cells, and hence:

$$P_{\mu \otimes \nu}(E \times F) \sim P_\mu(E)P_\nu(F).$$

This identity concerns the product assignment constructed from the two urns; it does not assert that an arbitrary joint probability assignment on the product type factorises. The construction of product urns and their scope are proved in Proposition B.5.8.

A.4 Finite causal models

This section defines the finite graphical and structural objects used for causal reasoning. It begins with node families, directed and bidirected edges, paths and acyclicity. It then builds structural causal models from an endogenous signature, an exogenous source, an all-directed latent skeleton and typed mechanisms, and records the induced solution map and observational assignment. Causal Bayesian networks appear after that structural package, as a parent-kernel presentation of the same finite setting. The section then defines d-separation, graph mutilation, intervention and the do-operator. A final subsection records a more general hidden-DAG layer used by later completeness arguments; it is not part of the structural-model spine. Throughout the section, the standard causal phrase *probability distribution* denotes a finite rational probability assignment of Section A.3; no additional measure-theoretic object is introduced.

Nodes, edges and paths

Definition A.4.1 (Finite node type and selection). The graph definitions below use a canonical finite presentation of the variable names and Boolean selections of the resulting formal nodes.

Node type. Let $\mathcal{V}$ be a finite family of variables with finite type I of variable names and value-type assignment $\text{Val} : I \rightarrow \mathcal{U}$, as in Definitions A.3.14 and A.3.15. Choose the finiteness presentation of I from Definition A.1.19, and write

$$V := \text{Fin}(n)$$

for its index type, with maps:

$$\text{encode} : I \rightarrow V, \quad \text{decode} : V \rightarrow I$$

and the two round-trip identity proofs of that definition. The type I contains the descriptive variable names, while V is the *formal node type* used by the finite graph records. An inhabitant $v : V$ represents the variable named by $\text{decode}(v) : I$, whose value type is $\text{Val}(\text{decode}(v))$. The round-trip proofs ensure that this indexed presentation contains every variable name exactly once.

Node selection. A finite node selection, called a *node set* in conventional causal-graph terminology, is a Boolean classifier:

$$S : V \rightarrow \text{Bool}.$$

Membership $v \in S$ abbreviates $S(v) = \text{true}$. For selections $S, T : V \rightarrow \text{Bool}$, union, intersection and difference are defined pointwise using the derived Boolean operations of Definition A.1.20:

$$\begin{aligned} (S \cup T)(v) &:= \text{or}_{\text{Bool}}(S(v), T(v)), \\ (S \cap T)(v) &:= \text{and}_{\text{Bool}}(S(v), T(v)), \\ (S \setminus T)(v) &:= \text{and}_{\text{Bool}}(S(v), \text{not}_{\text{Bool}}(T(v))). \end{aligned}$$

The empty and full selections are the constant classifiers:

$$\emptyset_V(v) := \text{false}, \quad \text{All}_V(v) := \text{true}.$$

A. Definitions

When the ambient node type is clear, the subscript on $\emptyset_V$ is omitted.

Selected-family presentation. Retain exactly those formal nodes $v : V$ for which $S(v) = \text{true}$, in their ambient index order. Filtering the canonical enumeration $0, \dots, n-1$ gives an embedding:

$$j_S : \text{Fin}(m_S) \rightarrow V, \quad j_S(k) = j_S(\ell) \rightarrow k = \ell.$$

Here $\text{Fin}(m_S)$ indexes the selected positions, while j_S identifies the formal node at each position. Because the filtered list retains the ambient order, this embedding also reflects strict order:

$$h_{\text{reflect}}^S : \Pi_{k, \ell : \text{Fin}(m_S)} \text{rank}_n(j_S(k)) <_{\mathbb{N}} \text{rank}_n(j_S(\ell)) \rightarrow \text{rank}_{m_S}(k) <_{\mathbb{N}} \text{rank}_{m_S}(\ell).$$

This field will allow an induced restriction to inherit the chosen topological order of its ambient graph. Composing j_S with the decoding map gives the selected variable-name embedding:

$$\bar{j}_S : \text{Fin}(m_S) \rightarrow I, \quad \bar{j}_S(k) := \text{decode}(j_S(k)),$$

which is the selected-subfamily data used in the cylinder-event construction of Definition A.3.16. Its joint-assignment type, coordinate projection and lifted events are exactly the constructions of that definition applied to $\bar{j}_S$. Pure graph definitions use only S and j_S . Conversely, finite search converts an embedding $j : \text{Fin}(m) \rightarrow V$ into the Boolean classifier that returns **true** exactly on its image. The constructive correspondence between these presentations is proved in Proposition B.6.2.

Example. Let I_{Mira} be the type of eight variable names

Pr, Qu, Top, Com, Fit, Short, Fund, Off.

Enumerating them in the displayed order gives the formal node type $V_{\text{Mira}} := \text{Fin}(8)$. Below, the same names also abbreviate their encoded inhabitants of V_{Mira} . For example, selecting Pr and Qu gives a classifier $S_{\text{PQ}} : V_{\text{Mira}} \rightarrow \text{Bool}$ and an order-preserving embedding $j_{\text{PQ}} : \text{Fin}(2) \rightarrow V_{\text{Mira}}$.

Definition A.4.2 (Edges). Directed and bidirected edge packages record the two kinds of graphical connection used below.

Data. Fix the formal node type $V = \text{Fin}(n)$ of Definition A.4.1.

Directed classifier. A directed edge relation is a Boolean classifier:

$$E^{\rightarrow} : V \times V \rightarrow \text{Bool}.$$

Write $X \rightarrow_E Y$ when $E^{\rightarrow}(X, Y) = \text{true}$. This says that there is a directed edge from X to Y ; it does not yet impose an order on the two nodes.

When an ordered directed relation is required, supply evidence of type:

$$\text{Topological}(E^{\rightarrow}) := \Pi_{X, Y : V} (E^{\rightarrow}(X, Y) = \text{true}) \rightarrow \text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(Y),$$

where rank_n and $<_{\mathbb{N}}$ are defined in Remark A.1.18.

Ordered directed package. Define $\text{OrderedDirectedEdges}(V)$ as a record type with fields:

$$\begin{aligned} E^{\rightarrow} &: V \times V \rightarrow \text{Bool}, \\ h_{\text{top}} &: \text{Topological}(E^{\rightarrow}). \end{aligned}$$

Thus h_{top} certifies that every selected directed edge points from an earlier node to a later one. A bare, directed classifier need not admit such evidence.

Bidirected package. Define $\text{BidirectedEdges}(V)$ as the record type with fields:

$$\begin{aligned} E^{\leftrightarrow} &: V \times V \rightarrow \text{Bool}, \\ h_{\text{sym}} &: \Pi_{X,Y:V} (E^{\leftrightarrow}(X,Y) = \text{true}) \rightarrow (E^{\leftrightarrow}(Y,X) = \text{true}), \\ h_{\text{irr}} &: \Pi_{X:V} E^{\leftrightarrow}(X,X) = \text{false}. \end{aligned}$$

Write $X \leftrightarrow_E Y$ when $E^{\leftrightarrow}(X,Y) = \text{true}$.

Adjacency. Let $D : \text{OrderedDirectedEdges}(V)$ and $B : \text{BidirectedEdges}(V)$. Directed adjacency forgets arrow orientation:

$$\text{Adj}_D^{\rightarrow}(X,Y) := \text{or}_{\text{Bool}}(E_D^{\rightarrow}(X,Y), E_D^{\rightarrow}(Y,X)).$$

Full adjacency additionally includes bidirected edges:

$$\text{Adj}_{D,B}(X,Y) := \text{or}_{\text{Bool}}(\text{Adj}_D^{\rightarrow}(X,Y), E_B^{\leftrightarrow}(X,Y)).$$

It returns **true** exactly when the two nodes are joined by a directed edge in either orientation or by a bidirected edge.

Proof-valued steps. For any Boolean relation $R : V \times V \rightarrow \text{Bool}$, define:

$$\text{Step}_R(X,Y) := \text{Id}_{\text{Bool}}(R(X,Y), \text{true}).$$

An inhabitant of $\text{Step}_R(X,Y)$ is evidence that R selects the ordered pair (X,Y) . If R is irreflexive, such evidence also entails that its endpoints are distinct.

Example. On V_{Mira} from Definition A.4.1, let $E_{\text{Mira}}^{\rightarrow}$ return **true** exactly on:

$$\begin{aligned} &(\text{Pr}, \text{Short}), \quad (\text{Qu}, \text{Short}), \quad (\text{Top}, \text{Fit}), \quad (\text{Com}, \text{Fit}), \\ &(\text{Fit}, \text{Short}), \quad (\text{Top}, \text{Fund}), \quad (\text{Short}, \text{Off}), \quad (\text{Fund}, \text{Off}). \end{aligned}$$

The displayed node order supplies $h_{\text{top}}^{\text{Mira}}$, so these data form $D_{\text{Mira}} : \text{OrderedDirectedEdges}(V_{\text{Mira}})$. Let $E_{\text{Mira}}^{\leftrightarrow}$ return **true** exactly on (Pr, Qu) and (Qu, Pr) . Its evident symmetry and irreflexivity form $B_{\text{Mira}} : \text{BidirectedEdges}(V_{\text{Mira}})$. These packages determine the directed, bidirected and adjacency relations used in the following examples; their observed graph is displayed in Figure A.1.

A. Definitions

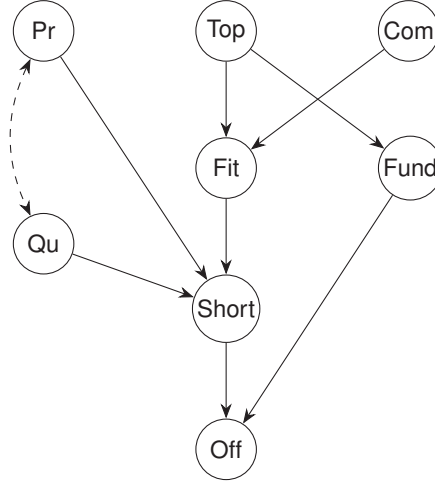


Figure A.1: The observed directed and bidirected edge data of the running Mira graph. The dashed double-headed line is the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$.

Definition A.4.3 (Chains, forks and colliders). Chains, forks and colliders are local configurations of three nodes in an ordered directed-edge package.

Data. Let $D : \text{OrderedDirectedEdges}(V)$ and let $A, B, C : V$. Using the proof-valued steps of Definition A.4.2, define:

$$\begin{aligned}
 \text{Chain}_D(A, B, C) &:= (\text{Step}_{E_D^{\rightarrow}}(A, B) \times \text{Step}_{E_D^{\rightarrow}}(B, C)) \\
 &\quad + (\text{Step}_{E_D^{\rightarrow}}(C, B) \times \text{Step}_{E_D^{\rightarrow}}(B, A)), \\
 \text{Fork}_D(A, B, C) &:= \text{Step}_{E_D^{\rightarrow}}(B, A) \times \text{Step}_{E_D^{\rightarrow}}(B, C), \\
 \text{Collider}_D(A, B, C) &:= \text{Step}_{E_D^{\rightarrow}}(A, B) \times \text{Step}_{E_D^{\rightarrow}}(C, B).
 \end{aligned}$$

Their graphical readings are:

- *chain*: $A \rightarrow B \rightarrow C$ or $A \leftarrow B \leftarrow C$;
- *fork*: $A \leftarrow B \rightarrow C$; and
- *collider*: $A \rightarrow B \leftarrow C$.

These three configurations are displayed in Figure A.2. Chains and forks are non-collider configurations; colliders behave differently under conditioning.

Example. For D_{Mira} , the fragments

$$\text{Top} \rightarrow \text{Fit} \rightarrow \text{Short}, \quad \text{Fit} \leftarrow \text{Top} \rightarrow \text{Fund}, \quad \text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$$

inhabit respectively a chain, a fork and a collider type.

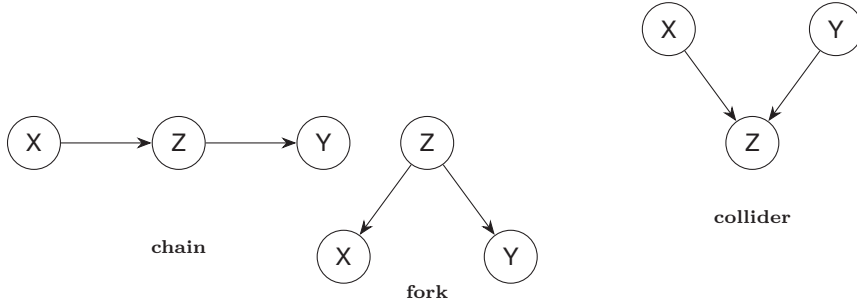


Figure A.2: The three local directed configurations of Definition A.4.3. The middle node is a non-collider in a chain or fork and a collider in the final configuration.

Definition A.4.4 (Paths). A path packages a finite node sequence together with evidence for its endpoints and every consecutive relation step.

Data. Fix the formal node type V of Definition A.4.1. Let $R : V \times V \rightarrow \text{Bool}$ temporarily denote any Boolean relation. It may later be instantiated by a directed classifier, a bidirected classifier or a full adjacency classifier.

Consecutive-step evidence. Using list elimination from Definition A.1.23, define the type family:

$$\text{Consecutive}_R : \text{List}(V) \rightarrow \mathcal{U}$$

by:

$$\begin{aligned} \text{Consecutive}_R(\text{nil}) &:= \text{Fin}(1), \\ \text{Consecutive}_R(\text{cons}(X, \text{nil})) &:= \text{Fin}(1), \\ \text{Consecutive}_R(\text{cons}(X, \text{cons}(Y, p))) &:= \text{Step}_R(X, Y) \times \text{Consecutive}_R(\text{cons}(Y, p)). \end{aligned}$$

The empty and singleton lists contain no consecutive pair that could fail the relation, so their evidence types are inhabited. For a longer list, an inhabitant proves that the first pair passes R and that every pair in the remaining list does likewise. Each recursive call removes the first entry of that finite list. Evaluation therefore terminates for every choice of R ; no termination property of R is required.

Relation paths. For $X, Y : V$, define the record type $\text{Path}_R(X, Y)$ with fields:

$$\begin{aligned} \text{nodes} &: \text{List}(V), \\ h_{\text{start}} &: \text{head}?(nodes) = \text{some}(X), \\ h_{\text{end}} &: \text{last}?(nodes) = \text{some}(Y), \\ h_{\text{step}} &: \text{Consecutive}_R(nodes). \end{aligned}$$

The endpoint fields use the optional list endpoints of Remark A.1.24; either equation entails that the node list is nonempty. A singleton list consequently gives a trivial path from a node to itself and contains no edge. For $q : \text{Path}_R(X, Y)$, named field projection in the sense of Remark A.1.11 gives $\text{nodes}(q) : \text{List}(V)$.

A. Definitions

Positive paths. When a path must contain at least one edge, define $\text{Path}_R^+(X, Y)$ as the record type with fields:

$$\begin{aligned} &\text{path} : \text{Path}_R(X, Y), \\ &\text{first} : V, \\ &\text{second} : V, \\ &\text{rest} : \text{List}(V), \\ &h_{\text{shape}} : \text{Id}_{\text{List}(V)} \left(\text{nodes}(\text{path}), \text{cons}(\text{first}, \text{cons}(\text{second}, \text{rest})) \right). \end{aligned}$$

Thus a positive path contains an ordinary path and evidence that its node sequence has at least two entries. The field **rest** is the possibly empty list of all nodes after the required first two; it is **nil** when the path contains exactly one edge. Transporting $h_{\text{step}}(\text{path})$ along h_{shape} exposes an inhabitant of $\text{Step}_R(\text{first}, \text{second})$ and the consecutive evidence for the remaining list.

Edge-specific paths. For $D : \text{OrderedDirectedEdges}(V)$ and $B : \text{BidirectedEdges}(V)$, write:

$$\begin{aligned} \text{Path}_D^{\rightarrow}(X, Y) &:= \text{Path}_{E_D^{\rightarrow}}(X, Y), \\ \text{Path}_B^{\leftrightarrow}(X, Y) &:= \text{Path}_{E_B^{\leftrightarrow}}(X, Y), \\ \text{Path}_{D,B}^{\text{adj}}(X, Y) &:= \text{Path}_{\text{Adj}_{D,B}}(X, Y). \end{aligned}$$

The corresponding positive forms are obtained by replacing **Path** with Path^+ .

Example. The node sequence **Top**, **Fit**, **Short**, **Off** underlies a positive directed path from **Top** to **Off** in D_{Mira} . The sequence **Pr**, **Qu** gives a positive bidirected path in B_{Mira} , while **Pr**, **Qu**, **Short**, **Off** gives a positive full-adjacency path.

Definition A.4.5 (Parents, ancestors and descendants). Parents, ancestors and descendants depend only on ordered directed edges and their paths.

Data. Fix $D : \text{OrderedDirectedEdges}(V)$ and let $Y : V$. Each family below is a finite node selection on V in the sense of Definition A.4.1.

Parents. The parent classifier selects exactly the nodes with a directed edge into Y :

$$X \in \text{Pa}_D(Y) \iff E_D^{\rightarrow}(X, Y) = \text{true}.$$

Ancestors and descendants. Using the positive path type of Definition A.4.4, define:

$$X \in \text{An}_D(Y) \iff X = Y \text{ or } \text{Path}_{E_D^{\rightarrow}}^+(X, Y) \text{ is inhabited,}$$

and:

$$X \in \text{De}_D(Y) \iff X = Y \text{ or } \text{Path}_{E_D^{\rightarrow}}^+(Y, X) \text{ is inhabited.}$$

Thus Y is its own ancestor and descendant; the positive path clause handles all other members.

Decidability. Equality and finite directed reachability are decidable, so these conditions determine Boolean classifiers. The checked finite reachability construction is recorded in Proposition B.6.8.

Example. For D_{Mira} :

$$\text{Pa}_{D_{\text{Mira}}}(\text{Short}) = \{\text{Pr}, \text{Qu}, \text{Fit}\}.$$

Moreover, **Top** is an ancestor of **Off** and **Off** is a descendant of **Top**.

Definition A.4.6 (Directed acyclic graph). The ordered directed-edge package already contains exactly the data required by the theorem-facing directed-acyclic-graph presentation.

Definition. For a formal node type $V = \text{Fin}(n)$, define:

$$\text{DAG}(V) := \text{OrderedDirectedEdges}(V).$$

Hence an inhabitant $D = (E_D^\rightarrow, h_{\text{top}}^D) : \text{DAG}(V)$ contains both its directed classifier and its chosen topological-order evidence.

Irreflexivity and acyclicity. The topological witness determines:

$$h_{\text{irr}}^\rightarrow : \Pi_{X:V} E_D^\rightarrow(X, X) = \text{false}.$$

A true self-edge would produce $\text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(X)$, contradicting irreflexivity of strict natural-number order. Similarly, a positive directed path from X to itself would make the rank strictly increase along every edge and, by transitivity, yield the same contradiction. These constructive consequences are proved in Proposition B.6.1. Because every finite cycle-free directed graph admits a topological ordering, this ordered presentation covers the usual finite directed acyclic graphs.

Example. The package D_{Mira} constructed in Definition A.4.2 inhabits $\text{DAG}(V_{\text{Mira}})$.

Lean implementation. Lean distributes this package across the fields `count`, `directed` and `directed_earlier` of `ObservedSignature`. The last field is used by recursive structural evaluation to obtain each parent value from an earlier index.

Definition A.4.7 (Causal graph). A finite causal graph combines an ordered directed part and a bidirected part over the same formal node type.

Data. Fix a formal node type V . Define $\text{CausalGraph}(V)$ as the record type with fields:

$$\begin{aligned} \text{directed} &: \text{DAG}(V), \\ \text{bidirected} &: \text{BidirectedEdges}(V). \end{aligned}$$

For $\mathcal{G} : \text{CausalGraph}(V)$, write $D_{\mathcal{G}} := \text{directed}(\mathcal{G})$ and $B_{\mathcal{G}} := \text{bidirected}(\mathcal{G})$. Their classifier projections are denoted $E_{\mathcal{G}}^\rightarrow := E_{D_{\mathcal{G}}}^\rightarrow$ and $E_{\mathcal{G}}^{\leftrightarrow} := E_{B_{\mathcal{G}}}^{\leftrightarrow}$.

Adjacency and paths. Define:

$$\text{Adj}_{\mathcal{G}} := \text{Adj}_{D_{\mathcal{G}}, B_{\mathcal{G}}}, \quad \text{Path}_{\mathcal{G}}(X, Y) := \text{Path}_{\text{Adj}_{\mathcal{G}}}(X, Y).$$

The first is the full adjacency classifier; the second is the graph-path type of Definition A.4.4 that may traverse either kind of edge while ignoring orientation.

Graphical cases. A graph is *Markovian* when $E_{\mathcal{G}}^{\leftrightarrow}$ is constantly `false`. It is *semi-Markovian* when the bidirected package may select symmetric, irreflexive bidirected edges. Thus a directed acyclic graph is the directed part of either causal-graph case, while a causal graph additionally records whether bidirected connections are present.

Example. Define $\mathcal{G}_{\text{Mira}} : \text{CausalGraph}(V_{\text{Mira}})$ by:

$$\begin{aligned} \text{directed}(\mathcal{G}_{\text{Mira}}) &:= D_{\text{Mira}}, \\ \text{bidirected}(\mathcal{G}_{\text{Mira}}) &:= B_{\text{Mira}}. \end{aligned}$$

A. Definitions

Its nonempty bidirected classifier makes it semi-Markovian. The definitions through Section A.8 reuse this graph.

Lean implementation. Lean makes the same division. An **ObservedSignature** contains the formal node and value types and the ordered directed part; an **ObservedGraph** over that signature contains the bidirected part.

Definition A.4.8 (Induced finite causal restriction). An induced restriction selects part of a causal graph, retains every directed or bidirected edge whose two endpoints remain selected, and ignores all other nodes and incident edges.

Data. Let $\mathcal{G} : \text{CausalGraph}(V)$. A finite node classifier $W : V \rightarrow \text{Bool}$ in the sense of Definition A.4.1 determines an embedding:

$$j_W : \text{Fin}(m_W) \rightarrow V$$

and order-reflection evidence h_{reflect}^W . The latter converts an ambient strict-rank comparison between selected nodes into the corresponding comparison between their filtered indices.

Construction. The induced restriction $\mathcal{G} \mid W$ has formal node type $\text{Fin}(m_W)$ and classifiers:

$$\begin{aligned} E_{\mathcal{G} \mid W}^{\rightarrow}(k, \ell) &:= E_{\mathcal{G}}^{\rightarrow}(j_W(k), j_W(\ell)), \\ E_{\mathcal{G} \mid W}^{\leftrightarrow}(k, \ell) &:= E_{\mathcal{G}}^{\leftrightarrow}(j_W(k), j_W(\ell)). \end{aligned}$$

Its topological evidence is:

$$h_{\text{top}}^{\mathcal{G} \mid W}(k, \ell, e) := h_{\text{reflect}}^W(h_{\text{top}}^{D_{\mathcal{G}}}(j_W(k), j_W(\ell), e)),$$

where e proves $E_{\mathcal{G} \mid W}^{\rightarrow}(k, \ell) = \text{true}$. Bidirected symmetry and irreflexivity are inherited by applying the corresponding fields of $B_{\mathcal{G}}$ at $j_W(k)$ and $j_W(\ell)$. These data therefore form a causal graph on $\text{Fin}(m_W)$.

Example. The restriction of $\mathcal{G}_{\text{Mira}}$ to $\{\text{Pr}, \text{Qu}\}$ retains their bidirected edge. Its restriction to $\{\text{Top}, \text{Fit}, \text{Com}\}$ retains the two directed arrows into Fit .

Causal models and distributions

The remaining definitions build a structural causal model in stages. The finitary setting is fixed throughout: endogenous and exogenous variables receive explicit finite value types, the exogenous source carries a finite product probability assignment, and an all-directed causal skeleton records which inputs each structural mechanism may read.

Definition A.4.9 (Endogenous signature). An endogenous signature assigns an inhabited finite value type to every node of an ordered directed graph.

Node and value types. Let $V := \text{Fin}(n)$ be a formal node type in the sense of Definition A.4.1. Provide a value-type family and its finiteness evidence in the sense of Definition A.1.19:

$$\begin{aligned} \text{Val} : V &\rightarrow \mathcal{U}, \\ h_{\text{finite}} &: \Pi_{i:V} \text{Finite}(\text{Val}(i)). \end{aligned}$$

Finiteness alone does not imply inhabitation: a finite type may be empty. Therefore also provide a default value at every node:

$$\text{default} : \Pi_{i:V} \text{Val}(i).$$

Each finiteness witness supplies a complete duplicate-free enumeration and decidable equality by Definition A.1.19 and Remark A.1.36.

Directed structure. Provide a directed parent graph:

$$D : \text{DAG}(V).$$

The joint endogenous-assignment type is:

$$\text{Val}(\mathcal{V}) := \prod_{i:V} \text{Val}(i).$$

Signature record. Define $\text{EndogenousSignature}(V)$ as the record type with fields:

$$\begin{aligned} \text{Val} &: V \rightarrow \mathcal{U}, \\ h_{\text{finite}} &: \prod_{i:V} \text{Finite}(\text{Val}(i)), \\ \text{default} &: \prod_{i:V} \text{Val}(i), \\ D &: \text{DAG}(V). \end{aligned}$$

Interpretation. The graph D alone records only formal nodes, directed edges and their topological order. The endogenous signature additionally specifies what value each node may take and how it can be initialized. It is therefore a typed directed acyclic graph, not merely a directed acyclic graph. The endogenous signature records only directed mechanism dependencies. Bidirected edges are not additional inputs to an endogenous mechanism.

Running example. Mira’s endogenous node type is V_{Mira} from Definition A.4.1; each of its eight nodes has value type Bool . Its directed parent structure is D_{Mira} from Definition A.4.2.

Lean implementation. These data form **ObservedSignature**. For executable finite search, Lean stores a complete duplicate-free enumeration and decidable equality for each value type explicitly. The finiteness witness above constructively supplies those fields; Lean’s final two fields provide the directed classifier and topological-order evidence of D .

Definition A.4.10 (Exogenous source). An exogenous source specifies the possible values of the background variables and their joint finite probability assignment before any structural mechanism is evaluated.

Root and value types. Let $U := \text{Fin}(r)$ be a formal node type in the sense of Definition A.4.1, now used to index the exogenous roots rather than the observed nodes. Provide a finite value type for every root:

$$\begin{aligned} \text{Val}_{\mathcal{U}} &: U \rightarrow \mathcal{U}, \\ h_{\text{finite}}^{\mathcal{U}} &: \prod_{\ell:U} \text{Finite}(\text{Val}_{\mathcal{U}}(\ell)). \end{aligned}$$

As in the endogenous case, these finiteness witnesses supply the enumerations and equality procedures needed for finite computation. The joint exogenous-assignment type is:

$$\text{Val}(\mathcal{U}) := \prod_{\ell:U} \text{Val}_{\mathcal{U}}(\ell).$$

Coordinate factors. For every $\ell : U$, provide an urn-generated probability assignment:

$$\nu_{\ell} : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}_{\mathcal{U}}(\ell)).$$

Joint assignment. Provide an urn-generated assignment:

$$\mu_{\mathcal{U}} : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{U})).$$

A. Definitions

Rectangular events. The exogenous roots U , together with their value-type assignment $\text{Val}_{\mathcal{U}}$, form a finite family of variables in the sense of Definition A.3.15. Rectangular events on $\text{Val}(\mathcal{U})$ are those of Remark A.3.18.

Product law. Require evidence:

$$h_{\text{prod}} : \prod_{A : \prod_{\ell : U} (\text{Val}_{\mathcal{U}}(\ell) \rightarrow \text{Bool})} P_{\mu_{\mathcal{U}}}(\text{Rect}_A) \sim \prod_{\ell : U} P_{\nu_{\ell}}(A_{\ell}).$$

It states that every rectangular event has the probability obtained by multiplying its coordinate probabilities. This is the finite, proof-carrying independence condition imposed on the exogenous roots.

Source record. Define $\text{ExogenousSource}(U)$ as the record type with fields:

$$\begin{aligned} \text{Val}_{\mathcal{U}} &: U \rightarrow \mathcal{U}, \\ h_{\text{finite}}^{\mathcal{U}} &: \prod_{\ell : U} \text{Finite}(\text{Val}_{\mathcal{U}}(\ell)), \\ \nu &: \prod_{\ell : U} \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}_{\mathcal{U}}(\ell)), \\ \mu_{\mathcal{U}} &: \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{U})), \\ h_{\text{prod}} &: \prod_{A : \prod_{\ell : U} (\text{Val}_{\mathcal{U}}(\ell) \rightarrow \text{Bool})} P_{\mu_{\mathcal{U}}}(\text{Rect}_A) \sim \prod_{\ell : U} P_{\nu_{\ell}}(A_{\ell}). \end{aligned}$$

The coordinate assignments ν_{ℓ} already witness that their value types are inhabited, so no separate default values are required.

Running example. Mira's exogenous source has six Boolean roots:

$$U_B, U_{\text{Pr}}, U_{\text{Qu}}, U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}.$$

Their coordinate factors satisfy:

$$\begin{aligned} P(U_B = \text{true}) &= \frac{1}{3}, \\ P(U_{\text{Pr}} = \text{true}) &= P(U_{\text{Qu}} = \text{true}) = P(U_{\text{Fund}} = \text{true}) = \frac{1}{4}, \\ P(U_{\text{Top}} = \text{true}) &= P(U_{\text{Com}} = \text{true}) = \frac{1}{2}. \end{aligned}$$

The joint source is their finite product urn.

Lean implementation. Lean distributes this conceptual package across **LatentExtension** and **FiniteLatentSCM**: the former stores the exogenous value family and explicit finite operations derived from its finiteness witnesses, while the latter stores **factor**, **prior** and **product_law**. The common-denominator representation used there is equivalent to the urn representation by Proposition B.5.10.

Definition A.4.11 (Latent causal skeleton). A latent causal skeleton combines an endogenous signature and an exogenous source with the directed incidence of exogenous roots into endogenous nodes. It contains no structural mechanisms yet.

Data. Let $\mathcal{V}$ be an endogenous signature in the sense of Definition A.4.9, with node type V and directed graph D . Let $\mathcal{U}$ be an exogenous source in the sense of Definition A.4.10, with root type U . Supply the Boolean incidence classifier:

$$L : U \times V \rightarrow \text{Bool}.$$

It records which exogenous roots may be read by each endogenous mechanism. An incidence is certified by an inhabitant of the type:

$$\text{Incident}_L(\ell, i) := \text{Id}_{\text{Bool}}(L(\ell, i), \text{true}).$$

For $i : V$, the values made available by the incidence classifier form the dependent input type:

$$\text{ExogenousInputs}(i) := \Pi_{\ell:U} (L(\ell, i) = \text{true}) \rightarrow \text{Val}_{\mathcal{U}}(\ell).$$

A complete exogenous assignment is written in boldface:

$$\mathbf{u} : \text{Val}(\mathcal{U}).$$

Its restriction to the inputs of i is the function:

$$\begin{aligned} \text{restrict}_i^{\mathcal{U}}(\mathbf{u}) &: \text{ExogenousInputs}(i), \\ \text{restrict}_i^{\mathcal{U}}(\mathbf{u})(\ell)(e) &:= \mathbf{u}_{\ell}. \end{aligned}$$

Here $\ell : U$ names one exogenous root, whereas $\mathbf{u}$ is a complete assignment of values to all roots. The evidence e certifies that the requested coordinate is incident to i ; if it is not, no such evidence can be supplied and the mechanism cannot request that value.

Skeleton record. These components form the record type:

$$\text{LatentCausalSkeleton}(V, U),$$

with fields:

$$\begin{aligned} \text{endogenous} &: \text{EndogenousSignature}(V), \\ \text{exogenous} &: \text{ExogenousSource}(U), \\ \text{incident} &: U \times V \rightarrow \text{Bool}. \end{aligned}$$

Thus the data introduced above determine:

$$\mathcal{K} := (\mathcal{V}, \mathcal{U}, L) : \text{LatentCausalSkeleton}(V, U).$$

The exogenous-input types and restriction functions are derived operations on this record; they are not additional fields. For $\mathcal{K} : \text{LatentCausalSkeleton}(V, U)$, write $\text{Val}_{\mathcal{K}}$ for the value-type family projected from its endogenous signature, and write $\text{ExogenousInputs}_{\mathcal{K}}(i)$ and $\text{restrict}_{\mathcal{K},i}^{\mathcal{U}}$ for the derived constructions above.

All-directed graph. On the combined node type $U + V$, define an edge from $\text{inl}(\ell)$ to $\text{inr}(i)$ exactly when $L(\ell, i) = \text{true}$, and an edge from $\text{inr}(i)$ to $\text{inr}(j)$ exactly when $E_{\vec{D}}(i, j) = \text{true}$. All other cases return **false**, so no edge enters an exogenous root. Giving every exogenous root rank 0 and every endogenous node i rank $\text{rank}_n(i) + 1$ supplies an acyclicity ranking: the rank strictly increases along every edge, although distinct exogenous roots may share rank 0.

Interpretation and scope. An endogenous node may read no exogenous root, one root or several roots; conversely, one exogenous root may feed several endogenous nodes. The last case represents a shared source of variation. In the model class used here, endogenous nodes are observed and the explicit exogenous roots are unobserved. Observability and the endogenous–exogenous distinction need not coincide in more general structural-model presentations.

A. Definitions

Running example. In Mira’s skeleton, the shared root is incident to both prestige and quality:

$$L(U_B, \text{Pr}) = \text{true}, \quad L(U_B, \text{Qu}) = \text{true}.$$

Each private root is incident only to its correspondingly named endogenous node. Together with D_{Mira} , these incidences give the all-directed skeleton in Figure A.3.

Lean implementation. The exogenous value data and incidence classifier form **LatentExtension**; its **expandedEdge** combines them with the directed part of **ObservedSignature** in exactly the two nonempty cases above. Probability fields are added by **FiniteLatentSCM**, as described in Definition A.4.10.

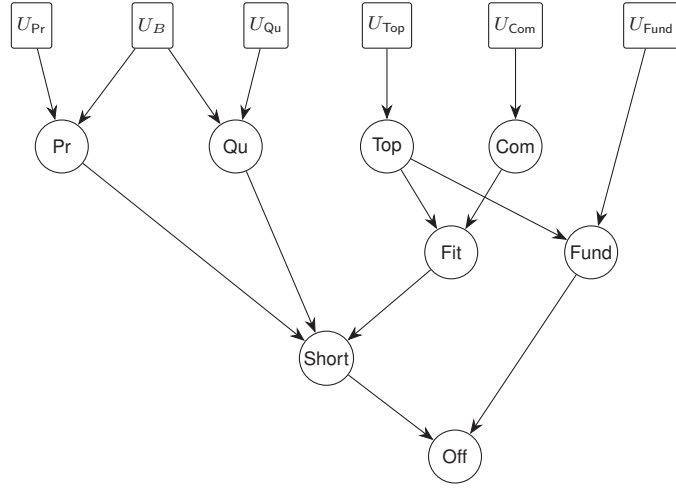


Figure A.3: The explicit all-directed graph of Mira’s latent causal skeleton. Rectangles are exogenous roots and circles are endogenous nodes. The shared root U_B is incident to both Pr and Qu ; no bidirected edge is primitive in this representation.

Definition A.4.12 (Latent projection). Latent projection hides the explicit exogenous roots of a latent causal skeleton and records their observable graphical consequences.

Data. Let $\mathcal{K}$ be a latent causal skeleton in the sense of Definition A.4.11, with endogenous graph D , exogenous-root index type U and incidence classifier $L : U \times V \rightarrow \text{Bool}$.

Projection. Define the observed causal graph in the sense of Definition A.4.7:

$$\pi(\mathcal{K}) : \text{CausalGraph}(V)$$

by retaining the directed package:

$$D_{\pi(\mathcal{K})} := D,$$

and defining its bidirected classifier by:

$$E_{\pi(\mathcal{K})}^{\leftrightarrow}(i, j) = \text{true} \iff i \neq j \wedge \Sigma_{\ell:U} \text{Incident}_L(\ell, i) \times \text{Incident}_L(\ell, j).$$

Thus two distinct endogenous nodes are joined bidirectionally exactly when they share an explicit exogenous root.

Interpretation. This operation depends only on the skeleton. It does not inspect the exogenous probabilities or structural mechanisms, and its result is an observed causal graph rather than a latent-free structural causal model. It is not an induced restriction in the sense of Definition A.4.8: hiding a shared root creates a bidirected edge between its observed children.

Reverse graph expansion. Conversely, a bidirected edge in an observed causal graph can be represented by a fresh hidden root. For every selected pair $i \leftrightarrow j$, introduce a node $U_{i,j}$ and the directed edges:

$$U_{i,j} \rightarrow i, \quad U_{i,j} \rightarrow j.$$

Hiding these fresh roots recovers the original bidirected classifier, as proved in Proposition B.6.3. This reverse construction supplies only nodes and edges; it does not recover the value types, probabilities or mechanisms of an original hidden model. Those data must be supplied separately before the expansion becomes a structural causal model.

Running example. Mira's root U_B is incident to Pr and Qu , so its projection produces the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$ displayed in Figures A.1 and 5.3. No other root in Mira's skeleton is incident to two endogenous nodes.

Lean implementation. In the namespace `LatentExtension`, the construction is `observedGraph` and its bidirected classifier is `projectedBidirected`. Symmetry and irreflexivity are proved when the resulting `ObservedGraph` is constructed. The direct finite projection results are stated in Proposition B.12.2.

Definition A.4.13 (Structural mechanisms). The latent causal skeleton determines exactly which inputs each endogenous mechanism may inspect.

Data. Let

$$\mathcal{K} : \text{LatentCausalSkeleton}(V, U)$$

be a skeleton in the sense of Definition A.4.11. Write $\mathcal{V}$, $\mathcal{U}$ and L for its endogenous signature, exogenous source and incidence classifier, and write D , Val and $\text{Val}_{\mathcal{U}}$ for the corresponding projected data.

Input types. For $c : V$, define the endogenous inputs and combine them with the exogenous-input type of Definition A.4.11:

$$\begin{aligned} \text{EndogenousInputs}_{\mathcal{K}}(c) &:= \Pi_{p:V} (E_D^{\rightarrow}(p, c) = \text{true}) \rightarrow \text{Val}(p), \\ \text{MechanismInputs}_{\mathcal{K}}(c) &:= \text{EndogenousInputs}_{\mathcal{K}}(c) \times \text{ExogenousInputs}_{\mathcal{K}}(c). \end{aligned}$$

The first component supplies values only for declared endogenous parents; the second supplies values only for declared incident roots. The parent classifier $\text{Pa}_D(c)$ of Definition A.4.5 selects the first family of coordinates, while $\text{EndogenousInputs}_{\mathcal{K}}(c)$ supplies one value for every selected parent. It therefore contains the same parent-value data as $\text{Val}(\text{Pa}_D(c))$, written in the dependent function form used by the structural evaluator.

Mechanism family. A finite structural mechanism family is:

$$\mathcal{F} : \Pi_{c:V} \text{MechanismInputs}_{\mathcal{K}}(c) \rightarrow \text{Val}(c).$$

Write $f_c := \mathcal{F}(c)$ for the mechanism at c .

A. Definitions

Running example. Mira’s complete Boolean mechanism family is:

$$\begin{aligned} f_{\text{Pr}} &:= U_B \vee U_{\text{Pr}}, & f_{\text{Qu}} &:= U_B \vee U_{\text{Qu}}, \\ f_{\text{Top}} &:= U_{\text{Top}}, & f_{\text{Com}} &:= U_{\text{Com}}, \\ f_{\text{Fit}} &:= \text{Top} \wedge \text{Com}, & f_{\text{Short}} &:= \text{Qu} \wedge (\text{Pr} \vee \text{Fit}), \\ f_{\text{Fund}} &:= \text{Top} \vee U_{\text{Fund}}, & f_{\text{Off}} &:= \text{Short} \wedge \text{Fund}. \end{aligned}$$

The first two mechanisms can read the shared root because the incidence evidence displayed in Definition A.4.11 supplies those inputs. In each of those two equations, either the shared root or the private root suffices to make the output true. The topic and committee mechanisms copy their private roots. The remaining four mechanisms combine only the endogenous parents and, where applicable, the private root selected by the skeleton. These eight equations therefore define f_c for every node $c : V_{\text{Mira}}$ used by the solution map below.

Lean implementation. In the namespaces `ObservedSignature` and `LatentExtension`, the two input types are respectively `ParentValues` and `Inputs`. The field `mechanism` of `FiniteLatentSCM` receives them in the equivalent curried form `ParentValues → Inputs → Value`; pairing and currying these inputs are mutually definable without any classical principle.

Definition A.4.14 (Structural causal model). In this thesis, a structural causal model is a finite latent-root model obtained by equipping a latent causal skeleton with one structural mechanism for every endogenous node.

Model record. Define $\text{SCM}(V, U)$ as the dependent record type with fields:

$$\begin{aligned} \text{skeleton} &: \text{LatentCausalSkeleton}(V, U), \\ \text{mechanism} &: \prod_{c:V} \text{MechanismInputs}_{\text{skeleton}}(c) \rightarrow \text{Val}_{\text{skeleton}}(c). \end{aligned}$$

Thus a skeleton $\mathcal{K}$ and a mechanism family $\mathcal{F}$ over that skeleton form the model:

$$\mathcal{M} = (\mathcal{K}, \mathcal{F}) : \text{SCM}(V, U).$$

Running example. Mira’s endogenous signature, six-root exogenous source and incidence classifier form the skeleton displayed in Figure A.3. Equipping that skeleton with the Boolean equations of Definition A.4.13 gives Mira’s structural causal model.

Lean implementation. These data form `FiniteLatentSCM`, abbreviated `ExactModel` in the transport development. Lean flattens the exogenous source, incidence classifier and mechanism family into this model and its `LatentExtension` field. Its explicit enumerations and common-denominator probability records carry information equivalent to the finiteness witnesses and urn assignments of the staged presentation above.

Definition A.4.15 (Solution map). The ordered directed signature makes every finite latent-root model recursively evaluable.

Prefix types. Let $\mathcal{M}$ be a model in the sense of Definition A.4.14, and write $\mathcal{K}$ for its latent causal skeleton. Its node type is $V = \text{Fin}(n)$. For $k \leq n$, let $\iota_k : \text{Fin}(k) \rightarrow V$ be the canonical inclusion and define the assignment type for the first k nodes:

$$\text{Prefix}_V(k) := \prod_{j:\text{Fin}(k)} \text{Val}(\iota_k(j)).$$

The empty prefix is the unique inhabitant $\star : \text{Prefix}_V(0)$.

Typed evaluation step. Fix a complete exogenous assignment $\mathbf{u} : \text{Val}(\mathcal{U})$. Suppose $k < n$, let $i_k : V$ be the node with index k , and let $\mathbf{s} : \text{Prefix}_{\mathcal{V}}(k)$ be an assignment to the already evaluated nodes. If

$$e : E_D^{\rightarrow}(p, i_k) = \text{true},$$

the topological evidence of D gives $\text{rank}_n(p) <_{\mathbb{N}} k$. It therefore determines an index $\bar{p}_e : \text{Fin}(k)$ naming the already evaluated parent p in the prefix. The prefix and the complete exogenous assignment therefore determine the typed mechanism input:

$$\begin{aligned} \text{parentInput}_k(\mathbf{s}) &: \text{EndogenousInputs}_{\mathcal{K}}(i_k), \\ \text{parentInput}_k(\mathbf{s})(p)(e) &:= \mathbf{s}(\bar{p}_e), \\ \text{latentInput}_k(\mathbf{u}) &: \text{ExogenousInputs}_{\mathcal{K}}(i_k), \\ \text{latentInput}_k(\mathbf{u}) &:= \text{restrict}_{\mathcal{K}, i_k}^{\mathcal{U}}(\mathbf{u}), \\ \text{mechanismInput}_{k, \mathbf{u}}(\mathbf{s}) &: \text{MechanismInputs}_{\mathcal{K}}(i_k), \\ \text{mechanismInput}_{k, \mathbf{u}}(\mathbf{s}) &:= (\text{parentInput}_k(\mathbf{s}), \text{latentInput}_k(\mathbf{u})), \\ \text{next}_{k, \mathbf{u}}(\mathbf{s}) &:= f_{i_k}(\text{mechanismInput}_{k, \mathbf{u}}(\mathbf{s})). \end{aligned}$$

Prefix extension. Define the operation:

$$\text{extend}_k : \text{Prefix}_{\mathcal{V}}(k) \rightarrow \text{Val}(i_k) \rightarrow \text{Prefix}_{\mathcal{V}}(k+1).$$

Write $j_k : \text{Fin}(k) \rightarrow \text{Fin}(k+1)$ for the canonical inclusion and $\text{last}_k : \text{Fin}(k+1)$ for the final index. The extension retains every earlier coordinate and places its supplied value at the new final coordinate:

$$\begin{aligned} \text{extend}_k(\mathbf{s}, x)(j_k(j)) &:= \mathbf{s}(j), \\ \text{extend}_k(\mathbf{s}, x)(\text{last}_k) &:= x. \end{aligned}$$

These two cases exhaust $\text{Fin}(k+1)$, so finite elimination determines the complete extended prefix.

Recursive evaluation. Define

$$\mathbf{s}_k(\mathbf{u}) : \text{Prefix}_{\mathcal{V}}(k)$$

by natural-number recursion:

$$\begin{aligned} \mathbf{s}_0(\mathbf{u}) &:= \star, \\ \mathbf{s}_{k+1}(\mathbf{u}) &:= \text{extend}_k(\mathbf{s}_k(\mathbf{u}), \text{next}_{k, \mathbf{u}}(\mathbf{s}_k(\mathbf{u}))). \end{aligned}$$

The construction still evaluates one node at a time: the mechanism at i_k may inspect endogenous parents, whose values become available only in the preceding prefix.

Solution map. After the n successor steps, the completed prefix is a joint endogenous assignment because $\text{Prefix}_{\mathcal{V}}(n)$ and $\text{Val}(\mathcal{V})$ have the same coordinates. Define:

$$\begin{aligned} \text{sol}_{\mathcal{M}} : \text{Val}(\mathcal{U}) &\rightarrow \text{Val}(\mathcal{V}), \\ \text{sol}_{\mathcal{M}}(\mathbf{u}) &:= \mathbf{s}_n(\mathbf{u}). \end{aligned}$$

A. Definitions

Thus the solution at each node is constructed only from an earlier prefix and the fixed exogenous assignment; no later endogenous value is used during evaluation.

Interpretation. The solution map sends each complete exogenous assignment to the uniquely determined endogenous assignment.

Running example. Consider the exogenous assignment in which $U_B = \text{true}$ and every private root is **false**. Evaluation in Mira’s displayed topological order gives:

$$\text{sol}_{\mathcal{M}}(\mathbf{u}) = (\text{true}, \text{true}, \text{false}, \text{false}, \text{false}, \text{true}, \text{false}, \text{false}),$$

in the coordinate order (Pr, Qu, Top, Com, Fit, Short, Fund, Off).

Lean implementation. In the namespace `FiniteLatentSCM`, recursive evaluation is `evalPrefixUnder` and `evalUnder`. Their pointwise and complete-assignment agreement is proved in Proposition B.6.5. The independent finite-source evaluator and this dependent interpretation are related by Theorem B.6.4 and Lemma B.6.6.

Definition A.4.16 (Observational assignment). The solution map of Definition A.4.15 transfers the exogenous source to an observational probability assignment on the endogenous variables.

Pushforward. By Definition A.3.7, the joint exogenous urn can be presented as:

$$\begin{aligned} \mu_{\mathcal{U}} &= (N, h_N, g), \\ g &: \text{Fin}(N) \rightarrow \text{Val}(\mathcal{U}), \end{aligned}$$

where g labels every urn cell by a complete exogenous assignment. Relabelling each of its cells by the resulting endogenous assignment gives the pushforward:

$$\mu_{\mathcal{M}} := (N, h_N, \text{sol}_{\mathcal{M}} \circ g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Hence every decidable event $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ satisfies:

$$P_{\mu_{\mathcal{M}}}(E) \sim P_{\mu_{\mathcal{U}}}(E \circ \text{sol}_{\mathcal{M}}).$$

This is the urn pushforward of Proposition B.5.11. Marginal, conditional and independence statements about the model are evaluated using $\mu_{\mathcal{M}}$.

Interpretation. Pushing the exogenous urn through the solution map transfers uncertainty over the background variables to the observational probability assignment over the endogenous variables.

Running example. Pushing Mira’s complete finite exogenous product urn through the solution map of Definition A.4.15 gives the probabilities computed in Example A.8.1.

Lean implementation. In the namespace `FiniteLatentSCM`, the pushforward is `observationalDist`. Its agreement with recursive evaluation is proved in Proposition B.6.5. The common-denominator and urn presentations agree by Proposition B.5.10.

Definition A.4.17 (Causal Bayesian network). A causal Bayesian network combines a directed acyclic graph with local probability assignments and a joint assignment that factorises through those local assignments.

Graphical data. Let $D : \text{DAG}(V)$ be a directed acyclic graph on the finite variable family $\mathcal{V}$ of Definition A.3.15. For $i : V$, the parent family $\text{Pa}_D(i)$ is the finite selection of Definition A.4.5.

Local probability kernels. For every $i : V$, provide:

$$\kappa_i : \text{Val}(\text{Pa}_D(i)) \rightarrow \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(i)).$$

Thus every parent assignment pa_i selects a normalised probability assignment $\kappa_i(\text{pa}_i)$ over the possible values of i . The kernel is supplied for every parent assignment; it is not obtained by conditioning a joint assignment and therefore remains defined when that parent assignment has probability zero.

Joint assignment. Provide an urn-generated joint assignment:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

For $v : \text{Val}(\mathcal{V})$, let v_i be its value at i and let $\text{pa}_i(v)$ be its restriction to the parents of i . For a parent assignment $\text{pa} : \text{Val}(\text{Pa}_D(i))$ and a value $x : \text{Val}(i)$, abbreviate the probability of the singleton event selecting x by

$$q_i(x \mid \text{pa}) := P_{\kappa_i(\text{pa})}(x).$$

Here applying $P_{\kappa_i(\text{pa})}$ directly to a value uses the same singleton-event convention as $P_{\mu}(v)$ in the factorisation below.

Factorisation. The joint assignment must satisfy:

$$P_{\mu}(v) \sim \prod_{i:V} q_i(v_i \mid \text{pa}_i(v)),$$

where $P_{\mu}(v)$ abbreviates the probability of the decidable singleton event selecting v .

Interpretation. The formula reconstructs the probability of a complete assignment by multiplying one local probability for every variable, evaluated at the parent values occurring in that same assignment. Consequently, the joint probability is governed by the parent-indexed local kernels rather than by one unrelated probability assignment for each variable. Its causal reading is that the parents are the direct graphical inputs to the local mechanism and that every variable is independent of its non-descendants conditional on its parents.

Lean implementation. The local-kernel part of this definition is the record `FiniteRationalCPT`, which uses an explicit finite code for each parent configuration and a finite rational row for each code. The construction `FiniteRationalCPT.toFunctionalCBN` turns those rows into finite response-function seeds, and `toSCM` produces the structural model of Definition A.4.14. The Lean record does not separately store the displayed observed joint assignment; evaluation of the resulting structural model constructs it. Both preservation of every supplied row and the displayed global factorisation of that observational assignment are established in Lemma B.12.1.

Graphical separation and probabilistic consequences

Definition A.4.18 (d-separation). d-separation is the graphical path-blocking criterion used for both Markovian and semi-Markovian causal graphs. The latter are evaluated through a canonical finite all-directed expansion.

Object flow. A full structural causal model supplies a latent causal skeleton; hiding the skeleton's explicit roots gives an observed causal graph; and expanding that graph again supplies the directed graph on which separation is checked:

$$\mathcal{M} = (\mathcal{K}, \mathcal{F}) \longrightarrow \mathcal{K} \xrightarrow{\pi} \mathcal{G} \longrightarrow \tilde{D}_{\mathcal{G}}.$$

A. Definitions

The first arrow selects the skeleton of the model, the projection π is the operation of Definition A.4.12, and the final arrow is the canonical expansion defined below. That final arrow is not an inverse reconstruction of $\mathcal{M}$: it restores enough directed graph structure to test separation, but it does not recover the original exogenous value types, probabilities or mechanisms.

Observed graph. Let $\mathcal{G} : \text{CausalGraph}(V)$ in the sense of Definition A.4.7. It may be supplied directly or obtained from a latent causal skeleton by Definition A.4.12. A later general hidden-DAG projection produces the same kind of observed graph. The separation relation below depends only on $\mathcal{G}$, not on which hidden model produced it.

Canonical directed expansion. Use the finite node type:

$$\tilde{V} := V + (V \times V).$$

This type is finite by Remark A.1.22; its finiteness presentation supplies the formal finite indexing used by the edge and path constructions above. The left summand contains the observed nodes, with embedding $o(i) := \text{inl}(i)$. An inhabitant $U_{A,B} := \text{inr}(A,B)$ of the right summand is a candidate latent root. Retain every observed directed edge of $D_{\mathcal{G}}$. The candidate $U_{A,B}$ has arrows to $o(A)$ and $o(B)$ exactly when

$$E_{\mathcal{G}}^{\leftrightarrow}(A,B) = \text{true} \quad \text{and} \quad \text{rank}_n(A) <_{\mathbb{N}} \text{rank}_n(B).$$

No arrow enters a candidate latent root. Write $\tilde{D}_{\mathcal{G}}$ for this all-directed graph. Giving every candidate rank 0 and every observed node $o(i)$ rank $\text{rank}_n(i) + 1$ proves that it is acyclic.

This construction is the canonical pair-root expansion of Definition A.4.12. There is one active hidden root for each bidirected edge, written with increasing rank. The reverse-ordered pair is not used as a second root. Candidates belonging to false bidirected pairs remain isolated. Hiding the active pair roots recovers $\mathcal{G}$ at graph level. In the Markovian case every candidate is isolated, so the active part of the expansion is just the observed directed graph. This is the exact finite representation used below and by the executable separation test [FK23; Pea09].

Data and paths. Let $X, Y, Z : V \rightarrow \text{Bool}$ be pairwise disjoint finite node selections in the sense of Definition A.4.1. For $x, y : V$ with $X(x) = \text{true}$ and $Y(y) = \text{true}$, a separation path is an inhabitant:

$$p : \text{Path}_{\text{Adj} \xrightarrow{\tilde{D}_{\mathcal{G}}} } (o(x), o(y))$$

whose node list contains no repeated node. It may traverse arrows in either direction; it need not itself be a directed path. The conditioning selection Z is lifted along o and contains no latent root.

Blocking. Such a path is blocked by Z if it contains either:

A.4.18.1. an internal non-collider in the sense of Definition A.4.3 that belongs to the lifted conditioning selection; or

A.4.18.2. an internal collider for which neither the collider nor any of its descendants under $\tilde{D}_{\mathcal{G}}$, in the sense of Definition A.4.5, belongs to that selection.

The three local configurations are displayed in Figure A.2: conditioning on the middle node blocks a chain or fork, whereas a collider blocks by default and is opened by conditioning on it or one of its descendants.

Graphical independence. If every separation path from X to Y is blocked by Z , then X and Y are *d-separated* by Z , written:

$$(X \perp\!\!\!\perp Y \mid Z)_G.$$

The graph subscript distinguishes this graphical relation from probabilistic conditional independence, whose subscript is a probability assignment, as specified in Remark A.3.20.

Example. In the directed expansion of $\mathcal{G}_{\text{Mira}}$, the path $\text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}$ is blocked by its collider when Z is empty. The remaining paths are likewise blocked, so:

$$(\text{Top} \perp\!\!\!\perp \text{Com} \mid \emptyset)_{\mathcal{G}_{\text{Mira}}}.$$

Conditioning on Fit opens the displayed collider path.

Checked implementation. The executable finite test computes the same graphical predicate by ancestral moralisation and finite reachability on the expanded graph, rather than by enumerating paths. Path-blocking remains the external interface: later transport uses the path-blocking relation, not the moralisation algorithm. Pairwise disjointness of X , Y and Z is part of the definition above; the executable test does not take that disjointness as a precondition.

Lean implementation. `SeparationNode` has the constructors `observed` and `latentPair`, corresponding exactly to the two summands of $\tilde{V}$. The path-specific structure `ActivePath` contains fields `nodes`, `starts`, `finishes`, `simple` and `adjacent`. The `adjacent` field uses the proposition-valued predicate `PathSpecification.Consecutive`, which expresses the same pairwise condition as `ConsecutiveR` above. Further fields record the conditioning requirements. The checked finite implementation computes the graphical predicate by explicit-latent expansion, ancestral moralisation and finite reachability. Its relationship to the path-blocking specification, and the external equivalence required by the published-result interface, are stated in Proposition B.6.8. The declarations are in `lean/Thesis/CausalTransport/DSeparation.lean`.

Definition A.4.19 (Causal Markov condition). The causal Markov condition relates the ordered directed graph to conditional independence in its joint probability assignment.

Data. Let $D : \text{DAG}(V)$ be a directed acyclic graph on a finite variable family $\mathcal{V}$, and let:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Non-descendant selection. For every node $i : V$, let $\text{ND}_D(i) : V \rightarrow \text{Bool}$ select the nodes that are neither descendants nor parents of i :

$$\text{ND}_D(i)(j) = \text{true} \iff j \notin \text{De}_D(i) \wedge j \notin \text{Pa}_D(i).$$

The parent and descendant selections are those of Definition A.4.5. Because i is its own descendant, it is not selected by $\text{ND}_D(i)$.

Condition. The assignment μ satisfies the causal Markov condition with respect to D when every $i : V$ satisfies:

$$\{i\} \perp\!\!\!\perp_{\mu} \text{ND}_D(i) \mid \text{Pa}_D(i).$$

A. Definitions

The singleton, parent and non-descendant classifiers are interpreted as finite variable selections and lifted to cylinder events as in Definition A.3.19. Thus every variable is conditionally independent of its non-descendants other than its parents once its parents are known [Pea09].

Scope. This condition is recorded for completeness and for comparison with classical presentations. The checked correspondence theorem of Theorem B.6.4 instead starts from deterministic structural mechanisms and an explicit product law for latent roots; it does not generically derive such a model from the kernels and factorisation of Definition A.4.17.

Remark A.4.20 (Faithfulness). *Data.* Let $\mathcal{G} : \text{CausalGraph}(V)$ be a causal graph on a finite variable family $\mathcal{V}$, and let:

$$\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Condition. The assignment μ is *faithful* to $\mathcal{G}$ when, for every three pairwise disjoint finite variable selections $X, Y, Z : V \rightarrow \text{Bool}$:

$$(X \perp\!\!\!\perp Y \mid Z)_{\mathcal{G}} \iff X \perp\!\!\!\perp_{\mu} Y \mid Z.$$

Interpretation. The forward implication from graphical d-separation to probabilistic conditional independence is the global Markov property. Faithfulness additionally requires the converse: every probabilistic conditional independence must be graphically represented, so no additional independence arises only from accidental parameter cancellation.

Scope. Faithfulness is standard background vocabulary in causal discovery, but it is not used by any later definition, assumption or result in this thesis. In particular, the identification and do-calculus completeness results of Section B.10 work with the graph and compatible model class as given.

Structural interventions

Definition A.4.21 (Graph mutilation). Graph mutilation is Pearl’s graphical representation of intervention.

Data. Let $\mathcal{G}$ be a causal graph in the sense of Definition A.4.7 and let X be a finite node selection in the sense of Definition A.4.1.

Incoming. The mutilated graph $\mathcal{G}_{\overline{X}}$ is obtained by deleting all directed arrows into X , because an intervention on X replaces its usual causes.

Outgoing. The graph $\mathcal{G}_{\underline{X}}$ is obtained by deleting all directed arrows out of X .

Combined. In the semi-Markovian reading, these operations are applied after passing to the canonical finite latent-variable directed acyclic graph of Definition A.4.18; equivalently, a bidirected edge incident with an intervened variable no longer contributes a latent arrowhead into that variable in $\mathcal{G}_{\overline{X}}$. These operations are combined when several marks occur together. Their finite computation and preservation under source interpretation are recorded in Proposition B.6.8.

Example. In the running example, $\mathcal{G}_{\text{Mira}, \overline{\text{Pr}}}$ removes the latent arrowhead into Pr represented by $\text{Pr} \leftrightarrow \text{Qu}$, but retains the outgoing edge $\text{Pr} \rightarrow \text{Short}$.

Definition A.4.22 (Intervention and interventional distribution). Intervention has a structural-model component and an induced probability assignment.

Intervened model. For a structural causal model $\mathcal{M} = (\mathcal{K}, \mathcal{F})$ in the sense of Definition A.4.14 and a joint assignment $x : \text{Val}(X)$ to a finite node selection X in the sense of Definition A.4.1, the *intervened model* is

$$\mathcal{M}_x := (\mathcal{K}_x, \mathcal{F}_x),$$

where $\mathcal{K}_x$ removes every incoming endogenous and exogenous incidence into a selected node i , and $\mathcal{F}_x$ replaces that node's mechanism by the corresponding constant value $x_i : \text{Val}(i)$. All other declarations, mechanisms and the exogenous source $\mu_{\mathcal{U}}$ are unchanged. The intervened model is again recursive over the same topological order.

Interventional assignment. If $\mu_{\mathcal{U}} = (N, h_N, g)$, the *interventional assignment* associated with $\text{do}(X = x)$ is the pushforward urn

$$\mu_{\mathcal{M}_x} := (N, h_N, \text{sol}_{\mathcal{M}_x} \circ g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\text{Val}(\mathcal{V})).$$

Thus, for any decidable event $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$,

$$P_{\mu_{\mathcal{M}_x}}(E) \sim P_{\mu_{\mathcal{U}}}(E \circ \text{sol}_{\mathcal{M}_x}).$$

This is again the urn pushforward of Proposition B.5.11. The interventional assignment $\mu_{\mathcal{M}_x}$ is the model-theoretic content of the intervention $\text{do}(X = x)$; its graph-level content is the mutilated graph $\mathcal{G}_{\overline{X}}$ of Definition A.4.21.

Running example. For $X = \{\text{Pr}\}$ and $x_{\text{Pr}} = \text{true}$, the intervened Mira model replaces the equation for Pr by the constant true . Its incoming latent use of U_B is removed, while the equation and latent input of Qu remain unchanged.

Remark A.4.23 (Evaluator representation of intervention). The checked finite implementation uses an extensionally equivalent evaluator presentation. Rather than constructing a second model record with empty parent and latent-input declarations at the intervened nodes, evaluation retains the source record and overrides the chosen equations during well-founded evaluation. The corresponding graph mutilation is computed separately. Thus record transformation, evaluator override and graph mutilation are distinct representations of the same hard-intervention semantics; the checked transport uses the latter two. Their equivalence and the atomic structural realization are established in Proposition B.7.5.

Lean implementation. The evaluator override is `equationUnder` and `evalNodeUnder`; the source record type is `FiniteLatentSCM`.

Definition A.4.24 (Do-operator). The expression $\text{do}(X = x)$ denotes an intervention that sets X to x . It differs from the observation $X = x$ because it changes the causal mechanism for X rather than merely conditioning on the event that X has value x .

Notation. $\text{do}(X = x)$ denotes the intervention operator.

Model. $\mathcal{M}_x$ denotes the resulting intervened structural causal model of Definition A.4.22.

Graph. $\mathcal{G}_{\overline{X}}$ denotes its graph-level representation from Definition A.4.21.

Thus the operator, transformed model and mutilated graph are related but distinct objects.

General hidden-DAG extensions

Definition A.4.25 (Partially observed directed acyclic graph). The direct-root projection of Definition A.4.12 suffices for the explicit structural models used in this thesis. The more general hidden-DAG layer of Proposition B.12.2 also permits hidden nodes with parents and hidden nodes lying along longer directed paths. Its graph data are separated from the projection operation below.

Total directed graph. Let the finite type of all graph nodes be written $T := \text{Fin}(q)$. Supply a directed edge classifier, a rank and evidence that the rank increases along every selected edge:

$$\begin{aligned} E_H &: T \times T \rightarrow \text{Bool}, \\ \text{rank}_H &: T \rightarrow \mathbb{N}, \\ h_{\text{top}}^H &: \Pi_{s,t:T} (E_H(s,t) = \text{true}) \rightarrow \text{rank}_H(s) <_{\mathbb{N}} \text{rank}_H(t). \end{aligned}$$

These fields make the total graph directed and acyclic even when its chosen rank is not the natural index order on T .

Observed nodes. Let $V = \text{Fin}(n)$ be the formal node type of the graph that remains visible after projection, in the sense of Definition A.4.1. Supply an embedding of these observed nodes into the total graph:

$$\begin{aligned} o &: V \rightarrow T, \\ h_o &: \Pi_{i,j:V} \text{Id}_T(o(i), o(j)) \rightarrow \text{Id}_V(i, j). \end{aligned}$$

Thus $o(i)$ identifies the total-graph node represented by the observed node i .

Hidden nodes. Supply a Boolean classifier and evidence that the embedded observed nodes are not hidden and that every total node has been classified:

$$\begin{aligned} H &: T \rightarrow \text{Bool}, \\ h_{\text{obs}} &: \Pi_{i:V} H(o(i)) = \text{false}, \\ h_{\text{cover}} &: \Pi_{t:T} (H(t) = \text{true}) + \Sigma_{i:V} \text{Id}_T(o(i), t). \end{aligned}$$

Hence every $t : T$ is either certified hidden or identified with an embedded observed node. A hidden node need not be a root: it may have incoming and outgoing directed edges. A *hidden root* is the special case certified by the type:

$$\text{HiddenRoot}_{H,E_H}(\ell) := (H(\ell) = \text{true}) \times \Pi_{s:T} (E_H(s, \ell) = \text{false}).$$

Hidden-graph record. Package the preceding data as the record type $\text{FiniteHiddenDAG}(V)$ with fields:

$$\begin{aligned} q &: \mathbb{N}, \\ E_H &: \text{Fin}(q) \times \text{Fin}(q) \rightarrow \text{Bool}, \\ \text{rank}_H &: \text{Fin}(q) \rightarrow \mathbb{N}, \\ h_{\text{top}}^H &: \Pi_{s,t:\text{Fin}(q)} (E_H(s,t) = \text{true}) \rightarrow \text{rank}_H(s) <_{\mathbb{N}} \text{rank}_H(t), \\ o &: V \rightarrow \text{Fin}(q), \\ h_o &: \Pi_{i,j:V} \text{Id}_{\text{Fin}(q)}(o(i), o(j)) \rightarrow \text{Id}_V(i, j), \\ H &: \text{Fin}(q) \rightarrow \text{Bool}, \\ h_{\text{obs}} &: \Pi_{i:V} H(o(i)) = \text{false}, \\ h_{\text{cover}} &: \Pi_{t:\text{Fin}(q)} (H(t) = \text{true}) + \Sigma_{i:V} \text{Id}_{\text{Fin}(q)}(o(i), t). \end{aligned}$$

For $\mathcal{H} : \text{FiniteHiddenDAG}(V)$, write $T_{\mathcal{H}}$, $o_{\mathcal{H}}$, $H_{\mathcal{H}}$ and $E_{\mathcal{H}}$ for the corresponding fields.

Scope. This is a graph record, not yet a structural causal model. A model over the total graph must additionally provide finite value types, structural mechanisms and probability data for its background variation. Hiding nodes then produces an observed graph and observed probability assignment; it does not generally produce a latent-free structural causal model of the same record type.

Lean implementation. These fields form **FiniteHiddenDAG**. The associated structural generator **FiniteHiddenDAGSCM** is described at the proof boundary in Proposition B.12.2.

Definition A.4.26 (General hidden-DAG projection). General latent projection packages the partially observed directed graph of Definition A.4.25 together with a causal graph over its observed node type and evidence that the latter records exactly the hidden-path projection defined below.

Data. Let $\mathcal{H} : \text{FiniteHiddenDAG}(V)$ and use its total node type $T_{\mathcal{H}}$, observed embedding $o_{\mathcal{H}}$, hidden classifier $H_{\mathcal{H}}$ and edge classifier $E_{\mathcal{H}}$.

Hidden-internal paths. A hidden-internal path from s to t is a directed path whose intermediate nodes are all hidden; the endpoints s and t need not be hidden. The inductive type $\text{HiddenInternalPath}_{\mathcal{H}}(s, t)$ is generated by:

$$\begin{aligned} \text{direct} &: \Pi_{s,t:T_{\mathcal{H}}} (E_{\mathcal{H}}(s, t) = \text{true}) \\ &\quad \rightarrow \text{HiddenInternalPath}_{\mathcal{H}}(s, t), \\ \text{tail} &: \Pi_{s,j,t:T_{\mathcal{H}}} \text{HiddenInternalPath}_{\mathcal{H}}(s, j) \rightarrow (H_{\mathcal{H}}(j) = \text{true}) \\ &\quad \rightarrow (E_{\mathcal{H}}(j, t) = \text{true}) \rightarrow \text{HiddenInternalPath}_{\mathcal{H}}(s, t). \end{aligned}$$

The second constructor extends an existing path through its hidden endpoint j . Consequently, a chain $o_{\mathcal{H}}(i) \rightarrow h_1 \rightarrow h_2 \rightarrow o_{\mathcal{H}}(j)$ supplies a hidden-internal path from $o_{\mathcal{H}}(i)$ to $o_{\mathcal{H}}(j)$ when h_1 and h_2 are hidden. Write $s \rightsquigarrow_{\mathcal{H}} t$ when this path type is inhabited.

Projected relations. Define the proposition selecting an observed directed edge by:

$$\text{ProjectedDirected}_{\mathcal{H}}(i, j) := o_{\mathcal{H}}(i) \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(j).$$

Define the proposition selecting an observed bidirected edge by:

$$\begin{aligned} \text{ProjectedBidirected}_{\mathcal{H}}(i, j) &:= (i \neq j) \times \Sigma_{\ell:T_{\mathcal{H}}} (H_{\mathcal{H}}(\ell) = \text{true}) \\ &\quad \times (\ell \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(i)) \times (\ell \rightsquigarrow_{\mathcal{H}} o_{\mathcal{H}}(j)). \end{aligned}$$

Projection record. Define $\text{FiniteLatentProjection}(V)$ as the record type with fields:

$$\begin{aligned} \text{hiddenDAG} &: \text{FiniteHiddenDAG}(V), \\ \text{graph} &: \text{CausalGraph}(V), \\ h_{\text{directed}} &: \Pi_{i,j:V} (E_{\text{graph}}^{\rightarrow}(i, j) = \text{true}) \iff \text{ProjectedDirected}_{\text{hiddenDAG}}(i, j), \\ h_{\text{bidirected}} &: \Pi_{i,j:V} (E_{\text{graph}}^{\leftrightarrow}(i, j) = \text{true}) \iff \text{ProjectedBidirected}_{\text{hiddenDAG}}(i, j). \end{aligned}$$

The directed field contracts an observed-to-observed path whose internal nodes are hidden into one observed arrow. The bidirected field selects two distinct

A. Definitions

observed endpoints when one hidden node has hidden-internal directed paths to both.

Examples. A path $o_{\mathcal{H}}(X) \rightarrow h_1 \rightarrow h_2 \rightarrow o_{\mathcal{H}}(Y)$ with hidden h_1, h_2 produces the observed arrow $X \rightarrow Y$. A hidden common ancestor with paths

$$\ell \rightarrow h_1 \rightarrow o_{\mathcal{H}}(X), \quad \ell \rightarrow h_2 \rightarrow o_{\mathcal{H}}(Y)$$

produces $X \leftrightarrow Y$. The bidirected classifier records this projected result; it is not itself the hidden path.

Relation to direct roots. For a latent causal skeleton $\mathcal{K}$, take the total nodes to be its exogenous and endogenous nodes, classify the exogenous nodes as hidden roots and the endogenous nodes as observed, and retain the incidence and endogenous directed edges of its all-directed graph. In this special case, every hidden-internal path from a root to an observed child consists of one incidence edge. The general projection therefore reduces to the direct projection $\pi(\mathcal{K})$ of Definition A.4.12. An observed intermediate node is not contracted: only paths whose internal nodes are hidden contribute new projected edges.

Purpose and Lean implementation. The direct-root case suffices for Mira’s structural model. The arbitrary finite hidden-DAG model class, its projection and the external identifiability-preservation boundary needed by the completeness transfer are stated precisely in Proposition B.12.2. Lean represents the graph and projection records by `FiniteHiddenDAG` and `FiniteLatentProjection`; the latter stores the matching evidence above and derives its observed graph.

A.5 Do-calculus

This section defines causal identification and its finite do-calculus certificate language. It introduces a finite table source presentation and its intrinsic dependent interpretation, supported kernel queries, compatible model classes, c-components, c-forests and hedges. It then defines the three do-calculus rules, finite probability expressions and proof-carrying identification certificates. Published completeness enters only through the explicitly stated external interface; the finite certificates and their source–target transport remain inspectable.

Finite source and target presentations

Definition A.5.1 (Finite table source presentation). The finite source presentation is organised as follows.

Signature and latent data. A *finite table signature* gives a finite ordered node family, an inhabited finite arity for every node, witnessed by a default code, and a Boolean directed-edge table whose arrows point strictly forward in the order. A *finite table latent family* independently declares explicitly enumerated finite latent-root types, their decidable equalities and a latent-incidence table.

Structural model. A *finite table structural causal model* over the signature consists of such a latent family, one finite rational probability record for each root, a joint finite rational prior with an exact product-law witness, and structural tables

$$f_i : \text{Val}(\text{Pa}(V_i)) \rightarrow \text{Val}(\text{Lat}(V_i)) \rightarrow \text{Fin}(a_i).$$

Its observational and interventional semantics are finite sums of the prior over the recursively evaluated table. Source assignments, parent and latent inputs, rectangular events, projected edges and the canonical semi-Markovian condition are defined on the source side rather than inherited from the target.

Lean implementation. The independently declared source definitions are `FiniteTableSignature`, `FiniteTableLatent`, `FiniteTableGraph` and `FiniteTableSCM`.

Definition A.5.2 (Intrinsic dependent causal target). The dependent target and its relation to the source are described in three stages.

Interpretation. The interpretation of a finite table signature replaces each code arity a_i by the node-dependent value type $\text{Fin}(a_i)$, producing an observed signature whose assignments are dependent functions. Interpreting a source structural causal model produces a finite latent-root model of Definition A.4.14 whose mechanism is typed only over the declared earlier observed parents and incident latent roots, equipped with the exact product-law witness required by the restricted fragment. This is not a second mathematical model class.

Adequacy. The interpreted model is Markovian when every latent root is incident to at most one observed mechanism and canonical semi-Markovian when every root is incident to at most two. Shared roots then induce the bidirected edges of Definition A.4.12. Source finite-sum semantics and target pushforward semantics agree pointwise, up to the rational equivalence $\sim$ of Definition A.3.6. This adequacy, including observational, interventional and partial-kernel agreement, is the content of Theorem B.6.4 and Proposition B.6.7.

Scope. The target is an intrinsic dependent representation of the finite type-theoretic structure used in the thesis, not a formal syntax and semantics theorem for every presentation of Martin-Löf-style type theory. Its constructive and intensional boundary is explained in Section B.4.

Running example. The executable Mira record realizes $\mathcal{M}_{\text{Mira}}$ as such a model: each observed coordinate has Boolean value type, each mechanism receives only its declared earlier parents and incident roots, and the prior carries the required product-law witness. Its checked realization is summarized in Corollary B.8.5.

Lean implementation. The interpretation is `FiniteTableSignature.toObserved`; the observed signature is `ObservedSignature`; the interpreted model is `ExactModel`. The executable record is `TenureTrack.model`.

Definition A.5.3 (Hard intervention). For any type A , write

$$\text{Option}(A) := \text{Fin}(1) + A,$$

using the sum type of Definition A.1.12. Its left inhabitant is written `none` and records that no value is defined; its right inhabitants are written `some(a)` for $a : A$.

A hard intervention is a dependent partial assignment

$$i \mapsto \text{Option}(\text{Val}(V_i)).$$

The value `none` leaves the structural mechanism at V_i unchanged, while `some(v)` replaces it by the constant value $v : \text{Val}(V_i)$.

Definition A.5.4 (Finite kernel queries and local support). The query language combines partial values, hard interventions and local support evidence.

A. Definitions

Partial values. The partial-value type $\text{Option}(A)$ and the hard interventions used below are those of Definition A.5.3.

Kernel queries. The distributional queries are joint and conditional kernels. Each contains finite action, outcome and, where applicable, conditioning node sets with their disjointness evidence. Their values are pointwise partial denotations of the whole kernel, valued in $\text{Option}(\mathbb{Q}_{\geq 0})$: a zero denominator gives **none** rather than an arbitrary rational value. The nonnegative rational presentation $\mathbb{Q}_{\geq 0}$ is that of Definition A.3.6, with equivalence given by cross multiplication as proved in Proposition B.5.4.

Local support. A support witness records that the relevant denominator is positive at the valuation and in the model being compared. An identification certificate carries local support recursively for every endpoint and intermediate expression reached at that valuation; it does not require every denominator to be positive at every valuation in every compatible model. Their constructive transport is proved in Proposition B.6.7.

The application-level interventional and conditional queries instead describe fixed decidable events and carry locality evidence for their selected outcome and conditioning nodes. They are derived event-level interfaces, not replacements for the whole-kernel queries used by completeness.

Lean implementation. The whole-kernel queries are `JointKernelQuery` and `ConditionalKernelQuery`. Lean’s `QProb` is the nonnegative rational presentation used above. The source support witness is `FiniteSourceLocalDerivationSupport`; the target witness is `LocalDerivationSupport`. The event-level interfaces are `InterventionalQuery` and `ConditionalQuery`.

Running example. Setting `Pr` to `true` is the hard intervention that returns `some(true)` at `Pr` and `none` elsewhere. The query for `Qu = true` under that action is an event-level interventional query; adding positive evidence produces a locally supported conditional query.

Definition A.5.5 (Restricted finite correspondence fragment). *Package.* The restricted correspondence fragment consists of the finite table source presentation of Definition A.5.1, its intrinsic dependent interpretation of Definition A.5.2, and the supported finite kernel queries of Definition A.5.4. It is this complete package, rather than an unrestricted class of classical or type-theoretic models, that enters the transport theorems.

Identification and hedge criteria

Definition A.5.6 (Compatibility and identification). Compatibility, observational equivalence and identification are specified separately.

FinSCM. Write `FinSCM` for the type of finite recursive structural causal models in the restricted fragment of Definition A.5.5.

Compatible models. Let $\mathcal{G}$ be a finite causal graph in the Markovian or semi-Markovian sense of Definition A.4.7. For a finite recursive structural causal model $\mathcal{M}$ in that fragment, let

$$\text{Compatible}_{\mathcal{G}}(\mathcal{M}) : \mathcal{U}$$

be the type of evidence that the directed and projected bidirected edges of $\mathcal{M}$ agree with $\mathcal{G}$, that every latent root reaches at most two observed mechanisms

in the canonical semi-Markovian case, and that the exact product-law and recursive fields of the fragment are present. The type of compatible models is

$$\text{Mod}(\mathcal{G}) := \Sigma_{\mathcal{M}:\text{FinSCM}} \text{Compatible}_{\mathcal{G}}(\mathcal{M}).$$

Thus $\text{Mod}(\mathcal{G})$ is a dependent type of models carrying compatibility evidence, not a set-theoretic class.

Observational equivalence. Two compatible models $\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})$ are observationally equivalent, written $\mathcal{M} \equiv_{\text{obs}} \mathcal{N}$, when every decidable event E on the full observed assignment type satisfies

$$P_{\mu_{\mathcal{M}}}(E) \sim P_{\mu_{\mathcal{N}}}(E).$$

Identification. The distributional query used by the completeness theorem is a kernel $Q = P_x(Y)$: for every valuation x, y , it asks for the corresponding interventional cylinder probability. It is *identifiable from $\mathcal{G}$* when, for every $\mathcal{M}, \mathcal{N} : \text{Mod}(\mathcal{G})$,

$$\mathcal{M} \equiv_{\text{obs}} \mathcal{N} \implies Q_{\mathcal{M}}(x, y) \sim Q_{\mathcal{N}}(x, y)$$

at every valuation. Conditional kernels $P_x(Y \mid Z)$ are read analogously on their local positive support in the sense of Definition A.5.4.

A fixed-event query $P_x(E)$, where E is a decidable event local to Y , is a derived application notion. In the finite setting, the full identifiable kernel determines every such event by summing its finitely many cylinder cells. The converse is not asserted: agreement on one event does not imply agreement of the whole distributional kernel. The semantic condition defined here is connected to action-free do-calculus expressions by the supplied completeness transport of Proposition B.10.4 and Theorem B.13.3; that connection is a theorem, not part of the definition [Pea09; SP06; TS09].

Compatibility, observational equivalence and identification are preserved and reflected by the finite source–target interpretation, as proved in Proposition B.6.9.

Running example. The expression $P_{\text{Pr}:=\text{true}}(\text{Qu})$ denotes the complete interventional kernel over the Boolean values of Qu , not only its probability at true . The displayed probability in Example A.8.1 is the corresponding fixed-event evaluation of that kernel.

Definition A.5.7 (c-component). The conventional and theorem-facing forms are distinguished explicitly.

Conventional c-components. In a semi-Markovian causal graph, a c-component is a maximal finite node set whose variables are connected by bidirected paths. In the notation of Definition A.4.4, a bidirected path from X to Y is an inhabitant of $\text{Path}_{B_{\mathcal{G}}}^{\leftrightarrow}(X, Y)$. The bidirected edges represent shared unobserved causes, so c-components mark the maximal parts of the graph tied together by latent confounding [TP02; TS09].

Internal connectedness. C-forests and hedges also require the non-maximal predicate saying that a selected node family is connected within its own induced restriction. Let $W : V \rightarrow \text{Bool}$ be a finite node classifier. The type $\text{BiConnectedWithin}_{\mathcal{G}}(W; i, j)$ is generated by

A.5.7.1. reflexive evidence at every selected node i ; and

A. Definitions

A.5.7.2. a step from already reached j to a selected node k whenever $E_{\mathcal{G}}^{\leftrightarrow}(j, k) = \text{true}$.

Define

$$\begin{aligned} \text{BidirectedComponent}_{\mathcal{G}}(W) &:= (\Sigma_{i:V} \text{Id}_{\text{Bool}}(W(i), \text{true})) \\ &\quad \times \Pi_{i,j:V} (W(i) = \text{true}) \rightarrow (W(j) = \text{true}) \\ &\quad \rightarrow \text{BiConnectedWithin}_{\mathcal{G}}(W; i, j). \end{aligned}$$

This record says that W is nonempty and internally bidirected-connected; it deliberately does not assert maximality. A conventional c-component is a maximal W satisfying this predicate.

In $\mathcal{G}_{\text{Mira}}$, the nodes $\{\text{Pr}, \text{Qu}\}$ form a c-component because of their bidirected edge. Every other node is a singleton c-component.

Lean implementation. The internal-connectedness predicate is `BidirectedComponent`.

Definition A.5.8 (c-forest). Let $\mathcal{G}$ be a finite semi-Markovian graph.

Data. A finite c-forest is represented by two Boolean node classifiers

$$W, R : V \rightarrow \text{Bool},$$

where W selects the forest nodes and R selects its roots.

Evidence. It carries the following evidence:

A.5.8.1. $\text{BidirectedComponent}_{\mathcal{G}}(W)$, so the selected nodes of W are nonempty and bidirected-connected;

A.5.8.2. every selected node has at most one selected directed child:

$$\begin{aligned} \Pi_{p,c_1,c_2:V} (W(p) = \text{true}) &\rightarrow (W(c_1) = \text{true}) \rightarrow (W(c_2) = \text{true}) \\ &\rightarrow (E_{\mathcal{G}}^{\rightarrow}(p, c_1) = \text{true}) \rightarrow (E_{\mathcal{G}}^{\rightarrow}(p, c_2) = \text{true}) \rightarrow \text{Id}_V(c_1, c_2); \end{aligned}$$

A.5.8.3. R selects exactly the nodes of W with no directed child in W :

$$R(r) = \text{true} \iff W(r) = \text{true} \wedge \Pi_{c:V} (W(c) = \text{true}) \rightarrow (E_{\mathcal{G}}^{\rightarrow}(r, c) = \text{false}).$$

Restriction. The evidence is stated on the selected nodes of W . Equivalently, it may be read on the induced restriction $\mathcal{G} \upharpoonright W$ of Definition A.4.8; the forest does not carry an independently chosen edge set.

Lean implementation. These fields are the `component`, `at_most_one_child` and `roots_exact` fields of `CForest` $\mathcal{G} W R$.

Definition A.5.9 (Hedge). *Data.* Let a joint kernel query select intervention nodes X and outcome nodes Y . A hedge witness in a finite semi-Markovian graph $\mathcal{G}$ consists of three Boolean node classifiers

$$W_{\text{large}}, W_{\text{small}}, R : V \rightarrow \text{Bool}.$$

Evidence. It carries the following evidence:

A.5.9.1. both (W_{large}, R) and (W_{small}, R) satisfy the c-forest record of Definition A.5.8;

A.5.9.2. the small node family is included in the large one:

$$\Pi_{i:V}(W_{\text{small}}(i) = \text{true}) \rightarrow (W_{\text{large}}(i) = \text{true});$$

A.5.9.3. the large family contains an intervention node, while the small family contains none:

$$\begin{aligned} \Sigma_{i:V}(W_{\text{large}}(i) = \text{true}) \times (X(i) = \text{true}), \\ \Pi_{i:V}(W_{\text{small}}(i) = \text{true}) \rightarrow (X(i) = \text{false}); \end{aligned}$$

A.5.9.4. every selected root reaches some outcome node by a directed path in the mutilated graph $\mathcal{G}_{\overline{X}}$.

The third clause together with inclusion makes the small family a proper part of the large family; no separate inequality or edge-set inclusion is required. It is the finite type-theoretical presentation of the conventional pair of nested c-forests with common roots.

Under the finite semi-Markovian model assumptions, the published hedge criterion uses such a pair to witness non-identifiability. Its finite-rational specialization and checked witness construction are stated in Lemma B.10.3 and Proposition B.10.7; this non-identifiability conclusion is not assumed merely from the graph definition itself [TS09].

Minimal example. Mira’s graph illustrates c-components but does not itself supply the minimal hedge used here. Instead take the two-node graph $X \rightarrow Y$ and $X \leftrightarrow Y$. Let W_{large} select both nodes and let W_{small} select only Y . Their induced restrictions automatically inherit the ambient edges whose endpoints they contain; no edge classifiers are selected separately. The two node classifiers satisfy the c-forest record with the common root classifier selecting Y . The large family contains X , the small family avoids X , and their common root reaches Y after incoming arrows into X are removed. They therefore form a hedge for the effect of X on Y .

Lean implementation. This dependent record is **HedgeWitness**.

The finite graph computations for internal bidirected connectivity, c-forests, mutilation and hedge witnesses, and their preservation under source interpretation, are collected in Proposition B.6.8.

Do-calculus syntax and certificates

Definition A.5.10 (Do-calculus rules). Let $\mathcal{G}$ be a causal graph.

Reading. The displays below use Pearl’s conventional equality notation for kernels. In the finite rational interpretation, each equality means pointwise equivalence $\sim$ of partial kernel denotations valued in $\text{Option}(\mathbb{Q}_{\geq 0})$ on their local positive support, as in Definition A.5.4; a zero denominator has no conditional value.

Pearl’s do-calculus consists of three transformation rules [Pea09; Pea12]:

A.5.10.1. insertion and deletion of observations:

$$P(y \mid \text{do}(x), z, w) = P(y \mid \text{do}(x), w)$$

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}}}$;

A. Definitions

A.5.10.2. action and observation exchange:

$$P(y \mid \text{do}(x), \text{do}(z), w) = P(y \mid \text{do}(x), z, w)$$

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}, \underline{Z}}}$;

A.5.10.3. insertion and deletion of actions:

$$P(y \mid \text{do}(x), \text{do}(z), w) = P(y \mid \text{do}(x), w)$$

whenever $(Y \perp\!\!\!\perp Z \mid X, W)_{\mathcal{G}_{\overline{X}, \overline{Z(W)}}}$, where $Z(W)$ contains the variables in Z that are not ancestors of any variable in W in $\mathcal{G}_{\overline{X}}$.

Definition A.5.11 (Finite probability-expression syntax). Let $\mathcal{G}$ be a finite causal graph.

Syntax. A symbolic kernel has the form

$$P(Y \mid \text{do}(X), W),$$

where X, Y, W are finite node sets. Finite probability expressions are generated inductively from zero, kernels, marginalisation, evaluation at a fixed complete assignment, addition, multiplication and defined division.

Events. Evaluation at fixed assignments and finite addition make every decidable Boolean event expressible: enumerate its satisfying complete assignments and add their singleton full-assignment kernel masses; the empty event is represented by zero.

Lean implementation. The shared syntax is **ProbabilityTerm**, with a partial finite-rational denotation. Its source–target preservation and its compilation of Boolean events are established in Proposition B.6.7; proof declarations are recorded there rather than treated as part of this syntax definition.

Definition A.5.12 (Do-calculus derivation and identification certificate). Derivations and the certificates built from them are specified separately.

Derivations. Let $\mathcal{G}$ be a finite causal graph in the Markovian or semi-Markovian sense of Definition A.4.7. A *do-calculus derivation* is an inductively generated equality between the expressions of Definition A.5.11 using the following constructors:

A.5.12.1. reflexivity, symmetry and transitivity;

A.5.12.2. marginalisation, formal conditioning and the chain rule, with finite disjointness conditions; division has an internal partial semantics that returns no value when its denominator is zero, while the primitive conditioning and chain identities used by a derivation belong to an explicit published-soundness interface;

A.5.12.3. congruence under marginalisation, fixed-assignment evaluation, addition, multiplication and division;

A.5.12.4. one of Pearl’s three rules from Definition A.5.10, carrying pairwise-disjointness proofs for X, Y, Z, W and a proof that the executable d-separation test of Definition A.4.18 succeeds in the required mutilated graph.

Identification certificates. An identification certificate consists of an action-free probability expression and such a derivation from the source query kernel to that expression. At each valuation where the certificate is used, it carries a recursive support witness: every primitive endpoint and every intermediate expression reached there has a defined partial denotation.

Published completeness remains an explicit external input, but its finite specialization must return this inspectable, locally support-carrying certificate rather than inhabit an arbitrary predicate. The transport of supplied certificates and the exact external boundary are stated in Propositions B.9.3 and B.10.4.

Lean implementation. The independently evaluated finite-table source uses `FiniteSourceLocalDerivationSupport`; its interpreted target uses `LocalDerivationSupport`. The shared model-independent definitions are `ProbabilityTerm`, `DoRuleApplication` and `DoCalculusDerivation`; the joint and conditional target certificate types are `JointIdentificationCertificate` and `ConditionalIdentificationCertificate`. The published-soundness interface is `PublishedSoundness`.

Definition A.5.13 (Modal do-calculus trace). The modal annotation has query-mode, rule-cell and scope components.

Query modes. A *causal query mode* records an outcome node set together with its current action and observation locks. There are two generating query modalities: `intervene( $X$ )`, which locks the action component by X , and `observe( $W$ )`, which locks the observation component by W . Thus the kernel $P(Y \mid \text{do}(X), W)$ is the query mode obtained from the base mode on Y by those two locks. These are thesis-specific finite query modes and modalities in the interpretive sense of Definitions A.2.2 and A.2.4 and Remark A.2.10; they do not add elimination rules to the introduction-only modal syntax of Definition A.2.7.

Rule cells and traces. A *modal rule cell* is a proof-carrying relation between two such locked query modes. The three constructors respectively change an observation lock, exchange action and observation locks, and change an action lock; each retains the exact disjointness and d-separation evidence of the corresponding rule in Definition A.5.10. A *modal do-calculus trace* recursively annotates an ordinary derivation with these cells and with its composition and probability-algebra structure.

Scope. A trace is an operation-indexed annotation, not an operation that rewrites a structural model. Later certificate transport relates such traces to executable operations; see Propositions B.7.1 and B.10.6.

Running example. The Mira kernel $P(\text{Qu} \mid \text{do}(\text{Pr} = \text{true}))$ carries an intervention lock selecting `Pr` and an outcome mode selecting `Qu`. A derivation involving that kernel records any change of action or observation locks as modal rule cells; the cells do not by themselves alter $\mathcal{M}_{\text{Mira}}$.

Lean implementation. The definitions are `CausalQueryModality`, `CausalQueryMode`, `ModalRuleCell` and `ModalDerivationTrace`.

A.6 Counterfactual semantics

This section defines finite counterfactual queries through abduction, action and prediction. It begins with potential outcomes and one-action conditional probabilities, then extends them to finite Boolean multiworld queries. Twin

A. Definitions

networks and occurrence-indexed multiworlds provide two explicit semantics in which every counterfactual world shares the same exogenous assignment while carrying its own intervention.

Counterfactual queries

Definition A.6.1 (Counterfactual query). The construction proceeds from potential outcomes to a one-action conditional probability.

Potential outcomes. Given a structural causal model $\mathcal{M}$ in the sense of Definition A.4.14, an exogenous assignment u , and an intervention $X = x$ in the sense of Definition A.4.22, the counterfactual $Y_x(u)$ is the value of Y in the solution of the intervened model $\mathcal{M}_x$ under u , that is, $\text{sol}_{\mathcal{M}_x}(u)$ from Definition A.4.15 projected to Y . With observed evidence, Pearl’s evaluation procedure is abduction, action and prediction: filter the exogenous urn by the evidence, replace the relevant equations, and push the filtered urn through the intervened solution map.

One-action probability. Concretely, let $E : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ be a decidable observed event whose pullback $E \circ \text{sol}_{\mathcal{M}}$ has positive probability under $\mu_{\mathcal{U}}$. Abduction produces the posterior exogenous urn

$$\mu_{\mathcal{U}|E} := (\mu_{\mathcal{U}})_{E \circ \text{sol}_{\mathcal{M}}}$$

by the filtering operation of Definition A.3.12. After the action $X = x$, the model-level counterfactual probability of a decidable target event $F : \text{Val}(\mathcal{V}) \rightarrow \text{Bool}$ is

$$P(F_x | E) := P_{\mu_{\mathcal{U}|E}}(F \circ \text{sol}_{\mathcal{M}_x}).$$

This is a calculation inside the given structural causal model. It is not, by itself, an identifiability claim saying that the value is determined by an observational distribution or a graph alone.

Running example. In $\mathcal{M}_{\text{Mira}}$, let E be the factual event $\{\text{Pr} = 0, \text{Qu} = 1, \text{Off} = 0\}$ and let x set Pr to 1. The one-action query

$$P(\text{Off}_{\text{Pr}=1} = 1 | E)$$

is obtained by taking F to select $\text{Off} = 1$. Its model-level value is calculated in Example A.8.1; the definition alone does not assert that the value is identifiable from the observed graph.

Lean implementation. The derived constructor `CounterfactualQuery.singleAction` compiles the ordinary one-action expression $P(F_x | E)$ into the finite Boolean event syntax of Definition A.6.2. Its agreement with the direct model-level and multiworld semantics is proved in Proposition B.8.3 and Corollary B.8.4.

Definition A.6.2 (Finite multiworld counterfactual query). The finite multiworld fragment extends the one-action query of Definition A.6.1 to Boolean combinations of atoms that may carry different interventions.

Atoms. A *counterfactual atom* is a triple consisting of a hard intervention x in the sense of Definition A.5.3, an observed variable V , and a value $v : \text{Val}(V)$, read as $[V_x = v]$.

Events. Counterfactual events are generated from truth, falsity and these atoms by finite conjunction, disjunction and negation. Every atom in one event

is evaluated at the same exogenous assignment u , although its intervention may differ. A counterfactual query is a pair of events (γ, δ) , interpreted as an outcome and a condition. Recursive evaluation in $\mathcal{M}$ gives Boolean classifiers

$$\llbracket \gamma \rrbracket_{\mathcal{M}}, \llbracket \delta \rrbracket_{\mathcal{M}} : \text{Val}(\mathcal{U}) \rightarrow \text{Bool},$$

by evaluating every atom occurrence at its indicated intervention world and the shared latent assignment.

Denotation. The partial denotation of the query is the urn conditional probability

$$P_{\mathcal{M}}(\gamma \mid \delta) := P_{\mu_u}(\llbracket \gamma \rrbracket_{\mathcal{M}} \mid \llbracket \delta \rrbracket_{\mathcal{M}}),$$

whenever the denominator is positive. A zero denominator has no rational denotation.

Identifiability. Relative to an observed graph $\mathcal{G}$, such a query is *observationally counterfactually identifiable* when any two compatible finite structural causal models with the same observational distribution give equivalent partial denotations. It is *uniformly query-supported* over $\mathcal{G}$ when its denominator is positive, and hence its denotation is defined, in every compatible model. This model-class support condition is stronger than merely knowing that the query is supported in one chosen structural causal model.

Lean implementation. The finite event syntax is implemented by `CounterfactualAtom`, `CounterfactualEvent` and `CounterfactualQuery`. The declarations `CounterfactualIdentifiable` and `CounterfactualSupported` encode the two model-class conditions.

Twin and multiworld semantics

Definition A.6.3 (Finite twin network). The twin network and its evaluation are constructed as follows.

Construction. Given a finite latent-root structural causal model $\mathcal{M}$ and hard intervention x , its finite twin network has two tagged copies V^f and V^c of every observed variable, carrying the same value type. Directed arrows are copied only within each world. The factual copy retains the original mechanisms; in the counterfactual copy every incoming observed and latent arrow at a target of x is deleted and its mechanism is replaced by the selected constant. Both copies share the same finite latent roots and prior.

Evaluation. Evaluation at one latent assignment is therefore

$$u \mapsto (\text{sol}_{\mathcal{M}}(u), \text{sol}_{\mathcal{M}_x}(u)).$$

The projected bidirected relation is derived from roots incident to two tagged nodes, including cross-world sharing.

Running example. For the intervention $\text{Pr} := 1$, the Mira twin network contains a factual and a counterfactual copy of every observed node. Both copies use the same U_B , but the counterfactual equation for Pr^c is constant and its incoming use of U_B is cut. The equation for Qu^c still uses that shared root, which is what preserves the relevant cross-world dependence.

Lean implementation. This construction is `TwinNetwork`; its generic constructor is `FiniteLatentSCM.twinNetwork`. The dependent tagged-node network is independently encoded as an ordinary finite-indexed structural

A. Definitions

causal model by `TwinNetwork.Encoding.model`. The two presentations agree pointwise and on event probabilities, as established in Proposition B.8.1.

Definition A.6.4 (Occurrence-indexed multiworld). The multiworld construction retains syntactic occurrences explicitly.

Construction. For an arbitrary finite Boolean counterfactual query, the construction retains one world for every *syntactic atom occurrence*, in left-to-right order, in addition to the factual world. Thus two occurrences carrying extensionally equal interventions remain distinct worlds without requiring equality testing for dependent intervention functions. All worlds share the source latent assignment. Directed arrows occur only within one world; the selected setting in that world cuts the corresponding directed and latent inputs. In the one-action case the construction agrees with the twin network of Definition A.6.3.

Running example. A compound Mira event containing both $[\text{Off}_{\text{Pr}:=1} = 1]$ and $[\text{Off}_{\text{Fit}:=1} = 1]$ receives two occurrence worlds in addition to the factual world. The worlds share the same exogenous assignment but carry their two different settings. Repeating either atom syntactically would create another occurrence world even though its intervention data are extensionally equal.

Lean implementation. The construction is `OccurrenceMultiworld`. `OccurrenceMultiworld.Encoding.model` materialises this dependent presentation as an ordinary finite-indexed structural causal model. The numerator and denominator read from that structural causal model are equivalent to `CounterfactualQuery.numerator` and `CounterfactualQuery.denominator`. Its agreement with the query semantics, and the relation between the direct twin and the occurrence-indexed construction in the one-action case, are established in Propositions B.8.1 and B.8.3.

Remark A.6.5. Counterfactual queries are placed here, after the identification machinery of Definitions A.5.6 to A.5.9, because they sit on Pearl’s third level of inquiry: the unconditional and conditional interventional fragment (Pearl’s second level) is closed by the identification definitions above, and counterfactuals add an extra step (abduction) on top of intervention. The ordinary transfer of Theorem B.13.3 remains restricted to interventional distributions. The separate result of Theorem B.11.1 covers supported finite multiworld queries only after an explicit source-native specialization of the published counterfactual-identification theorem is supplied; it does not internally formalise the counterfactual-identification algorithms conventionally named `ID*` and `IDC*` [SP08].

A.7 Modal causal representation

This section gives the finite causal constructions their modal interpretation. A causal mode carries a structural model, a current finite belief and a current intervention. Modalities mark admissible transitions between such modes, while separately defined proof-carrying operations construct the corresponding record changes. The section develops fixed-signature transitions, signature-changing structural edits, endpoint semantics, atomic intervention, counterfactual execution, learning and forgetting. This is the thesis-specific interpretive fragment of multimodal type theory; it does not reproduce the full multimodal term calculus.

Causal epistemic records and modes

Definition A.7.1 (Finite causal epistemic record). *Data.* A finite causal epistemic record $R = (\mathcal{M}, b, x)$ consists of:

A.7.1.1. a finite latent-root structural causal model $\mathcal{M}$ in the intrinsic target presentation of Definition A.5.2;

A.7.1.2. a current finite rational belief b over its latent assignments, represented by the common-denominator probability record of Definition A.3.9;

A.7.1.3. a dependent hard intervention x on its observed variables in the sense of Definition A.5.3.

These fields jointly contain the current causal and epistemic information. In particular, no separate relation graph is maintained: directed and latent links are part of the structural causal model, the belief records the presently available uncertainty, and the intervention records the structural settings currently in force.

Running example. Mira’s initial record is

$$R_{\text{Mira}} = (\mathcal{M}_{\text{Mira}}, \mu_{\mathcal{U}}, \emptyset),$$

containing the structural model of Example A.8.1, its exogenous prior and the empty hard intervention. Receiving evidence changes the second field; performing a setting changes the third; structural editing may change the first.

Lean implementation. This structure is `CausalEpistemicRecord`.

Definition A.7.2 (Epistemic causal mode). *Data.* An epistemic causal mode is a pair consisting of a name and a finite causal epistemic record from Definition A.7.1.

Interpretation. It is interpreted as a mode in the sense of Definitions A.2.1 and A.2.2 and Remark A.2.9. The name distinguishes endpoints in a transition but has no causal or probabilistic effect. This thesis-specific construction uses Gratzner-style modes to organise Pearl-style causal states. Moving between such modes may change the structural model, the current belief, or the intervention in force.

Example. For the running example, a name such as m_{Mira} paired with R_{Mira} forms the initial mode. A posterior mode after receiving evidence has a different record and may have a different name, but the name itself contributes no information to the posterior.

Proof-carrying transitions

Definition A.7.3 (Finite epistemic transition layer). The fixed-signature layer separates executable operations from their proof-carrying transitions and identification consequences.

Model-preserving operations. On a fixed observed and latent signature, the model-preserving record transformations are:

A.7.3.1. conditioning, which updates the belief from explicit positive event evidence;

A. Definitions

A.7.3.2. setting and unsetting, which respectively insert and remove one component of the dependent hard intervention.

A simultaneous finite setting and a selected-node observation are derived multi-node forms.

Running example. Observing $E = \{\text{Pr} = 0, \text{Qu} = 1, \text{Off} = 0\}$ conditions Mira’s current belief, while setting Pr to 1 inserts that value into the intervention field. Both transformations leave the stored structural model unchanged at this fixed-signature layer.

Proof-carrying transitions. Each corresponding transition contains source and target modes together with a proof that the stated record transformation produced the target. The transition label is therefore a separator between modes, not an operation that discovers or performs a context-native change by itself. A realization witness records that these operations realize the corresponding intervene and observe locks of Definition A.5.13; from an empty intervention, the multi-node operation is pointwise equal to the intervention used by the kernel semantics.

Identification at a mode. These model-preserving steps preserve the structural model. Hence static joint and conditional identifiability, and the transported completeness/soundness equivalences for that model, are invariant along a transition. Conditioning and intervention still change the record’s current numerical belief or evaluation state; this invariance is only the graph-and-model-level identification statement. Separately, transported do-calculus certificates retain a modal derivation trace and a proof that the indexing epistemic mode is compatible with the source graph. With finite-source primitive soundness, operation-realized joint and conditional certificates additionally execute each supported source kernel and every Pearl-rule leaf. Mode-indexed variants combine that executable certificate with a proof that the indexing epistemic mode is compatible with the source graph, and specialize the uniform realization to the model stored at that mode. Hence the transport includes the operation-sensitive query locks without identifying a lock with the executable record operation itself. The preservation, realization and certificate results are stated in Propositions B.7.1, B.7.2 and B.10.6.

Lean implementation. The transition record is `CausalTransition`; the step proof is `CausalRecordStep`; realization is `CausalTransition.Realizes`. Certificates retain a `ModalDerivationTrace`. The executable certificates are `OperationRealizedJointCertificate` and its conditional counterpart; the mode-indexed variants are `ModeIndexedOperationRealizedJointCertificate` and its conditional counterpart, specialized by `atMode`.

Definition A.7.4 (Typed causal editing and belief reindexing). Model-changing edits are organised by their indexed inputs, causal links and belief-reindexing obligations.

Indexed operations. Model-changing causal edits use a dependent-signature edit transition. It contains an edit operation indexed by the concrete source record and dependent target signature. Each constructor carries its operation-specific inputs rather than a proposed target record: conditioning carries positive evidence, causal linking carries the typed replacement mechanism, learning carries its variable specification, certified belief reindexing carries a causal origin, coordinate map and endpoint evaluator, and compact surgery carries

the action. An eliminator computes the unique target record from these inputs and the indexed source. Paired forgetting constructors are available only at the definitional output of their corresponding learning constructor and return its recorded source. Thus neither an arbitrary result nor an independently selected label can be attached to an operation. The transition’s equality field certifies that its named target is this computed result. The canonical builders used below prove the remaining operation-specific semantic laws. This one family includes fixed-signature epistemic updates, structural linking, variable creation, provenance-guided forgetting, belief reindexing and compact reference surgery.

Causal links. A causal link is either a rank-respecting endogenous-to-endogenous parent link or an exogenous-to-endogenous latent-input link. Relating or unrelating changes the corresponding signature or incidence table and requires an explicitly typed replacement mechanism for the affected child. Thus causal linkage is not inferred from an arbitrary function and is not merely epistemic metadata.

Belief reindexing. When a construction changes the latent coordinates, the reindexing operation constructor supplies an explicit origin and a map into the new root coordinates. The origin is either an epistemic record, whose current belief is used, or a structural causal model, whose prior is used; the finite distribution is therefore derived from causal data rather than supplied independently. A declared endpoint evaluator and pointwise realization proof certify the coordinate map. Under the preserving policy this proof uses the retained override. The clearing policy has two separate obligations on every transported origin assignment: evaluation with the retained override must equal evaluation by the target structural equations alone, and that structural evaluation must realize the declared endpoint evaluator. Thus clearing is admitted only after the override has been shown semantically absorbed. The unrestricted finite pushforward remains available as a low-level operation, but it is not admitted directly by the typed edit family. Reindexing does not pretend that equation replacement itself performed an epistemic update.

Correctness. The operation typing, structural realization and clearing obligations are established in Propositions B.7.5 and B.7.7.

Running example. The structural realization of $\text{do}(\text{Pr} = 1)$ replaces Mira’s equation for Pr by a constant and unrelates its incoming latent link from U_B . It retains the outgoing link $\text{Pr} \rightarrow \text{Short}$ and the separate link $U_B \rightarrow \text{Qu}$. The replacement mechanism and every retained or removed link are therefore explicit operation inputs.

Lean implementation. The dependent-signature transition is `CausalEditTransition`; the indexed operation family is `CausalEditOperation`; the eliminator is `result`. A causal link is `CausalLink`. The reindexing constructor is `reindexingBelief`; the low-level pushforward is `BeliefReindexing.Operation`.

Definition A.7.5 (Operation realization of modal traces). Modal traces become executable by assigning operations to their locks.

Realization. A modal do-calculus trace from Definition A.5.13 becomes executable by assigning a supported finite kernel operation to each of its query locks. An intervention lock is realized by a setting transition, an observation lock by a conditioning transition with positive evidence, and each Pearl-rule leaf by its corresponding source-kernel operation. The lock still marks the source

A. Definitions

and target modes; the assigned operation computes the record change, and a realization proof certifies that this change has the denotation named by the lock.

Lean implementation. In Lean, `OperationRealizedJointCertificate` and its conditional counterpart execute the supported source kernels and rule leaves. Their mode-indexed variants additionally certify compatibility of the source mode with the graph. This produces operation-sensitive modal certificates while keeping modal syntax and executable transformations distinct. Their transport and locality properties are proved in Propositions B.7.1 and B.10.6.

Endpoint and intervention semantics

Definition A.7.6 (Current-state endpoint semantics). Endpoint semantics reads probabilities from the literal current record.

Current observed probability. Let $R = (\mathcal{M}, b, x)$ be a finite causal epistemic record, where b is its current latent belief and x its accumulated hard intervention. Its current observed assignment is obtained by the map

$$u \longmapsto \text{sol}_{\mathcal{M}_x}(u),$$

so a decidable observed event E has current-state probability

$$P_R(E) := P_b(E \circ \text{sol}_{\mathcal{M}_x}).$$

Endpoint queries. An endpoint query consists of a numerator event N and a denominator event D . The event N is the complete event whose mass forms the numerator; for a conditional kernel, it already combines the queried outcome with the conditioning event. Provided $P_R(D) > 0$, the partial denotation is therefore

$$\text{endpointDenote}_R(N, D) := P_R(N) / P_R(D).$$

Thus endpoint semantics reads the current belief and accumulated intervention rather than silently returning to the model’s original product prior.

Kernels and paths. For an ordinary kernel, endpoint evaluation first overlays the kernel’s action on the record’s accumulated intervention, with the later action taking precedence at selected coordinates, and then evaluates the kernel numerator and conditioning events under the retained current belief. Path-level endpoint evaluation evaluates an endpoint query at the literal target record of a proof-carrying edit path whose signatures may vary. At an initial record, this agrees with the existing model-level kernel denotation. These agreements are established in Propositions B.7.7 and B.7.8.

Scope. This is an executable semantics for the concrete endpoint histories constructed below. It is not a definition of identifiability after an arbitrary epistemic program and does not assert a new completeness theorem for arbitrary interleavings of observation, intervention, learning and forgetting.

Running example. At the initial Mira record, $P_R(E)$ is the ordinary observational probability of E . After conditioning the belief on the evidence from Definition A.6.1 and setting Pr to 1, endpoint evaluation of $\text{Off} = 1$ gives the corresponding abduction–action–prediction value.

Definition A.7.7 (Atomic structural intervention). A hard intervention has an atomic execution and a compact reference presentation.

Atomic compilation. For a finite causal epistemic record and hard intervention x , atomic compilation constructs an ordered path of the primitive model edits: replace every selected structural equation by its constant and then unrelate every incoming directed parent and latent input at those selected nodes. Its endpoint may use a different dependent-signature presentation, so the path carries explicit coordinate and latent-root transports. Its realization certificate first relates its evaluator to the source model’s hard-intervention evaluator and then supplies a proof-carrying function from every source latent assignment to a target latent assignment. The derived endpoint record transports the current finite epistemic belief through that function.

Compact reference. The one-shot surgical intervention constructs the corresponding graph-mutilated structural causal model directly and is retained as a compact reference presentation. Its plan carries directed-cut, latent-cut and effectiveness obligations, while the atomic path and this compact presentation agree semantically, as proved in Propositions B.7.5 and B.7.7.

Running example. Compiling $\text{Pr} := 1$ in Mira’s model first installs the constant equation and then cuts the incoming latent use of U_B . The compiled endpoint and the compact intervened evaluator therefore agree on the value of every observed event, including $\text{Off} = 1$.

Counterfactual execution

Definition A.7.8 (Executed modal counterfactual trace). The construction combines recursive action traces with executable occurrence-indexed models.

Recursive modal trace. An atom trace assigns the atomic execution and its compact reference transition to one potential-outcome atom $[V_x = v]$. An event trace follows the finite Boolean syntax recursively. The corresponding query trace packages the traces for the outcome and condition. Thus a multiworld query retains every action world as an operation-indexed modal trace rather than as an unstructured annotation. In parallel, a combined construction carries the actual from-factual and from-empty occurrence-indexed executions. Its endpoint evaluation reads the query numerator and denominator from the epistemic record produced by the from-factual atomic endpoint; it does not select an independently specified materialised model. The two generated endpoints are compared through their common occurrence coordinates, including their numerator and denominator probabilities. This comparison agrees with the general counterfactual query denotation and, for one action, with the independent twin-network semantics. The mode-indexed counterfactual transport certificate carries both the recursive action trace and this executed combined construction.

Executed construction. The underlying executed occurrence construction starts from a fresh factual mode. It appends one observed copy for every syntactic atom occurrence, installs the within-world directed links, shared latent incidences and source equations, reindexes the source prior as the endpoint belief, and compiles each occurrence setting and incoming-link cut. The from-empty construction gives the stronger literal route: it begins with no observed variables and no exogenous roots, creates every source root and every factual and counterfactual node, and then performs the same linking, equation, belief-reindexing and intervention stages. Every stage is a typed causal-edit transition, and each composite is a dependent-signature edit path.

A. Definitions

Endpoint agreement. The two endpoints have separately generated dependent signatures and therefore need not be record-identical. Their evaluators are instead compared after transport to the common occurrence coordinates, where their numerator and denominator probabilities agree. This is a coordinate-wise adequacy statement for the query’s fixed finite occurrence universe, not a claim that arbitrary dependent-signature duplication is definitionally identical to repeated terminal learning. The construction and its probability laws are proved in Propositions B.8.1 and B.8.3.

Lean implementation. The recursive traces are `ModalCounterfactualAtomTrace`, `ModalCounterfactualEventTrace` and `ModalCounterfactualQueryTrace`. The combined construction is `ModalCombinedCounterfactualConstruction`, with evaluation `endpointDenote`. The underlying constructions are `ExecutedOccurrenceConstruction` and `FromEmptyExecutedOccurrenceConstruction`. Every stage is a `CausalEditTransition`; each composite is a `CausalEditPath`.

Definition A.7.9 (Modal abduction–action–prediction). The one-action construction has an executable sequence and an agreement result.

Execution. For a supported one-action query, the concrete abduction–action–prediction sequence starts at the model’s prior, conditions the current latent belief on factual evidence, compiles the action into atomic edits, transports the posterior belief to the actual endpoint record, and evaluates the transported target event there. The public action is this atomic execution, and prediction is its endpoint probability.

Agreement. A compact one-shot action is used only as an independently checked reference. The endpoint probability is identified with the model-level counterfactual value, with the compact probability and with the direct twin-network value. These equalities are established in Proposition B.8.3 and Corollary B.8.4.

Running example. For Mira’s evidence E and action $\text{Pr} := 1$, abduction changes the latent belief, atomic action changes the causal evaluation state, and prediction evaluates $\text{Off} = 1$ at the resulting endpoint. The resulting probability agrees with the twin-network value and is calculated as $1/2$ in Example A.8.1.

Lean implementation. The sequence is `AbductionActionPrediction`. The public fields are `action` and `prediction`; the compact reference is `compactAction`; the model-level value is `counterfactualValue`.

Learning and forgetting

Definition A.7.10 (Constructive causal learning and forgetting). The general learning layer treats endogenous and exogenous extensions separately and pairs each with provenance-guided forgetting.

Endogenous learning. Learning may change the dependent observed or latent signature, so it uses a typed causal-edit transition rather than a model-preserving record transition. An endogenous variable specification supplies a finite value type for a new terminal observed variable, old observed parents, any finite subset of the existing exogenous roots, and a mechanism typed exactly over those declared inputs. The extension appends the variable in topological order, preserves every old equation and link, and adds the selected incoming links.

Exogenous learning. An exogenous variable specification supplies an enumerated finite value type and finite rational factor. Its operation appends a fresh root, initially incident to no endogenous variable, extends the current belief by the independent product with that factor, and constructs the corresponding product prior for the extended structural causal model. Subsequent latent-input relating may connect the root to any number of endogenous mechanisms, with each affected equation replaced explicitly.

Forgetting. Forgetting is provenance-guided. The inverse transition is available for exactly the terminal endogenous variable or fresh exogenous root produced by its paired learning transition and restores the stored source record. It does not claim a choice-based deletion operation for an arbitrary dependent variable. The two provenance-guided round trips are established in Proposition B.7.6.

Lean implementation. The edit transition is `CausalEditTransition`; the model-preserving transition is `CausalTransition`. The executable specifications are `EndogenousVariableSpec` and `ExogenousVariableSpec`.

Definition A.7.11 (Conservative terminal learning). The conservative subcase is specified by its data, transitions and preservation properties.

Data. A narrower terminal-variable specification remains useful for conservative query transport. It specifies a finite value type for a new variable L , a set of old observed parents and a deterministic mechanism for L from those parent values. Its extension S^+ appends L as the last variable in the topological order, retains every old directed edge, permits only selected old variables to point into L , and gives L no incident latent root. The extended structural causal model reuses the old latent factors and prior, retains every old mechanism and adds only the specified terminal mechanism.

Transitions. The learning operation performs this conservative extension and lifts the current intervention along the inclusion $S \hookrightarrow S^+$. A typed causal-edit transition contains the source and target signatures, an indexed edit operation, and an equality proving that the operation produced the target. Its operation constructor supplies the learning or forgetting label. The packaged learning transition records the extension; its provenance-guided inverse is available only with the original mode and the terminal specification, and restores that original record. It is not arbitrary deletion of a variable.

Preservation. Old events, interventions and queries are lifted without selecting the new coordinate. For these lifts, old observational, interventional and current-belief probabilities are preserved. Moreover, joint and locally supported conditional identifiability of an old query imply identifiability of its lift to S^+ . This is a conservative one-way preservation theorem, not a claim that arbitrary causal learning preserves identifiability or that every signature extension has this form. The more general endogenous and exogenous edit constructors above supply causal expressivity; this conservative subcase supplies the stronger old-query preservation theorem. These properties are proved in Proposition B.7.6.

Lean implementation. The specification is `TerminalVariableSpec`. The learning operation is `learnRecord`; the packaged transitions are `learnTerminal` and `forgetTerminal`. The transition record is `CausalEditTransition`; the operation family is `CausalEditOperation`.

A.8 Examples

This section applies the finite definitions to Mira's tenure-track model. The same model illustrates directed and bidirected causal structure, d-separation, observation, intervention, counterfactual evaluation and modal execution. Its probabilities are calculated exactly using finite rational assignments and checked by the corresponding Lean implementation. The example introduces no additional foundational syntax or semantic framework.

Example A.8.1 (Mira's tenure-track search, finite calculation). This is the fully finite version of the tenure-track model used in Example 5.4.1. It is assembled from Definitions A.4.9 to A.4.11, A.4.13 to A.4.16, A.4.22 and A.6.1. The calculation is organised by its model, graph, observational and interventional values, counterfactual value and checked realization.

Model. All endogenous value types are the two-element Boolean type. In the numerical formulas below, 1 abbreviates **true** and 0 abbreviates **false**. The endogenous node type is $V_{\text{Mira}} := \text{Fin}(8)$, with names

Pr, Qu, Top, Com, Fit, Short, Fund, Off

in that order. Throughout the example, P denotes probability under the assignment indicated by its argument: exogenous events use the displayed product prior, observational events use the assignment induced by the structural model, and expressions containing **do** or a counterfactual subscript use the corresponding intervened or abducted assignment.

The exogenous variables are independent Boolean variables

$$U_B, U_{\text{Pr}}, U_{\text{Qu}}, U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}},$$

with finite rational masses

$$P(U_B = 1) = \frac{1}{3}, \quad P(U_{\text{Pr}} = 1) = P(U_{\text{Qu}} = 1) = P(U_{\text{Fund}} = 1) = \frac{1}{4},$$

and

$$P(U_{\text{Top}} = 1) = P(U_{\text{Com}} = 1) = \frac{1}{2}.$$

The structural functions are

$$\begin{aligned} \text{Pr} &:= U_B \vee U_{\text{Pr}}, & \text{Qu} &:= U_B \vee U_{\text{Qu}}, \\ \text{Top} &:= U_{\text{Top}}, & \text{Com} &:= U_{\text{Com}}, \\ \text{Fit} &:= \text{Top} \wedge \text{Com}, & \text{Short} &:= \text{Qu} \wedge (\text{Pr} \vee \text{Fit}), \\ \text{Fund} &:= \text{Top} \vee U_{\text{Fund}}, & \text{Off} &:= \text{Short} \wedge \text{Fund}. \end{aligned}$$

Graph. In the full finite directed acyclic graph, U_B is a common cause of **Pr** and **Qu**. If U_B is hidden, the observed semi-Markovian graph has the bidirected edge $\text{Pr} \leftrightarrow \text{Qu}$, so $\{\text{Pr}, \text{Qu}\}$ is a c-component. The graph also contains the colliders

$$\text{Top} \rightarrow \text{Fit} \leftarrow \text{Com}, \quad \text{Pr} \rightarrow \text{Short} \leftarrow \text{Fit}, \quad \text{Short} \rightarrow \text{Off} \leftarrow \text{Fund}.$$

Thus **Top** and **Com** are d-separated by the empty conditioning set, but the path through **Fit** is opened when one conditions on **Fit** or one of its descendants. Numerically, the deterministic equation $\text{Fit} := \text{Top} \wedge \text{Com}$ gives

$$P(\text{Top} = 1 \mid \text{Com} = 1) = \frac{1}{2} = P(\text{Top} = 1),$$

while conditioning on the collider event $\text{Fit} = 0$ changes the relation:

$$P(\text{Top} = 1 \mid \text{Fit} = 0) = \frac{1}{3}, \quad P(\text{Top} = 1 \mid \text{Com} = 1, \text{Fit} = 0) = 0.$$

This is the collider phenomenon in a finite deterministic form.

Observation and intervention. The distinction between observation and intervention is also visible by finite summation. First,

$$P(\text{Pr} = 1) = 1 - P(U_B = 0, U_{\text{Pr}} = 0) = 1 - \frac{2}{3} \cdot \frac{3}{4} = \frac{1}{2}.$$

Similarly $P(\text{Qu} = 1) = 1/2$, and

$$P(\text{Pr} = 1, \text{Qu} = 1) = 1 - \frac{1}{2} - \frac{1}{2} + \frac{2}{3} \cdot \frac{3}{4} \cdot \frac{3}{4} = \frac{3}{8}.$$

Hence

$$P(\text{Qu} = 1 \mid \text{Pr} = 1) = \frac{P(\text{Pr} = 1, \text{Qu} = 1)}{P(\text{Pr} = 1)} = \frac{3/8}{1/2} = \frac{3}{4}.$$

But the intervention $\text{do}(\text{Pr} = 1)$ replaces the equation for Pr by the constant value 1 and leaves the mechanism for Qu unchanged, so

$$P(\text{Qu} = 1 \mid \text{do}(\text{Pr} = 1)) = P(\text{Qu} = 1) = \frac{1}{2}.$$

Committee fit. A second calculation shows why observing committee fit is not the same as forcing committee fit. Since $\text{Fit} = 1$ implies $\text{Top} = 1$ and $\text{Com} = 1$, it also implies $\text{Fund} = 1$. Under $\text{Fit} = 1$, the shortlist equation reduces to $\text{Short} = \text{Qu}$, and therefore

$$P(\text{Off} = 1 \mid \text{Fit} = 1) = P(\text{Qu} = 1) = \frac{1}{2}.$$

By contrast, in the intervened model $\text{do}(\text{Fit} = 1)$, the variable Top is not forced. The offer requires both $\text{Qu} = 1$ and $\text{Fund} = 1$, and these are independent because they depend on disjoint exogenous variables. Since

$$P(\text{Fund} = 1) = 1 - P(\text{Top} = 0, U_{\text{Fund}} = 0) = 1 - \frac{1}{2} \cdot \frac{3}{4} = \frac{5}{8},$$

one gets

$$P(\text{Off} = 1 \mid \text{do}(\text{Fit} = 1)) = \frac{1}{2} \cdot \frac{5}{8} = \frac{5}{16}.$$

Counterfactual calculation. Finally, the same finite model permits a model-level counterfactual calculation. Let

$$E := \{\text{Pr} = 0, \text{Qu} = 1, \text{Off} = 0\}.$$

The braces abbreviate the decidable conjunction of these three Boolean equalities. Under the structural equations, $\text{Pr} = 0$ forces $U_B = 0$ and $U_{\text{Pr}} = 0$. Together with $\text{Qu} = 1$, this forces $U_{\text{Qu}} = 1$. Moreover, when $\text{Pr} = 0$ and $\text{Qu} = 1$, the equation for Short reduces to $\text{Short} = \text{Fit}$. Since $\text{Fit} = 1$ implies $\text{Top} = 1$, hence $\text{Fund} = 1$, the additional evidence $\text{Off} = 0$ forces $\text{Fit} = 0$. Thus

$$P(E) = \frac{2}{3} \cdot \frac{3}{4} \cdot \frac{1}{4} \cdot \frac{3}{4} = \frac{3}{32},$$

A. Definitions

so the abduction step is well defined.

Condition on E , then perform the action $\text{do}(\text{Pr} = 1)$. In the intervened model, $\text{Short} = 1$, because $\text{Qu} = 1$ is retained from the posterior evidence and Pr has been set to 1. Therefore the counterfactual offer depends only on

$$\text{Fund} = \text{Top} \vee U_{\text{Fund}}.$$

The evidence about Pr and Qu is independent of $U_{\text{Top}}, U_{\text{Com}}, U_{\text{Fund}}$, and the remaining relevant information is $\text{Fit} = 0$, i.e. not both Top and Com . Hence

$$P(\text{Fund} = 1 \mid E) = P(\text{Fund} = 1 \mid \text{Fit} = 0) = \frac{1/4 + 1/8}{3/4} = \frac{1}{2}.$$

Consequently

$$P(\text{Off}_{\text{Pr}=1} = 1 \mid E) = \frac{1}{2}.$$

The same posterior also gives

$$P(\text{Off}_{\text{Fit}=1} = 1 \mid E) = \frac{1}{2},$$

because setting $\text{Fit} = 1$ likewise makes $\text{Short} = 1$ while leaving the same posterior uncertainty about Fund . This is Pearl's abduction–action–prediction procedure in the finite model: first condition the exogenous distribution, then change the structural equation, then evaluate the target. It is not a proof that arbitrary counterfactuals are identifiable from the observational distribution alone.

Type-theoretic and Lean implementation. Internally to the thesis framework, this example is a finite structural-causal-model record at an epistemic causal mode:

$$\Gamma_{\text{ten}} \vdash \mathcal{M}_{\text{ten}} : \text{FinSCM}@m_{\text{ten}},$$

where FinSCM is the type of finite recursive structural causal models in the restricted fragment, as introduced in Definition A.5.6. The structural equations are terms in the Boolean value types; the exogenous assignment is a finite rational probability type; and the displayed probability calculations are equalities of finite sums and quotients with positive denominators. The complete executable instance is `TenureTrack.model` in `lean/Thesis/Examples/TenureTrack.lean`. It is an `ExactModel` from the default theorem-facing library, so its observational, interventional, conditional, counterfactual and modal calculations use the same probability records, recursive evaluator and transformations as the correspondence development. Its principal checked values and its agreement with the generic twin network, atomic abduction–action–prediction and the from-factual and from-empty occurrence constructions are established in Corollary B.8.5.

APPENDIX B

Proofs and arguments

“The important thing is life — life and nothing else! What is any ‘discovery’ whatever compared with the incessant, eternal discovery of life?”

Hippolyte in *Idiot*

B. Proofs and arguments

The appendix contains several different kinds of support: philosophical motivation, metatheoretic limitation, a Lean verification boundary, finite probability reconstruction, causal-model correspondence and completeness transfer. The route through these arguments is summarised in Table B.1.

Proof or argument	Purpose	Depends on	Used by
Causality argument	Explains why mathematical inference for science should include causal reasoning, not only probabilistic prediction.	Scientific explanation and Pearl’s seeing–doing–imagining distinction.	Motivates Section 5.4 and the causal transfer arguments.
Physics argument	Defends the constructive and epistemological stance: finite measurements should not force a commitment to completed infinities or fixed ontology.	Gisin-style finite information and Spekkens-style epistemic readings.	Supports the constructive stance used in Sections 5.3 and 5.4.
Excluded middle proof	Shows why global excluded middle is not available in the base constructive theory.	Martin-Löf-style type theory and Kripke/presheaf semantics.	Justifies restricting probability to finite decidable events.
Lean verification boundary	Fixes the meaning of Lean-checked, conditionally transported and externally supported claims.	Lean’s kernel, the focused axiom audit and explicit theorem parameters.	Governs the reading of every Lean-supported section below.
Finite probability reconstruction	Reconstructs finite probability from urn principles, then derives finite probability laws, conditioning and Bayesian update.	Definition A.3.22 and decidable events on finite sample types.	Supports Section 5.3 and the Lean probability checks.
Finite causal correspondence and execution	Connects finite source tables to the intrinsic dependent model, then checks causal edits, conservative extension and finite multiworld execution.	Finite probability records, recursive SCMs and causal modes.	Supplies Sections B.6 to B.9.
Completeness transports and limits	Transports ordinary and counterfactual certificates, then states the exact checked and external boundary.	The internal correspondences and explicit published-theorem specializations.	Supports Theorems B.11.1 and B.13.3 without claiming new formal proofs of ID, IDC, ID* or IDC*.

Table B.1: Route through the proofs and arguments appendix.

B.1 Causality argument

Human beings reason causally almost before they reason formally. We ask what made the glass fall, why the patient recovered, what caused the model to fail, which policy changed the outcome, and what would have happened had the

intervention been different. This is not an exotic philosophical habit. It is one of the ordinary grammars of human orientation.

Science can be described, at least in part, as the disciplined extension of this causal grammar. It discovers, tests, rejects and stabilises causal relations in the material world, the biological world and the social world. A science without causality would not simply be incomplete in a technical sense. It would fail to speak the language in which many scientific questions are actually asked.

None of which is to say that any causal story will do. Scientific inquiry is communal, historical and bounded. With Huizinga, one may say that it contains a playful and creative element: scientists construct models, concepts and experimental situations together [Hui49]. With Erasmus, one may add that such constructions retain an ironic or provisional character: they are humanly made, and therefore should not be confused with reality itself [Era93]. With Kierkegaard, one may add that irony is not limitless: it is bounded by what has already been inherited [Kie80; Kie83; Kie89]. Fleck sharpens this social point: causal judgement is stabilised inside communities, inherited styles of thought, shared techniques and historically formed expectations [Fle79]. Hacking sharpens the practical point: science is not only representational, but interventionist, because scientists isolate, manipulate, produce, measure and stabilise phenomena [Hac83]. To this I add that scientific construction is bounded by intersubjective accountability: claims must pass through other people, practices, instruments, institutions and traditions of criticism.

The result is a middle position. Against a crude Feyerabendian reading, not anything goes [Fey75]. Against a crude Humean scepticism, causal talk need not be metaphysically guaranteed in order to be scientifically indispensable [Hum75; Hum78]. Against a purely Kantian framing, the question whether causality originates in the world or in human cognition becomes secondary once science is understood as a human practice of shared orientation, intervention and explanation [Kan98].

Pearl's framework is therefore justified pragmatically before it is justified metaphysically [Pea09; PM18]. It gives a rigorous language for something science already needs to do: distinguish seeing from doing, association from intervention, and prediction from counterfactual explanation. This thesis uses Pearl not because Pearl solves every philosophical problem about causality, but because any formal theory of mathematical inference for science must give causal reasoning a home.

B.2 Physics argument

The physics argument begins with a modest observation: measurements contain finite information. A measured value is not an actually completed real number handed down from the heavens. It is a finite construction, with precision, procedure, instrument and context. Gisin's work is valuable because it presses this point against the habitual use of completed real numbers in physics [Bar26; BDG25; DG19; Gis20].

On a finite-information picture, measurement can actualise new information and the future can be genuinely open. One can recover analogues of quantum-style indeterminism, measurement-problem-like tensions, Wigner's-friend-like situations and no-cloning phenomena. This is a helpful direction. Yet it does

B. Proofs and arguments

not recover the whole quantum core: incompatible observables, Bell violations, contextuality, the precise Born-rule structure and full quantum entanglement remain outside the classical indeterministic reconstruction.

The difficulty is that Gisin’s proposed numbers still risk importing infinity through the back door. If an observed number is treated as an infinite sequence of propensities, then the number may look constructive while depending on an indefinitely extended object. That is not the constructivist instinct taken seriously enough. A genuinely constructive reading should say that measurement constructs a value up to the precision of the measurement, not that it reveals part of a hidden infinite propensity sequence.

Spekkens’ toy theory supports the epistemological half of the argument [HS10; Spe07]. Many features often advertised as distinctively quantum can arise from restrictions on knowledge: interference-like behaviour, noncommuting measurements, no-cloning, no-broadcasting, teleportation-like structure and epistemic update. Yet Spekkens also does not recover full quantum mechanics. Pusey, Barrett and Rudolph then sharpen the warning: one cannot keep a completely definite underlying ontic state and expect the epistemic view to do all the work [PBR12].

The natural conclusion is agnostic and constructive. Theories should be treated as epistemological vehicles for finite observations. There is no need, for the purposes of this thesis, to decide what the ultimate ontology of physics is. Measurements are finite; formal reasoning about them should therefore begin constructively. The epistemological reading gives room for Spekkens-style insight; the constructive reading gives room for Gisin-style finite information; and the refusal to fix a hidden ontology leaves space for the quantum phenomena that neither classical finite information nor purely epistemic toy theory fully captures.

B.3 Excluded middle is not derivable

The claim used in the main text is modest. It is not that Martin-Löf-style type theory proves the negation of excluded middle as an internal theorem. The claim is metatheoretic: global excluded middle is not derivable in the bare constructive theory. If one wants to use it, one must add it as an additional classical principle.

Proposition B.3.1 (Global excluded middle is not derivable). *In the Martin-Löf-style fragment used in this thesis, the uniform principle, read over small propositions or proposition-like types in the universe,*

$$\prod_{A:\mathcal{U}}(A + \neg A)$$

is not derivable without adding a classical axiom or restricting A to a decidable class of types.

Proof. The argument is by soundness with respect to standard Kripke/presheaf semantics for dependent type theory [AR11; Gra23]. The model used here validates the underlying non-modal fragment needed in this argument: contexts and substitution, dependent function types, finite product and sum types, the empty type, negation as function into the empty type and a small universe

containing the proposition-like type used below. It is not a Boolean model, and it does not validate uniform excluded middle.

The countermodel is used only for the propositional fragment of the argument, so the intensional/extensional distinction for identity types does not affect the failure of uniform excluded middle.

Consider the two-stage Kripke frame

$$w_0 \leq w_1,$$

and let P be an atomic proposition which is not forced at w_0 but is forced at w_1 . Forcing is persistent along extensions: if a proposition is forced at an earlier stage, it remains forced at later stages. Thus information may grow from w_0 to w_1 , so it is coherent for P not to be forced at w_0 while being forced at w_1 .

At w_0 , P is not forced by construction. Moreover, $\neg P$ is not forced at w_0 , because forcing $\neg P$ would mean that no extension of w_0 could force P ; but w_1 is an extension of w_0 where P is forced. Hence neither disjunct of

$$P + \neg P$$

is forced at w_0 . Thus excluded middle is not valid in this sound model of the constructive propositional fragment.

If the displayed uniform excluded-middle term were derivable in bare Martin-Löf-style type theory for a universe containing this small propositional type P , its interpretation would give, at every stage and for every such proposition, either a witness of the proposition or a witness of its negation. Instantiating it with P at w_0 would force $P + \neg P$, contradicting the model above. Therefore no such term is derivable in the base theory. *Formal status.* This is a metatheoretic countermodel argument supported by the cited Kripke/presheaf soundness results; it is deliberately not presented as a new Lean theorem inside the object theory. ■

The same warning applies to double-negation elimination as a global principle. Constructively one always has

$$A \rightarrow \neg\neg A,$$

by sending $a : A$ and $n : A \rightarrow \perp$ to $n(a) : \perp$. The converse

$$\neg\neg A \rightarrow A$$

is a classical principle and is not available uniformly in the base theory.

This does not conflict with the finite probability constructions used in this thesis. For particular finite types, equality, membership and Boolean predicates are decidable by construction. Thus one may use $E + \neg E$ for a finite decidable event E , and one may take complements inside finite event algebras. What is rejected is the global move from an arbitrary proposition A to $A + \neg A$ without a decision procedure or an explicit classical assumption.

B.4 Lean verification and foundational boundary

The excluded-middle argument above is metatheoretic and deliberately not formalised as a Lean theorem. From the finite probability reconstruction in the following section onward, the appendix repeatedly pairs mathematical arguments with Lean declarations. This section fixes how those references are to be read. Its purpose is not to introduce another mathematical assumption, but to separate what Lean checks internally, what Lean checks conditionally from explicit inputs and what remains a cited mathematical or metatheoretic result.

Scope and proof status

Lean is used here as a proof checker for the finite constructions: probability records and urns, structural causal models, source–target correspondence, modal record transformations, finite derivation certificates, counterfactual execution and the tenure-track calculations. A declaration accepted by Lean has been checked from its definitions, imported declarations and explicit parameters. This does not by itself show that every parameter has been constructed in the development. In particular, structures packaging published completeness, soundness, latent-projection or counterfactual-reduction results are deliberately supplied as theorem parameters where the thesis transports rather than reproves those results.

The status expressions used below have the following meanings.

Status	What is established	What is not thereby established
Lean-checked declaration	Lean has accepted the stated type from the cited definitions, prior declarations and displayed parameters.	That every parameter was independently derived in this project.
Checked conditional transport	Lean proves that an explicitly packaged source theorem yields the stated target certificate, equivalence or counterexample.	That the package is inhabited merely because the original paper is accepted; its specialization to the exact finite source interface remains explicit input.
Executable checked example	A closed finite instance is built from the theorem-facing probability and causal library, and Lean verifies the displayed computation or equality.	A theorem about arbitrary models beyond the quantified declaration being instantiated.
Mathematical or metatheoretic argument	The prose proof is supported by the displayed reasoning and cited mathematical results.	A new internal Lean formalisation, unless a separate checked declaration is also named.

Table B.2: Proof-status conventions for the Lean-supported arguments.

A source reference therefore has two parts. A path such as `lean/Thesis/CausalTransport/Correspondence.lean` identifies the module, while a name such as `finiteSource_completeness_transport` identifies the checked declaration. Phrases such as “external package”, “explicit parameter” or “published specialization” identify premises of a checked implication, not

conclusions independently proved by Lean. Declaration names are used instead of source line numbers because they remain stable when surrounding documentation changes.

Intensional kernel and constructive audit

Lean’s judgemental equality is intensional. If Lean has a proof $h : a = b$, the kernel does not thereby treat a and b as definitionally equal, and Lean does not validate equality reflection. In this respect, it shares the central intensional distinction stated in Definitions A.1.5 and A.1.13. Lean is nevertheless not raw Martin-Löf-style type theory: its standard theory has a proof-irrelevant and impredicative proposition sort, quotient types and extensional principles used by ordinary library infrastructure.

The theorem-facing source uses function extensionality when replacing pointwise equality by equality of functions. In Lean’s standard environment, the axiom audit for `funext` reduces to `Quot.sound`; it does not introduce a third reported axiom. The focused audit in `lean/Thesis/AxiomAudit.lean` asks Lean to print the axioms of representative probability, causal, modal, transport, counterfactual and example declarations. Some declarations are axiom-free. For the audited declarations that are not, the reported foundation is contained in

`{propext, Quot.sound}`.

The source contains no executable use of `Classical.*`, no assumed global excluded-middle principle, no explicit `Classical.choice` and no `native_decide`. Finite case distinctions instead use explicit decidability data or structurally recursive Boolean procedures. The claim is scoped to the audited theorem surface and its dependencies; it is not a claim that every theorem in Lean’s imported libraries is constructive.

Consequently, “constructive” here means that the checked thesis declarations do not obtain witnesses by excluded middle or classical choice. It does not mean “accepted by Lean with an empty axiom report”. Even elementary imported arithmetic can inherit `propext` through library implementation: the audit records `Nat.mul_assoc` as a control example for precisely this reason.

Relation to Martin-Löf-style type theory

The formal development is therefore described as *constructive-extensional Lean*. The adjective “constructive” records the absence of excluded middle and classical choice from the audited thesis results. The adjective “extensional” records the use of propositional and functional extensional reasoning and quotient soundness. It does not mean that Lean has extensional judgemental equality or equality reflection.

The target records are intrinsic dependent Lean structures, not an encoding of the raw syntax and derivation judgements of Martin-Löf-style type theory. Their finite types, dependent fields, inductive certificates and proof-carrying transformations follow the intended constructive discipline, but the present development does not prove a syntactic interpretation theorem from Lean into that raw theory. The extensional uses identified by the audit have standard intensional reformulations: equality of functions can be retained pointwise, predicates and finite probability values can be compared by explicit equivalence

B. Proofs and arguments

relations, and quotient representatives can be carried together with setoid-respect proofs. These observations provide a route to a stricter intensional reconstruction, but that reconstruction has not been mechanised here.

The resulting claim is deliberately limited. Lean checks the finite constructions and the stated transports under their explicit inputs within its constructive-extensional foundation. The thesis does not claim that the files themselves constitute a literal formalisation in raw intensional Martin-Löf-style syntax. Conversely, the presence of `propext` or `Quot.sound` must not be misreported as a hidden use of excluded middle: the audit distinguishes those principles explicitly.

The finite joint-assignment construction gives a concrete example of this translation boundary. Lean represents an assignment by the dependent function type `FiniteProduct.Assignment`; its enumeration and equality proofs use function extensionality only when pointwise equal coordinates are assembled into an equality of functions. An intensional reconstruction may instead represent the same finite assignment by the corresponding finite telescope, built recursively from dependent pairs. The recursive enumeration, encoding, decoding and coordinate computations are unchanged, while final equalities of dependent functions are retained as pointwise identities or as an explicit setoid relation. Thus the shorter Lean representation has a direct intensional recipe even though that alternative syntax is not reproduced in the files.

B.5 Clayton–Waddington probability proof

This proof is deliberately finite. It recovers the ordinary finite probability calculus from urn-generated probability types, not the full measure-theoretic theory of countably additive probability. The constructive point is also important: no global law of excluded middle is used. Complements appear only inside finite decidable event algebras. The urn rescaling below is a representation result for the stated Clayton–Waddington-style assumptions; it does not claim that every possible plausibility calculus must satisfy those assumptions. The corresponding Lean 4 formalisation is in the module `Thesis.Probability`, in `lean/Thesis/Probability.lean`.

Finite events, urn rescaling and records

Proposition B.5.1 (Finite-type closure and joint assignments). *Finite types are closed under finite sums, dependent sums and ordinary products. Consequently, if I is finite and $A : I \rightarrow \mathcal{U}$ assigns a finite type $A(i)$ to every $i : I$, the finite telescope obtained by listing one coordinate of type $A(i)$ for every $i : I$ is finite. With pointwise or function-extensional equality, its familiar dependent-function presentation*

$$\prod_{i:I} A(i)$$

is finite as well.

Proof. Finite types stay finite when they are placed side by side or paired. A binary sum is one finite list followed by another. An ordinary product is a pair of finite indices. A dependent sum concatenates the fibres and keeps the fibre index with its local position. Each of these encodings has an inverse by finite case analysis, so the round-trip proofs of Definition A.1.19 are available.

If I is listed as $X_0, \dots, X_{n-1}$, a joint assignment is the ordinary product of the fibres $A(X_k)$, built one coordinate at a time from the one-element empty assignment. That telescope is finite by induction on n . The shorter dependent-function presentation needs one extra step: pointwise equal coordinates become equal functions. The telescope presentation avoids that step.

Lean checks the function presentation as `FiniteWitness` and `FiniteProduct.finiteWitness` in `lean/Thesis/Probability/Construction.lean`. ■

Proposition B.5.2 (Finite rectangular-event characterization). *Let $U = \text{Fin}(n)$, let $\text{Val} : U \rightarrow \mathcal{U}$ be a finite value-type family, let*

$$A : \Pi_{\ell:U} (\text{Val}(\ell) \rightarrow \text{Bool})$$

be a family of coordinate events, and let $\mathbf{u} : \Pi_{\ell:U} \text{Val}(\ell)$ be a complete assignment. The rectangular classifier of Definition A.4.10 satisfies:

$$\text{Id}_{\text{Bool}}(\text{Rect}_A(\mathbf{u}), \text{true}) \iff \Pi_{\ell:U} \text{Id}_{\text{Bool}}(A_\ell(\mathbf{u}_\ell), \text{true}).$$

Proof. A rectangular event holds exactly when every coordinate event holds. Induct on n . The empty family is true, and there is nothing to check. At a successor step the classifier is the Boolean conjunction of the last coordinate with the preceding rectangle. Conjunction is true exactly when both parts are true, and the induction hypothesis reads the second part as the remaining coordinates.

Lean checks this induction as `FiniteProduct.rectangularEvent_eq_true_iff` in `lean/Thesis/Probability/Construction.lean`. ■

Proposition B.5.3 (Finite-sum reindexing). *Let $n : \mathbb{N}$. Let M be a type with an equivalence relation $\approx$, an addition operation and an additive identity 0 , as in Definition A.3.5. Let*

$$u : \text{Fin}(n) \rightarrow M$$

and let

$$\sigma : \text{Fin}(n) \rightarrow \text{Fin}(n)$$

be a permutation, meaning that σ is equipped with an inverse and the corresponding round-trip identity proofs. Then

$$\sum_{i:\text{Fin}(n)} u(\sigma(i)) \approx \sum_{i:\text{Fin}(n)} u(i).$$

Proof. A finite sum does not depend on the order of its terms. Lean first reconstructs a list permutation from equal occurrence counts, then factors that permutation into adjacent swaps. The last index of $\text{Fin}(n)$ is bubbled into place, and the remaining $n - 1$ positions are handled by induction. One adjacent swap replaces a fragment $\dots + x + y + \dots$ by $\dots + y + x + \dots$. Associativity isolates the two summands, and commutativity gives $x + y \approx y + x$. Addition respects $\approx$, so each swap preserves the total, and the whole permutation does as well.

Lean checks the generic fact as `AdditiveCarrier.finSum_reindex` in `lean/Thesis/Probability/Reindexing.lean`. The later urn count argument uses the separate list-count form `count_reindex` in `lean/Thesis/Probability/Core.lean`. ■

B. Proofs and arguments

Proposition B.5.4 (Lean correspondence for nonnegative rational values). *Under the fieldwise correspondence*

$$(a, b, h) \mapsto \{\text{num} := a, \text{den} := b, \text{den_pos} := h\},$$

Lean’s **QProb** is the presentation type $\mathbb{Q}_{\geq 0}$ of Definition A.3.6, and

$$\text{QProb.Equiv}(q, r) \iff q \sim r.$$

Proof. Lean’s **QProb** is exactly the triple of a numerator, a positive denominator and the proof that the denominator is positive. Two such triples are **QProb.Equiv** when their cross-multiplications agree, which is the relation $\sim$ of Definition A.3.6. The same cross-multiplication comparison, with $\leq$ in place of $=$, is **QProb.LE**. Addition, multiplication and division by a positive value respect $\sim$, and positivity of a numerator is independent of the chosen presentation. No quotient type is formed.

Lean checks the equivalence and the operations in `lean/Thesis/Probability/Core.lean`. ■

Classical Cox / Van Horn route	This thesis (finite, constructive)
desiderata: real-valued, common sense, consistency	finite cell types constructed; four qualitative assumptions stated explicitly
associativity equation $\Rightarrow$ product rule first	urn rescaling $U_N(k) \mapsto k/N \Rightarrow$ ratio and sum rule first
negation function $S \Rightarrow$ sum rule second	Cartesian product urns $\Rightarrow$ product rule second
<i>universality</i> (R4) density assumed/derived	<i>universality eliminated</i> : values are k/N by construction
logical equivalence and structural symmetries stated separately	pointwise and relabelling invariance derived from finite counts
continuity on $[0, 1]$	no continuum; complements only on decidable events
construct rescaling p	record–urn equivalence $\rightarrow$ Kolmogorov laws $\rightarrow$ Bayes

Table B.3: Where this thesis’s finite derivation departs from the Cox–Jaynes–Van Horn argument refined by Clayton and Waddington [CW17]: only qualitative coherence, count invariance, strict monotonicity and refinement invariance remain primitive; logical equivalence, relabelling and product urns are derived or constructed.

Proposition B.5.5 (Resolution-status partition). *Let Ω be a finite sample type, let $A : \Omega \rightarrow \mathcal{U}$ and let $\iota_A : \Omega \rightarrow \text{Status}_A$ be a finite inspection procedure as in Definition A.3.2. Then the events Prov_A , Ref_A and Unr_A of Definition A.3.4 are decidable, pairwise disjoint and jointly exhaustive.*

Proof. Each outcome receives exactly one of the three statuses **proved**, **refuted** or **unresolved**. Equality on that three-element type is decidable, so the three events are decidable. If the status is **proved**, the outcome lies in Prov_A and in neither of the other two events. The other two cases are the same. The three events are therefore pairwise disjoint and cover Ω .

Lean checks the partition as `Inspection.status_count_partition` and the probability form as `UrnProb.status_partition`. ■

Proposition B.5.6 (Derived urn-event invariances). *Assume count invariance from Definition A.3.22. Let $N : \mathbb{N}$ and let $E, F : \text{Fin}(N) \rightarrow \text{Bool}$. Then:*

B.5.6.1. *if $E(i) = F(i)$ for every $i : \text{Fin}(N)$, then*

$$\text{plaus}_N(E) \approx_U \text{plaus}_N(F);$$

B.5.6.2. *if $\sigma : \text{Fin}(N) \rightarrow \text{Fin}(N)$ is a permutation, then*

$$\text{plaus}_N(E \circ \sigma) \approx_U \text{plaus}_N(E);$$

B.5.6.3. *if m_E is the number of cells selected by E , then*

$$\text{plaus}_N(E) \approx_U U_N(m_E).$$

Proof. If E and F agree pointwise, their indicators agree pointwise. Summing those equal natural numbers gives $m_E = m_F$, so count invariance gives the first conclusion.

For a permutation σ , apply Proposition B.5.3 to the indicator function $i \mapsto \mathbb{1}_E(i)$. This gives

$$\sum_{i:\text{Fin}(N)} \mathbb{1}_E(\sigma(i)) = \sum_{i:\text{Fin}(N)} \mathbb{1}_E(i).$$

The left side is the count of $E \circ \sigma$, so count invariance gives the second conclusion. Finally, the canonical event C_{N,m_E} selects its first m_E cells and therefore has count m_E . Applying count invariance to E and C_{N,m_E} proves the third conclusion.

Lean checks the pointwise step as `count_congr` and the permutation-of-counts step as `count_reindex` in `lean/Thesis/Probability/Core.lean`. The abstract finite-sum reindexing of Proposition B.5.3 is a separate theorem; the urn argument uses the count form directly. ■

Proposition B.5.7 (Qualitative urn rescaling). *Assume the four clauses of Definition A.3.22. Let*

$$\begin{aligned} N, M, k, \ell &: \mathbb{N}, \\ h_N &: N > 0, \quad h_M : M > 0, \\ h_k &: k \leq N, \quad h_\ell : \ell \leq M. \end{aligned}$$

The qualitative equivalence and strict order of the represented urn plausibilities are characterized exactly by cross multiplication:

$$\begin{aligned} U_N(k) \approx_U U_M(\ell) &\iff kM = \ell N, \\ U_N(k) \prec_U U_M(\ell) &\iff kM < \ell N. \end{aligned}$$

Consequently, the assignment

$$p(U_N(k)) := (k, N, h_N) : \mathbb{Q}_{\geq 0}$$

is well-defined on qualitative-equivalence classes and both preserves and reflects the strict cross-multiplication order of the represented ratios. Now let X be any type, let

$$\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X),$$

B. Proofs and arguments

and let a decidable event $E : X \rightarrow \text{Bool}$ select k cells of μ . Its complement selects $N - k$ cells and

$$P_\mu(E) + P_\mu(E^c) \sim 1.$$

Proof. First suppose that $kM < \ell N$. Refining the N -cell urn by the positive factor M gives

$$U_N(k) \approx_U U_{MN}(Mk),$$

while refining the M -cell urn by the positive factor N gives

$$U_M(\ell) \approx_U U_{NM}(N\ell).$$

The two refined urns have the same size because $MN = NM$. Their selected-cell counts satisfy

$$Mk = kM < \ell N = N\ell.$$

Strict monotonicity in this common urn, followed by compatibility of $\prec_U$ with $\approx_U$, therefore gives

$$U_N(k) \prec_U U_M(\ell).$$

The equality argument is analogous. If $kM = \ell N$, the same two refinements have equal urn sizes and equal selected-cell counts. Same-count invariance and refinement invariance consequently give

$$U_N(k) \approx_U U_M(\ell).$$

Conversely, suppose $U_N(k) \approx_U U_M(\ell)$. Natural-number comparison is decidable, so exactly one of

$$kM < \ell N, \quad kM = \ell N, \quad \ell N < kM$$

holds. The first and third cases would, by the strict comparison just established, make one of the equivalent plausibilities strictly smaller than the other. Qualitative coherence excludes this. Hence $kM = \ell N$.

Finally, suppose $U_N(k) \prec_U U_M(\ell)$. The equality case is impossible because equal ratios give qualitatively equivalent values, and equivalent values cannot be strictly ordered. The reverse inequality $\ell N < kM$ is also impossible: the first part of the proof would give $U_M(\ell) \prec_U U_N(k)$, contradicting asymmetry of the strict order. Therefore $kM < \ell N$. This proves both displayed equivalences.

The first equivalence says precisely that replacing one representative of a qualitative-equivalence class by another replaces (k, N, h_N) by a $\sim$ -equivalent rational presentation. The second equivalence proves both preservation and reflection of the strict cross-multiplication order. For complements, the two selected-cell counts add to $k + (N - k) = N$, and the fraction arithmetic of Definition A.3.6 gives

$$P_\mu(E) + P_\mu(E^c) \sim (N, N, h_N) \sim 1.$$

Checked declarations. The corresponding declarations in `lean/Thesis/Probability/Urn.lean` are:

- `QualitativeUrnScale.eqv_iff_sameRatio` for qualitative equivalence;
- `QualitativeUrnScale.lt_iff_ratio_lt` for the two-way strict-order characterization;

- `rational_rescaling_well_defined` for independence of the rational representative;
- `rational_rescaling_order_iff` for the combined order result;
- `rational_rescaling_order_preserving` and `rational_rescaling_order_reflecting` for its two directions.

The scope remains decidable finite urn events rather than arbitrary propositions. ■

Proposition B.5.8 (Product urns after rescaling). *Let X, Y be types and let*

$$\mu = (M, h_M, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X), \quad \nu = (N, h_N, g) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y).$$

Let $k, \ell : \mathbb{N}$ with $h_k : k \leq M$ and $h_\ell : \ell \leq N$. If $E : X \rightarrow \text{Bool}$ selects k cells of μ and $F : Y \rightarrow \text{Bool}$ selects ℓ cells of ν , their independently constructed rectangular product satisfies

$$P_{\mu \otimes \nu}(E \times F) \sim P_\mu(E)P_\nu(F).$$

Proof. Construct the product cell type $\text{Fin}(M) \times \text{Fin}(N)$. Its row-major encoding is

$$\text{encode}(i, j) := iN + j : \text{Fin}(MN).$$

Conversely, for $q : \text{Fin}(MN)$, define

$$\text{decode}(q) := (\lfloor q/N \rfloor, q \bmod N) : \text{Fin}(M) \times \text{Fin}(N).$$

The bound $q < MN$ and positivity of N ensure that $\lfloor q/N \rfloor < M$ and $q \bmod N < N$. The quotient–remainder identities give

$$\text{decode}(\text{encode}(i, j)) = (i, j) \quad \text{and} \quad \text{encode}(\text{decode}(q)) = q.$$

These maps therefore exhibit the required equivalence

$$\text{Fin}(M) \times \text{Fin}(N) \simeq \text{Fin}(MN).$$

The labelled events induce cell events

$$E_f(i) := E(f(i)), \quad F_g(j) := F(g(j)).$$

Their rectangular cell event is the Boolean classifier

$$(E_f \times F_g)(i, j) := \text{and}_{\text{Bool}}(E_f(i), F_g(j)).$$

Enumerating the product cells row by row, every cell selected by E_f is paired with each of the ℓ cells selected by F_g . Since E_f selects k cells, the rectangular event consequently selects

$$\sum_{i : \text{Fin}(M)} \mathbb{1}_{E_f}(i) \ell = k\ell$$

cells. Transport through the displayed equivalence preserves this count and gives a Boolean event on $\text{Fin}(MN)$ selecting the same $k\ell$ cells. The proofs h_M and h_N determine a proof $h_{MN} : MN > 0$. Therefore the rescaling of Proposition B.5.7 gives

$$P_{\mu \otimes \nu}(E \times F) \sim (k\ell, MN, h_{MN}) \sim P_\mu(E)P_\nu(F).$$

B. Proofs and arguments

The second equivalence follows directly by cross multiplication. This constructs the product of the two given urn assignments; it does not assert that an arbitrary joint assignment on the product type is independent.

Lean checks the cell count as `FiniteCellProduct.selectedCells_product` and the probability equation as `UrnProb.product_probVal`. This constructs the product of two given urns; it does not say that every joint assignment is independent. ■

Proposition B.5.9 (Finite rational records admit a common denominator). *Let Ω be a finite sample type and let*

$$\mu = (p, h) : \text{Prob}_{\mathbb{Q}}(\Omega)$$

be a finite rational probability record in the sense of Definition A.3.8. Then μ has an extensionally equal presentation in the common-denominator form of Definition A.3.9: there are a positive natural number D and natural numbers m_{ω} indexed by $\omega : \Omega$ such that

$$p(\omega) = \frac{m_{\omega}}{D} \quad \text{for every } \omega : \Omega,$$

and

$$\sum_{\omega : \Omega} m_{\omega} = D.$$

Proof. Enumerate Ω , multiply the finitely many positive denominators, and scale each numerator by the other denominator factors. Normalisation then says that the resulting natural weights sum to that common denominator.

Lean checks the conversion as `CommonDenominator.FiniteQMass.toRecord_preserves` in `lean/Thesis/Probability/Construction.lean`. ■

Proposition B.5.10 (Common-denominator record–urn equivalence). *Let X be any type. For weighted finite-support records in the common-denominator form of Definition A.3.9, urn presentations and record presentations are extensionally equivalent. More precisely:*

B.5.10.1. *every urn-generated assignment $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ determines a weighted record R_{μ} such that*

$$P_{R_{\mu}}(E) \sim P_{\mu}(E)$$

for every decidable event $E : X \rightarrow \text{Bool}$;

B.5.10.2. *every weighted common-denominator record R determines an urn-generated assignment μ_R such that*

$$P_R(E) \sim P_{\mu_R}(E)$$

for every decidable event $E : X \rightarrow \text{Bool}$.

Consequently, after choosing a common denominator, finite rational records on finite sample types may be treated as urn presentations without changing any event probabilities.

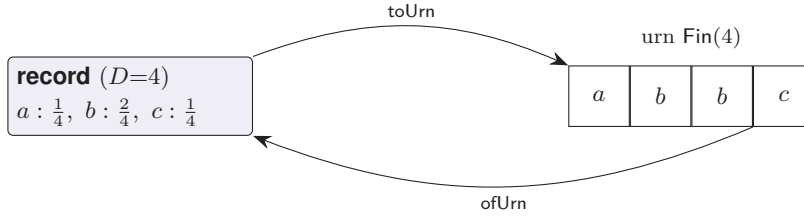


Figure B.1: Record and urn presentations of a finite rational distribution agree on every decidable event (Proposition B.5.10); the maps `toUrn/ofUrn` are the computable witnesses checked in Lean.

Proof. An urn becomes a record by giving each labelled cell unit weight. A record becomes an urn by repeating each label according to its natural weight. Both constructions count the same selected cells, so event probabilities agree even though the presentations are not the same object.

Lean checks the two directions as `FiniteProbRecord.urn_to_record_extensional` and `FiniteProbRecord.record_to_urn_extensional`. ■

Proposition B.5.11 (Pushforward of an urn assignment). *Let $\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ and let $g : X \rightarrow Y$. Relabelling every cell by g gives the pushforward urn*

$$g_*\mu := (N, h_N, g \circ f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(Y).$$

For every decidable event $E : Y \rightarrow \text{Bool}$,

$$P_{g_*\mu}(E) \sim P_{\mu}(E \circ g).$$

Proof. Both sides inspect the same N cells. A cell i is selected on the left exactly when $E(g(f(i)))$ is true, which is the condition used on the right. The selected-cell counts and denominators are therefore identical.

Lean checks the event equation as `UrnProb.probVal_map`. ■

Finite probability laws

Theorem B.5.12 (Finite Kolmogorov laws). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events. Then:*

B.5.12.1. $P_{\mu}(E) \geq 0$;

B.5.12.2. $P_{\mu}(\top_X) \sim 1$, where $\top_X(x) := \text{true}$ is the total event;

B.5.12.3. if E and F are disjoint, then

$$P_{\mu}(E \cup F) \sim P_{\mu}(E) + P_{\mu}(F).$$

More generally, every finite pairwise disjoint family of decidable events is finitely additive.

B. Proofs and arguments

Proof. Event probability is a natural cell count divided by the positive urn size, so it is nonnegative. The total event counts every cell, hence has probability 1. If E and F are disjoint, their indicators add pointwise, and so do their counts. The finite-family statement is the same argument by induction. No countable additivity is claimed.

Lean checks the one- and two-event facts as `UrnProb.probNum_nonneg`, `UrnProb.normalization` and `UrnProb.finite_additivity`. ■

Corollary B.5.13 (Probability monotonicity). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events with $E \subseteq F$, that is, $E(x) = \text{true}$ implies $F(x) = \text{true}$ for every $x : X$. Then*

$$P_{\mu}(E) \leq P_{\mu}(F).$$

Proof. If E is contained in F , every selected cell of E is a selected cell of F . The finite counts are therefore ordered, and division by the same positive denominator preserves that order.

Lean checks the rational inequality as `UrnProb.monotonicity_probVal`, using `QProb.LE`. ■

Corollary B.5.14 (Two-event inclusion-exclusion). *Let $\mu = (N, h, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events. Then*

$$P_{\mu}(E \cup F) + P_{\mu}(E \cap F) \sim P_{\mu}(E) + P_{\mu}(F).$$

Proof. Proof idea. For every urn cell, the Boolean indicators satisfy $\mathbb{1}_{E \cup F} + \mathbb{1}_{E \cap F} = \mathbb{1}_E + \mathbb{1}_F$. Summing this pointwise identity gives the result. *Checked declaration.* The equality is `UrnProb.inclusion_exclusion` in `lean/Thesis/Probability.lean`. ■

Corollary B.5.15 (Resolution-status probabilities). *Let Ω be a finite sample type, let $A : \Omega \rightarrow \mathcal{U}$, let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(\Omega)$, and let $\iota_A : \Omega \rightarrow \text{Status}_A$ be a finite inspection procedure. Then*

$$P_{\mu}(\text{Prov}_A) + P_{\mu}(\text{Ref}_A) + P_{\mu}(\text{Unr}_A) \sim 1.$$

Proof. By Proposition B.5.5, the three resolution-status events are pairwise disjoint and jointly exhaustive. Finite additivity from Theorem B.5.12 gives

$$P_{\mu}(\text{Prov}_A \cup \text{Ref}_A \cup \text{Unr}_A) \sim P_{\mu}(\text{Prov}_A) + P_{\mu}(\text{Ref}_A) + P_{\mu}(\text{Unr}_A).$$

The union is the total event $\top_{\Omega}$, whose probability is 1 by normalisation.

Lean checks this as `UrnProb.status_partition`. ■

Conditioning and Bayesian update

Theorem B.5.16 (Bayes' theorem for decidable finite events). *Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, and let $E, F : X \rightarrow \text{Bool}$ be decidable events with $P_{\mu}(E) > 0$ and $P_{\mu}(F) > 0$. With conditional probability defined by*

$$P_{\mu}(E \mid F) := \frac{P_{\mu}(E \cap F)}{P_{\mu}(F)},$$

one has

$$P_{\mu}(E \mid F) \sim \frac{P_{\mu}(F \mid E)P_{\mu}(E)}{P_{\mu}(F)}.$$

Proof. Proof idea. Expand both conditionals as intersection counts, use commutativity of intersection, and cancel only the explicitly positive denominators. *Checked declarations.* The cross-multiplied identity is `UrnProb.bayes_product_rule`; the quotient form is `UrnProb.bayes_formula`. ■

Proposition B.5.17 (Product and conditional characterisations of independence). *Let $\mu : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$ and let $A, B, C : X \rightarrow \text{Bool}$ be decidable events. Write m_A, m_B, m_C and $m_{B \cap C}$ for the selected-cell counts of A, B, C and $B \cap C$, respectively. Whenever the displayed positivity witnesses are supplied, the following equivalences hold:*

B.5.17.1. *if $h_B : m_B > 0$, then*

$$P_{\mu}(A \cap B) \sim P_{\mu}(A)P_{\mu}(B) \iff P_{\mu}(A \mid B) \sim P_{\mu}(A);$$

B.5.17.2. *if $h_A : m_A > 0$, then*

$$P_{\mu}(A \cap B) \sim P_{\mu}(A)P_{\mu}(B) \iff P_{\mu}(B \mid A) \sim P_{\mu}(B);$$

B.5.17.3. *if $h_C : m_C > 0$, then*

$$\begin{aligned} P_{\mu}(A \cap B \cap C)P_{\mu}(C) &\sim P_{\mu}(A \cap C)P_{\mu}(B \cap C) \\ \iff P_{\mu}(A \cap B \mid C) &\sim P_{\mu}(A \mid C)P_{\mu}(B \mid C); \end{aligned}$$

B.5.17.4. *if $h_C : m_C > 0$ and $h_{B \cap C} : m_{B \cap C} > 0$, then*

$$\begin{aligned} P_{\mu}(A \cap B \cap C)P_{\mu}(C) &\sim P_{\mu}(A \cap C)P_{\mu}(B \cap C) \\ \iff P_{\mu}(A \mid B \cap C) &\sim P_{\mu}(A \mid C). \end{aligned}$$

Proof. Expand every probability into its selected-cell count and positive urn denominator. In the first two cases, the product equation and the conditional equation both reduce, after cancelling the positive urn size or conditioning count, to

$$m_{A \cap B}N = m_A m_B.$$

In the third and fourth cases, both sides reduce to

$$m_{A \cap B \cap C}m_C = m_{A \cap C}m_{B \cap C}.$$

Boolean associativity identifies the two parenthesisations of $A \cap B \cap C$. All cancellations use the displayed positivity witnesses.

Lean checks the four equivalences in `lean/Thesis/Probability/Urn.lean`, beginning with `UrnProb.independent_iff_condVal_left`. ■

Proposition B.5.18 (Bayesian update is conditional probability). *Let $\mu = (N, h_N, f) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$, let $E : X \rightarrow \text{Bool}$ be a decidable event with selected-cell count m_E , and suppose $h_E : m_E > 0$. Let μ_E be the filtered urn of Definition A.3.12. Then, for every decidable event $F : X \rightarrow \text{Bool}$,*

$$P_{\mu_E}(F) \sim P_{\mu}(F \mid E).$$

B. Proofs and arguments

Proof. The posterior urn keeps exactly the original cells that satisfy E , so its denominator is m_E . Among those cells, F selects the cells that satisfy both E and F . Both sides are therefore the ratio $m_{F \cap E}/m_E$.

Lean checks the eventwise equation as `UrnProb.conditionOn_probVal`. The common-denominator form used later is `FiniteProbRecord.conditionOn_probVal`. ■

Proposition B.5.19 (Bayesian update is a probability assignment). *Under the assumptions of Proposition B.5.18, the filtered assignment*

$$\mu_E = (m_E, h_E, f_E) : \text{Prob}_{\mathbb{Q}}^{\text{urn}}(X)$$

is a nonempty urn-generated probability assignment and is normalised:

$$P_{\mu_E}(\top_X) \sim 1.$$

Proof. The proof $h_E : m_E > 0$ makes the filtered cell type nonempty. Every one of those m_E cells satisfies the total event, so the posterior probability is $m_E/m_E \sim 1$.

Lean checks this as `UrnProb.conditionOn_normalization`. ■

The theorem applies immediately to the resolution-status events Prov_A , Ref_A and Unr_A , provided the conditioning event has positive probability. It does not turn an arbitrary proposition A into a decidable proposition. In particular, there is no general constructive theorem

$$P(\neg A \mid I) = 1 - P(A \mid I).$$

What is available is the finite resolution-status decomposition:

$$P(\text{Prov}_A \mid B) + P(\text{Ref}_A \mid B) + P(\text{Unr}_A \mid B) \sim 1$$

whenever B is a decidable conditioning event of positive probability. Thus the probability of proof and refutation may sum to less than 1; the remaining mass is exactly $P(\text{Unr}_A \mid B)$, the probability assigned to unresolved cases under the chosen finite inspection procedure.

B.6 Finite causal-model correspondence

This appendix records the proof architecture checked by Lean. It does not repeat the published identification proofs. The central distinction is between the finite model correspondence, which is internal to the Lean development, and the classical completeness theorem, which is supplied to the final transport theorem as an explicit parameter.

The source and target fragments used below are defined in Definition A.5.5. Direct-root projection is defined in Definition A.4.12; its Lean constructions are `LatentExtension.projectedBidirected` and `LatentExtension.observedGraph` in `lean/Thesis/Causality/Model.lean`. General hidden-DAG data and their projection are defined in Definitions A.4.25 and A.4.26; their Lean records are `FiniteHiddenDAG` and `FiniteLatentProjection` in `lean/Thesis/Causality/Reductions.lean`.

The principal declarations for the finite model correspondence are `FiniteTableSignature`, `FiniteTableLatent`, `FiniteTableSCM`, `FiniteTableSCM.interpret`, `ObservedSignature` and `ExactModel`. The independent source interpretation and its pointwise preservation theorems are in `lean/Thesis/CausalTransport/FiniteSource.lean`; the intrinsic target model is in `lean/Thesis/Causality/Model.lean`, with `ExactModel` and hard interventions in `lean/Thesis/Causality/Core.lean`.

Proposition B.6.1 (Finite-index order, directed irreflexivity and ordered acyclicity). *The strict natural-number order of Remark A.1.18 is transitive and irreflexive. Consequently, the directed classifier of a topologically ordered graph is irreflexive, and a directed acyclic graph of Definition A.4.6 has no positive directed path from a node to itself.*

Proof. The usual order of natural numbers is transitive and has no loops. Transitivity is induction on the second witness, with a case split on the first. Irreflexivity is induction on the number: nothing proves $0 < 0$, and a successor loop would shrink to a smaller loop.

If a directed edge is required to raise rank, it cannot run from a node to itself. A longer directed path would then raise rank at every step, so a path from X back to X would give $\text{rank}_n(X) <_{\mathbb{N}} \text{rank}_n(X)$, which is impossible.

Lean records the rank constraint as `directed_earlier` on an observed signature. On a hidden DAG the same fact is the field `edge_rank_lt`, and `HiddenInternalPath.irreflexive` says that a hidden-internal path cannot begin and end at the same node. ■

Proposition B.6.2 (Finite node classifiers and selected subfamilies). *Let $V = \text{Fin}(n)$ be a finite node type. Every Boolean node classifier $S : V \rightarrow \text{Bool}$ determines a natural number m_S and an embedding $j_S : \text{Fin}(m_S) \rightarrow V$ whose image consists exactly of the nodes classified as `true`. The canonical construction enumerates this image in ambient index order and therefore reflects strict order:*

$$\text{rank}_n(j_S(k)) <_{\mathbb{N}} \text{rank}_n(j_S(\ell)) \implies \text{rank}_{m_S}(k) <_{\mathbb{N}} \text{rank}_{m_S}(\ell).$$

Consequently, pulling a topologically ordered directed edge classifier back along j_S supplies a topologically ordered classifier on $\text{Fin}(m_S)$. Conversely, every embedding $j : \text{Fin}(m) \rightarrow V$ determines a Boolean classifier selecting exactly its image. Two such presentations of the same selected nodes differ at most by a finite reindexing, so their dependent joint-assignment types are equivalent by coordinate transport. If V is the indexed presentation of a finite variable-name type I , then $\text{decode} \circ j_S : \text{Fin}(m_S) \rightarrow I$ is also an embedding.

Proof. Use the canonical enumeration $0, \dots, n-1$ of V and filter it by S . Let m_S be the length of the resulting list. Its k th entry defines $j_S(k)$; duplicate-freeness makes this map injective, and the filter law gives

$$(\exists k : \text{Fin}(m_S), j_S(k) = v) \iff S(v) = \text{true}.$$

Filtering does not change the relative order of the retained entries. Hence, if $j_S(k)$ occurs earlier than $j_S(\ell)$ in the ambient enumeration, then k occurs earlier than ℓ in the filtered enumeration. This is precisely the displayed order-reflection

B. Proofs and arguments

map. If a directed edge from $j_S(k)$ to $j_S(\ell)$ carries ambient topological-order evidence, applying this map gives the required inequality between k and ℓ .

Conversely, finite search through $\text{Fin}(m)$ decides whether $j(k) = v$ for some k , because finite types have decidable equality by Remark A.1.36. If two embeddings have the same image, finite search and injectivity give mutually inverse index maps; precomposition with these maps transports their dependent assignments coordinate by coordinate. Finally, if $\text{decode}(j_S(k)) = \text{decode}(j_S(\ell))$, applying encode and then the round-trip proof ε of Definition A.1.19 gives $j_S(k) = j_S(\ell)$; injectivity of j_S therefore gives $k = \ell$.

Lean checks the length, embedding, injectivity, image and order-reflection facts as `selectedNodeCount`, `selectedEmbed`, `selectedEmbed_inj`, `mem_iff_exists_selectedEmbed` and `selectedEmbed_lt_reflect` in `lean/Thesis/Causality/Modalities.lean`. The last part of the proposition, about equivalent assignment types and $\text{decode} \circ j_S$, is left as prose. ■

Proposition B.6.3 (Canonical pair-root expansion). *Let $\mathcal{G} : \text{CausalGraph}(V)$ be finite. Introduce one hidden root $U_{i,j}$ for every bidirected edge whose endpoints satisfy $\text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j)$, and give that root precisely the two outgoing edges:*

$$U_{i,j} \rightarrow i, \quad U_{i,j} \rightarrow j.$$

Retain the directed part of $\mathcal{G}$. The resulting graph is a finite all-directed acyclic graph, and hiding the introduced roots recovers exactly the directed and bidirected classifiers of $\mathcal{G}$.

Proof. Enumerate the finite type $V \times V$ and retain exactly those pairs satisfying the decidable rank and bidirected-edge tests displayed in the proposition. Indexing this filtered list by $\text{Fin}(m)$ gives the finite type of introduced roots. Put every such root before every observed node; the retained observed arrows follow the existing topological order, so the expanded graph is acyclic.

Its directed observed classifier is unchanged by construction. For the bidirected classifier, suppose first that $E_{\mathcal{G}}^{\leftrightarrow}(i, j) = \text{true}$. Bidirected irreflexivity gives $i \neq j$, and comparison in the finite total order gives either $\text{rank}_n(i) <_{\mathbb{N}} \text{rank}_n(j)$ or the reverse inequality. In the first case $U_{i,j}$ is present; in the second, symmetry of the classifier makes $U_{j,i}$ present. Either root has arrows into both i and j , so latent projection selects the pair.

Conversely, a projected bidirected edge must be witnessed by one of the introduced roots. Such a root exists only for an originally selected bidirected pair and is incident only to its two endpoints. The queried pair is therefore that edge, possibly with its endpoints reversed; in the latter case symmetry supplies the required orientation. Hence no bidirected edge is lost or added. This proposition concerns graph structure only and supplies no probability assignment or structural mechanism for the introduced roots.

Lean checks the graph-only expansion as `pairRootExtension_projected` and the hidden-DAG form as `pairRootHiddenDAG_directed` and `pairRootHiddenDAG_bidirected` in `lean/Thesis/Causality/PairRoot.lean`. No probability or mechanism data are supplied. ■

Theorem B.6.4 (Finite causal-model adequacy). *Every finite table SCM of Definition A.5.5 has an intrinsic dependent target interpretation. Source-table evaluation and target evaluation agree at every node, and the interpretation*

preserves every observational and hard-interventional event value pointwise. On an interpreted finite-table signature, every target exact model can in turn be presented as a finite table SCM. Source and target observational equivalence, joint identifiability and conditional identifiability are therefore equivalent.

Proof. A finite table SCM can be read as an intrinsic dependent model, and that reading does not change node values or event probabilities. Source-table evaluation and the target's well-founded evaluator agree at every node. Observational, interventional and kernel event values are therefore the same, up to rational equivalence. Going back from an interpreted table signature to a table SCM is the reverse presentation. Identifiability questions asked on either side are therefore the same questions.

Lean checks the nodewise agreement as `FiniteTableSCM.evalNodeUnder_preserved` and the event values as `observational_preserved` in `lean/Thesis/CausalTransport/FiniteSource.lean`. This is correspondence for the coded finite source class, not a syntax theorem for every classical SCM presentation. ■

Proposition B.6.5 (Agreement of prefix and well-founded structural evaluation). *Let $\mathcal{M}$ be a structural causal model in the sense of Definition A.4.14, let*

$$\tau : \Pi_{i:V} \text{Option}(\text{Val}(i))$$

be an equation override, and let $\mathbf{u} : \text{Val}(\mathcal{U})$ be a complete exogenous assignment. Write $\mathbf{s}_k^\tau$ for the prefix recursion of Definition A.4.15 with the equation override τ applied at each newly evaluated node. For every $k \leq n$ and $j : \text{Fin}(k)$, this recursion agrees with direct well-founded evaluation at the corresponding node:

$$\mathbf{s}_k^\tau(\mathbf{u})(j) = \text{evalNodeUnder}_{\mathcal{M},\tau}(\mathbf{u}, \iota_k(j)).$$

Consequently, the complete assignments agree:

$$\mathbf{s}_n^\tau(\mathbf{u}) = \text{evalUnder}_{\mathcal{M},\tau}(\mathbf{u}).$$

Mapping the exogenous prior through either evaluator therefore gives the same interventional probability record. For the empty override, both constructions give the observational record `observationalDist $\mathcal{M}$` .

Proof. Induct on k . The empty prefix has no coordinate. At a successor stage, the topological-order field sends every parent of the new final node to an index in the already constructed prefix. The induction hypothesis identifies each of those stored parent values with direct well-founded evaluation. Both evaluators read the same exogenous coordinates and apply the same equation override and structural mechanism, so they agree at the new final coordinate. Every earlier coordinate is retained unchanged by the constructive finite-prefix extension, where the induction hypothesis applies directly.

Taking $k = n$ gives pointwise agreement on the complete assignment. Function extensionality packages those coordinatewise equalities as equality of the two Lean assignment functions. Congruence of `FiniteProbRecord.map` then gives equality of the corresponding pushforward records, both for an arbitrary override and for the empty observational override.

Lean checks the pointwise induction as `FiniteLatentSCM.evalPrefixUnder_eq_evalNodeUnder` in `lean/Thesis/Causality/Model.lean`. ■

B. Proofs and arguments

Lemma B.6.6 (Pointwise finite semantic agreement). *Interpreting a finite source table preserves recursive node values, observational event values, interventional event values and the projected graph pointwise.*

Proof. The same nodewise and eventwise agreement used in Theorem B.6.4 is the whole claim. Whole records need not be identical; rational probabilities are compared by cross multiplication.

Lean checks this as the preservation theorems of `FiniteTableSCM` in `lean/Thesis/CausalTransport/FiniteSource.lean`. ■

Proposition B.6.7 (Finite probability-expression transport). *Observational and interventional event probabilities are preserved by the encoding. Consequently, any expression formed from those values by the same finite rational operations on both sides has the same value whenever its displayed denominators are positive.*

Proof. Source and target evaluators agree at every node, so the same finite rational expression has the same value on both sides whenever its displayed denominators are positive. A zero denominator is reported as undefined, not replaced by an arbitrary rational. Support is checked along the source derivation and then transported.

Lean checks the composite expressions as `FiniteTableSCM.termDenote_preserved`. ■

Proposition B.6.8 (Checked graph-predicate transport). *Interpreting a source graph preserves the observed directed–bidirected graph and therefore preserves predicates and computations built directly from the graph objects formalised in the theorem-facing modules: directed reachability, bidirected connectivity, internally connected induced node restrictions, c-forests, graph mutilation, the finite ancestral-moralisation d-separation test and hedge witnesses.*

Proof. Interpreting a source graph does not change the observed directed or bidirected edges. Predicates built from those edges—reachability, bidirected components, c-forests, mutilation, the finite d-separation test and hedge witnesses—are therefore the same predicates after interpretation.

Lean defines those graph objects in `lean/Thesis/Causality/Identification.lean` and `lean/Thesis/Causality/Derivation.lean`. The two shared-cause calculations `shared_cause_is_d_connected` and `intervention_cut_d_separates_shared_cause` are two-node examples in `lean/Thesis/Examples/TenureTrack.lean`, not general theorems. Equivalence of the executable test with the external active-path specification remains the field of `DSeparationCorrectness`. ■

Proposition B.6.9 (Compatible-model-class equivalence). *Compatibility with a fixed observed graph is preserved and reflected by interpretation, as are observational equivalence and joint or locally supported conditional kernel identifiability.*

Proof. Compatibility with a fixed observed graph, observational equivalence and the two identifiability questions are the same on the source table and on its interpreted model.

Lean checks the three equivalences as `finiteSource_observational_iff`, `finiteSource_identifiable_iff` and `finiteSource_conditionalIdentifiable_iff`. ■

B.7 Modal and structural causal execution

This section concerns executable, proof-carrying changes of finite causal records. It is separate from the source–target correspondence above: `Modalities.lean` defines the core modes and transitions, the modules under `Causality/Structural/` define indexed dependent-signature edits and their semantics, and `CausalTransport/Modal.lean` together with `CausalTransport/ModalRealization.lean` connects those operations to identification certificates. The modality records the valid source–target transition; the separately defined operation constructs the target record; and the transition witness certifies that the operation was performed.

Proposition B.7.1 (Modal locality of the correspondence). *The correspondence applies to the exact finite causal model stored at a mode. A modality is not itself a graph-rewriting oracle: an operation constructs the target record, and a transition carries a witness that its source and target are related by the named operation. Every symbolic kernel also determines its action and observation locks, and every checked do-calculus derivation determines a modal trace between the corresponding locked query modes. When local support and primitive soundness are supplied, the source kernel and every Pearl-rule leaf of that trace also carry the concrete intervention–observation execution and its probability equality. Likewise, every potential-outcome atom in a finite counterfactual query carries the executable atomic structural intervention for its action world. The query as a whole carries an occurrence-indexed atomic construction from both a fresh factual mode and a genuinely empty mode. The two generated dependent endpoints agree after coordinate transport and have equivalent query probabilities; the epistemic endpoint record, rather than a parallel reference model, supplies the probability used by the mode-indexed reduction.*

Proof. Record locality is enforced by `CausalRecordStep`, `CausalTransition.valid` and `CausalRecordStep.model_eq` in `lean/Thesis/Causality/Modalities.lean`. Query locality is `CausalQueryMode.kernel_built_from_locks`, and derivation annotation is `DoCalculusDerivation.toModalTrace` in `lean/Thesis/Causality/ModalDerivation.lean`. Concrete endpoint execution is `KernelOperationRealization`; recursive rule-leaf execution is `ModalDerivationTrace.realizeOperations` in `lean/Thesis/Causality/ModalRealization.lean`. Counterfactual-world locality is `ModalCounterfactualQueryTrace.canonical`. The combined construction is `ModalCombinedCounterfactualConstruction.canonical`; its endpoint comparison is `endpointEvaluations_agree` and its probability-level bridge is `semanticAgreement`, in `lean/Thesis/Causality/ModalCounterfactual.lean`. ■

The proof-carrying record transformations used here are defined in Definitions A.7.3 to A.7.5.

B. Proofs and arguments

Proposition B.7.2 (Finite modal transition preservation). *Each model-preserving fixed-signature finite epistemic transition preserves every record component it is not specified to change, and its target is definitionally the output of the corresponding record operation. Conditioning, setting and unsetting preserve the exact structural model. Simultaneous finite intervention and selected-node observation are the multi-node specializations used to realise query locks. Simultaneous intervention is extensionally the duplicate-free sequential fold of its selected single-node settings; hence the multi-node operation is a convenience macro, not an additional causal primitive. This proposition does not apply to `CausalEditTransition`, whose purpose is precisely to permit an explicit model or signature change.*

Proof. Model and prior preservation under conditioning are `conditioning_keeps_model` and `conditioning_keeps_product_prior`. Belief preservation under the action updates is `setting_keeps_belief` and `unsetting_keeps_belief`. Uniform model preservation is `CausalRecordStep.model_eq`; endpoint validity is the `valid` field of every transition. Simultaneous intervention is connected to its lock by `CausalTransition.intervening_realizes_intervention`. Observation uses `CausalTransition.observing_realizes_observation`. Agreement of an empty-record action with kernel intervention is `HardIntervention.empty_setNodes_eq_kernel_intervention`. The fold theorem is `CausalEpistemicRecord.interveneNodes_eq_sequential`; its duplicate-free enumeration is `selectedNodeList`. ■

Proposition B.7.3 (Modal identification invariance). *Let a proof-carrying fixed-signature finite epistemic transition connect two causal modes. A joint or conditional kernel is identifiable at the source mode exactly when it is identifiable at the target mode. Compatibility with any fixed observed graph is preserved as well. At either endpoint compatible with a finite source graph, supplied finite-source completeness and soundness packages identify fixed-graph identifiability with the existence of a mode-indexed certificate that contains the corresponding encoded do-calculus derivation and its modal trace. If primitive soundness is also used to realize the trace, the corresponding mode-indexed operation-realized certificate is equivalent to identifiability as well.*

Proof. Every `CausalRecordStep` preserves the exact structural model, so its observed graph and compatible-model identification class are unchanged. Joint endpoint transport is `CausalMode.finiteSourceTransported_joint_iff`. Conditional endpoint transport is `CausalMode.finiteSourceTransported_conditional_iff`. Transition invariance is `CausalTransition.jointIdentifiable_iff`, `CausalTransition.conditionalIdentifiable_iff` and `CausalTransition.compatibleWith_iff` in `lean/Thesis/CausalTransport/Modal.lean`. The joint certificate type is `ModeIndexedJointDerivation`. Its conditional counterpart is `ModeIndexedConditionalDerivation`. The combined mode-indexed forward theorem is `CausalMode.finiteSource_modal_completeness_transport`. The stronger two-way results combining the compatible modal index with executable kernel endpoints are `finiteSource_modeIndexedOperationRealized_joint_iff` and its conditional counterpart in `lean/Thesis/CausalTransport/ModalRealization.lean`. Their certificate types are `ModeIndexedOperationRealizedJointCertificate` and

ModeIndexedOperationRealizedConditionalCertificate. These integrate the modal index and rule-by-rule locks with separately defined executable transformations, as required by Remark A.2.10. ■

Proposition B.7.4 (Bayesian conditioning as a modal transition). *For positive-probability latent or observed evidence, conditioning replaces the current belief by its finite Bayesian posterior and leaves the causal model, its independent-root prior and the intervention unchanged.*

Proof. The latent- and observed-evidence operations are `conditionLatent` and `conditionObservation`. They use `FiniteProbRecord.conditionOn`; the checked numerical equation is `conditionLatent_probVal`. ■

Proposition B.7.5 (Typed causal editing and structural intervention). *Every `CausalEditOperation` computes its result from its indexed source and operation-specific input data. Its constructor determines the edit label, and every `CausalEditTransition` identifies its named target with that computed result. In particular, no transition can certify an arbitrary target merely by assigning it a causal-edit label. Public belief reindexing derives its finite distribution from a certified epistemic-record belief or SCM prior. Its coordinate map is accompanied by a pointwise endpoint-realization proof. Clearing an override requires both a direct before/after equality between evaluation with the retained override and structural evaluation without it, and a proof that the latter realizes the endpoint evaluator.*

Directed relating inserts one rank-respecting endogenous-parent link; latent relating inserts one exogenous-input link. Their unrelating counterparts remove the selected link. Every such edit carries the replacement mechanism required by its new input type. Endogenous and exogenous learning followed by its paired provenance-guided forgetting restores the source record.

For a hard intervention x , `AtomicIntervention.compile` replaces the selected mechanisms and removes all of their incoming directed and latent links by primitive edits. The compact `SurgicalIntervention` presents the same graph surgery in one record constructor. Both have the hard-intervention evaluation of the source model at unit level.

Proof. The intrinsic operation family, its `result` and `label` eliminators, and the transition equality are in `lean/Thesis/Causality/Structural/Core.lean`. For fixed-signature transitions, `CausalEditOperation.ofRecordStep_result` checks that the intrinsic result is exactly the endpoint certified by the underlying record step. Certified belief provenance and endpoint realization are checked by `BeliefReindexing.CertifiedOperation.apply_belief` and `BeliefReindexing.CertifiedOperation.apply_realizes`. The direct before/after equality is `BeliefReindexing.CertifiedOperation.apply_preserves_evaluation`, and `source_realizes_reference` composes it with endpoint realization. Atomic compilation preserves intervention emptiness by `AtomicIntervention.compile_noActiveIntervention`; its certified clearing operation is constructed by `AtomicIntervention.Execution.DeterministicRealizesEvaluation.endpointReindexing`. The ordinary kernel, from-factual and from-empty probability endpoints expose their belief origins in `Kernel.atomicEndpoint_belief_provenance`, `ExecutedOccurrenceConstruction.probabilityTarget_`

B. Proofs and arguments

`belief_provenance` and `FromEmptyExecutedOccurrenceConstruction`. `probabilityTarget_belief_provenance`. Directed and latent edit operations and their edge witnesses are in `DirectedLink` and `LatentLink`. The two exact learning and forgetting round trips are `EndogenousVariableSpec.learn_forget_record_roundTrip` and `ExogenousVariableSpec.learn_forget_record_roundTrip`. Structural atomic compilation and its semantics are `AtomicIntervention.compile` and `AtomicIntervention.compileRealizes`; the deterministic endpoint assignment and its probability theorem are `AtomicIntervention.compileDeterministicRealizes` and `AtomicIntervention.Execution.DeterministicRealizesEvaluation`. `endpointRecord_observedValue`. Compact cuts and effectiveness are packaged by `SurgicalIntervention.plan`; its evaluator equality is `SurgicalIntervention.eval_eq_evalUnder`, and `AtomicIntervention.compile_semanticallyEquivalent_surgery` connects the two presentations. Link operations are in `lean/Thesis/Causality/Structural/Links.lean`, atomic compilation is in `lean/Thesis/Causality/Structural/Atomic.lean`, variable creation is in `lean/Thesis/Causality/Structural/Learning.lean`, the compact surgery constructor and plan are in `lean/Thesis/Causality/Structural/SurgeryCore.lean`, and its comparison with atomic compilation is in `lean/Thesis/Causality/Structural/SurgeryEquivalence.lean`. (The former `lean/Thesis/Causality/Structural/Surgery.lean` is retained as a compatibility facade.) Unsetting separately removes the current intervention at one target; it is not claimed to restore an arbitrary setting that may have existed before the last setting step. ■

Proposition B.7.6 (Conservative terminal learning and forgetting). *The general constructors of Definition A.7.10 append an endogenous variable with any declared old observed parents and existing latent inputs, or an initially unlinked independent exogenous root. Their paired provenance-guided forgetting transitions restore the exact source record. For the narrower conservative terminal observed extension, learning additionally preserves every old observational, hard-interventional and current-belief event value. Every compatible model over the extended graph restricts to a compatible old model. Consequently, old joint and locally supported conditional identifiability imply identifiability of the corresponding lifted queries.*

Learning, forgetting and the other structural edits are constructors of the same indexed `CausalEditOperation` family. Their labels are derived from those constructors, and their `CausalEditTransitions` certify the computed endpoints. No corresponding preservation claim is made for an arbitrary causal extension, and no arbitrary interior deletion operation is claimed.

Proof. Model compatibility is preserved by `TerminalVariableSpec.extendModel_compatible` and reflected to the old subsystem by `TerminalVariableSpec.restrictModel_compatible`. Old event semantics is checked by `extendModel_observationalValue_old`, `extendModel_interventionalValue_old` and `learnRecord_observedValue_old`. These declarations are in `lean/Thesis/Causality/Learning.lean`. Query preservation is `liftJointQuery_identifiable` and `liftConditionalQuery_identifiable` in `lean/Thesis/CausalTransport/Learning.lean`. The

conservative transition round trip is `CausalEditTransition.learn_forget_record_roundTrip` in `lean/Thesis/Causality/Structural/Core.lean`. The more general endogenous and exogenous executable specifications are `EndogenousVariableSpec` and `ExogenousVariableSpec` in `lean/Thesis/Causality/Structural/Learning.lean`; their intrinsically indexed transitions and exact round trips are in `lean/Thesis/Causality/Structural/Core.lean`. ■

Proposition B.7.7 (Canonical endpoint-record agreement). *Every deterministically realised structural execution whose endpoint intervention is proved absorbed on its transported latent assignments has a canonical one-step closure whose literal target record is its probability-bearing endpoint record. The canonical ordinary-kernel, from-factual and from-empty executions discharge this condition by proving that their fresh empty intervention remains empty through every construction and atomic-compilation step. Before that closure, eventwise agreement with a separately specified literal target follows whenever the source belief has been pushed through the endpoint assignment and the target has no residual intervention. Thus later probability theorems need neither definitional equality of arbitrary record presentations nor an unproved identification of their distributions.*

Proof. Checked declarations. In the namespace `AtomicIntervention.Execution.DeterministicRealizesEvaluation`, `canonicalExecution` appends the belief-reindexing endpoint, and `canonicalExecution_target_record` and `canonicalRealizes_endpointRecord` identify its two record presentations definitionally. Its premise is `EndpointInterventionAbsorbed`; empty interventions supply it through `EndpointInterventionAbsorbed.ofEmpty`. The three canonical discharge proofs are:

```
Kernel.atomicBaseEndpointInterventionAbsorbed;
ExecutedOccurrenceConstruction.endpointInterventionAbsorbed;
FromEmptyExecutedOccurrenceConstruction.endpointInterventionAbsorbed.
```

The more general eventwise criterion is `TargetRecordAgreement`, with conclusion `endpointRecord_target_observedValue`. All are in `lean/Thesis/Causality/Structural/Atomic.lean`. ■

Proposition B.7.8 (Current-state endpoint semantics). *Every proof-carrying causal edit path admits the endpoint-query semantics of Definition A.7.6. It evaluates the query at the path’s literal target record and therefore retains that record’s accumulated finite belief and hard intervention. A later kernel action overrides only its selected intervention coordinates. At an initial mode this semantics is equivalent to the ordinary model-level joint or conditional kernel denotation.*

The concrete histories used in this thesis agree with this common endpoint reading: supported do-calculus kernels, modal abduction-action-prediction and the occurrence-indexed counterfactual construction all reduce to their previously stated semantics. This proposition does not assert identifiability or completeness for arbitrary sequential epistemic programs.

Proof. The endpoint evaluator and initial-mode equivalences are:

B. Proofs and arguments

CausalEditPath.endpointDenote
Kernel.endpointDenote_initial
JointKernelQuery.endpointDenote_initial
ConditionalKernelQuery.endpointDenote_initial.

They are defined in `lean/Thesis/Causality/EndpointSemantics.lean`. Agreement with a supported kernel realization is `KernelOperationRealization.endpointDenotation`. Agreement with one-action AAP is `AbductionActionPrediction.endpointDenote_eq_singleAction`. Agreement of the generic endpoint query read along the actual from-factual multiworld path is `ModalCombinedCounterfactualConstruction.pathEndpoint_semanticAgreement`. These three bridge theorems are in `lean/Thesis/Causality/EndpointAgreement.lean`. ■

B.8 Counterfactual and multiworld execution

The next results concern shared-latent two-world and arbitrary finite multiworld constructions. They use the structural operations of Section B.7, but are logically distinct from ordinary do-calculus certificate transport.

Proposition B.8.1 (Finite twin-network adequacy). *The construction of Definition A.6.3 is finite and acyclic. Its factual marginal is the observational distribution of the original model, its counterfactual marginal is the distribution under the selected hard intervention, and its joint events are evaluated by the same latent assignment in both worlds. After conditioning on positive factual evidence, its counterfactual-event value is equivalent to the model-level value of Definition A.6.1.*

More generally, the occurrence-indexed multiworld of Definition A.6.4 gives each syntactic atom occurrence in a finite Boolean counterfactual event a separate world sharing the same latent assignment. Its dependent multiworld evaluator and its ordinary finite-indexed SCM encoding agree pointwise and on event probabilities. For a complete query, the encoded numerator and denominator agree with the general query semantics. The occurrence-indexed causal model can be executed atomically as in Definition A.7.8, either by appending the missing occurrence worlds to a fresh factual mode or by creating all roots and worlds from an empty mode. The generated dependent endpoints agree pointwise after coordinate transport, and their numerator and denominator probabilities are equivalent. The epistemic record produced by the factual-extension endpoint supplies the probability consumed by the modal counterfactual transport.

For a one-action query, the executable modal abduction–action–prediction sequence transports the posterior belief through the actual atomic endpoint and agrees with the model helper, the compact surgery and the twin value. Its query is supported at a model exactly when its factual evidence has positive probability there.

Proof. Finiteness is witnessed by `TwinNode.enumeration`; copied arrows strictly increase `TwinNode.rank` by `TwinNetwork.directed_rank_lt`. The intervention-cut theorems are `intervention_cuts_counterfactual_directed` and `intervention_cuts_counterfactual_incident`. The copied structural equations are checked by `TwinNetwork.eval_satisfies_mechanism`; the marginal and joint statements are

`factual_marginal`, `counterfactual_marginal` and `joint_event_eq`. Finally, `abductedCounterfactualValue_eq_counterfactualValue` proves the claimed abduction-action-prediction equality. The derived single-action syntax is connected to both presentations by `CounterfactualQuery.singleAction_denote_eq_counterfactualValue` and `CounterfactualQuery.singleAction_denote_eq_twinNetwork`. These declarations are in `lean/Thesis/Causality/Counterfactual.lean`. The encoded ordinary SCM and its pointwise and probability-level adequacy are `TwinNetwork.Encoding.model`, `TwinNetwork.Encoding.model_eval_eq_encodeAssignment` and `TwinNetwork.Encoding.model_observationalValue_encodeEvent`. The general occurrence semantics is `OccurrenceMultiworld`; its ordinary SCM encoding and probability theorem are `OccurrenceMultiworld.Encoding.model` and `OccurrenceMultiworld.Encoding.model_eventProbability` in `lean/Thesis/Causality/Multiworld.lean`. The two atomic construction routes are `ExecutedOccurrenceConstruction.canonical` and `FromEmptyExecutedOccurrenceConstruction.canonical`. Their endpoint evaluator and probability theorems are compared by

- `executedRoutes_endpoint_eval_agree`;
- `executedRoutes_eventProbability_equiv`;
- `executedRoutes_denominator_equiv`;
- `executedRoutes_numerator_equiv`.

The generic event theorem makes the numerator and denominator comparisons consequences rather than isolated coincidences. In each route, `ExecutedOccurrenceConstruction.endpointRecord_probabilityTarget` and `FromEmptyExecutedOccurrenceConstruction.endpointRecord_probabilityTarget` state that the probability-bearing endpoint is the literal target of the canonically closed execution. The two constructions are in `lean/Thesis/Causality/ExecutedMultiworld/FromFactual.lean` and `lean/Thesis/Causality/ExecutedMultiworld/FromEmpty.lean`; their comparison is in `lean/Thesis/Causality/ExecutedMultiworld/Agreement.lean`. The occurrence-by-occurrence connection between the retained modal trace, both atomic construction routes and their counterfactual endpoint coordinates is `ModalCombinedCounterfactualConstruction.occurrencePathMatch`. The encoded query denominator and numerator are connected to the original semantics by `combinedDenominator_equiv` and `combinedNumerator_equiv` in `CounterfactualQuery`. The primary atomic action and compact surgery are connected by `action_semanticallyEquivalent_compactAction`. The generic AAP equalities in `AbductionActionPrediction` are `prediction_eq_counterfactualValue` and `prediction_eq_twinNetwork`. The combined executed endpoint is connected directly to the independent twin by `ModalCombinedCounterfactualConstruction.singleAction_semantics_eq_twinNetwork`. These are checked in `lean/Thesis/Causality/ModalCounterfactual.lean`. Exact one-action support is checked by `singleAction_supportedAt_iff` in the `CounterfactualQuery` namespace of `lean/Thesis/CausalTransport/ModalCounterfactual.lean`. ■

Proposition B.8.2 (Finite unit-level counterfactual laws). *Let u be a latent assignment of a finite SCM. Evaluation satisfies: no intervention returns the*

B. Proofs and arguments

factual assignment; an intervened variable takes its selected value (effectiveness); setting variables to the values they already have factually leaves the complete assignment unchanged (consistency); and extending a base intervention only by values already produced under that base intervention leaves the complete assignment unchanged, provided no base setting is removed (composition).

Proof. In the `FiniteLatentSCM` namespace, the checked declarations are:

- `evalUnder_noIntervention`;
- `evalUnder_effectiveness`;
- `evalUnder_consistency`; and
- `evalUnder_composition`.

The last theorem is proved by well-founded induction along the topological node order; consistency is its no-intervention instance. ■

Proposition B.8.3 (Finite counterfactual correspondence). *Every finite Boolean multiworld event has the same truth value at each latent assignment in a finite source table and its intrinsic target interpretation. Hence every partial conditional counterfactual query has equivalent source and target denotations. Compatibility, observational counterfactual identifiability and uniform query support are each preserved and reflected.*

Proof. The checked declarations are:

- `CounterfactualAtom.finiteSourceHolds_preserved`, for atoms;
- `CounterfactualEvent.finiteSourceHolds_preserved`, by Boolean induction;
- `CounterfactualQuery.finiteSourceDenote_preserved`, for the partial quotient;
- `finiteSource_counterfactualIdentifiable_iff`, for identifiability; and
- `finiteSource_counterfactualSupported_iff`, for uniform support.

They are defined in `lean/Thesis/CausalTransport/Counterfactual.lean`. ■

Corollary B.8.4 (Interventional-query adequacy). *Every local finite ordinary interventional query has the same rational value in a finite source model and its intrinsic target interpretation. Moreover, identifiability of the whole distributional kernel implies identifiability of every decidable fixed event local to its outcome nodes.*

Proof. The first statement is the pointwise encoding theorem used above. For the second statement, Lean projects finite assignments to the outcome coordinates, proves that singleton masses determine every decidable event, and sums the resulting cylinder cells. The checked declarations are `FiniteProbRecord.probVal_extensional_of_singletons`, `kernel_identifiable_implies_event_identifiable` and `transport_joint_event_soundness`. Independently, `ProbabilityTerm.eventTerm` compiles any Boolean event on the complete finite assignment type into the shared expression language, and `ProbabilityTerm.eventTerm_denote` proves its exact observational denotation. The converse is deliberately absent. ■

Corollary B.8.5 (Executable tenure-track realization). *The finite model of Example A.8.1 is an inhabitant of the intrinsic target presentation and uses the same probability records, recursive evaluator and structural operations as the correspondence development. Its principal observational, conditional and interventional values are checked by the generic finite machinery. Its evidence event has probability $3/32$, and the two displayed one-action counterfactual offer probabilities are both $1/2$. For the prestige action, the direct model calculation, finite twin network, atomic modal abduction–action–prediction sequence and the occurrence-indexed from-factual and from-empty executions have equivalent denotations.*

Proof. The executable model is `TenureTrack.model` in `lean/Thesis/Examples/TenureTrack.lean`. The declarations `prob_prestige_true`, `prob_quality_true`, `prob_prestige_and_quality_true`, `prob_quality_given_prestige`, `prob_quality_do_prestige_true`, `prob_offer_given_fit` and `prob_offer_do_fit_true` check the principal ordinary calculations. Evidence and the two direct counterfactual values are `evidence_probability`, `counterfactual_offer_do_prestige_given_evidence` and `counterfactual_offer_do_fit_given_evidence`. The generic twin and atomic modal comparisons are `twin_counterfactual_offer_do_prestige_given_evidence` and `modal_aap_counterfactual_offer_do_prestige`. Finally, `prestigeModalCombinedConstruction_semantics` identifies the occurrence-indexed endpoint with the general query denotation, and `prestigeModalCombinedConstruction_value` evaluates it to $1/2$. ■

B.9 Finite-source certificate transport

The following results return from executable target records to the independently evaluated finite source presentation. They record the exact source–target identifiability equivalence and compilation of externally supplied derivation certificates. The intrinsic conservative-learning construction itself appears with the other structural edits in Proposition B.7.6; only its certificate consequence belongs here.

Proposition B.9.1 (Finite-source identifiability correspondence). *Let T be a finite table signature and G a finite table graph. A joint distributional kernel query is identifiable under the source-native finite-table evaluator over all source models compatible with G if and only if it is identifiable under the intrinsic dependent evaluator over all exact models compatible with the interpretation of G . The same equivalence holds for conditional distributional kernels, with local support premises retained on both sides.*

Proof. These are `finiteSource_identifiable_iff` and `finiteSource_conditionalIdentifiable_iff` in `lean/Thesis/CausalTransport/FiniteSource.lean`. From source to target, an arbitrary exact target model is converted by `FiniteTableSCM.ofExact`; from target to source, a finite table SCM is mapped by `FiniteTableSCM.interpret`. The independently defined source evaluator is connected to the target evaluator by the pointwise preservation theorems for kernels, expressions, support and observational equivalence. Thus the result is not an identification of two differently named wrappers around one evaluator. ■

B. Proofs and arguments

Corollary B.9.2 (Conservative-learning certificate transport). *If completeness and primitive-soundness packages are supplied for the conservatively extended signature and graph, an identifiable lifted joint or locally supported conditional query receives an operation-realized modal certificate. The package retains the exact unified learning and forgetting transitions, compatibility of the source, learned and restored modes, and current-belief preservation.*

Proof. The checked packages are:

```
CausalEditTransition.terminalLearningJointTransport  
CausalEditTransition.terminalLearningConditionalTransport.
```

They are defined in `lean/Thesis/CausalTransport/Learning.lean`. Their completeness and soundness arguments remain explicit external inputs for the extended graph. ■

Proposition B.9.3 (Transport of supplied derivation certificates). *If the explicit finite-source completeness specialization supplies a derivation certificate for every identifiable finite-table query, then an identifiable query in the interpreted type-theoretic target has the corresponding encoded certificate.*

Proof. Proof idea. Apply `finiteSource_identifiable_iff` to move the query to the independently declared source presentation, apply the supplied source theorem and compile its path certificate. *Checked declaration.* This is `finiteSource_completeness_transport`. Its result contains an action-free `ProbabilityTerm` and a `DoCalculusDerivation`; the external package therefore cannot replace derivability by an unconstrained predicate such as `True`. ■

B.10 Do-calculus identification and completeness transport

The Shpitser–Pearl, Huang–Valtorta and Tian–Shpitser soundness, interventional and counterfactual completeness, and latent-projection results remain external mathematical inputs [HV06; Pea12; SP06; SP08; TS09]. Lean represents the required finite specializations by the ordinary structures listed in Table B.4; no axiom with any of that content is declared. Inhabitants must be supplied when the corresponding transport theorem is applied. Completeness returns concrete path-based derivation trees over the finite source class. The `DSeparationCorrectness` field compiles every active-path side condition into the executable ancestral-moralisation test used by the concrete derivation syntax. Soundness supplies only the primitive finite SCM laws; recursive support evidence and certificate-to-identifiability soundness are then proved internally first for source tables and then preserved by interpretation. The generic `PublishedCompleteness` and `PublishedSoundness` structures in the lower-level correspondence module are derived adapters; the primary finite-source interfaces do not assume an already transported conclusion. Their identifiability, expression denotation, conditional support, derivation-support trees, primitive semantic laws and counterexamples are source-native. The counterfactual interface is source-native as well: it returns a finite multiworld reduction, followed by a path-based ordinary do-calculus derivation to an action-free

observational term and a support tree for that derivation. The probability-expression and derivation syntax is a shared model-independent intermediate language, rather than a shared evaluator. `FiniteTableSCM.termDenote_preserved`, `FiniteSourceLocalDerivationSupport.toTarget` and `FiniteSourceCounterexample.toTarget` are the ordinary checked bridges; `CounterfactualQuery.finiteSourceDenote_preserved` and `FiniteSourceCounterfactualCertificate.toTarget` are their multiworld counterparts.

Constructing an inhabitant of the finite-source interfaces from the cited papers requires a precise metatheoretic specialization. One must verify that the papers' recursive semi-Markovian model class contains the finite coded table models used here; identify their graphical criterion with `PathDSeparated`; translate every ID/IDC or do-calculus output into `PathDoCalculusDerivation`; prove local support for each division used by that output; supply the finite global-Markov and primitive probability laws; and show that every represented hedge has the required finite rational countermodels. The published binary fair-coin hedge construction and finite-domain statements make this specialization mathematically plausible, but the current Lean development does not formalise the papers' ID/IDC algorithms or their metaproofs. The counterfactual interface additionally requires translating the published ID*/IDC* reduction into the shared-latent finite source semantics and observationalising every resulting interventional term by a supported ordinary derivation. For the Boolean event syntax of Definition A.6.2, the specialization must also justify the finite reduction from Boolean combinations to the query fragment covered by the published algorithm. The shared ordinary expression language can now represent every resulting finite assignment event through `ProbabilityTerm.eventTerm`; this does not by itself prove that the published multiworld algorithm performs the required reduction. Accepting the papers as correct is therefore not, by itself, a kernel term of these structures; the finite specialization is the deliberately explicit external theorem boundary. Once those source-native semantic structures over the shared finite syntax are supplied, the corresponding interpretation, certificate compilation, source and target soundness induction and completeness transports are checked internally. Modal annotation and compatible-mode indexing are additionally checked for the ordinary do-calculus certificates described below. For counterfactuals, every potential-outcome atom receives its structural-intervention trace and the resulting query trace is packaged with both executed occurrence-indexed construction routes. The probability is read from the epistemic record produced by the factual-extension endpoint, and `ModeIndexedCounterfactualDerivation.endpointReduction` composes that endpoint's checked denotation with the transported observational reduction. The external source-native reduction itself remains supplied data rather than an internally generated modal algorithm.

B. Proofs and arguments

	Layer	Status	Lean declaration
1	Independent finite source and dependent target	Source latent data, compatibility, evaluators, kernel and expression semantics and support are independently declared; their target preservation is checked internally.	FiniteTableLatent; FiniteTableSCM; ExactModel; FiniteTableSCM. termDenote_preserved
2	Rational records, products and CPT responses	Checked internally.	CommonDenominator. FiniteQMass.toRecord; FiniteRationalCPT.toSCM
3	Latent projection and executable d-separation	Graph projection and Boolean ancestral-moralisation computation checked internally.	FiniteLatentProjection; ObservedGraph.dSeparated
4	Standard active-path equivalence	Independent specification internal; equivalence theorem supplied explicitly and used to compile path certificates.	PathSpecification. PathDSeparated; DSeparationCorrectness; PathDoCalculusDerivation. compile
5	Distributional queries, events, hedges and derivation syntax	Checked internally as records and inductive witness types; finite kernel-to-event aggregation is checked.	JointKernelQuery; HedgeWitness; DoCalculusDerivation
6	Partial expression semantics, local support and certificate soundness	Source and target evaluators and recursive support trees checked internally once primitive laws are supplied; preservation is explicit.	FiniteTableSCM. termDenote; FiniteSource JointCertificate .source_identifiable; FiniteSource ConditionalCertificate .source_identifiable

B.10. Do-calculus identification and completeness transport

	Layer	Status	Lean declaration
7	Primitive do-rule and probability semantics/global Markov	Explicit source-native external argument with source support premises; target adaptation checked separately.	FiniteSourcePath PrimitiveSoundness; PublishedFiniteSource Soundness
8	Finite-source completeness/hedges	Exact external specialization of the published result; it returns certificates with source-native support over shared finite syntax.	PublishedFiniteSource Completeness
9	Source-target correspondence/two-way transport	Source-target equivalence is checked internally. Two-way certificate transport is checked conditional on explicit finite-source completeness and soundness packages.	finiteSource_ identifiable_iff; finiteSource_transported_ joint_iff
10	Counterfactual transport	Twin reference, occurrence-indexed semantics, both atomic construction routes, endpoint-record probability semantics, AAP and source-target correspondence are checked internally. The mode-indexed reduction begins at the executed endpoint denotation. The source-only ID*/IDC* reduction remains external.	TwinNetwork; ModalCombined Counterfactual Construction; PublishedFinite Counterfactual Completeness; finiteSource_modeIndexed_ transported_ counterfactual_iff
11	Modal operations and causal editing	Executable kernel/rule-cell realizations, compatible mode indexing, typed directed and latent link edits, endogenous and exogenous terminal creation, atomic intervention compilation, compact-surgery equivalence, old-event semantics, lifted identifiability and exact provenance-guided forgetting are checked internally; extended-graph completeness and soundness remain explicit inputs.	ModeIndexed OperationRealized JointCertificate; CausalEditTransition; AtomicIntervention. compile; CausalEditTransition. terminalLearning JointTransport
12	Structurally generated hidden-DAG class and latent projection	Model structure checked; projection theorem external; application checked.	FiniteHiddenDAGSCM; PublishedLatentProjection; hiddenDAG_joint_transport

Table B.4: Exact boundary of the checked causality-completeness transfer.

Lemma B.10.1 (Disjoint exogenous independence). *In an exact finite latent-root model, every rectangular event has joint probability equal to the product of its source-wise probabilities.*

Proof. Proof idea. Independence is evidence carried by the model, not an inference from separate normalisation facts. *Checked declaration.* This is the `product_law` field of `FiniteLatentSCM`; the generically constructed product record satisfies the same equation by `FiniteProduct.record_rectangular_probVal`. ■

B. Proofs and arguments

Lemma B.10.2 (External finite do-calculus soundness specialization). *To interpret a concrete `DoCalculusDerivation` semantically, the finite specialization must prove that the executable d-separation side conditions entail the required conditional independencies in every compatible finite latent SCM and must supply the primitive marginalisation, conditioning and chain identities on their stated support.*

Proof. Formal status. `FiniteTableSCM.termDenote`, its congruence cases and recursive `FiniteSourceLocalDerivationSupport` are internal source-table definitions. Joint-query support is obtained directly from finite-table normalization by `JointKernelQuery.finiteSourceSupportedAt`; it does not pass through the interpreted target. At the valuation being interpreted, every primitive law receives source support for both endpoints and the marginalisation, conditioning and chain constructors receive the full four-way disjointness premise. The external obligations are fields of `FiniteSourcePathPrimitiveSoundness`, supplied source model by source model through `PublishedFiniteSourceSoundness`. Its path-based do-rule premise is compiled by the supplied `DSeparationCorrectness` equivalence. Lean’s `DoCalculusDerivation.finiteSource_denotational_soundAt` then proves source denotational equality of the whole derivation by induction. The target laws and support trees are obtained only afterward through `FiniteSourcePathPrimitiveSoundness.toTarget` and `FiniteSourceLocalDerivationSupport.toTarget`.

There are no separate external `joint_sound` or `conditional_sound` fields. For an action-free final formula, observational equivalence makes that formula invariant between two compatible models. Lean composes this invariant with the two recursive source witnesses in `FiniteSourceJointCertificate.source_identifiable` and `FiniteSourceConditionalCertificate.source_identifiable`; the corresponding joint target declaration is `JointIdentificationCertificate.identifiable`. The conditional counterpart is `ConditionalIdentificationCertificate.identifiable`. The remaining external mathematical content is the active-path equivalence, the finite global Markov justification of the do-rules and the stated primitive finite probability identities. ■

Lemma B.10.3 (External finite-rational hedge specialization). *To instantiate `PublishedFiniteSourceCompleteness` for the intended class, every accepted hedge must yield two compatible finite source models with equal source observational distributions and unequal source-kernel values.*

Proof. This is precisely the type of the source-native hedge-counterexample field in `PublishedFiniteSourceCompleteness`; its interpretation is checked by `FiniteSourceCounterexample.toTarget`. The familiar binary fair-coin construction from the published hedge proof explains why the intended specialization is rational, but that construction is not regenerated for an arbitrary hedge by the current Lean code. Consequently this lemma states the external obligation required to build the package, not an additional kernel theorem. ■

Proposition B.10.4 (Constructive finite identification). *Given $P : \text{PublishedFiniteSourceCompleteness}$, an identifiable type-theoretic joint distributional kernel yields `EncodedJointDerivation G q`.*

Proof. Proof idea. Convert target identifiability to finite-source identifiability and apply the supplied source completeness field. *Checked declaration.* This is the joint component of `finiteSource_completeness_transport`. It is defined in `lean/Thesis/CausalTransport/FiniteSource.lean`. ■

Remark B.10.5 (Identification traces). The current formalisation transports an inspectable derivation tree, but does not implement the branch trace or termination proof of the ID or conditional-ID algorithm that produces it. Finite d-separation search itself is implemented; branch selection and certificate generation remain part of the supplied completeness theorem.

Proposition B.10.6 (Operation-sensitive modal certificate transport). *Let an epistemic causal mode be compatible with the interpreted finite source graph. Given finite-source completeness and primitive-soundness packages, joint identifiability is equivalent to the existence of an `ModeIndexedOperationRealizedJointCertificate`; the analogous equivalence holds for locally supported conditional queries. Such a certificate contains the mode’s compatibility proof, the compiled do-calculus derivation and its modal trace. For every compatible target model, valuation and supported source kernel, it also executes that kernel and both endpoints of every Pearl-rule leaf by a hard intervention followed by positive observation, and proves that the resulting finite rational values agree as required by primitive soundness. The kernel endpoint is obtained through the same atomic structural compiler used elsewhere: its canonically closed target is a literal probability-bearing record, and conditional values are formed from its atomic numerator and positive atomic denominator. Its `atMode` operation specializes these uniform realizations to the model stored at the indexed mode.*

Proof. Checked declarations. The two equivalences in `lean/Thesis/CausalTransport/ModalRealization.lean` are:

```
finiteSource_modeIndexedOperationRealized_joint_iff
finiteSource_modeIndexedOperationRealized_conditional_iff.
```

The annotation compiler is `DoCalculusDerivation.toModalTrace`; executable kernel semantics is `KernelOperationRealization.ofSupported`; and `ModalDerivationTrace.realizeOperations` recursively constructs the rule-leaf realizations. These executable declarations are in `lean/Thesis/Causality/ModalRealization.lean`. The structural kernel endpoint is `Kernel.atomicExecution`; its literal target record is `Kernel.atomicRecord`. The event probability and conditional quotient are checked by:

- `AtomicKernelActionRealization.probability`;
- `KernelOperationRealization.atomicConditionedValue`.

Probability-algebra constructors retain their recursive support and denotational-soundness proofs; they are not misrepresented as additional causal record transitions. ■

Proposition B.10.7 (Constructive hedge witnesses). *The finite graph data of a hedge are represented by `HedgeWitness`. Given the external hedge-counterexample field, such a witness proves failure of type-theoretic identifiability.*

Proof. The checked hedge-failure transport proof is in `lean/Thesis/CausalTransport/Correspondence.lean` as `transport_hedge_failure`. ■

B. Proofs and arguments

Proposition B.10.8 (Conditional-effect correspondence). *Conditional distributional kernels have a partial finite-rational denotation. Their correspondence compares a valuation only when the source kernel is supported at that valuation in each of the two models. A supplied certificate must then produce local support for every intermediate denominator reached by its derivation at that valuation.*

Proof. This pointwise support implication is built into both conditional identifiability definitions. Their source-target equivalence is `finiteSource_conditionalIdentifiable_iff`. The separate application-level `ConditionalQuery` for arbitrary outcome and conditioning events is not silently identified with this whole-kernel certificate type. ■

Corollary B.10.9 (Transferred conditional-identification completeness). *Given $P : \text{PublishedFiniteSourceCompleteness}$, every conditionally identifiable type-theoretic query yields an `EncodedConditionalDerivation` $G\ q$.*

Proof. Checked declaration. The conditional component is `finiteSource_completeness_transport`. It is defined in `lean/Thesis/CausalTransport/FiniteSource.lean`. ■

B.11 Counterfactual completeness transport

Ordinary do-calculus transport does not by itself identify multiworld counterfactual queries. The next theorem therefore keeps the counterfactual source reduction, its observationalisation, the modal occurrence traces and both executed finite multiworld routes in a separate package.

Theorem B.11.1 (Finite counterfactual-completeness transport). *Let T be a finite table signature, G a finite table graph and $Q : \text{PublishedFiniteSourceSoundness } T\ G$. If an explicit source-native specialization*

$$P : \text{PublishedFiniteCounterfactualCompleteness } T\ G\ Q$$

is supplied, then a finite multiworld query in the intrinsic target is observationally identifiable and uniformly supported over the compatible model class if and only if it has an `EncodedCounterfactualDerivation`. That encoded object retains a source-native multiworld reduction, a path-based do-calculus derivation from its interventional expression to an action-free observational expression and recursive local support for the derivation; its `toTarget` map yields the intrinsic target certificate. Moreover, for any epistemic causal mode compatible with the interpreted graph, the same conjunction of identifiability and uniform support is equivalent to the existence of a `ModeIndexedCounterfactualDerivation`. This package retains the encoded source derivation, the compatibility witness, a structural modal trace and the executed occurrence-indexed construction from both a fresh factual mode and an empty mode. Its `endpointReduction` starts from the partial probability read from the epistemic record produced by that atomic execution and ends at the transported action-free observational formula.

Proof. Multiworld source and target semantics, support and identifiability are connected by `finiteSource_counterfactualIdentifiable_iff` and `finiteSource_counterfactualSupported_iff`. The external package supplies `FiniteSourceCounterfactualReduction` and `FiniteSourceCounterfactualObservationalization`. Lean compiles the latter's path derivation, checks its finite-table denotational soundness with Q , combines it with the multiworld reduction in `toCertificate`, and interprets the result by `FiniteSourceCounterfactualCertificate.toTarget`. The two-way statement is `finiteSource_transported_counterfactual_iff` in `lean/Thesis/CausalTransport/Counterfactual.lean`. Its compatible-mode strengthening is `finiteSource_modeIndexed_transported_counterfactual_iff` in `lean/Thesis/CausalTransport/ModalCounterfactual.lean`; the recursive trace is constructed by `ModalCounterfactualQueryTrace.canonical`; the combined construction is `ModalCombinedCounterfactualConstruction.canonical` in `lean/Thesis/Causality/ModalCounterfactual.lean`. Its endpoint probability is connected to the target certificate by `ModeIndexedCounterfactualDerivation.endpointReduction`, and `ModeIndexedCounterfactualDerivation.endpointSupportedAt` carries the supplied support through the same endpoint semantics.

The qualification by P is essential. Lean does not construct an inhabitant from the cited paper and does not formalise the ID^*/IDC^* algorithms or their counterfactual-graph completeness proof [SP08]. It checks that, once the exact finite source theorem is supplied, its supported observational reduction is carried into the intrinsic finite setting without an additional identification assumption. ■

B.12 Finite SCM construction and latent projection

The preceding transport theorems quantify over compatible finite source and target models. This section records two complementary ways models enter that class: constructive functionalisation of finite rational conditional probability tables and structural projection of explicit hidden DAGs.

Lemma B.12.1 (Finite rational causal-Bayesian-network functionalisation). *Every finite rational conditional probability table with explicitly enumerated parent configurations has a functional representation with one independent finite response-function seed per observed node. The deterministic mechanism selects the response-table coordinate indexed by the realised parent configuration. Every original row is preserved, the observational probability of every complete assignment factorises as the product of its selected conditional-table entries, and the resulting latent-root SCM is Markovian.*

Proof. Proof idea. Convert each rational row to a common-denominator record. For one node, sample a complete response table from the product of all its rows; selecting one coordinate has exactly that row as its marginal. Then take the dependent product of the response-table laws over all observed nodes. *Checked declarations.* Generic denominator conversion is `CommonDenominator.FiniteQMass.toRecord` with preservation theorem `toRecord_preserves`. The finite product and coordin-

B. Proofs and arguments

ate marginal theorems are `FiniteProduct.record_rectangular_probVal` and `FiniteProduct.record_coordinate_probVal`. The CPT conversion is `FiniteRationalCPT.toFunctionalCBN`. Row preservation is `responseFactor_preserves_row` and `toSCM_preserves_row`. The preimage of an observed singleton is identified with a rectangular response-seed event by `observationalSingleton_preimage_eq_rectangular`; combining this with the product law and row preservation gives the global factorisation theorem `toSCM_observational_singleton_factorizes`. Markovianity is `toSCM_isMarkovian`. These declarations are in `lean/Thesis/Causality/Reductions.lean`. Finally, the declaration `enters_finite_transport` in the namespace `FiniteRationalCPT` and the file `lean/Thesis/CausalTransport/Construction.lean` checks that the constructed SCM is compatible with its own projected graph and that, given the same explicit identifiability and published-completeness premises as the general theorem, it enters the certificate-transport framework. ■

Proposition B.12.2 (Finite semi-Markovian projection). *The direct-root semi-Markovian projection of Definition A.4.12 is represented and checked. A Markovian model has no projected bidirected edges, while a shared latent root can induce one. For the partially observed hidden graph and its general projection in Definitions A.4.25 and A.4.26, `FiniteHiddenDAGSCM` requires an enumerated value type and an independent private-noise factor at every full-DAG node, together with a mechanism typed only over that node's declared DAG parents. Its observational and interventional distributions are derived by well-founded evaluation and finite pushforward. The graph projection through hidden-only directed paths is represented separately, including the proof that its projected directed arrows agree with the declared observed signature.*

Proof. Checked declarations. Private roots imply no bidirected edge by `markovian_projection_has_no_bidirected`; a shared root induces one in `shared_source_projects_to_bidirected`. The general graph construction is `FiniteHiddenDAG` together with `FiniteLatentProjection`; the associated model structure is `FiniteHiddenDAGSCM`, and `hidden_path_projects_to_bidirected` checks a two-step example. `HiddenDAGModelClass` now packages only models carrying such a structural generator; it is not an arbitrary table of distributions indexed by a graph. *External boundary.* These definitions and graph theorems must not be inflated into a probability-preserving conversion of each arbitrary hidden-DAG model to independent pairwise roots. The published reduction instead preserves identifiability of the full model class; that exact obligation is `PublishedLatentProjection`, and its checked application is `hiddenDAG_joint_transport` and `hiddenDAG_conditional_transport` in `lean/Thesis/CausalTransport/HiddenDAG.lean`. ■

B.13 Completeness consequences and boundary

The final statements collect the consequences of the ordinary finite-source completeness and soundness packages. They do not add another external assumption; they expose the joint, conditional, modal and failure consequences already checked by the preceding correspondence.

Theorem B.13.1 (Transferred do-calculus completeness). *Let P be an explicit finite-source completeness package of type `PublishedFiniteSourceCompleteness`, specialized to the applicable finite classical theorem and the source tables of Definition A.5.5. Every type-theoretically identifiable joint distributional kernel then has an action-free probability expression and a concrete encoded do-calculus derivation of it.*

Proof. Checked declaration. Apply the joint component of `finiteSource_completeness_transport`. The qualification by P is essential: Lean checks the certificate type and transport, but does not prove the cited completeness theorem that produces the certificate. ■

Corollary B.13.2 (Transferred failure witnesses). *Under the same explicit package, every represented hedge witnesses failure of type-theoretic identifiability.*

Proof. Apply `transport_hedge_failure`. ■

Theorem B.13.3 (Finite causal-completeness transport). *For every finite table signature T , source graph G , and explicit finite-source completeness package P for T and G , Lean proves:*

B.13.3.1. *every type-theoretically identifiable joint distributional kernel has the joint derivation certificate supplied by P ;*

B.13.3.2. *every locally supported type-theoretically identifiable conditional distributional kernel has the conditional certificate supplied by P ;*

B.13.3.3. *every represented hedge implies failure of type-theoretic identifiability.*

If, in addition, an epistemic causal mode is supplied with a proof of compatibility with G , the first two certificates lift to mode-indexed certificates carrying their operation-sensitive modal traces. If a finite-source primitive-soundness package is also supplied, joint and locally supported conditional identifiability are equivalent to the existence of mode-indexed certificates that concretely realize the supported source kernel and every Pearl-rule leaf by record operations.

Proof. Checked declaration. This is `finiteSource_completeness_transport` in `lean/Thesis/CausalTransport/FiniteSource.lean`. Its three components are the joint, conditional and hedge transport theorems above. The stated compatible-mode strengthening is `CausalMode.finiteSource_modal_completeness_transport` in `lean/Thesis/CausalTransport/Modal.lean`. The mode-indexed, operation-realized two-way strengthenings are `finiteSource_modeIndexedOperationRealized_joint_iff` and its conditional counterpart in `lean/Thesis/CausalTransport/ModalRealization.lean`. ■

Corollary B.13.4 (Finite two-way derivability transport). *Given both a finite-source completeness package P and a finite-source soundness package Q for T and G , a finite joint distributional kernel is type-theoretically identifiable exactly when it has an encoded action-free derivation certificate. The same equivalence holds for locally supported conditional distributional kernels.*

B. Proofs and arguments

Proof. Proof idea. Completeness supplies the forward implication; semantic soundness supplies the reverse implication; the source–target semantic and identifiability equivalences transport both. *Checked declarations.* These are `finiteSource_transported_joint_iff` and `finiteSource_transported_conditional_iff` in `lean/Thesis/CausalTransport/FiniteSource.lean`. The inhabitants P and Q remain explicit external mathematical inputs. ■

The core checked model class is finite, rational, recursive and canonical semi-Markovian. It covers joint distributional interventional kernels, locally supported conditional kernels, derived local fixed-event queries, source-native and target partial semantics for their probability expressions, modal annotations of do-calculus derivations, executable realizations of finite action and positive observation locks, typed directed and latent-input edits, endogenous and exogenous terminal creation, provenance-guided forgetting, structural intervention and its hard-evaluator equivalence, generic finite rational CPT functionalisation and conversion to independently seeded Markovian SCMs. It also covers an explicit shared-latent two-world construction, arbitrary finite Boolean multiworld query semantics, modal action-world traces, from-factual and literal from-empty occurrence-indexed execution, a generic structural abduction–action–prediction theorem, their finite-table to intrinsic correspondence, and the mode-indexed conditional counterfactual-completeness transport whose reduction starts at the executed atomic endpoint record, as stated in Theorem B.11.1. The conservative learning theorem preserves lifted old queries in one direction; it does not assert invariance under arbitrary signature change. A structurally generated finite hidden-DAG model class and its graph-level latent projection are also internal; the theorem that projection preserves identifiability of that class is an explicit external input. The development does not cover cyclic SCMs, an internal construction of the published counterfactual-completeness package, transportability, path-specific effects, mechanised **ID**, **IDC**, **ID*** or **IDC*** algorithms, an internal proof of the active-path–moralisation equivalence, an internal finite global Markov theorem and the semantic validity of all three do-calculus rules, or a probability-preserving individual-model reduction from every arbitrary hidden-DAG SCM to independent pairwise latent roots. These are not harmless omissions: each requires a separate formal object or reduction theorem.

Bibliography/sources

“Contrarily, it is characteristic of a truly wise man that, being just a mortal, he doesn’t want to transgress any boundaries with his wisdom and together with humanity either turns a blind eye, or, out of courtesy, wanders along”

Folly in *The Praise of Folly*

B. Proofs and arguments

- [Agr+25a] Agricola, I. et al. ‘Fraudulent Publishing in the Mathematical Sciences’. In: *Notices of the American Mathematical Society* vol. 72, no. 9 (Oct. 2025), pp. 1038–1049.
- [Agr+25b] Agricola, I. et al. ‘How to Fight Fraudulent Publishing in the Mathematical Sciences: Joint Recommendations of the IMU and the ICIAM’. In: *Mitteilungen der Deutschen Mathematiker-Vereinigung* vol. 33, no. 4 (1st Dec. 2025), pp. 249–252.
- [Ait13] Aitkenhead, D. *Peter Higgs: I Wouldn’t Be Productive Enough for Today’s Academic System*. The Guardian. 6th Dec. 2013. URL: <https://www.theguardian.com/science/2013/dec/06/peter-higgs-boson-academic-system> (visited on 12/05/2026).
- [AK91] Allis, V. and Koetsier, T. ‘On Some Paradoxes of the Infinite’. In: *The British Journal for the Philosophy of Science* vol. 42, no. 2 (1st June 1991), pp. 187–194. URL: <https://www.jstor.org/stable/688052>.
- [AK95] Allis, V. and Koetsier, T. ‘On Some Paradoxes of the Infinite II’. In: *The British Journal for the Philosophy of Science* vol. 46, no. 2 (1st June 1995), pp. 235–247. URL: <https://www.jstor.org/stable/687814>.
- [Al-14] Al-Khalili, J. *Higgs Would Not Find His Boson in Today’s ‘Publish or Perish’ Research Culture*. The Guardian. 14th Feb. 2014. URL: <https://www.theguardian.com/commentisfree/2014/feb/14/higgs-boson-publish-or-perish-science-culture> (visited on 12/05/2026).
- [Alp+26] Alper, J. et al. *Leiden Declaration on Artificial Intelligence and Mathematics*. Leiden Declaration on Artificial Intelligence and Mathematics. 2nd June 2026. URL: <https://leidendeclaration.ai/> (visited on 24/07/2026).
- [AR11] Awodey, S. and Rabe, F. ‘Kripke Semantics for Martin-Löf’s Extensional Type Theory’. In: *Logical Methods in Computer Science* vol. 7, no. 3, 18 (2011), pp. 1–25. URL: <https://lmcs.episciences.org/1184>.
- [AS00] Arnborg, S. and Sjödin, G. ‘Bayes Rules in Finite Models’. In: *Proceedings of the 14th European Conference on Artificial Intelligence*. Berlin, 2000, pp. 571–575. URL: <https://frontiersinai.com/ecai/ecai2000/pdf/p0571.pdf>.
- [AS01] Arnborg, S. and Sjödin, G. ‘On the Foundations of Bayesianism’. In: *AIP Conference Proceedings*. Vol. 568. AIP Publishing, 2001, pp. 61–71.
- [AS14] Alvesson, M. and Sandberg, J. ‘Habitat and Habitus: Boxed-in versus Box-Breaking Research’. In: *Organization Studies* vol. 35, no. 7 (2014), pp. 967–987.
- [AS99] Arnborg, S. and Sjödin, G. *A Note on Foundations of Bayesianism*. Tech. rep. Royal Institute of Technology, 1999.
- [Bar26] Barber, G. *What Can We Gain by Losing Infinity?* Quanta Magazine. 29th Apr. 2026. URL: <https://www.quantamagazine.org/what-can-we-gain-by-losing-infinity-20260429/> (visited on 12/05/2026).

- [BDG25] Bentzen, B., Del Santo, F. and Gisin, N. *Naturalistic Intuitionism for Physics*. Preprint, PhilSci-Archive. 2025. URL: <https://philsci-archive.pitt.edu/26743/>.
- [BDK02] Beasley, M. R., Datta, S. and Kinyoun, H. H. *Report of the Investigation Committee on the Possibility of Scientific Misconduct in the Work of Hendrik Schön and Coauthors*. Tech. rep. Bell Labs, Lucent Technologies, 2002. URL: <https://s3.amazonaws.com/org.aps-content-assets/671ef6a6-c78d-4436-9e1c-e8135315bf30/lucentrep.pdf>.
- [BE12] Begley, C. G. and Ellis, L. M. ‘Drug Development: Raise Standards for Preclinical Cancer Research’. In: *Nature* vol. 483 (2012), pp. 531–533.
- [Bec+01] Beck, K. et al. *Manifesto for Agile Software Development*. 2001. URL: <https://agilemanifesto.org/> (visited on 26/05/2026).
- [BHM21] Bornmann, L., Haunschild, R. and Mutz, R. ‘Growth Rates of Modern Science: A Latent Piecewise Growth Curve Approach to Model Publication Numbers from Established and New Literature Databases’. In: *Humanities and Social Sciences Communications* vol. 8 (2021), p. 224.
- [Bik26a] Bik, E. *Is Science Self-Correcting? Not in This Elsevier Journal*. Science Integrity Digest. 24th June 2026. URL: <https://scienceintegritydigest.com/2026/06/24/is-science-self-correcting-not-in-this-elsevier-journal/> (visited on 01/07/2026).
- [Bik26b] Bik, E. *Science Integrity Digest*. Science Integrity Digest. 2nd July 2026. URL: <https://scienceintegritydigest.com/> (visited on 02/07/2026).
- [Bol25] Bollen, T. *Geld genoeg, maar niet voor jou: Hoe heroveren we de macht over ons geld?* Amsterdam: Follow the Money Publishing B.V., 2025.
- [Bot+20] Botvinik-Nezer, R. et al. ‘Variability in the Analysis of a Single Neuroimaging Dataset by Many Teams’. In: *Nature* vol. 582 (2020), pp. 84–88.
- [BR16] Baltag, A. and Renne, B. ‘Dynamic Epistemic Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. Winter 2016. Metaphysics Research Lab, Stanford University, 2016. URL: <https://plato.stanford.edu/archives/win2016/entries/dynamic-epistemic/>.
- [Bre+22] Breznau, N. et al. ‘Observing Many Researchers Using the Same Data and Hypothesis Reveals a Hidden Universe of Uncertainty’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 119, no. 44 (2022), e2203150119.
- [Bri25] Brierley, O. *How Bad Science Is Becoming Big Business*. Phys.org. 18th Aug. 2025. URL: <https://phys.org/news/2025-08-bad-science-big-business.html> (visited on 12/05/2026).

B. Proofs and arguments

- [BT24] Banach, S. and Tarski, A. ‘Sur la décomposition des ensembles de points en parties respectivement congruentes’. In: *Fundamenta Mathematicae* vol. 6, no. 1 (1924), pp. 244–277. URL: <https://eudml.org/doc/214280>.
- [But+13] Button, K. S. et al. ‘Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience’. In: *Nature Reviews Neuroscience* vol. 14 (2013), pp. 365–376.
- [Cam+16] Camerer, C. F. et al. ‘Evaluating Replicability of Laboratory Experiments in Economics’. In: *Science* vol. 351, no. 6280 (2016), pp. 1433–1436.
- [Cam98] Campanario, J. M. ‘Peer Review for Journals as It Stands Today—Part 2’. In: *Science Communication* vol. 19, no. 4 (1998), pp. 277–306.
- [Cas26] Castelvechi, D. ‘“It is incredible”: How AI is transforming mathematics’. In: *Nature* vol. 653, no. 8115 (19th May 2026), pp. 664–665. URL: <https://www.nature.com/articles/d41586-026-01553-1> (visited on 29/05/2026).
- [Cen26a] Center for Open Science. *Open Science Framework*. Center for Open Science. 2nd July 2026. URL: <https://osf.io/> (visited on 02/07/2026).
- [Cen26b] Center for Open Science. *Registered Reports*. Center for Open Science. 2nd July 2026. URL: <https://www.cos.io/initiatives/registered-reports> (visited on 02/07/2026).
- [CFN22] Cintula, P., Fermüller, C. G. and Noguera, C. ‘Fuzzy Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. Spring 2022. Metaphysics Research Lab, Stanford University, 2022. URL: <https://plato.stanford.edu/archives/spr2022/entries/logic-fuzzy/>.
- [Cle26] Clear Skies Ltd. *Clear Skies: Your Partner in Research Integrity*. Clear Skies Ltd. 2nd July 2026. URL: <https://clear-skies.co.uk/> (visited on 02/07/2026).
- [Col04] Colyvan, M. ‘The Philosophical Significance of Cox’s Theorem’. In: *International Journal of Approximate Reasoning* vol. 37, no. 1 (2004), pp. 71–85.
- [Coq25] Coquand, T. ‘Type Theory’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Fall 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/fall2025/entries/type-theory/>.
- [Cou23] Council of the European Union. *Council Calls for Transparent, Equitable, and Open Access to Scholarly Publications*. Council of the European Union. 23rd May 2023. URL: <https://www.consilium.europa.eu/en/press/press-releases/2023/05/23/council-calls-for-transparent-equitable-and-open-access-to-scholarly-publications/> (visited on 12/05/2026).
- [Cox46] Cox, R. T. ‘Probability, Frequency and Reasonable Expectation’. In: *American Journal of Physics* vol. 14, no. 1 (1946), pp. 1–13.

- [Cox61] Cox, R. T. *The Algebra of Probable Inference*. Baltimore: Johns Hopkins Press, 1961. URL: <https://www.press.jhu.edu/books/title/1089/algebra-probable-inference>.
- [CW17] Clayton, A. and Waddington, T. ‘Bridging the Intuition Gap in Cox’s Theorem: A Jaynesian Argument for Universality’. In: *International Journal of Approximate Reasoning* vol. 80 (2017), pp. 36–51.
- [Cyr06] Cyranoski, D. ‘Verdict: Hwang’s Human Stem Cells Were All Fakes’. In: *Nature* vol. 439, no. 7073 (2006), p. 122.
- [DG19] Del Santo, F. and Gisin, N. ‘Physics Without Determinism: Alternative Interpretations of Classical Physics’. In: *Physical Review A* vol. 100 (2019), p. 062107.
- [DIA26] DIAMAS Project. *Developing Institutional Open Access Publishing Models to Advance Scholarly Communication*. DIAMAS. 2nd July 2026. URL: <https://diamasproject.eu/> (visited on 02/07/2026).
- [DOR12] DORA. *San Francisco Declaration on Research Assessment*. DORA. 16th Dec. 2012. URL: <https://sfdora.org/read/> (visited on 12/05/2026).
- [DP24] Dybjer, P. and Palmgren, E. ‘Intuitionistic Type Theory’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Winter 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/win2024/entries/type-theory-intuitionistic/>.
- [Era93] Erasmus, D. *Praise of Folly and Letter to Maarten van Dorp*. Ed. by Levi, A. H. T. Trans. by Radice, B. Penguin Classics. Latin original *Moriae Encomium*, Paris: Gilles de Gourmont, 1511. Harmondsworth: Penguin, 1993.
- [Eur26] European Commission. *Open Research Europe*. European Commission. 2nd July 2026. URL: <https://open-research-europe.ec.europa.eu/> (visited on 02/07/2026).
- [Fan09] Fanelli, D. ‘How Many Scientists Fabricate and Falsify Research? A Systematic Review and Meta-Analysis of Survey Data’. In: *PLOS ONE* vol. 4, no. 5 (2009), e5738.
- [Fey75] Feyerabend, P. *Against Method: Outline of an Anarchistic Theory of Knowledge*. London: New Left Books, 1975.
- [FK23] Fritz, T. and Klingler, A. ‘The d-Separation Criterion in Categorical Probability’. In: *Journal of Machine Learning Research* vol. 24, no. 46 (2023), pp. 1–49. URL: <https://jmlr.org/papers/v24/22-0916.html>.
- [Fle79] Fleck, L. *Genesis and Development of a Scientific Fact*. Ed. by Trenn, T. J. and Merton, R. K. Trans. by Bradley, F. and Trenn, T. J. Foreword by Thomas S. Kuhn. German original *Entstehung und Entwicklung einer wissenschaftlichen Tatsache*, Schwabe, Basel, 1935. Chicago: University of Chicago Press, 1979.

B. Proofs and arguments

- [FSC12] Fang, F. C., Steen, R. G. and Casadevall, A. ‘Misconduct Accounts for the Majority of Retracted Scientific Publications’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 109, no. 42 (2012), pp. 17028–17033.
- [Gar24] Garson, J. ‘Modal Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Spring 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/spr2024/entries/logic-modal/>.
- [GB24] Gross, K. and Bergstrom, C. T. ‘Rationalizing Risk Aversion in Science: Why Incentives to Work Hard Clash with Incentives to Take Risks’. In: *PLOS Biology* vol. 22, no. 8 (15th Aug. 2024), e3002750.
- [Gel14] Gelman, A. *Scientists Behaving Badly*. Statistical Modeling, Causal Inference, and Social Science. 7th Nov. 2014. URL: <https://statmodeling.stat.columbia.edu/2014/11/07/scientists-behaving-badly/> (visited on 12/05/2026).
- [GHR17] Gyenis, Z., Hofer-Szabó, G. and Rédei, M. ‘Conditioning Using Conditional Expectations: The Borel–Kolmogorov Paradox’. In: *Synthese* vol. 194, no. 7 (2017), pp. 2595–2630.
- [Gis20] Gisin, N. ‘Mathematical Languages Shape Our Understanding of Time in Physics’. In: *Nature Physics* vol. 16 (2020), pp. 114–116.
- [GL13] Gelman, A. and Loken, E. *The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No “Fishing Expedition” or “p-Hacking” and the Research Hypothesis Was Posited Ahead of Time*. Tech. rep. Department of Statistics, Columbia University, 2013. URL: https://www.stat.columbia.edu/~gelman/research/unpublished/p_hacking.pdf.
- [GL14] Gelman, A. and Loken, E. ‘The Statistical Crisis in Science’. In: *American Scientist* vol. 102, no. 6 (2014), pp. 460–465.
- [Gol12] Goldacre, B. *Bad Pharma: How Drug Companies Mislead Doctors and Harm Patients*. London: Fourth Estate, 2012.
- [GR25] Goranko, V. and Rumberg, A. ‘Temporal Logic’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/sum2025/entries/logic-temporal/>.
- [Gra+21] Gratzer, D. et al. ‘Multimodal Dependent Type Theory’. In: *Logical Methods in Computer Science* vol. 17, no. 3 (28th July 2021), 11:1–11:67. URL: <https://lmcs.episciences.org/7571>.
- [Gra23] Gratzer, D. ‘Syntax and Semantics of Modal Type Theory’. PhD thesis. Aarhus University, 2023. URL: <https://iris-project.org/pdfs/2023-phd-gratzer.pdf>.
- [GSB19] Gratzer, D., Sterling, J. and Birkedal, L. ‘Implementing a Modal Dependent Type Theory’. In: *Proceedings of the ACM on Programming Languages* vol. 3, no. ICFP (2019), 107:1–107:29.

- [GSM11] Godlee, F., Smith, J. and Marcovitch, H. ‘Wakefield’s Article Linking MMR Vaccine and Autism Was Fraudulent’. In: *BMJ* vol. 342 (2011), p. c7452. URL: <https://www.bmj.com/content/342/bmj.c7452>.
- [Hac83] Hacking, I. *Representing and Intervening: Introductory Topics in the Philosophy of Natural Science*. Cambridge: Cambridge University Press, 1983.
- [Hal99a] Halpern, J. Y. ‘A Counterexample to Theorems of Cox and Fine’. In: *Journal of Artificial Intelligence Research* vol. 10 (1999), pp. 67–85. URL: <https://www.jair.org/index.php/jair/article/view/10223>.
- [Hal99b] Halpern, J. Y. ‘Cox’s Theorem Revisited’. In: *Journal of Artificial Intelligence Research* vol. 11 (1999), pp. 429–435. URL: <https://arxiv.org/abs/cs/9911012>.
- [Hof23] Hofweber, T. ‘Logic and Ontology’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2023. Metaphysics Research Lab, Stanford University, 2023. URL: <https://plato.stanford.edu/archives/sum2023/entries/logic-ontology/>.
- [HP07] Halpern, J. Y. and Pucella, R. ‘Dealing with Logical Omniscience’. In: *Proceedings of the Tenth Conference on Theoretical Aspects of Rationality and Knowledge*. Singapore, 2007, pp. 169–181. URL: http://www.tark.org/proceedings/tark_jun25_07/p169-halpern.pdf.
- [HS10] Harrigan, N. and Spekkens, R. W. ‘Einstein, Incompleteness, and the Epistemic View of Quantum States’. In: *Foundations of Physics* vol. 40, no. 2 (2010), pp. 125–157.
- [Hui49] Huizinga, J. *Homo Ludens: A Study of the Play-Element in Culture*. London: Routledge & Kegan Paul, 1949.
- [Hum75] Hume, D. *An Enquiry concerning Human Understanding*. Ed. by Selby-Bigge, L. A. and Nidditch, P. H. 3rd ed. Originally published as *Philosophical Essays concerning Human Understanding*, London: A. Millar, 1748. Oxford: Clarendon Press, 1975.
- [Hum78] Hume, D. *A Treatise of Human Nature: Being an Attempt to Introduce the Experimental Method of Reasoning into Moral Subjects*. Ed. by Selby-Bigge, L. A. and Nidditch, P. H. 2nd ed. Originally published London: John Noon, 1739–1740. Oxford: Clarendon Press, 1978.
- [HV06] Huang, Y. and Valtorta, M. ‘Pearl’s Calculus of Intervention Is Complete’. In: *Proceedings of the Twenty-Second Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2006, pp. 217–224. URL: <https://arxiv.org/abs/1206.6831>.
- [Ibá99] Ibáñez, C. F. ‘Researchers Face ‘Catch-22’ Grants Trap’. In: *Nature* vol. 398 (1999), p. 280.
- [Iem24] Iemhoff, R. ‘Intuitionism in the Philosophy of Mathematics’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Summer 2024. Metaphysics Research Lab, Stanford University, 2024. URL: <https://plato.stanford.edu/archives/sum2024/entries/intuitionism/>.

B. Proofs and arguments

- [Ioa05] Ioannidis, J. P. A. ‘Why Most Published Research Findings Are False’. In: *PLOS Medicine* vol. 2, no. 8 (2005), e124.
- [Jay03] Jaynes, E. T. *Probability Theory: The Logic of Science*. Cambridge: Cambridge University Press, 2003.
- [Jef61] Jeffreys, H. *Theory of Probability*. 3rd ed. Oxford: Clarendon Press, 1961.
- [JLP12] John, L. K., Loewenstein, G. and Prelec, D. ‘Measuring the Prevalence of Questionable Research Practices with Incentives for Truth Telling’. In: *Psychological Science* vol. 23, no. 5 (2012), pp. 524–532.
- [Kak26] Kakaes, K. *The AI Revolution in Math Has Arrived*. Quanta Magazine. 13th Apr. 2026. URL: <https://www.quantamagazine.org/the-ai-revolution-in-math-has-arrived-20260413/> (visited on 12/05/2026).
- [Kan98] Kant, I. *Critique of Pure Reason*. Ed. and trans. by Guyer, P. and Wood, A. W. The Cambridge Edition of the Works of Immanuel Kant. German original *Kritik der reinen Vernunft*, Riga: Hartknoch, 1st ed. 1781 (A) / 2nd ed. 1787 (B). Cambridge: Cambridge University Press, 1998.
- [Key21] Keynes, J. M. *A Treatise on Probability*. London: Macmillan, 1921. URL: <https://archive.org/details/treatiseonprobab007528mbp>.
- [Kie80] Kierkegaard, S. *The Concept of Anxiety: A Simple Psychologically Orienting Deliberation on the Dogmatic Issue of Hereditary Sin*. Ed. and trans. by Thomte, R. and Anderson, A. B. Kierkegaard’s Writings VIII. Danish original *Begrebet Angest*, Reitzel, Copenhagen, 1844. Princeton, NJ: Princeton University Press, 1980.
- [Kie83] Kierkegaard, S. *Fear and Trembling / Repetition*. Ed. and trans. by Hong, H. V. and Hong, E. H. Kierkegaard’s Writings VI. Danish original *Frygt og Bæven*, Reitzel, Copenhagen, 1843. Princeton, NJ: Princeton University Press, 1983.
- [Kie89] Kierkegaard, S. *The Concept of Irony, with Continual Reference to Socrates*. Ed. and trans. by Hong, H. V. and Hong, E. H. Kierkegaard’s Writings II. Danish original *Om Begrebet Ironi med stadigt Hensyn til Socrates*, Reitzel, Copenhagen, 1841. Princeton, NJ: Princeton University Press, 1989.
- [Kle+14] Klein, R. A. et al. ‘Investigating Variation in Replicability: A “Many Labs” Replication Project’. In: *Social Psychology* vol. 45, no. 3 (2014), pp. 142–152.
- [Kon15] Konnikova, M. *How a Gay-Marriage Study Went Wrong*. The New Yorker. 22nd May 2015. URL: <https://www.newyorker.com/science/maria-konnikova/how-a-gay-marriage-study-went-wrong> (visited on 12/05/2026).
- [Lan+22] Lane, J. N. et al. ‘Conservatism Gets Funded? A Field Experiment on the Role of Negative Information in Novel Project Evaluation’. In: *Management Science* vol. 68, no. 6 (2022), pp. 4478–4495.

- [Lan16] Landhuis, E. ‘Information Overload’. In: *Nature* vol. 535 (2016), pp. 457–458.
- [Los11] Los Angeles Times. *Dutch Scientist Accused of Falsifying Data*. Los Angeles Times. 6th Nov. 2011. URL: <https://www.latimes.com/science/la-sci-science-fraud-20111106-story.html> (visited on 12/05/2026).
- [Mar84] Martin-Löf, P. *Intuitionistic Type Theory*. Naples: Bibliopolis, 1984.
- [MAV05] Martinson, B. C., Anderson, M. S. and Vries, R. de. ‘Scientists Behaving Badly’. In: *Nature* vol. 435 (2005), pp. 737–738.
- [McE20] McElreath, R. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. 2nd ed. Boca Raton: CRC Press, 2020. URL: <https://xcelab.net/rm/>.
- [MO15] Marcus, A. and Oransky, I. *How the Biggest Fabricator in Science Got Caught*. Nautilus. 18th June 2015. URL: <https://nautil.us/how-the-biggest-fabricator-in-science-got-caught-235421/> (visited on 12/05/2026).
- [MU21] Moura, L. de and Ullrich, S. ‘The Lean 4 Theorem Prover and Programming Language’. In: *Automated Deduction — CADE 28*. Ed. by Platzer, A. and Sutcliffe, G. Vol. 12699. Lecture Notes in Computer Science. Cham: Springer, 2021, pp. 625–635. URL: https://doi.org/10.1007/978-3-030-79876-5_37.
- [Mun+17] Munafò, M. R. et al. ‘A Manifesto for Reproducible Science’. In: *Nature Human Behaviour* vol. 1 (10th Jan. 2017), p. 0021.
- [Nad26] Naddaf, M. *How Much for a Fake Authorship? Ad Database Reveals Secrets of Scientific Fraud*. Nature. 24th Apr. 2026. URL: <https://www.nature.com/articles/d41586-026-01340-y> (visited on 12/05/2026).
- [Nat26] Nature Editorial. ‘We Need to Talk about Failure in Science’. In: *Nature* vol. 652 (23rd Apr. 2026), p. 1091.
- [NDR18] NDR. *More than 5,000 German Scientists Have Published Papers in Pseudo-Scientific Journals*. NDR. 19th July 2018. URL: https://www.ndr.de/der_ndr/presse/More-than-5000-German-scientists-have-published-papers-in-pseudo-scientific-journals,fakescience178.html (visited on 12/05/2026).
- [NI12] Nicholson, J. M. and Ioannidis, J. P. A. ‘Conform and Be Funded’. In: *Nature* vol. 492 (2012), pp. 34–36.
- [Nos+15] Nosek, B. A. et al. ‘Promoting an Open Research Culture’. In: *Science* vol. 348, no. 6242 (2015), pp. 1422–1425.
- [NPS90] Nordström, B., Petersson, K. and Smith, J. M. *Programming in Martin-Löf’s Type Theory: An Introduction*. Oxford: Oxford University Press, 1990. URL: <https://intuitionistic.wordpress.com/wp-content/uploads/2010/07/nordstrom-et-al-type-theory.pdf>.

B. Proofs and arguments

- [NR19] Nguyen, H. N. and Rakib, A. ‘A Probabilistic Logic for Resource-Bounded Multi-Agent Systems’. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2019, pp. 521–527.
- [NSM12] Nosek, B. A., Spies, J. R. and Motyl, M. ‘Scientific Utopia: II. Restructuring Incentives and Practices to Promote Truth over Publishability’. In: *Perspectives on Psychological Science* vol. 7, no. 6 (2012), pp. 615–631.
- [OC10] Oreskes, N. and Conway, E. M. *Merchants of Doubt: How a Handful of Scientists Obscured the Truth on Issues from Tobacco Smoke to Global Warming*. New York: Bloomsbury Press, 2010.
- [OGr24] O’Grady, C. *Failure at Every Level: How Science Sleuths Exposed Massive Ethics Violations at a Famed French Institute*. Science. 5th Sept. 2024. URL: <https://www.science.org/content/article/failure-every-level-how-science-sleuths-exposed-massive-ethics-violations-famed-french> (visited on 12/05/2026).
- [OGr25] O’Grady, C. *Scientific Fraud Has Become an ‘Industry,’ Alarming Analysis Finds*. Science. 4th Aug. 2025. URL: <https://www.science.org/content/article/scientific-fraud-has-become-industry-alarming-analysis-finds> (visited on 12/05/2026).
- [OM23] Oransky, I. and Marcus, A. *There’s Far More Scientific Fraud than Anyone Wants to Admit*. The Guardian. 9th Aug. 2023. URL: <https://www.theguardian.com/commentisfree/2023/aug/09/scientific-misconduct-retraction-watch> (visited on 12/05/2026).
- [Ope15] Open Science Collaboration. ‘Estimating the Reproducibility of Psychological Science’. In: *Science* vol. 349, no. 6251 (2015), aac4716.
- [Orn26] Ornes, S. *Math Long Resisted a Digital Disruption. AI Is Poised to Change That*. Science News. 15th Apr. 2026. URL: <https://www.sciencenews.org/article/math-disrupted-by-ai-verify-proofs> (visited on 12/05/2026).
- [PBR12] Pusey, M. F., Barrett, J. and Rudolph, T. ‘On the Reality of the Quantum State’. In: *Nature Physics* vol. 8 (2012), pp. 475–478.
- [Pea09] Pearl, J. *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge: Cambridge University Press, 2009.
- [Pea12] Pearl, J. ‘The Do-Calculus Revisited’. In: *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2012, pp. 4–11. URL: https://ftp.cs.ucla.edu/pub/stat_ser/r402.pdf.
- [PLF23] Park, M., Leahey, E. and Funk, R. J. ‘Papers and Patents Are Becoming Less Disruptive over Time’. In: *Nature* vol. 613 (2023), pp. 138–144.
- [PM18] Pearl, J. and Mackenzie, D. *The Book of Why: The New Science of Cause and Effect*. New York: Basic Books, 2018. URL: <https://www.basicbooks.com/titles/judea-pearl/the-book-of-why/9780465097616/>.

B.13. Completeness consequences and boundary

- [Pub26] PubPeer Foundation. *PubPeer*. PubPeer Foundation. 2nd July 2026. URL: <https://pubpeer.com/> (visited on 02/07/2026).
- [Ram31] Ramsey, F. P. ‘Truth and Probability’. In: *The Foundations of Mathematics and Other Logical Essays*. Ed. by Braithwaite, R. B. Written in 1926. London: Kegan Paul, Trench, Trubner & Co., 1931, pp. 156–198.
- [Ret26] Retraction Watch. *Retraction Watch*. Retraction Watch. 2nd July 2026. URL: <https://retractionwatch.com/> (visited on 02/07/2026).
- [Ric+25] Richardson, R. A. K. et al. ‘The Entities Enabling Scientific Fraud at Scale Are Large, Resilient, and Growing Rapidly’. In: *Proceedings of the National Academy of Sciences of the United States of America* vol. 122, no. 32 (12th Aug. 2025), e2420092122.
- [Sch+21] Schweinsberg, M. et al. ‘Same Data, Different Conclusions: Radical Dispersion in Empirical Results When Independent Analysts Operationalize and Test the Same Hypothesis’. In: *Organizational Behavior and Human Decision Processes* vol. 165 (2021), pp. 228–249.
- [Sch13] Schekman, R. *How Journals Like Nature, Cell and Science Are Damaging Science*. The Guardian. 9th Dec. 2013. URL: <https://www.theguardian.com/commentisfree/2013/dec/09/how-journals-nature-science-cell-damage-science> (visited on 12/05/2026).
- [Sha04] Shafer, G. ‘Comments on “Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem”, by Kevin S. Van Horn’. In: *International Journal of Approximate Reasoning* vol. 35 (2004), pp. 97–105.
- [Sil+18] Silberzahn, R. et al. ‘Many Analysts, One Dataset: Making Transparent How Variations in Analytical Choices Affect Results’. In: *Advances in Methods and Practices in Psychological Science* vol. 1, no. 3 (2018), pp. 337–356.
- [Sim55] Simon, H. A. ‘A Behavioral Model of Rational Choice’. In: *The Quarterly Journal of Economics* vol. 69, no. 1 (1955), pp. 99–118.
- [Sku26] Skuse, B. *What It Means to Be a Mathematician When AI Does the Math*. IEEE Spectrum. 25th June 2026. URL: <https://spectrum.ieee.org/ai-in-mathematics> (visited on 09/07/2026).
- [Slo26] Sloman, L. *In Math, Rigor Is Vital. But Are Digitized Proofs Taking It Too Far?* Quanta Magazine. 25th Mar. 2026. URL: <https://www.quantamagazine.org/in-math-rigor-is-vital-but-are-digitized-proofs-taking-it-too-far-20260325/> (visited on 12/05/2026).
- [SM16] Smaldino, P. E. and McElreath, R. ‘The Natural Selection of Bad Science’. In: *Royal Society Open Science* vol. 3 (2016), p. 160384.
- [Smi06] Smith, R. ‘Peer Review: A Flawed Process at the Heart of Science and Journals’. In: *Journal of the Royal Society of Medicine* vol. 99, no. 4 (2006), pp. 178–182.

B. Proofs and arguments

- [SN25] Steup, M. and Neta, R. ‘Epistemology’. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Zalta, E. N. and Nodelman, U. Spring 2025. Metaphysics Research Lab, Stanford University, 2025. URL: <https://plato.stanford.edu/archives/spr2025/entries/epistemology/>.
- [Sno98] Snow, P. ‘On the Correctness and Reasonableness of Cox’s Theorem for Finite Domains’. In: *Computational Intelligence* vol. 14, no. 3 (1998), pp. 452–459.
- [SNS11] Simmons, J. P., Nelson, L. D. and Simonsohn, U. ‘False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant’. In: *Psychological Science* vol. 22, no. 11 (2011), pp. 1359–1366.
- [SP06] Shpitser, I. and Pearl, J. ‘Identification of Joint Interventional Distributions in Recursive Semi-Markovian Causal Models’. In: *Proceedings of the Twenty-First National Conference on Artificial Intelligence*. AAAI Press, 2006, pp. 1219–1226.
- [SP08] Shpitser, I. and Pearl, J. ‘Complete Identification Methods for the Causal Hierarchy’. In: *Journal of Machine Learning Research* vol. 9 (2008), pp. 1941–1979. URL: <https://jmlr.org/papers/v9/shpitser08a.html>.
- [Spa18] Spaink, K. *Profileer- en Publicatiedwang: Wetenschap in de Greep van het Geld*. Follow the Money. 8th Aug. 2018. URL: <https://www.ftm.nl/artikelen/publicatiedwang> (visited on 12/05/2026).
- [Spe07] Spekkens, R. W. ‘Evidence for the Epistemic View of Quantum States: A Toy Theory’. In: *Physical Review A* vol. 75, no. 3 (2007), p. 032110. arXiv: [quant-ph/0401052](https://arxiv.org/abs/quant-ph/0401052).
- [SR14] Spicer, A. and Roulet, T. *Hate the Peer-Review Process? Einstein Did Too*. The Conversation. 2nd June 2014. URL: <https://theconversation.com/hate-the-peer-review-process-einstein-did-too-27405> (visited on 12/05/2026).
- [SS20] Schwaber, K. and Sutherland, J. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. 2020. URL: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf> (visited on 26/05/2026).
- [The16] The New Yorker. *The Stem-Cell Scandal*. The New Yorker. 29th Feb. 2016. URL: <https://www.newyorker.com/magazine/2016/02/29/the-stem-cell-scandal> (visited on 12/05/2026).
- [The24] ThePrint. *India’s Research Crime Is Getting Worse. Scientists Are Gaming Peer Review System*. ThePrint. 14th Sept. 2024. URL: <https://theprint.in/ground-reports/indias-research-crime-is-getting-worse-scientists-are-gaming-peer-review-system/2261884/> (visited on 12/05/2026).
- [The26] The Black Spatula Project. *The Black Spatula Project*. Project website. 2nd July 2026. URL: <https://the-black-spatula-project.github.io/> (visited on 02/07/2026).

- [Tho54] Thomson, J. F. ‘Tasks and Super-Tasks’. In: *Analysis* vol. 15, no. 1 (1st Oct. 1954), pp. 1–13. URL: <https://academic.oup.com/analysis/article-abstract/15/1/1/170170>.
- [TP02] Tian, J. and Pearl, J. ‘A General Identification Condition for Causal Effects’. In: *Proceedings of the Eighteenth National Conference on Artificial Intelligence*. AAAI Press, 2002, pp. 567–573.
- [TS09] Tian, J. and Shpitser, I. ‘On Identifying Causal Effects’. In: *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. AUAI Press, 2009, pp. 722–729.
- [Van03] Van Horn, K. S. ‘Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem’. In: *International Journal of Approximate Reasoning* vol. 34, no. 1 (2003), pp. 3–24.
- [Vit05] Vitali, G. *Sul problema della misura dei gruppi di punti di una retta: Nota*. Bologna: Tip. Gamberini e Parmeggiani, 1905, p. 5. URL: <https://web.unica.it/unica/protected/417040/0/def/ref/MAT246835/>.
- [Wel26] Wellcome Open Research. *Wellcome Open Research*. Wellcome Open Research. 2nd July 2026. URL: <https://wellcomeopenresearch.org/> (visited on 02/07/2026).
- [Zha26] Zhao, C. ‘AI Has Helped Scientists—but May Have Hurt Science’. In: *Science* vol. 391, no. 6783 (22nd Jan. 2026), p. 337.
- [Zub19] Zuboff, S. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. London: Profile Books, 2019.

